Deep Neural Networks
Scanning for patterns
(aka convolutional networks)

Bhiksha Raj

11-785, Fall 2025

Attendance poll: @623

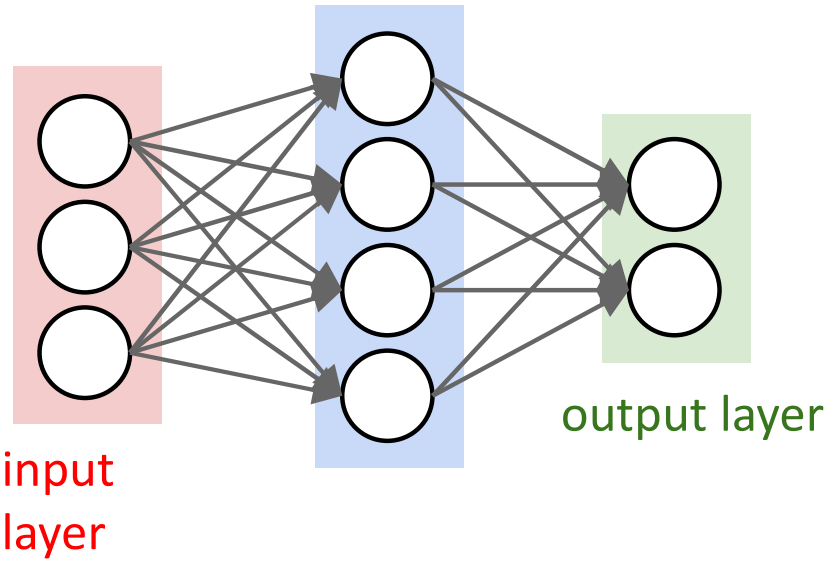
Story so far

- **MLPs are universal function approximators**
  - Boolean functions, classifiers, and regressions
- **MLPs can be trained through variations of gradient descent**
  - Gradients can be computed by backpropagation

The model so far

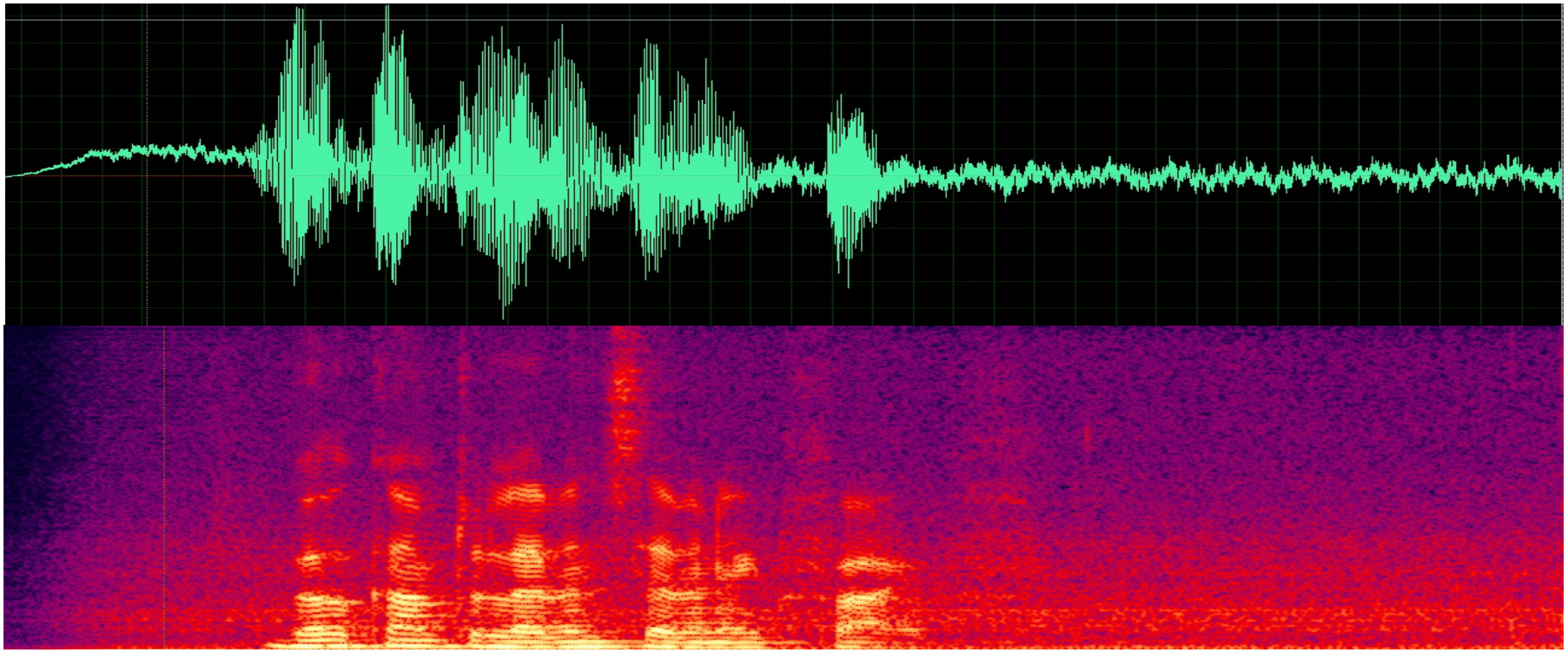


Or, more generally
a vector input



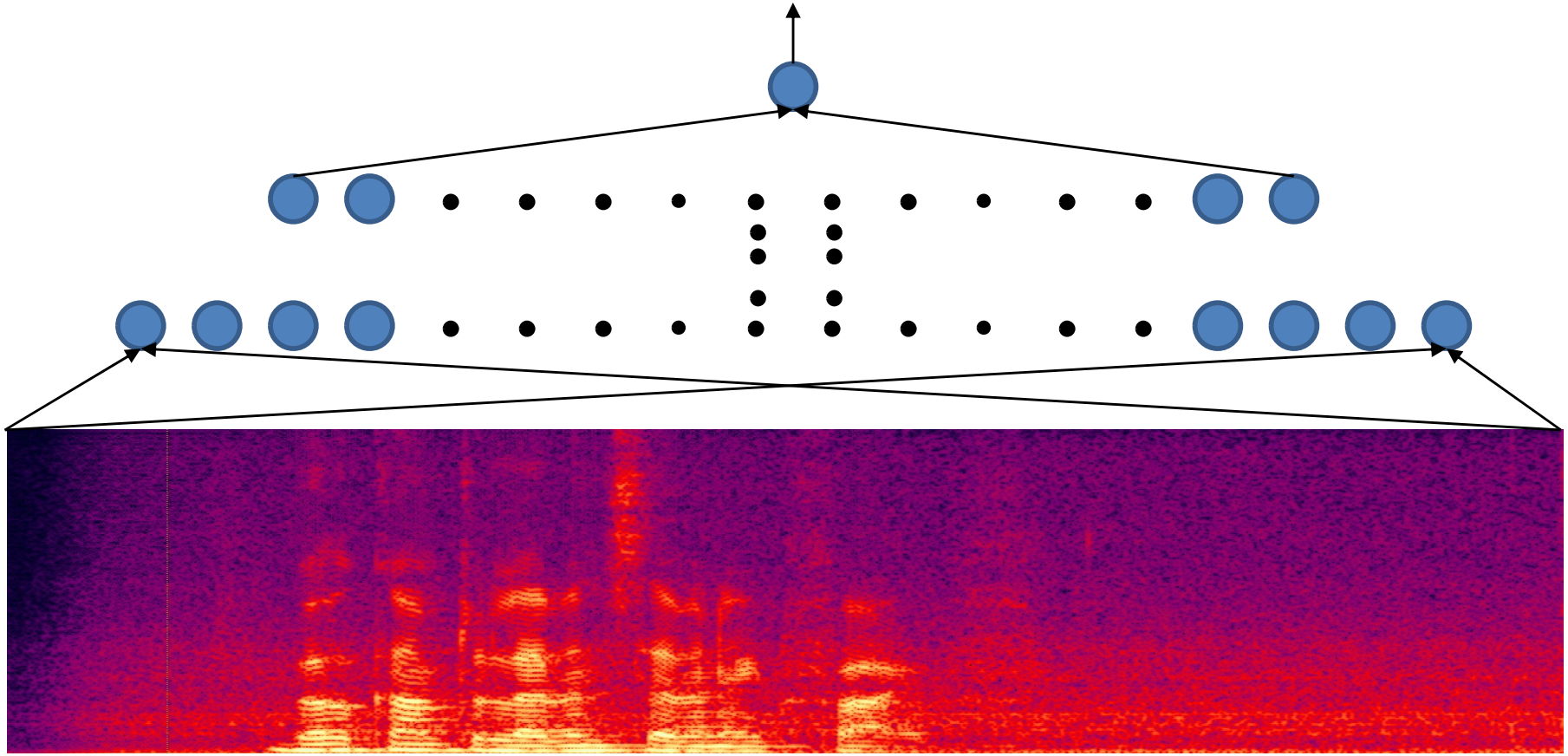
- Can recognize patterns in data
  - E.g. digits
  - Or any other vector data

A new problem



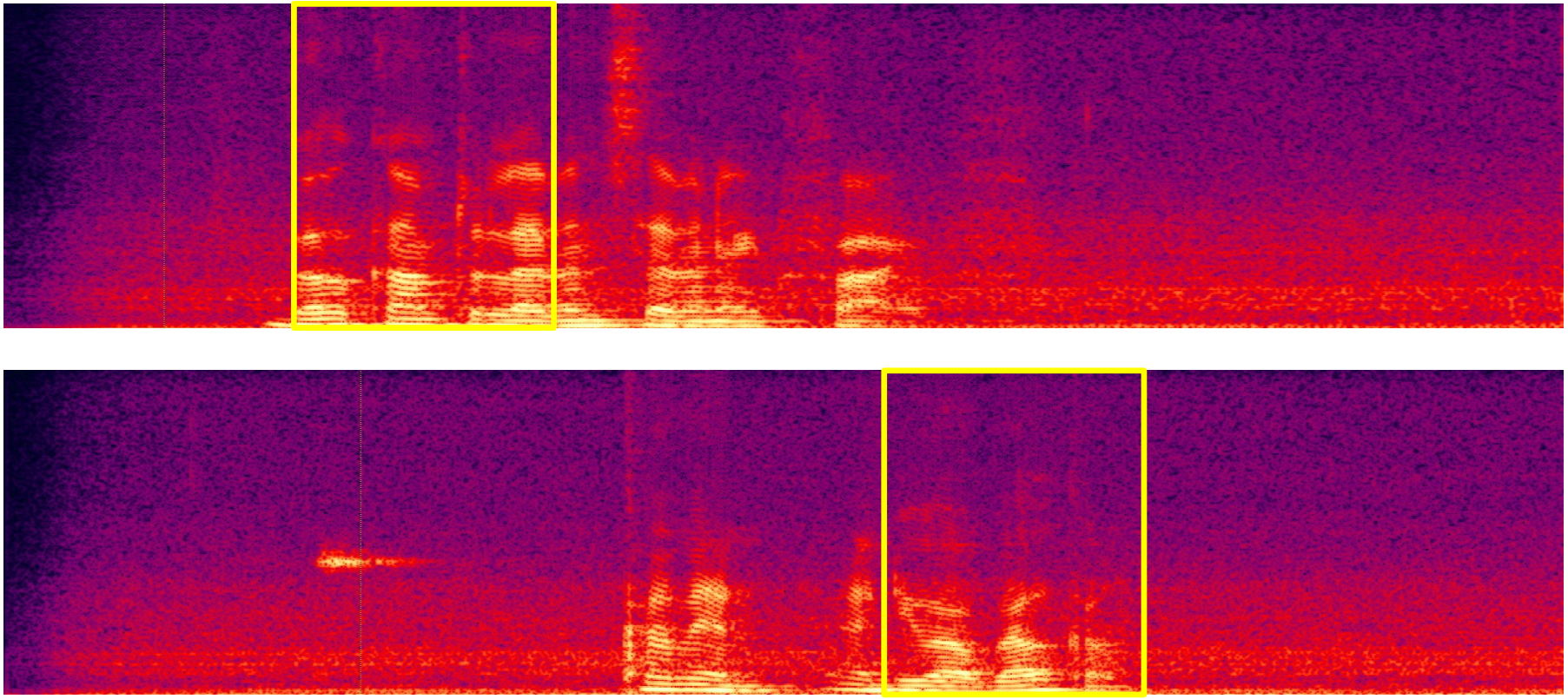
- Does this signal contain the word “Welcome”?
- Compose an MLP for this problem.
  - Assuming all recordings are exactly the same length..

Finding a Welcome



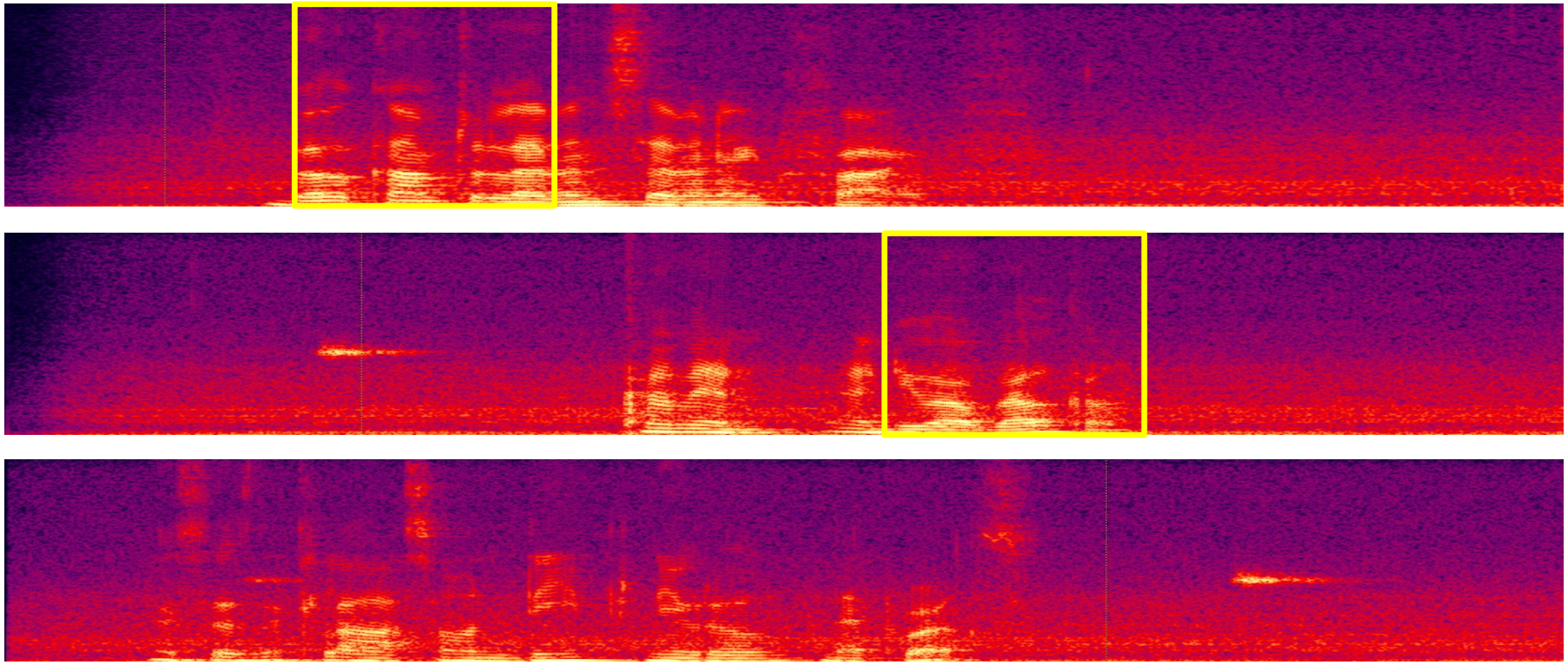
- Trivial solution: Train an MLP for the entire recording

Finding a Welcome



- Problem with trivial solution: Network that finds a “welcome” in the top recording will not find it in the lower one
  - Unless trained with both
  - Will require a very large network and a large amount of training data to cover every case

Finding a Welcome



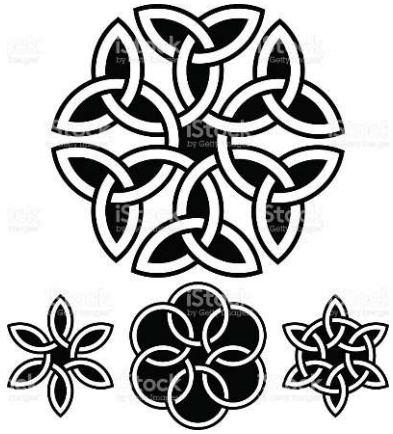
- Need a *simple* network that will fire regardless of the location of “Welcome”
  - and not fire when there is none

Flowers

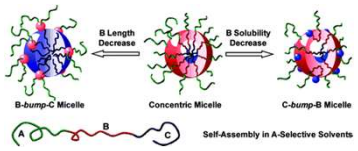






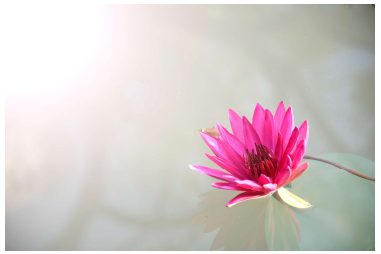






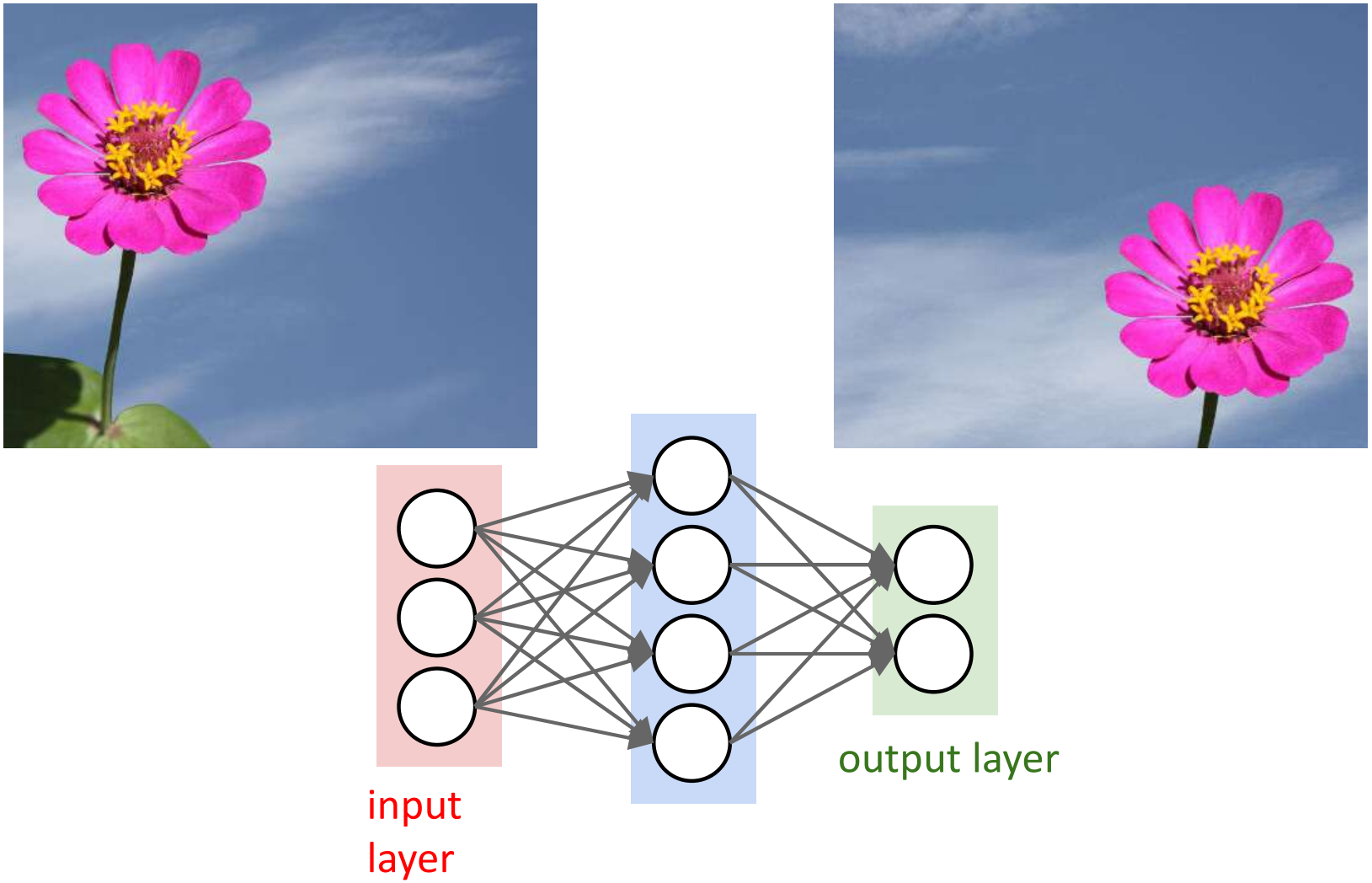






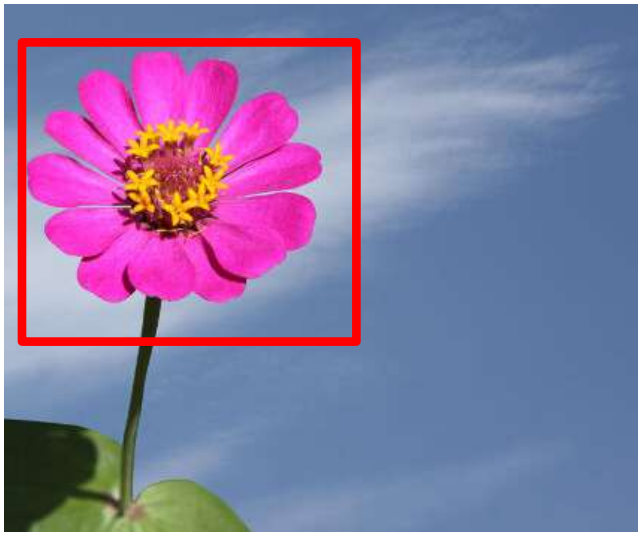
- Is there a flower in any of these images

A problem



- Will an MLP that recognizes the left image as a flower also recognize the one on the right as a flower?

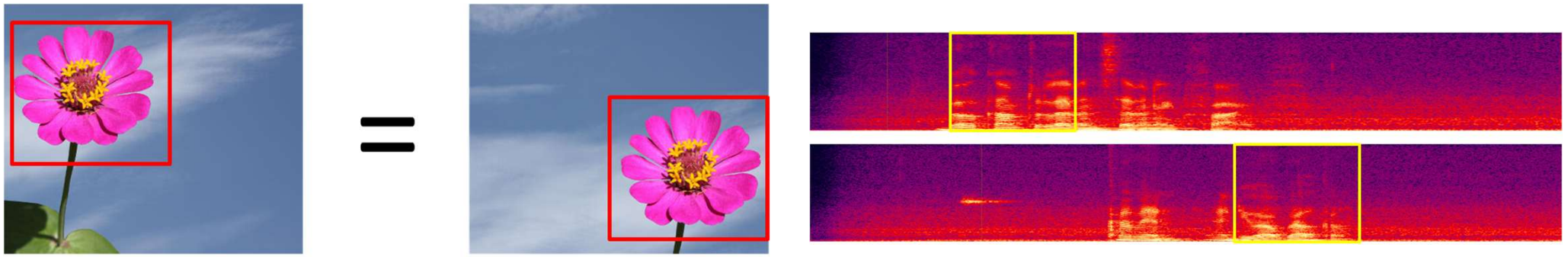
A problem





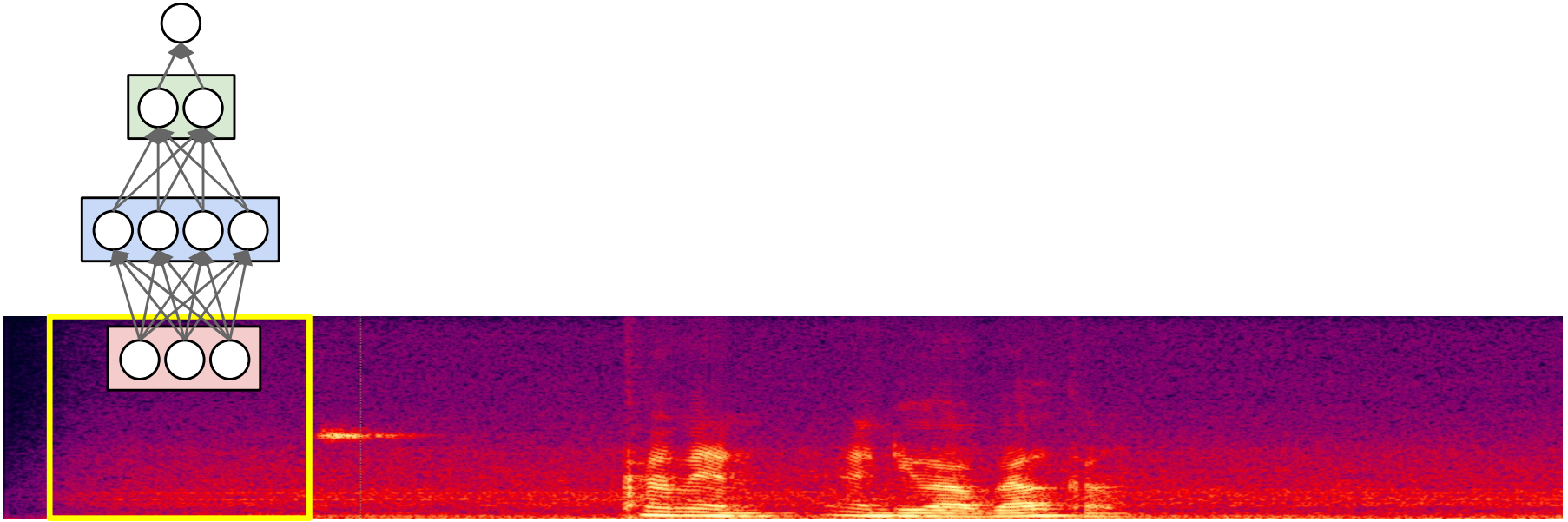
- Need a network that will “fire” regardless of the precise location of the target object

The need for *shift invariance*



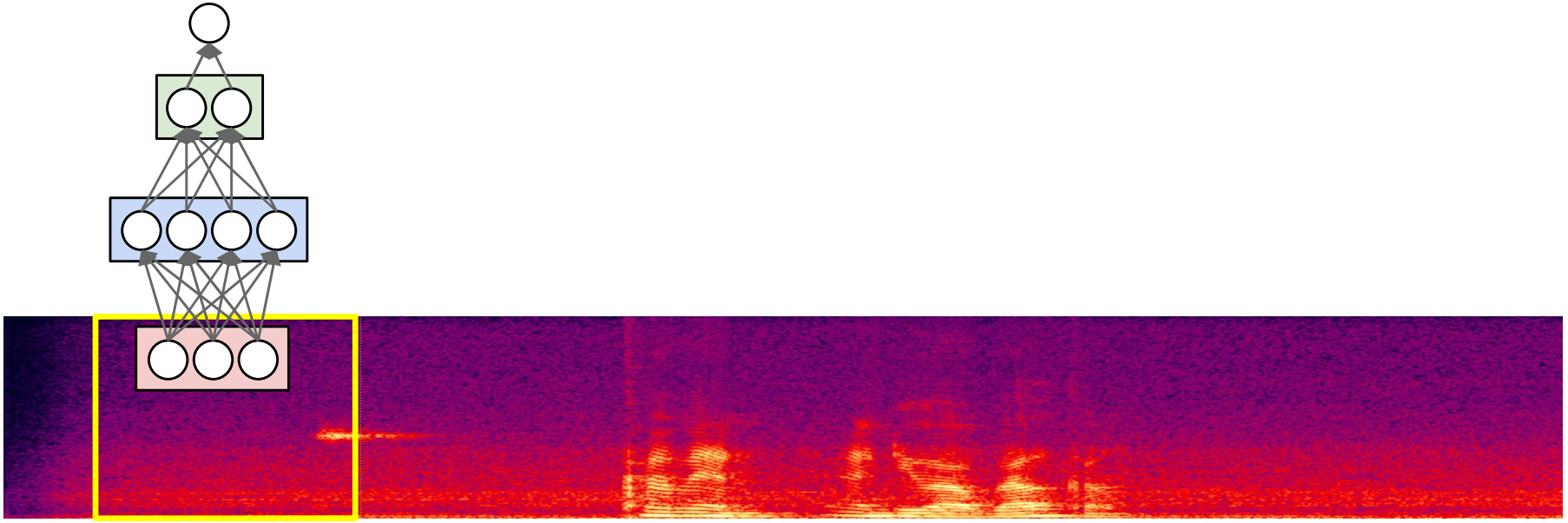
- In many problems the *location* of a pattern is not important
  - Only the presence of the pattern
- Conventional MLPs are sensitive to the location of the pattern
  - Moving it by one component results in an entirely different input that the MLP won't recognize
- Requirement: Network must be *shift invariant*

Solution: Scan



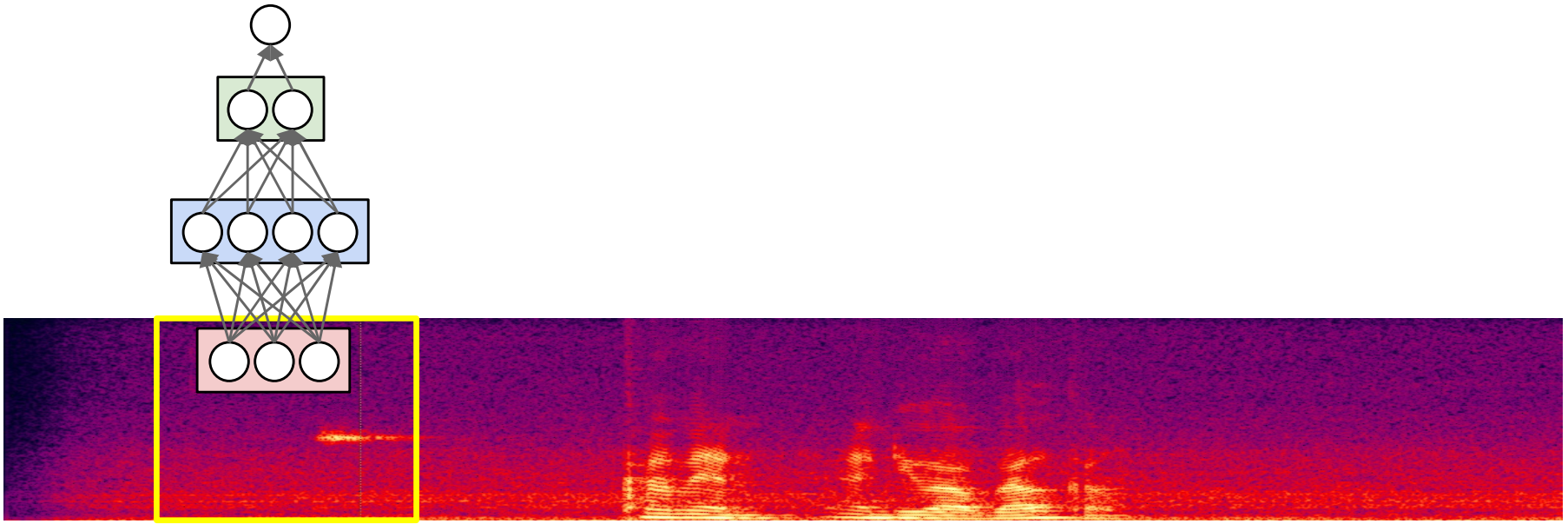
- Scan for the target word
  - The spectral time-frequency components in a "window" are input to a "welcome-detector" MLP

Solution: Scan



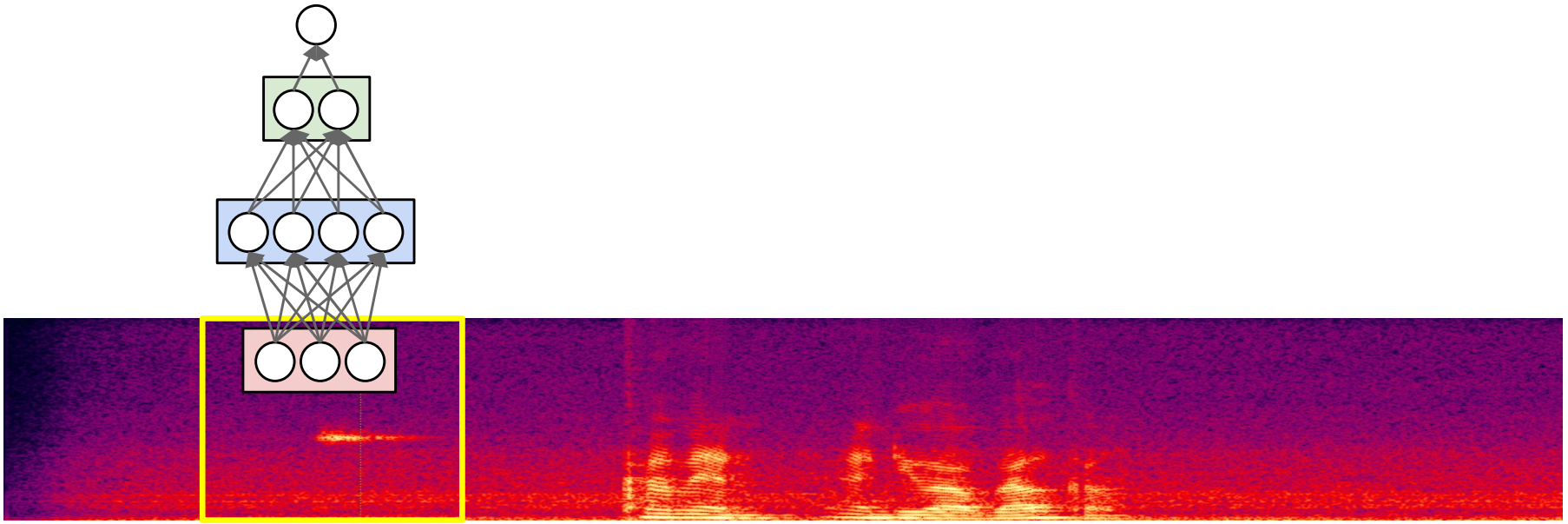
- Scan for the target word
  - The spectral time-frequency components in a "window" are input to a "welcome-detector" MLP

Solution: Scan



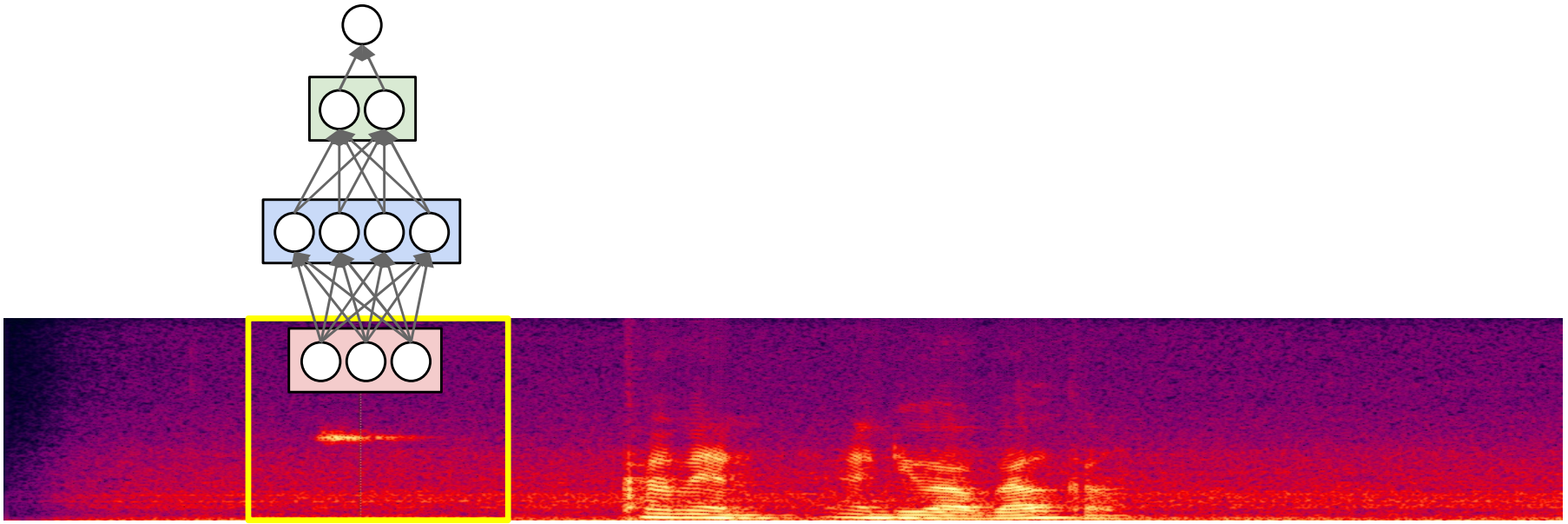
- Scan for the target word
  - The spectral time-frequency components in a "window" are input to a "welcome-detector" MLP

Solution: Scan



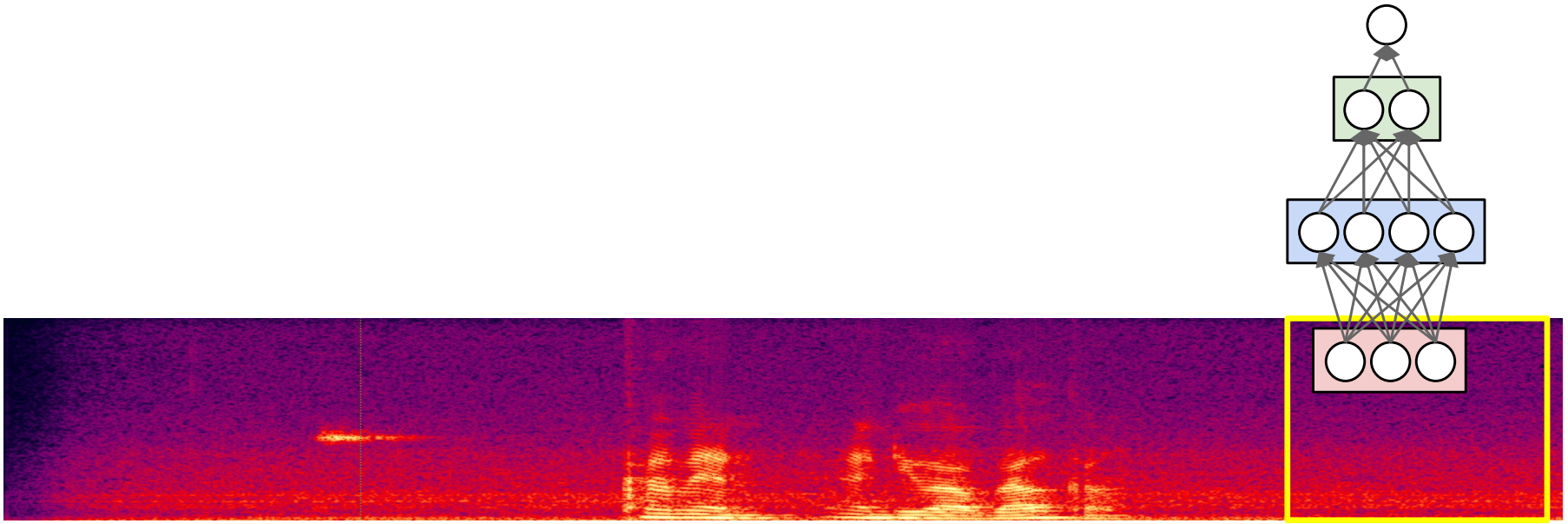
- Scan for the target word
  - The spectral time-frequency components in a "window" are input to a "welcome-detector" MLP

Solution: Scan



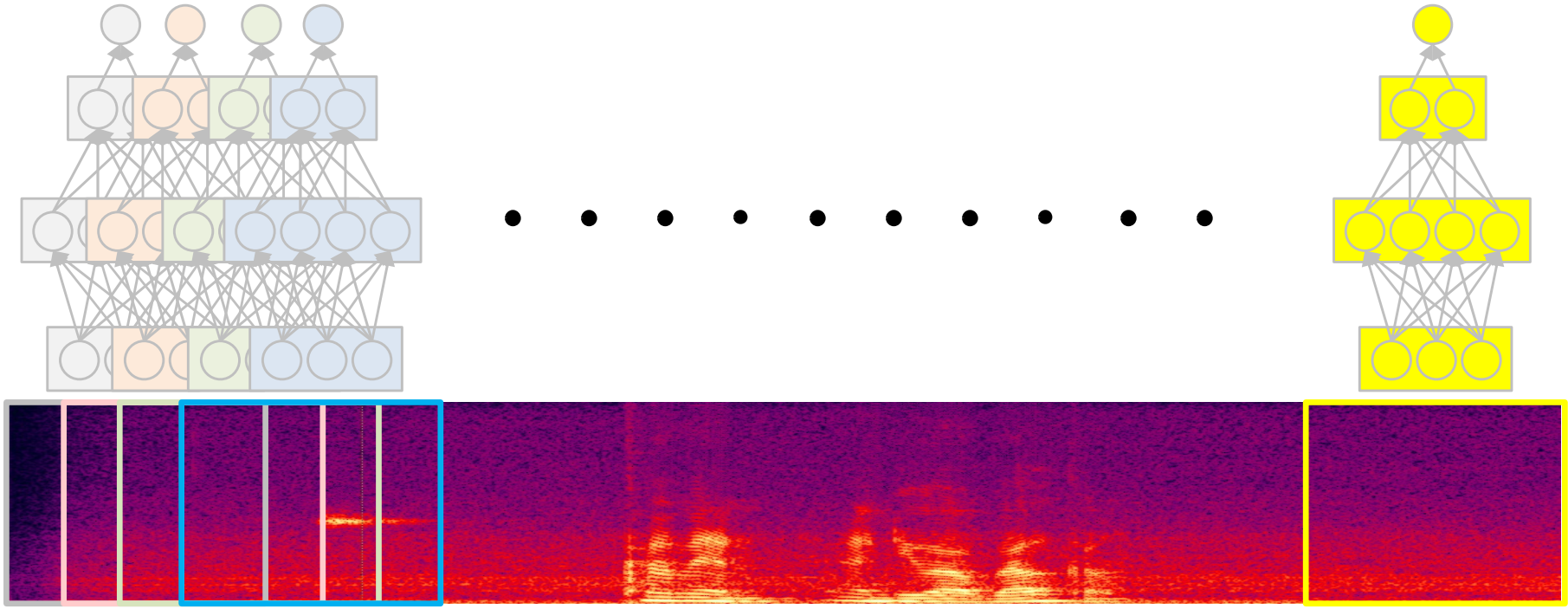
- Scan for the target word
  - The spectral time-frequency components in a "window" are input to a "welcome-detector" MLP

Solution: Scan



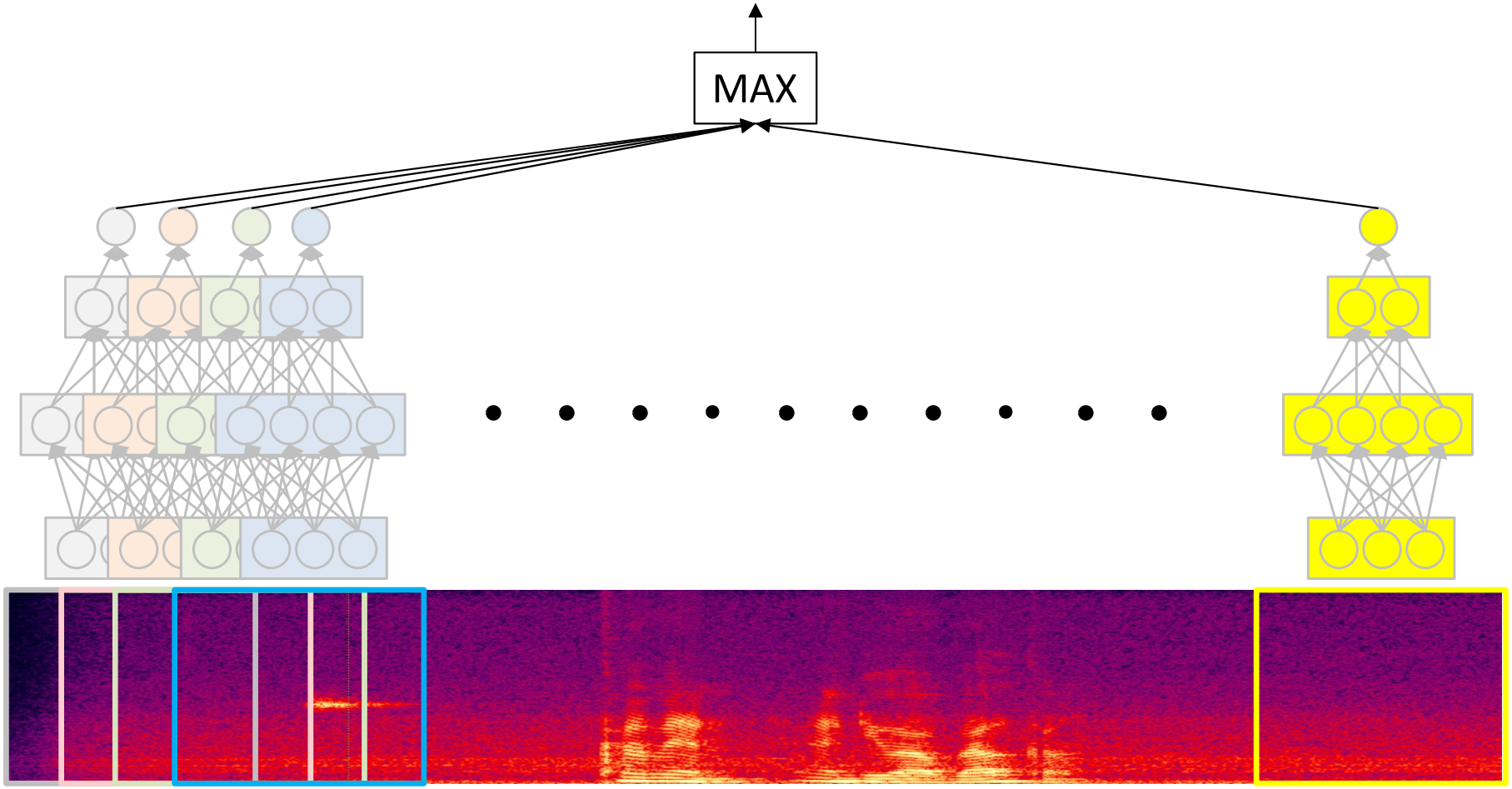
- Scan for the target word
  - The spectral time-frequency components in a "window" are input to a "welcome-detector" MLP

Solution: Scan



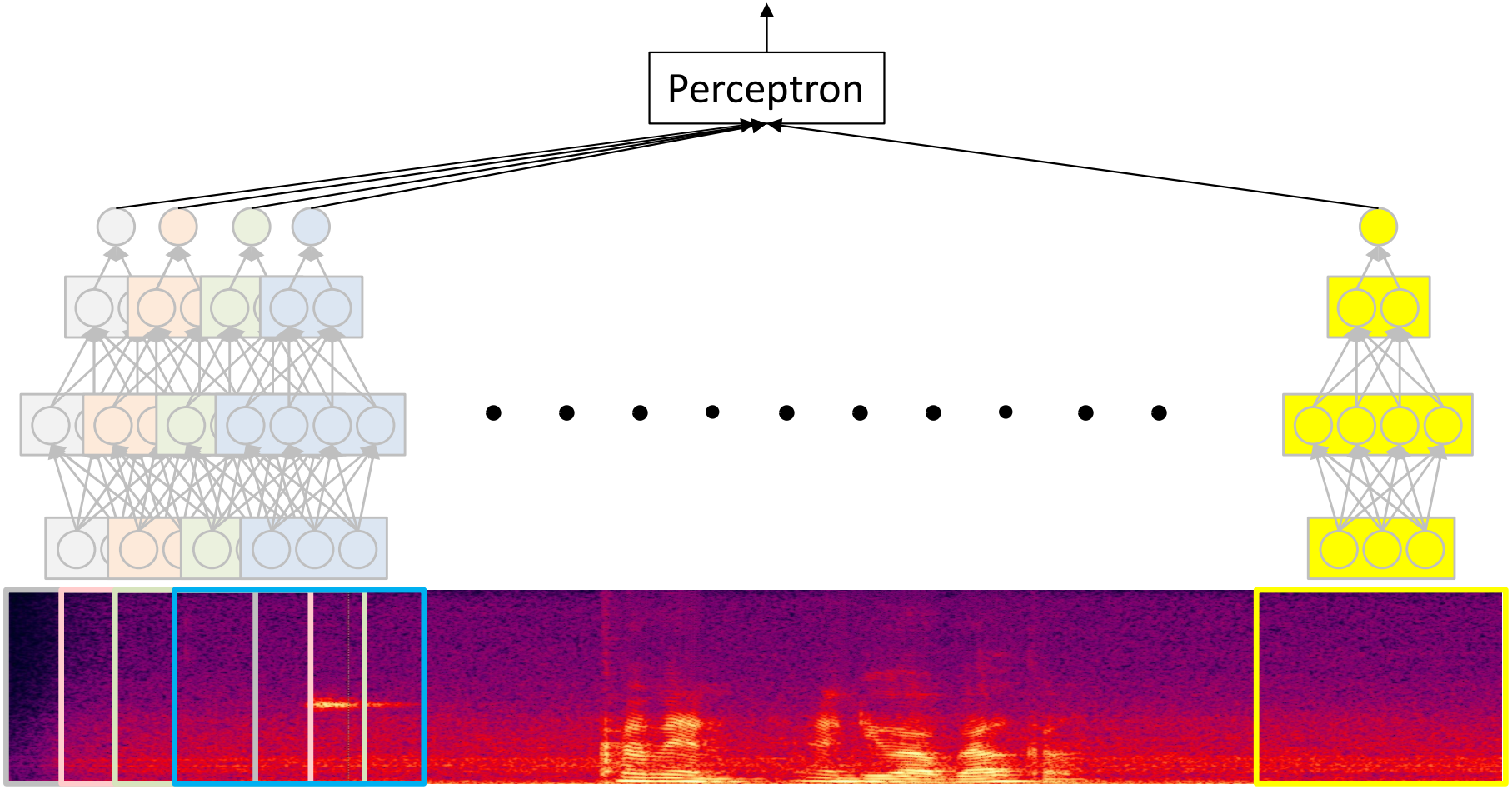
- "Does welcome occur in this recording?"
  - We have classified many "windows" individually
  - "Welcome" may have occurred in any of them

Solution: Scan



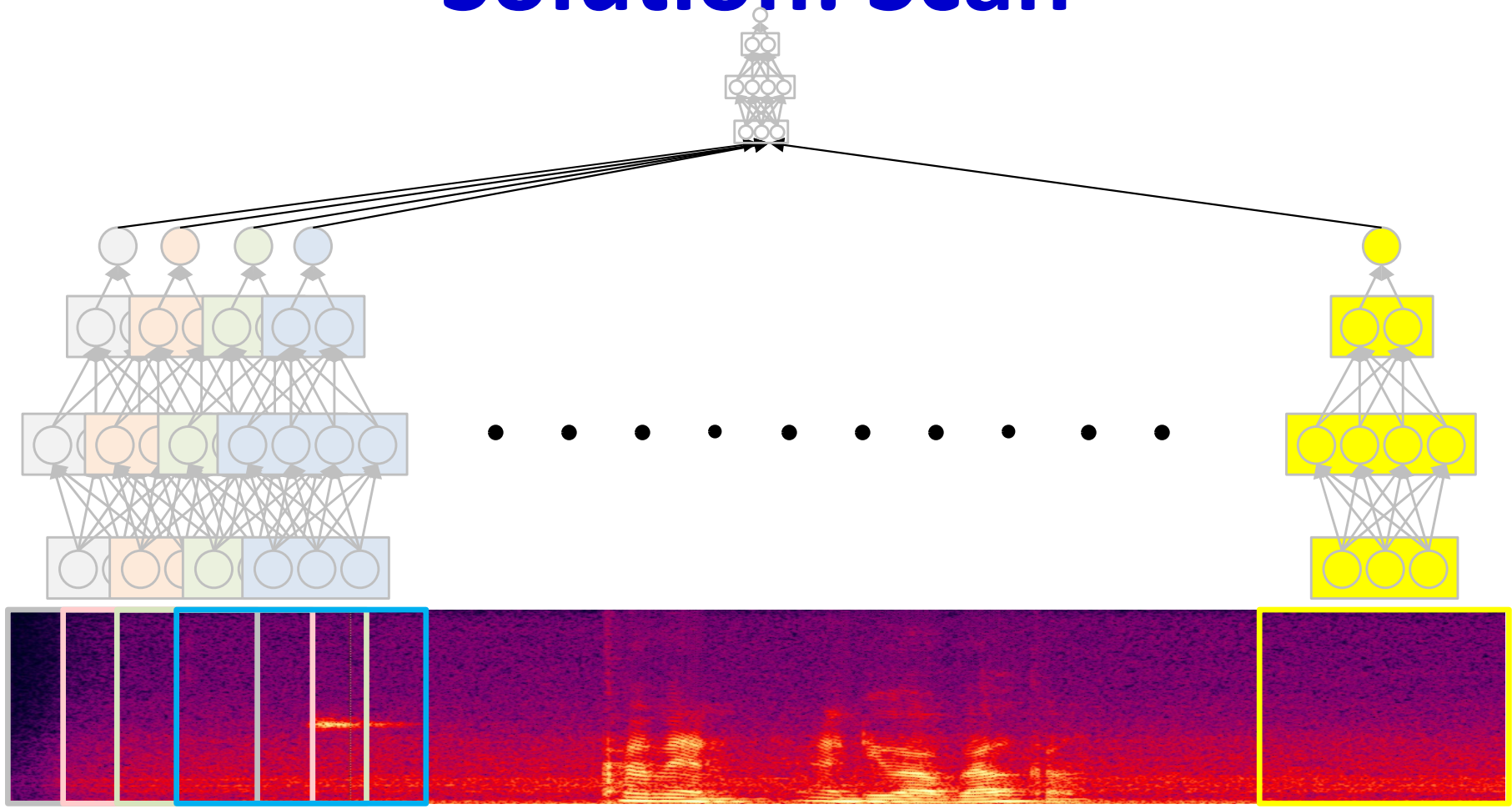
- "Does welcome occur in this recording?"
  - Maximum of all the outputs (Equivalent of Boolean OR)

Solution: Scan



- "Does welcome occur in this recording?"
  - Maximum of all the outputs (Equivalent of Boolean OR)
  - Or a proper softmax/logistic
    - Finding a welcome in adjacent windows makes it more likely that we didn't find noise

Solution: Scan



- "Does welcome occur in this recording?"
  - Maximum of all the outputs (Equivalent of Boolean OR)
  - Or a proper softmax/logistic
    - Adjacent windows can combine their evidence
  - Or even an MLP

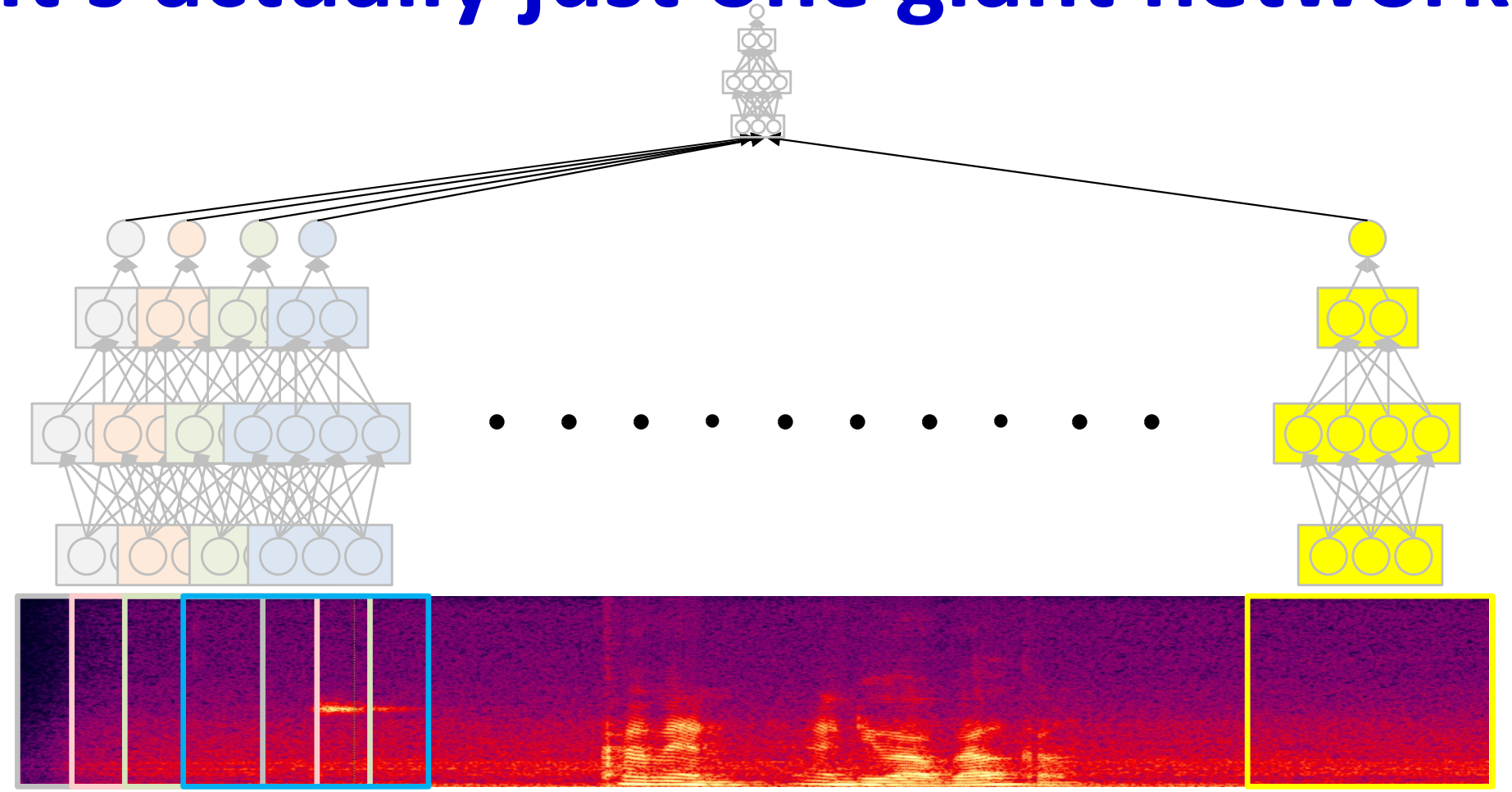
Scanning with an MLP

- `K` = width of "patch" evaluated by MLP

```
For t = 1:T-K+1
   XSegment = x(:, t:t+K-1)
   y(t) = MLP(XSegment)

Y = softmax(y(1)..y(T-K+1))
```

It’s actually just one giant network



- The entire operation can be viewed as one giant network
  - With many subnetworks, one per window
  - Restriction: All subnets are identical
- The network is *shift-invariant!*

Scanning with an MLP

- `K` = width of "patch" evaluated by MLP

```
For t = 1:T-K+1
   XSegment = x(:, t:t+K-1)
   y(t) = MLP(XSegment)

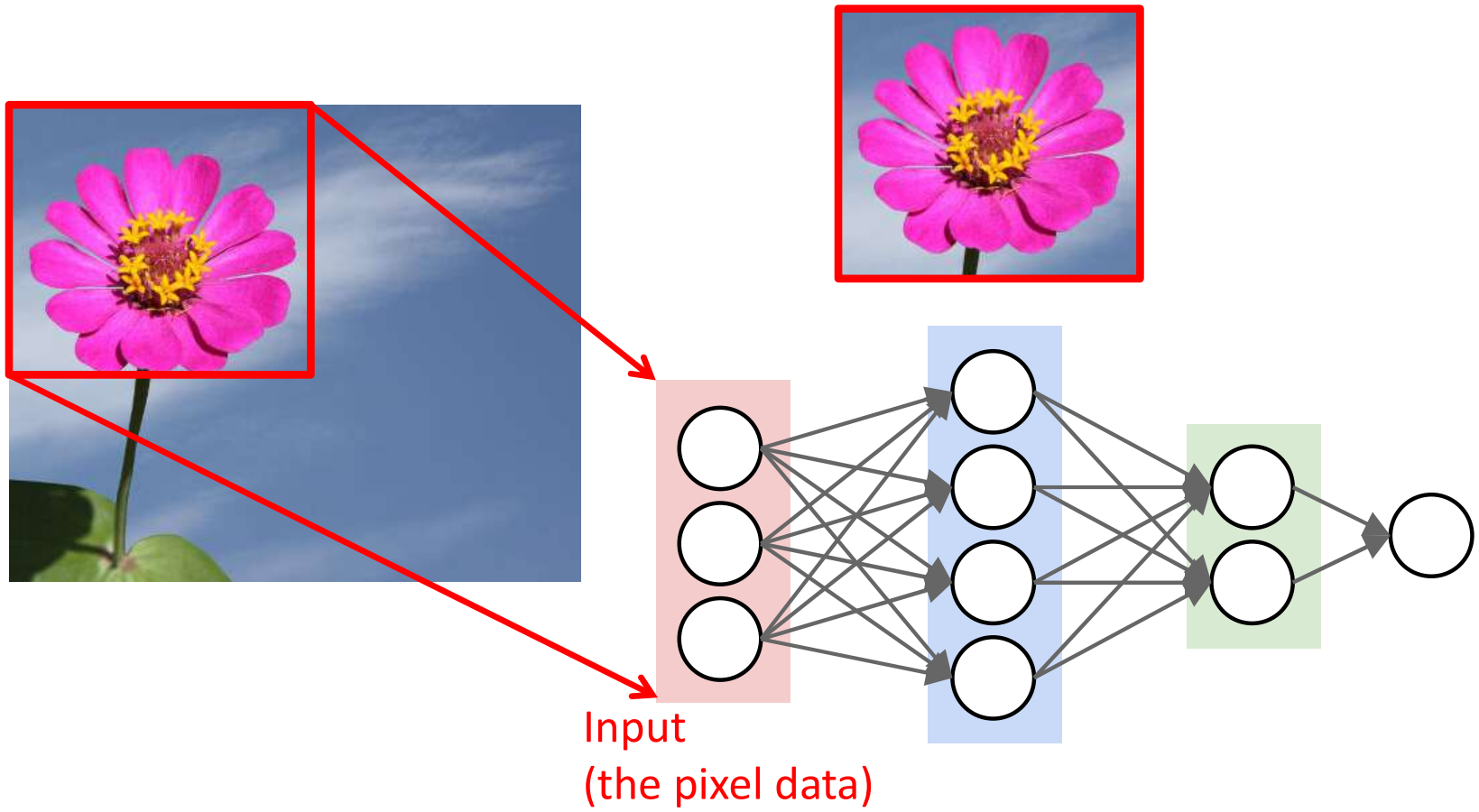
Y = softmax(y(1)..y(T-K+1))
```

Just the final layer of the overall MLP

Scanning with an MLP

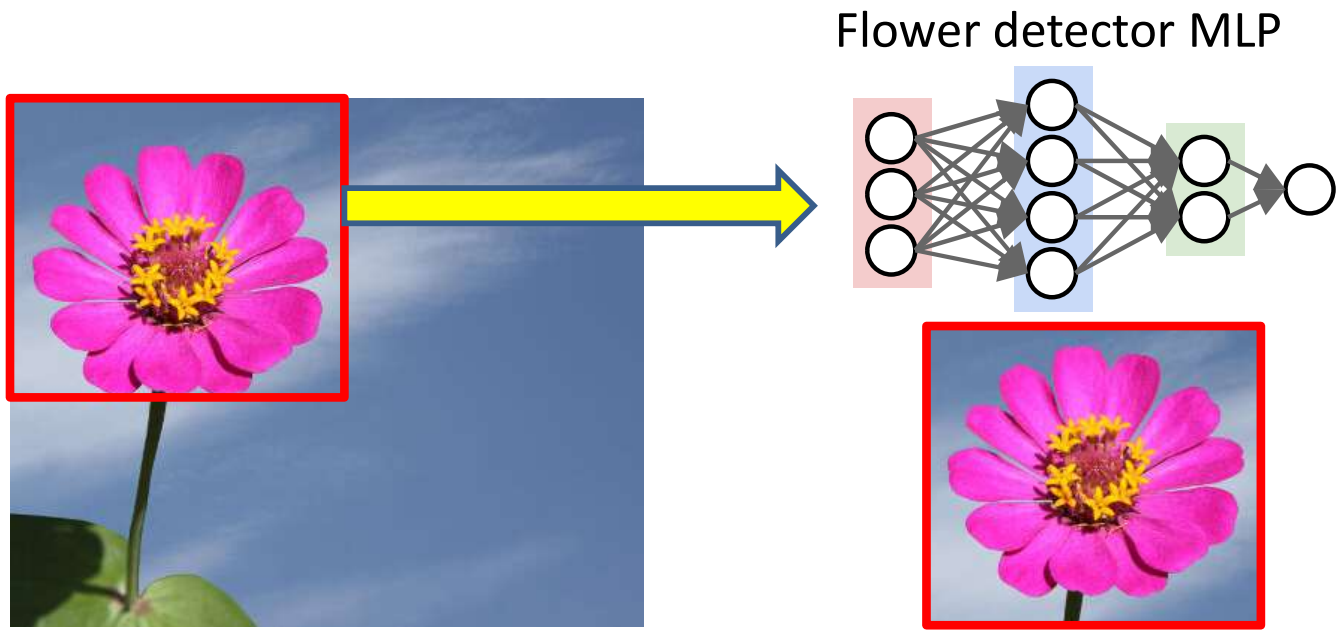
```
Y = giantMLP(x)
```

The 2-d analogue: Does this picture have a flower?



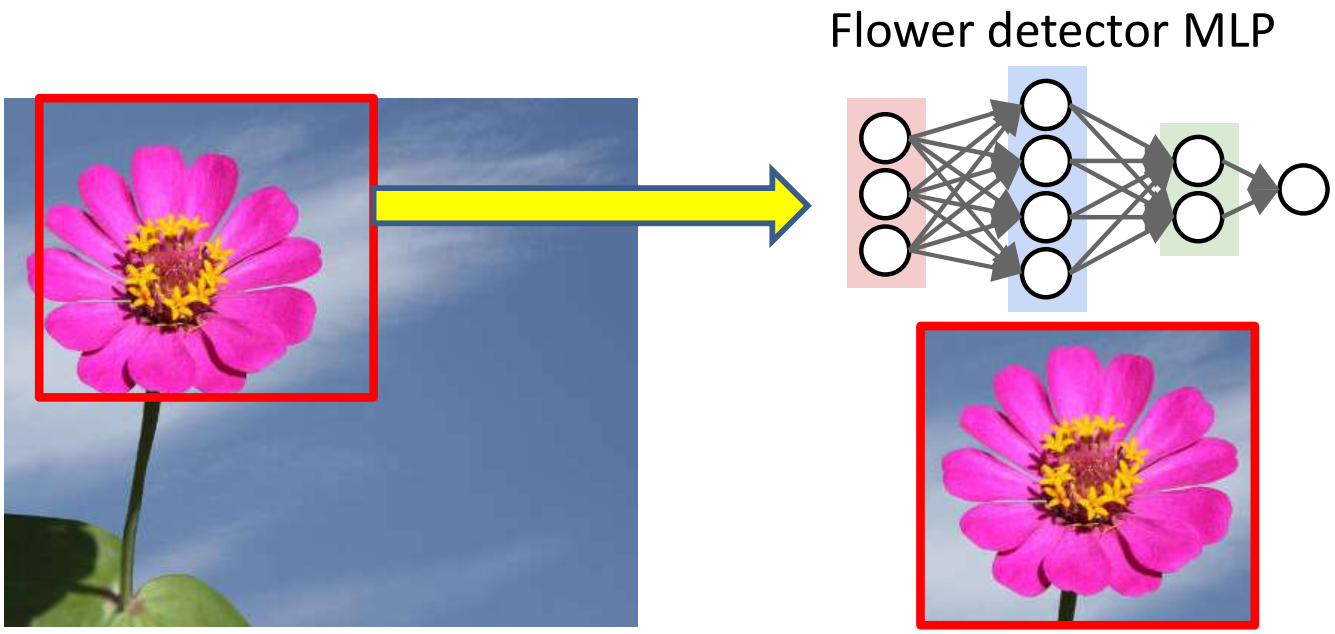
- *Scan* for the desired object
  - "Look" for the target object at each position

Solution: Scan



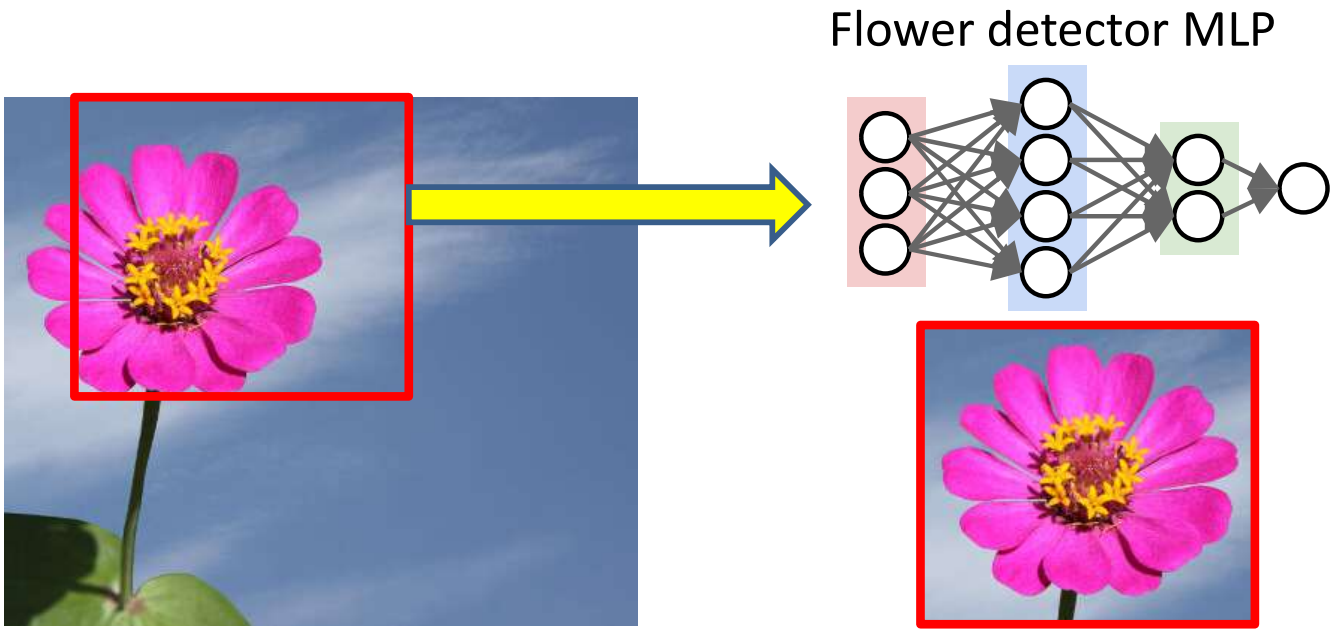
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



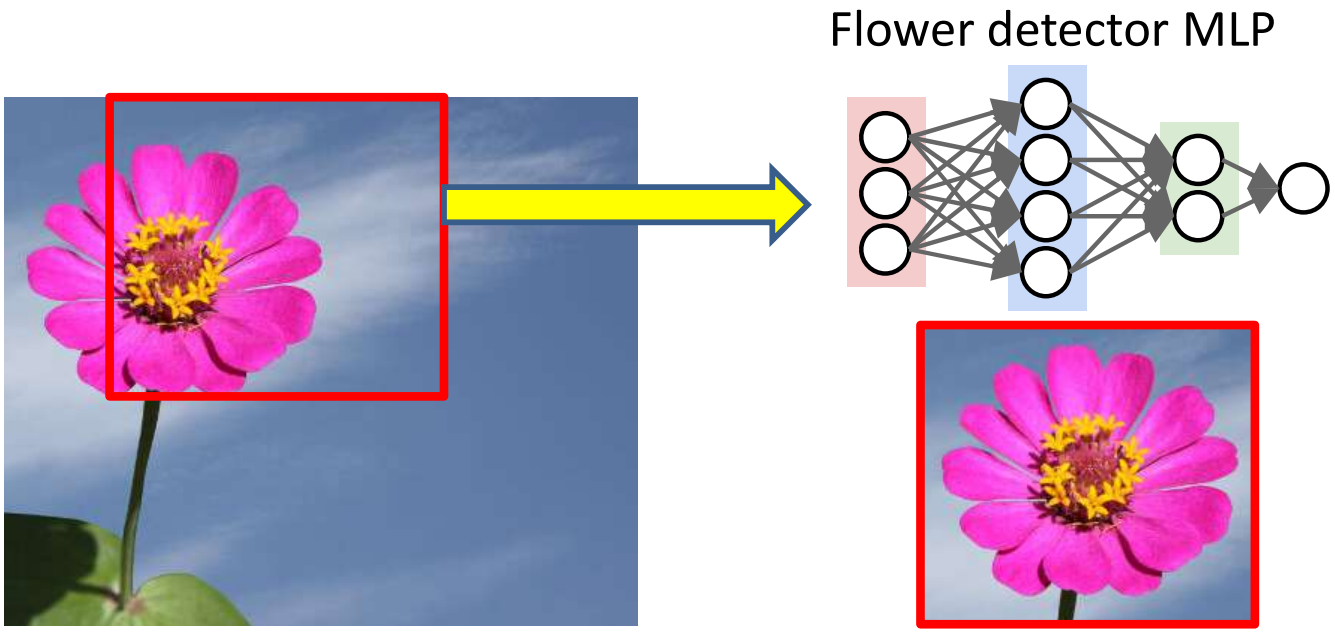
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



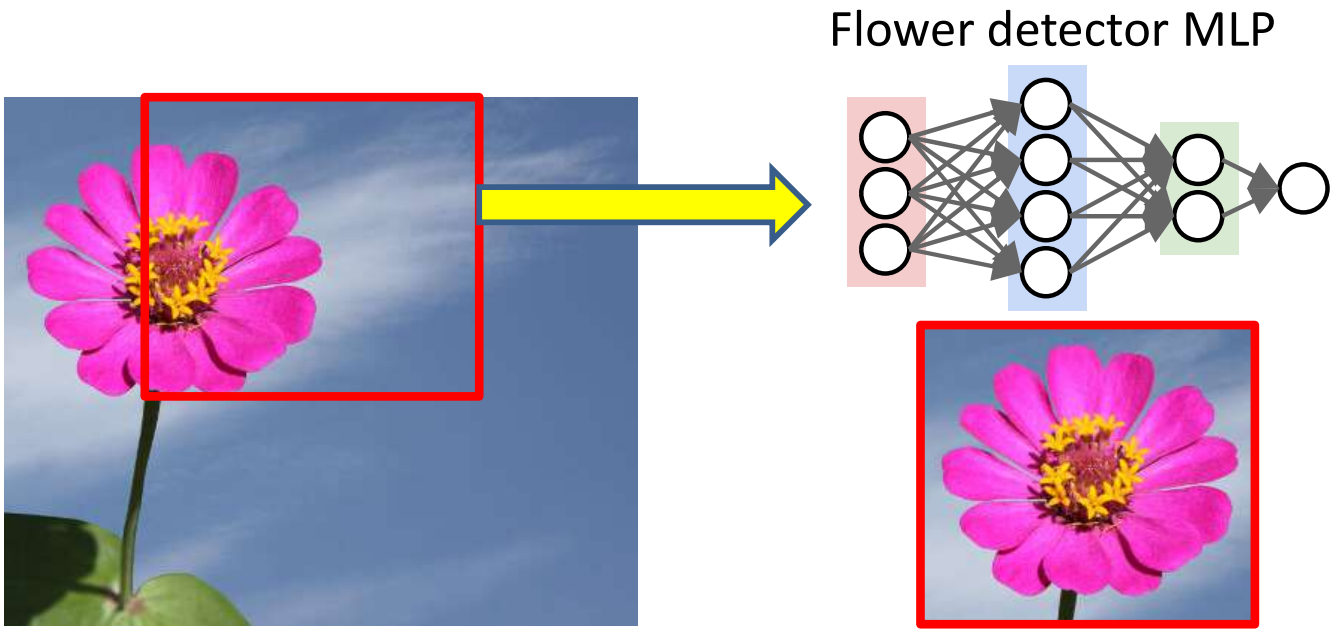
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



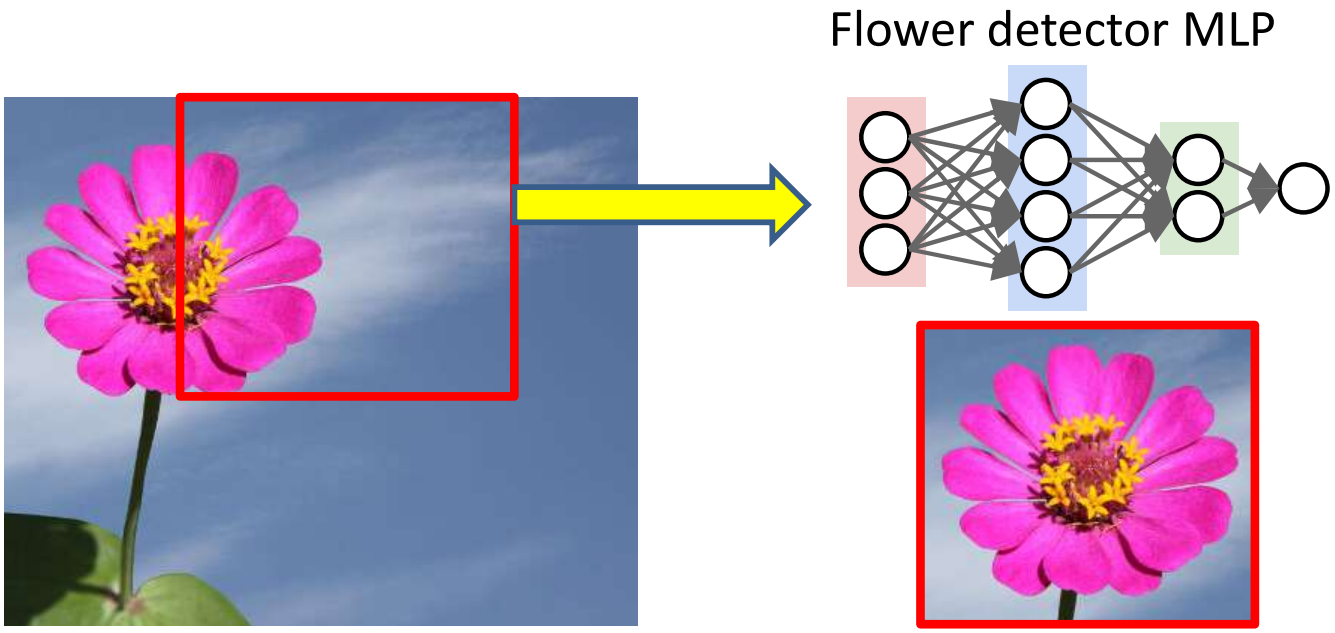
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



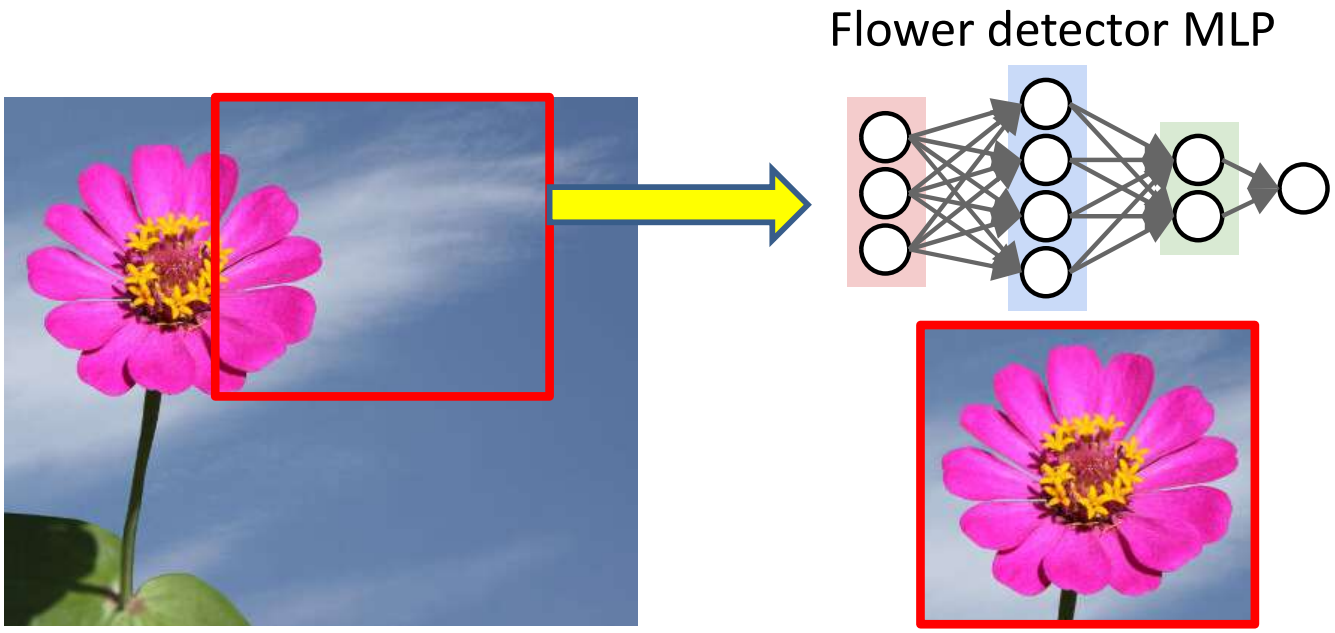
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



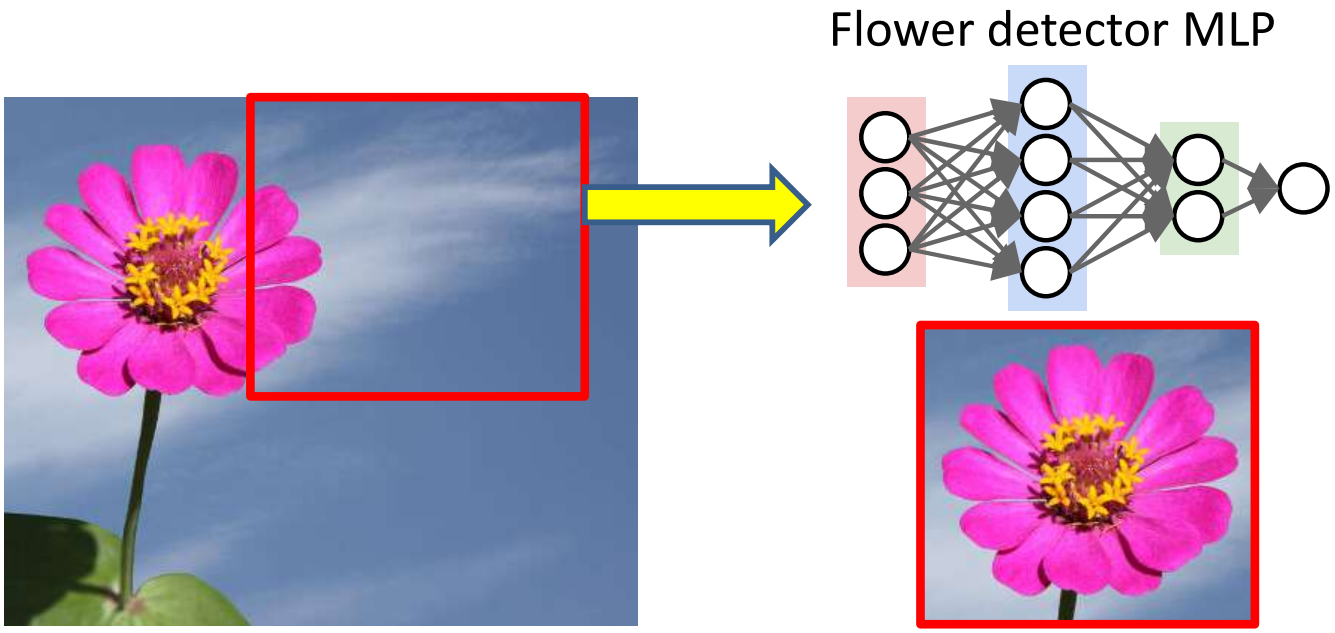
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



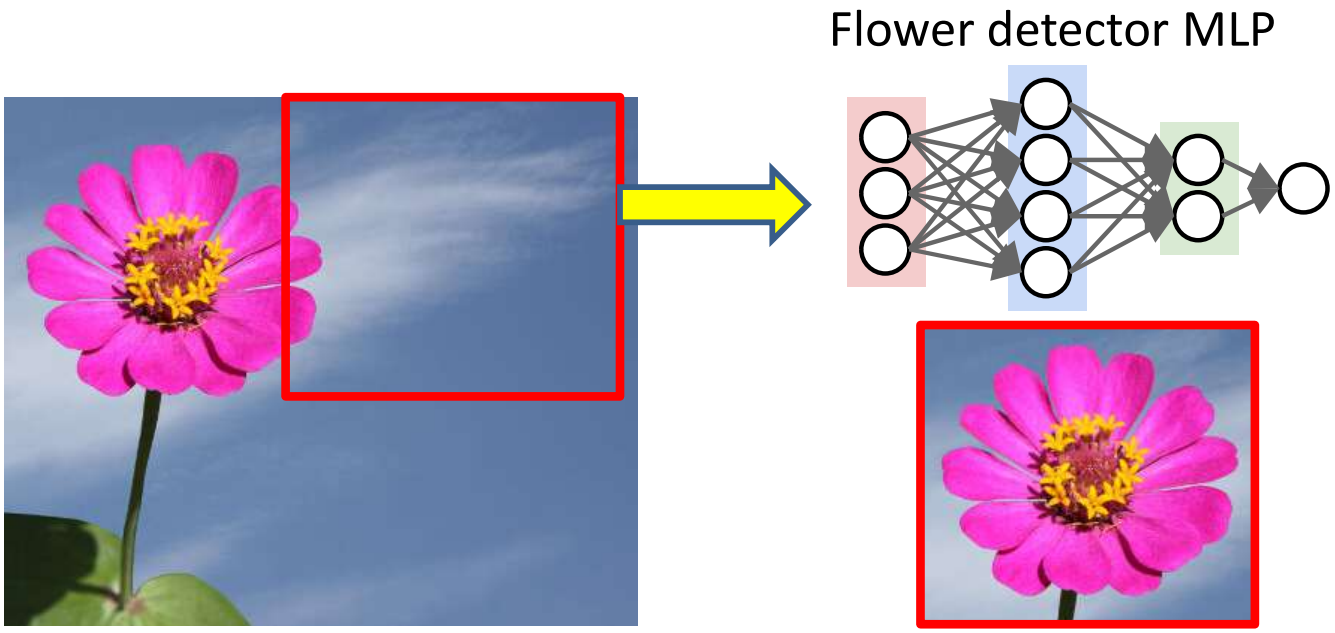
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



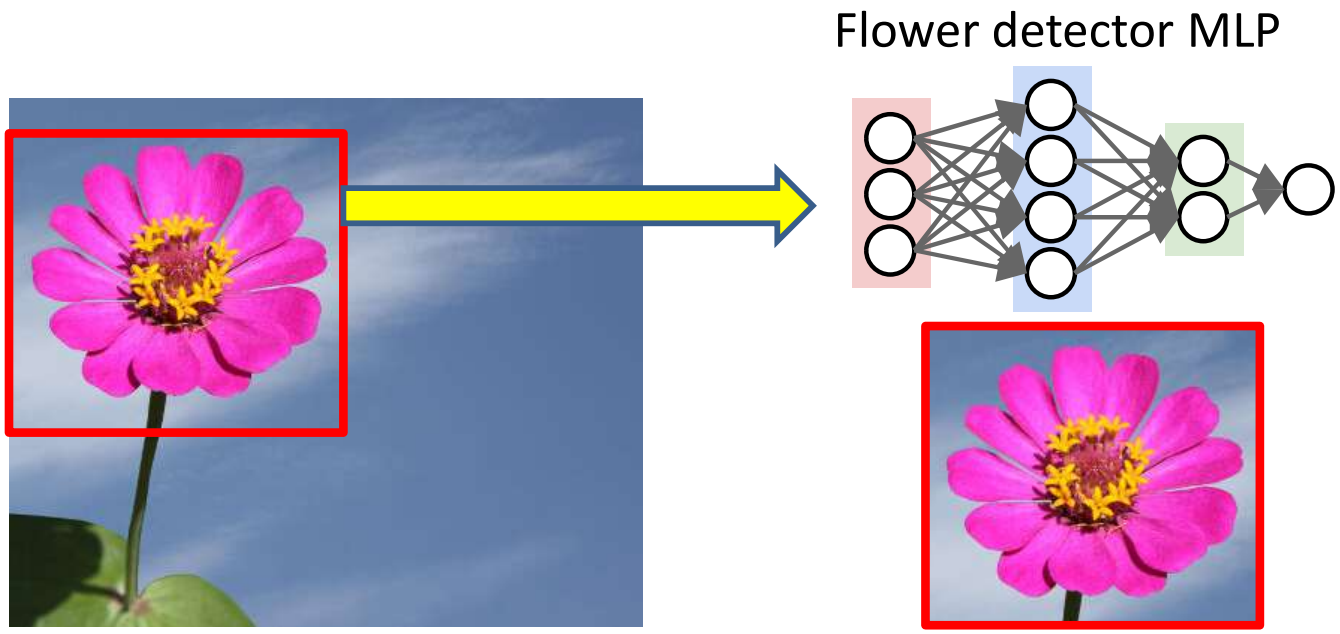
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



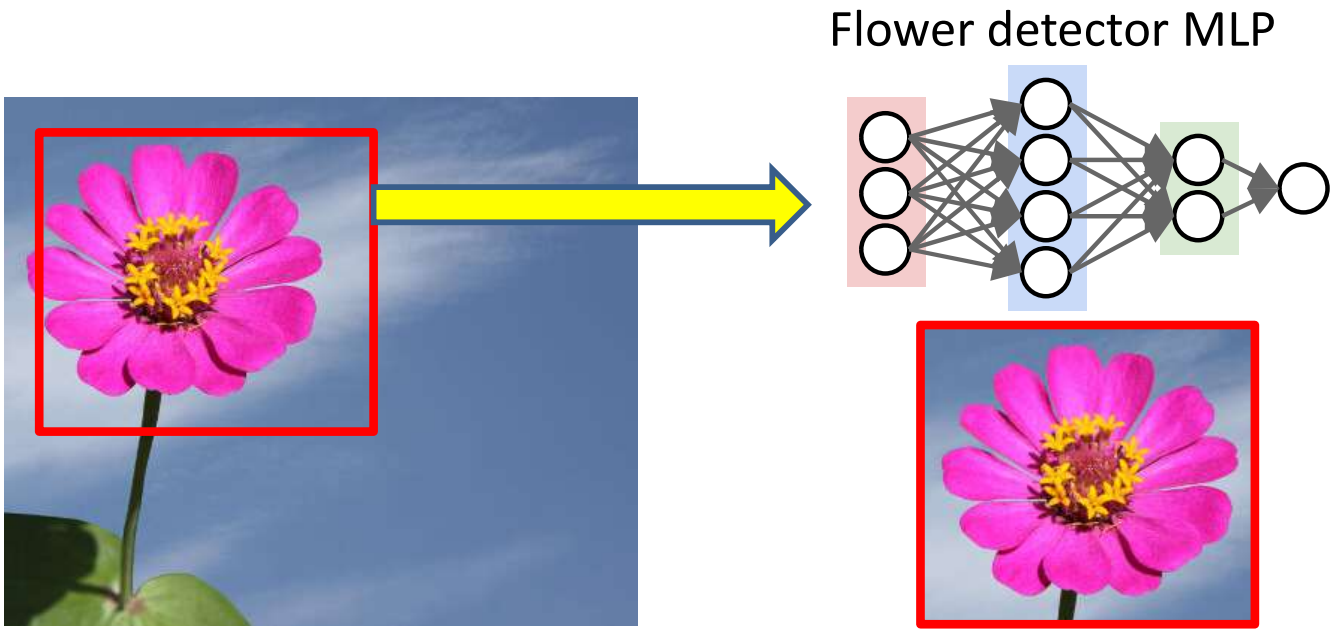
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



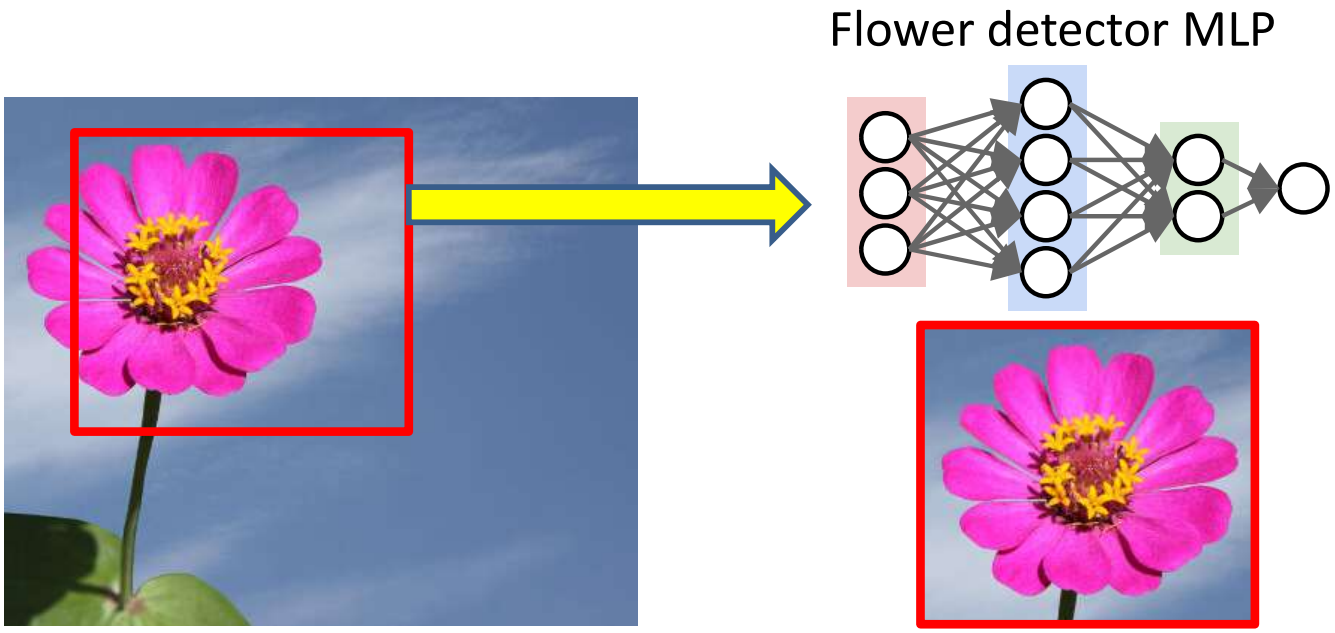
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



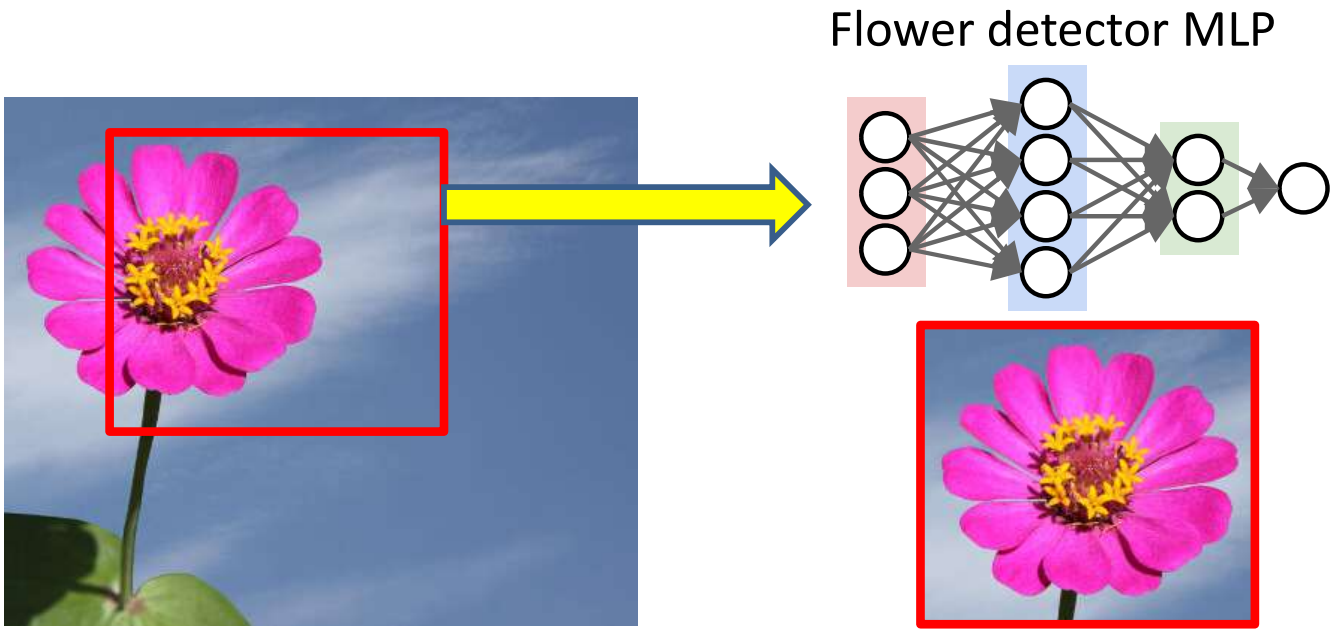
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



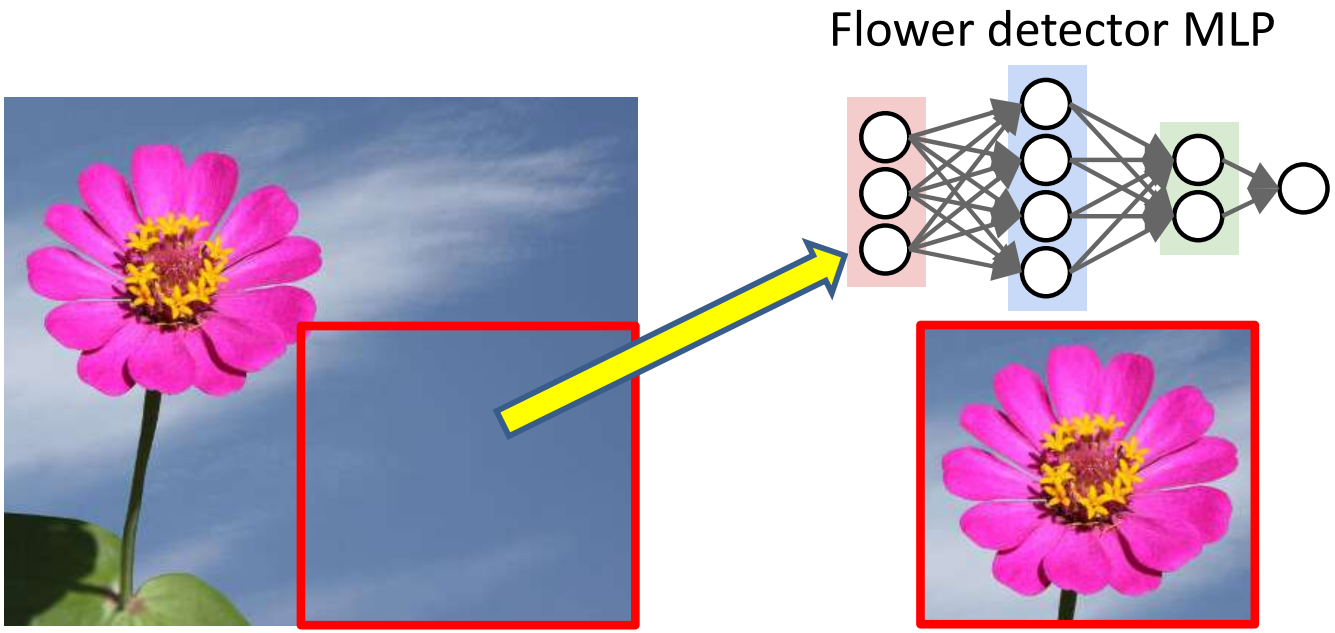
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



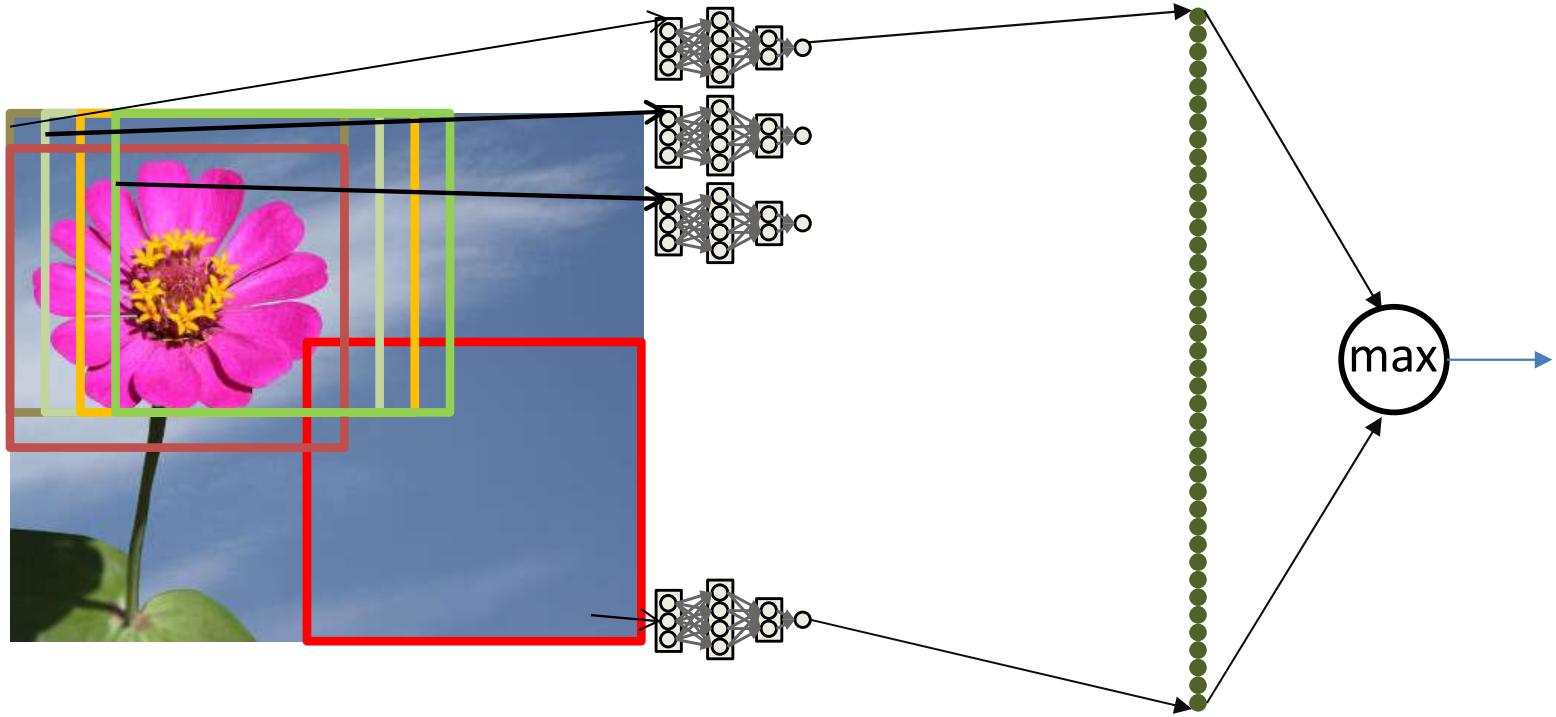
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Solution: Scan



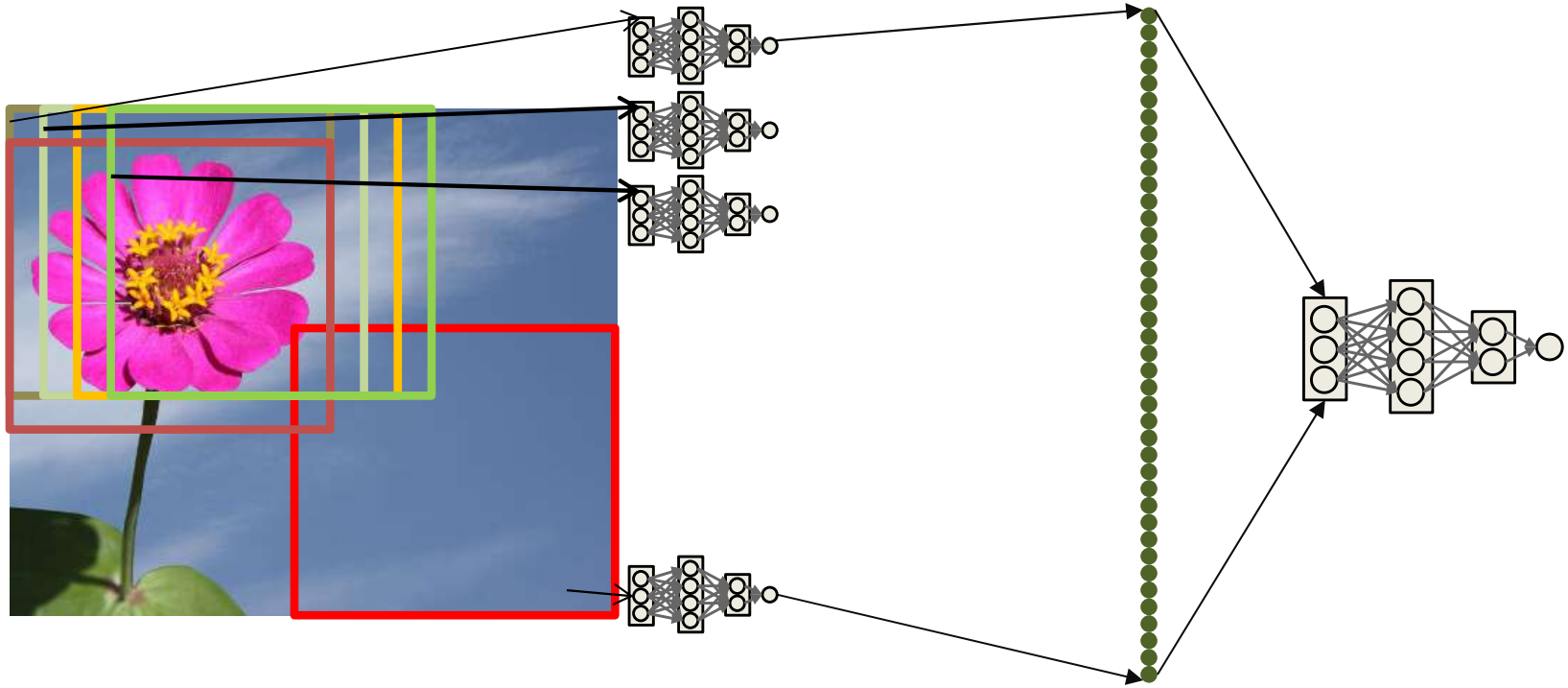
- *Scan* for the desired object
- At each location, the entire region is sent through the MLP

Scanning the picture to find a flower



- Determine if any of the locations had a flower
  - We get one classification output per scanned location
    - Each dot in the right represents the output of the MLP when it classifies one location in the input figure
      - The score output by the MLP
  - Look at the maximum value
    - If the picture has a flower, the location with the flower will result in high output value

Scanning the picture to find a flower



- Determine if any of the locations had a flower
  - Each dot in the right represents the output of the MLP when it classifies one location in the input figure
    - The score output by the MLP
  - Look at the maximum value
  - Or pass it through a softmax or even an MLP

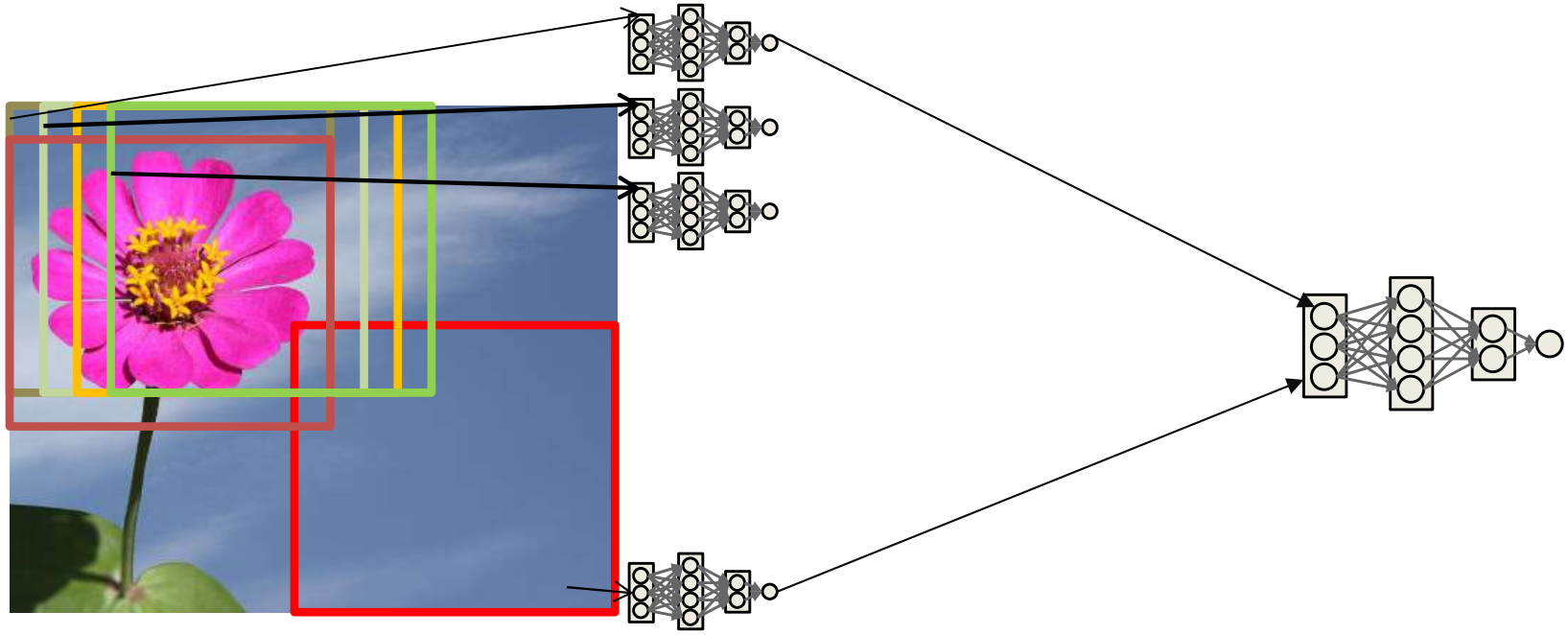
Scanning with an MLP

- `KxK` = size of "patch" evaluated by MLP
- `W` is width of image
- `H` is height of image

```
For i = 1:W-K+1
    For j = 1:H-K+1
        ImgSegment = Img(i:i+K-1, j:j+K-1)
        y(i,j) = MLP(ImgSegment)

Y = softmax( y(1,1)..y(W-K+1,H-K+1) )
```

It's just a giant network with common subnets



- The entire operation can be viewed as a single giant network
  - Composed of many "subnets" (one per window)
  - With one key feature: all subnets are identical
- The network is *shift invariant*.

Scanning with an MLP

- KxK = size of "patch" evaluated by MLP
- W is width of image
- H is height of image

```
For i = 1:W-K+1
    For j = 1:H-K+1
        ImgSegment = Img(i:i+K-1, j:j+K-1)
        y(i,j) = MLP(ImgSegment)

Y = softmax( y(1,1)..y(W-K+1,H-K+1) )
```

Just the final layer of the overall MLP

Scanning with an MLP

```
Y = giantMLP(img)
```

Poll 1 (@624, @625)

- We can determine if a picture has a flower by scanning it for a flower with an MLP
  - True
  - False

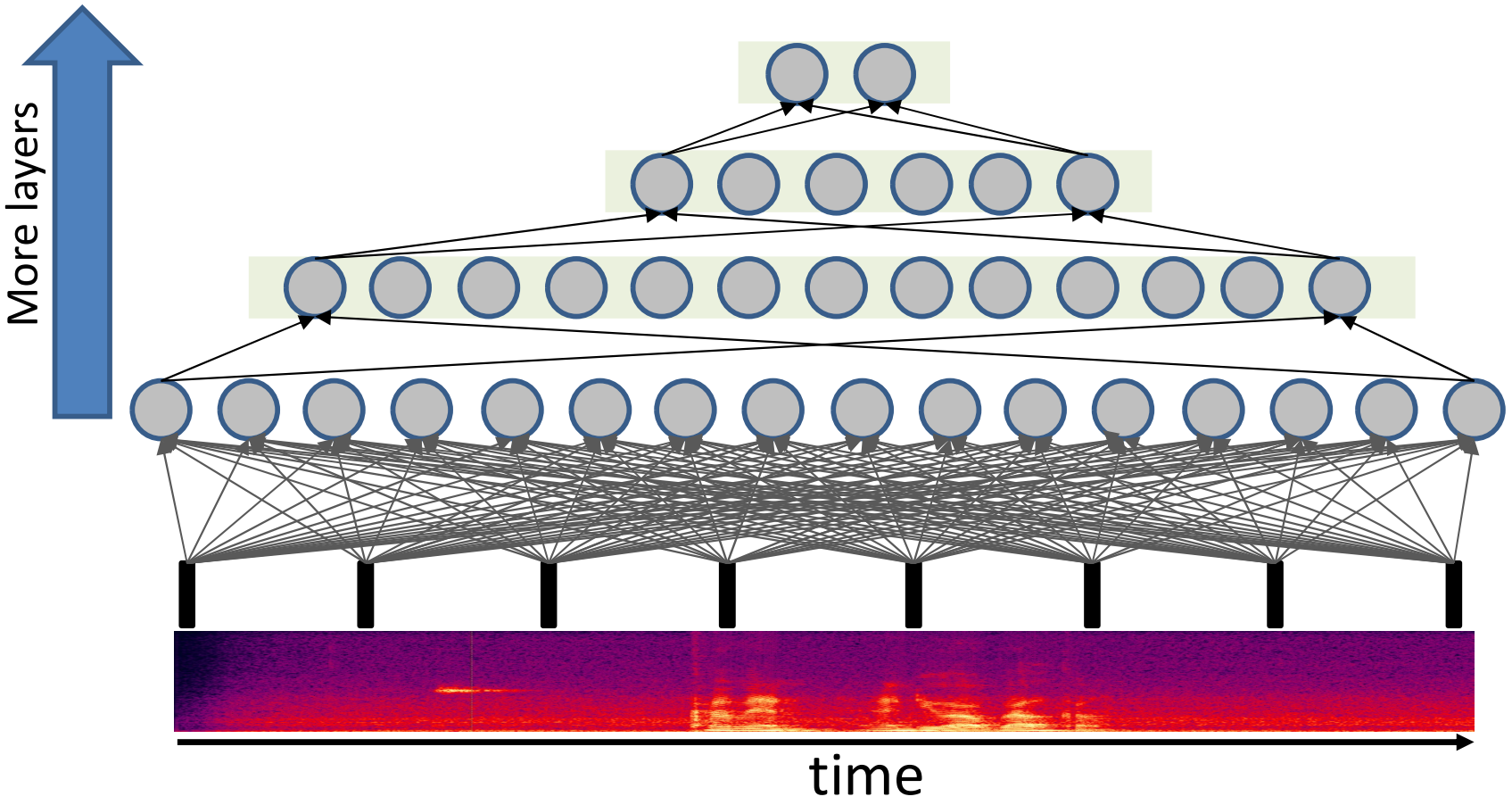
- Scanning a picture for a flower to determine if the picture has a flower in it is strictly the same as analyzing the entire picture with a single large shared-parameter MLP
  - True
  - False

Poll 1

- We can determine if a picture has a flower by scanning it for a flower with an MLP
  - **True**
  - False

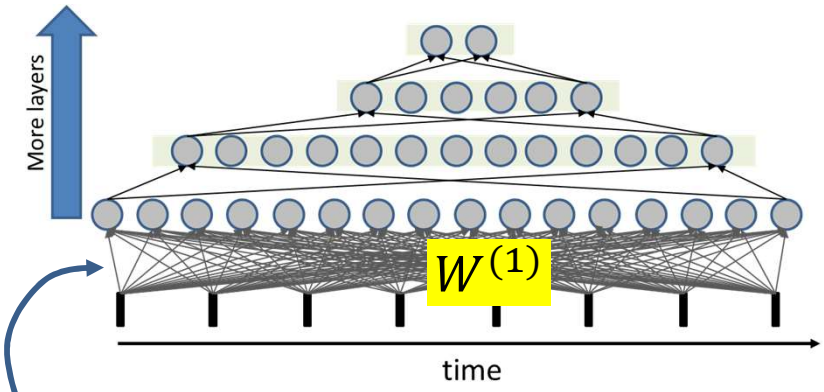
- Scanning a picture for a flower to determine if the picture has a flower in it is strictly the same as analyzing the entire picture with a single large shared-parameter MLP
  - **True**
  - False

Regular networks vs. scanning networks



- In a ***regular MLP*** every neuron in a layer is connected by a unique weight to every unit in the previous layer
  - All entries in the weight matrix are unique
  - The weight matrix is (generally) full

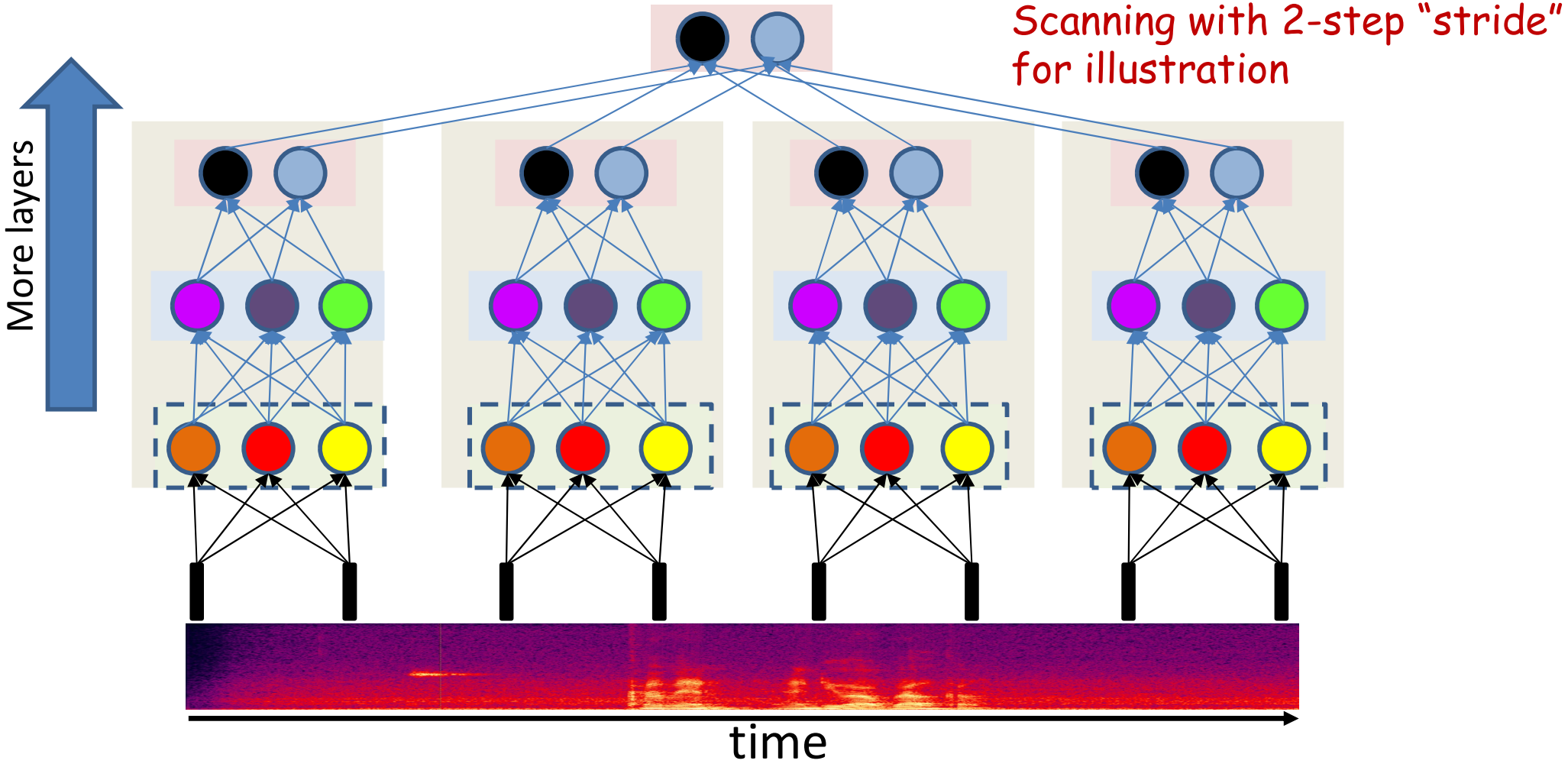
Regular network



$$W^{(1)} = \begin{bmatrix} w_{11} & w_{21} & w_{31} & w_{41} & \cdots & \cdots & \cdots & w_{N1} \\ w_{12} & w_{22} & w_{32} & w_{42} & \cdots & \cdots & \cdots & w_{N2} \\ w_{13} & w_{23} & w_{33} & w_{43} & \cdots & \cdots & \cdots & w_{N3} \\ w_{14} & w_{24} & w_{34} & w_{44} & \cdots & \cdots & \cdots & w_{N4} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ w_{1M} & w_{2M} & w_{3M} & w_{4M} & \cdots & \cdots & \cdots & w_{NM} \end{bmatrix}$$

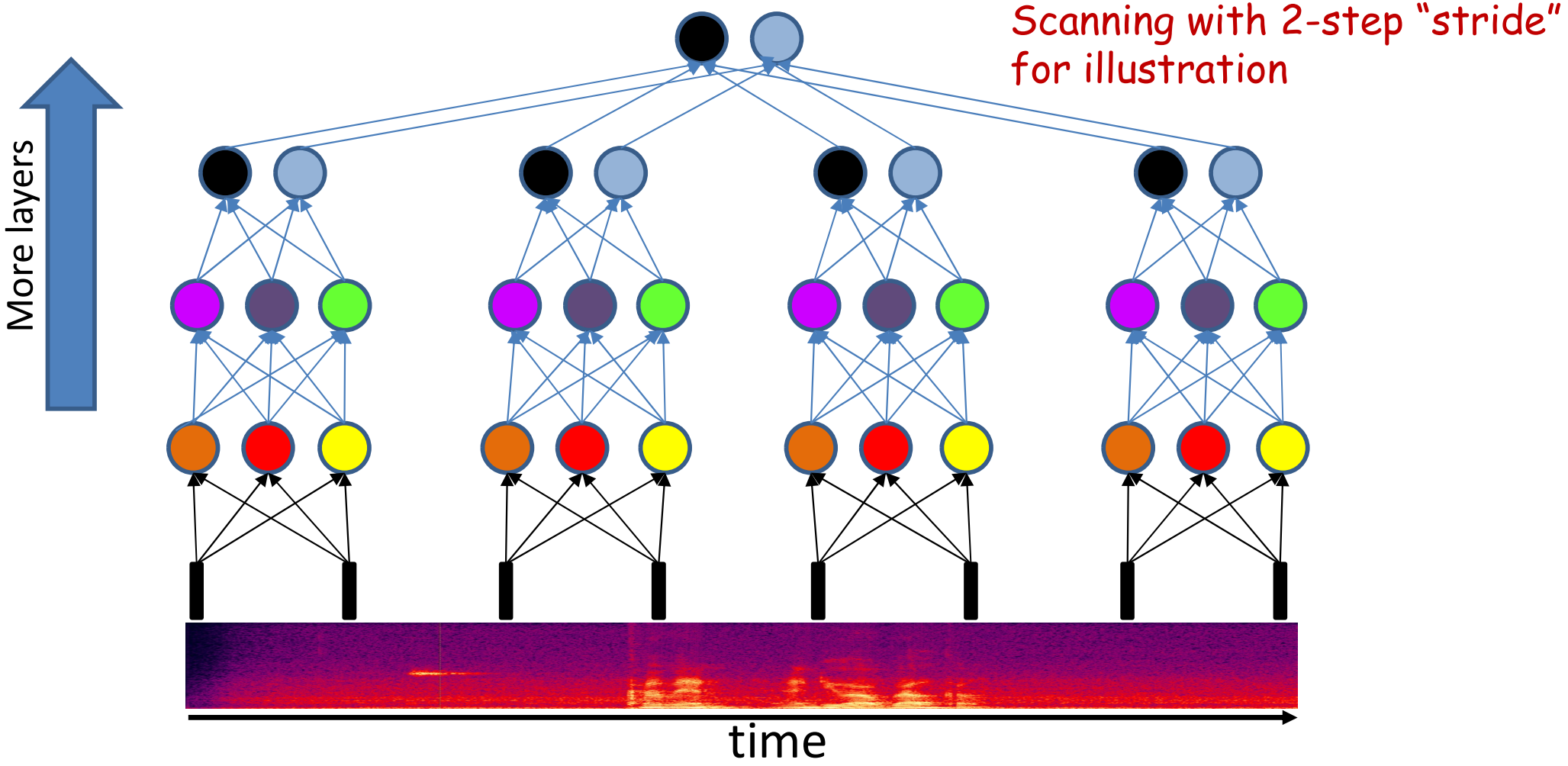
- Consider the first layer
  - Assume N inputs and M outputs
- The weights matrix is a full $M \times N$ matrix
  - Requiring MN unique parameters

Scanning networks



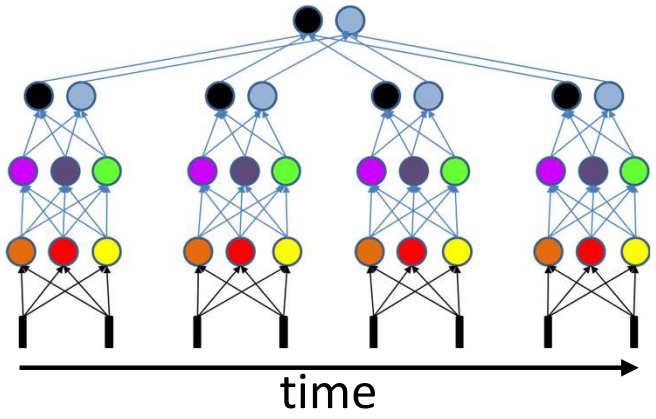
- In a ***scanning MLP*** each neuron is connected to a subset of neurons in the previous layer
  - The weights matrix is sparse
  - The weights matrix is block structured ***with identical blocks***

Scanning networks



- In a ***scanning MLP*** each neuron is connected to a subset of neurons in the previous layer
  - The weights matrix is sparse
  - The weights matrix is block structured ***with identical blocks***

Scanning networks

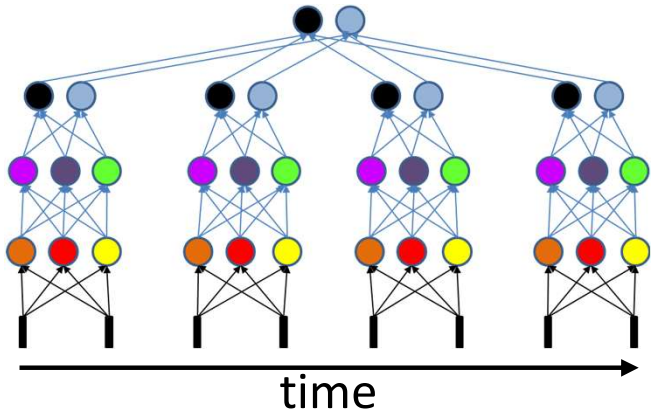


$W^{(1)}$

$$W^{(1)} = \begin{bmatrix} w_{11} & w_{12} & 0 & 0 & 0 & \cdots & \cdots & \cdots & 0 \\ w_{21} & w_{22} & 0 & 0 & 0 & \cdots & \cdots & \cdots & 0 \\ w_{31} & w_{32} & 0 & 0 & 0 & \cdots & \cdots & \cdots & 0 \\ 0 & 0 & w_{11} & w_{12} & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & w_{21} & w_{22} & 0 & 0 & 0 & \vdots & \vdots \\ 0 & 0 & w_{31} & w_{32} & 0 & 0 & 0 & \vdots & \vdots \\ 0 & 0 & 0 & 0 & w_{11} & w_{12} & 0 & 0 & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 0 & 0 & w_{31} & w_{32} \end{bmatrix}$$

- In a ***scanning MLP*** each neuron is connected to a subset of neurons in the previous layer
  - The weights matrix is sparse
  - The weights matrix is block structured ***with identical blocks***
  - **The network is a *shared parameter* model**

Scanning networks



$W^{(1)}$

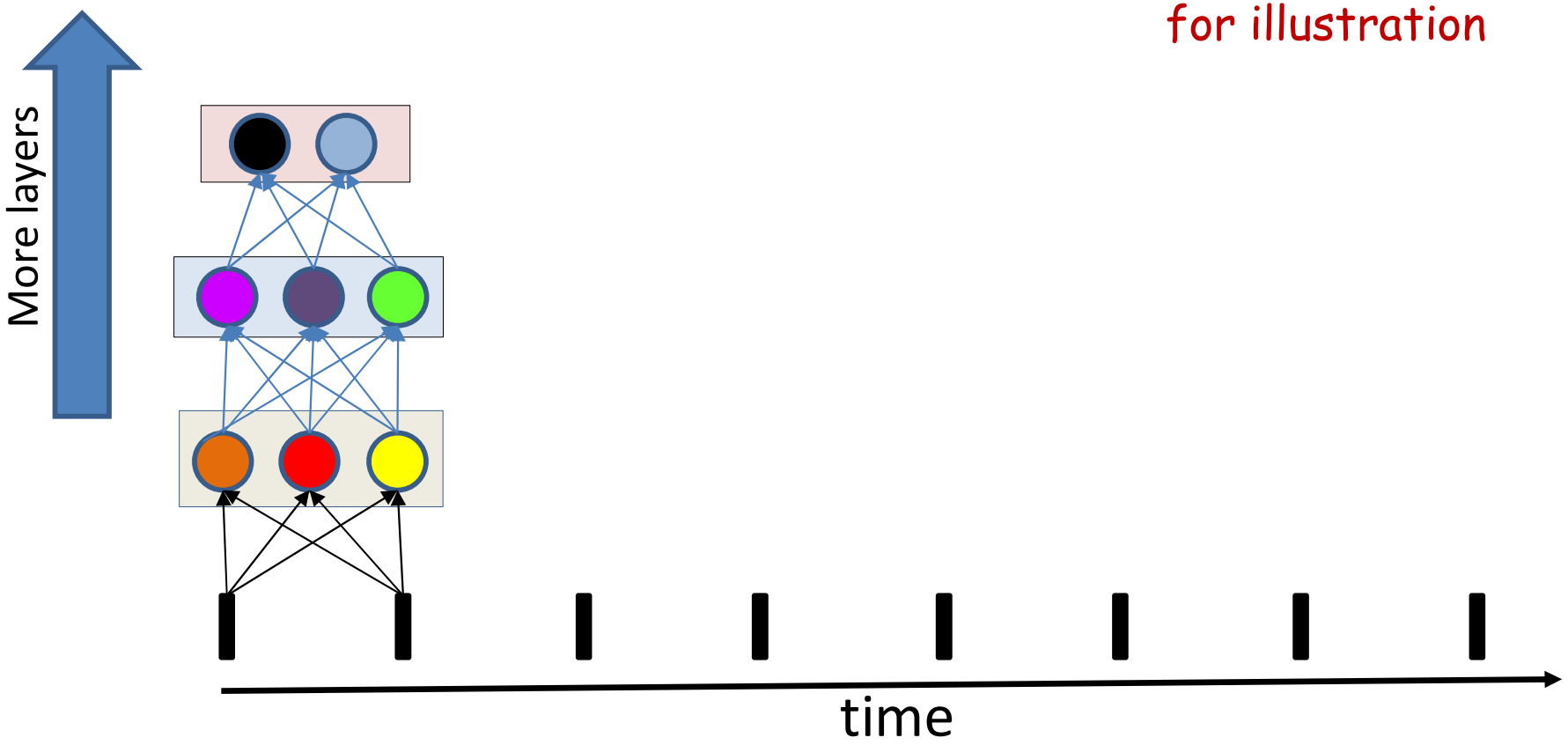
$$W^{(1)} = \begin{bmatrix} w_{11} & w_{12} & 0 & 0 & 0 & \cdots & \cdots & \cdots & 0 \\ w_{21} & w_{22} & 0 & 0 & 0 & \cdots & \cdots & \cdots & 0 \\ w_{31} & w_{32} & 0 & 0 & 0 & \cdots & \cdots & \cdots & 0 \\ 0 & 0 & w_{11} & w_{12} & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & w_{21} & w_{22} & 0 & 0 & 0 & \vdots & \vdots \\ 0 & 0 & w_{31} & w_{32} & 0 & 0 & 0 & \vdots & \vdots \\ 0 & 0 & 0 & 0 & w_{11} & w_{12} & 0 & 0 & \vdots \end{bmatrix}$$

Effective in any situation where the data are expected to be composed of similar structures at different locations

- In a ***scanning MLP*** each neuron is connected to a subset of neurons in the previous layer
  - The weights matrix is sparse
  - The weights matrix is block ***structured with identical blocks***
  - *The network is a shared-parameter model*
- *Also, far fewer parameters (we return to this topic shortly)*

Scanning networks

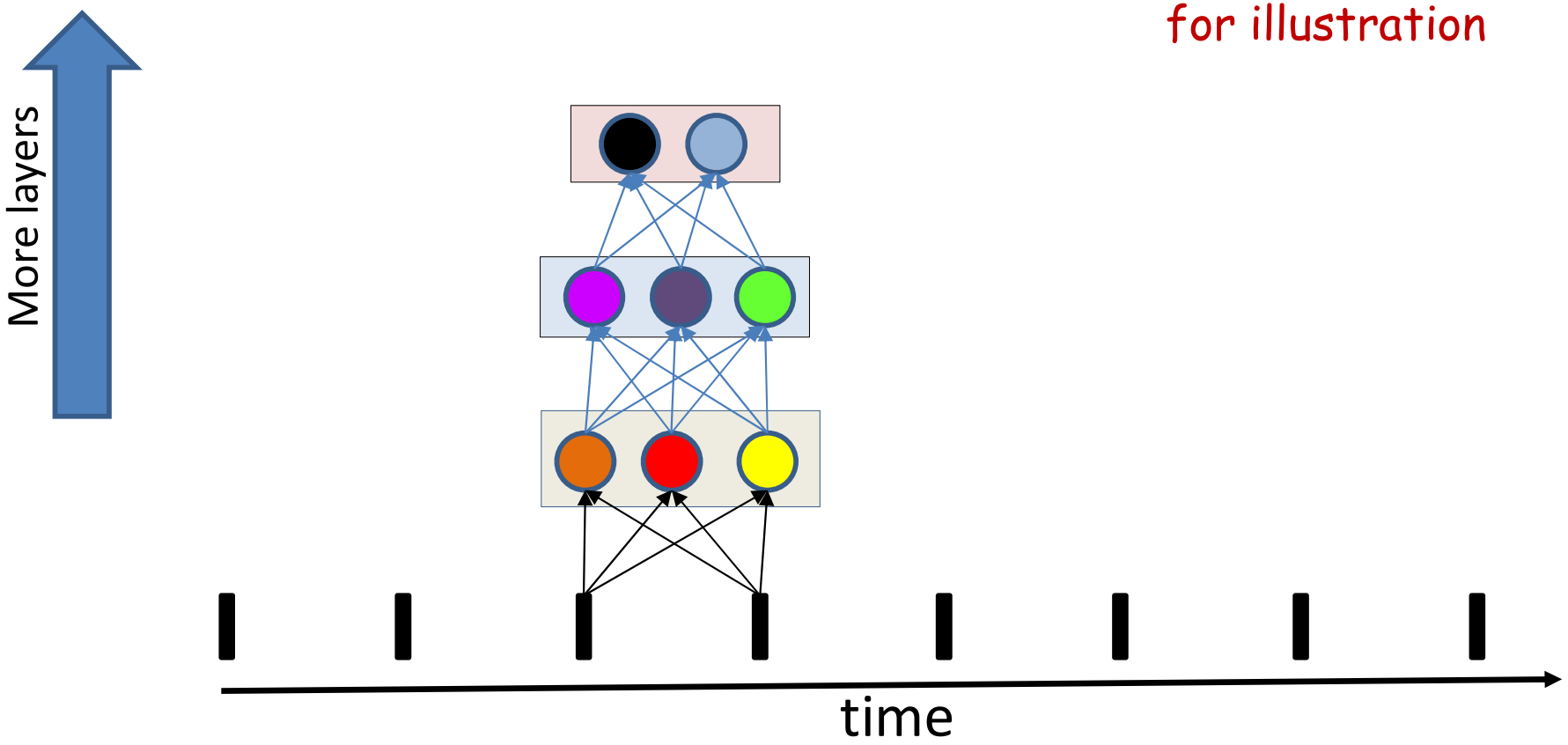
Scanning with 2-step "stride" for illustration



- Changing the illustration
  - Illustration of scanning with layers laid horizontally give erroneous impression of time (left to right) within a layer

Scanning networks

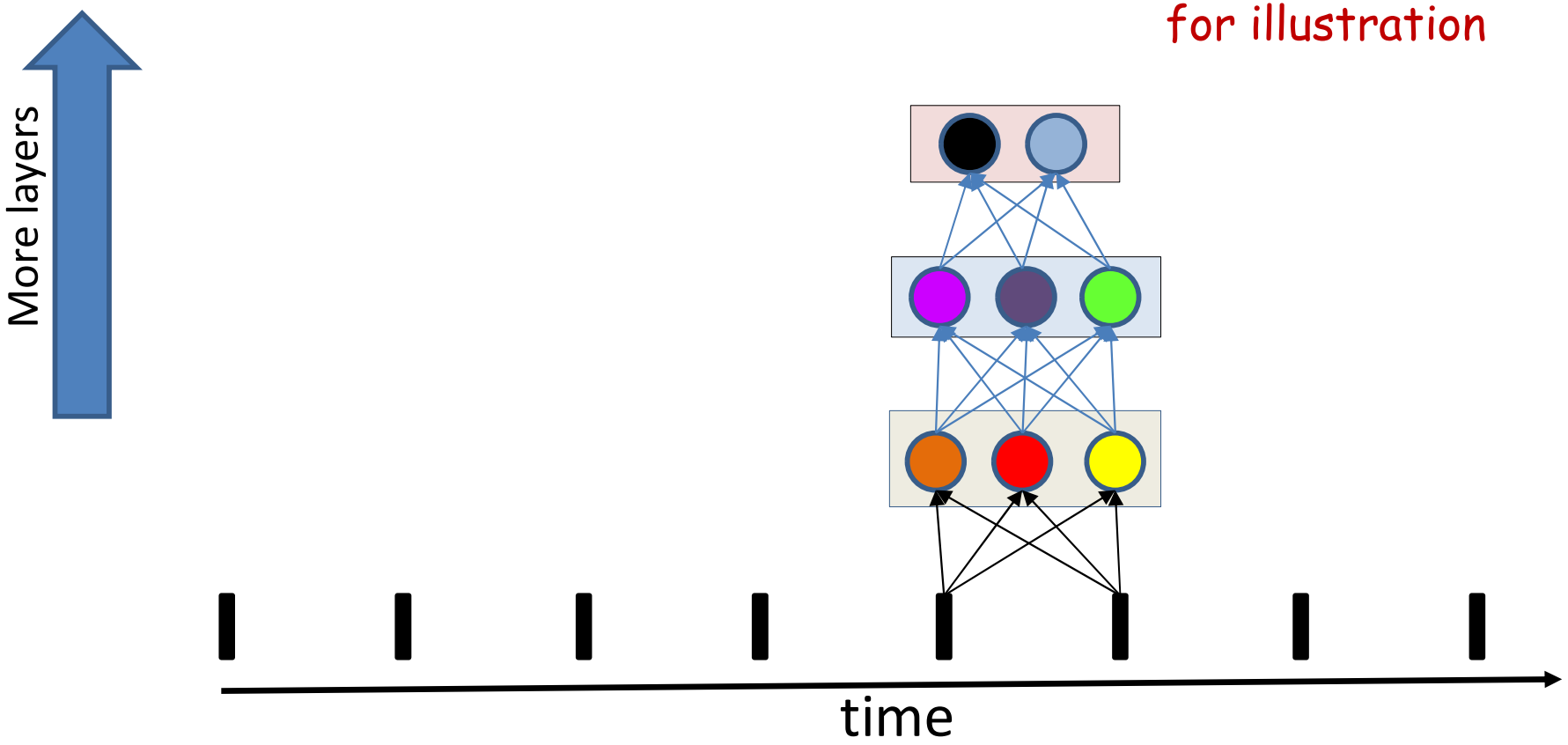
Scanning with 2-step "stride" for illustration



- Changing the illustration
  - Illustration of scanning with layers laid horizontally give erroneous impression of time (left to right) within a layer

Scanning networks

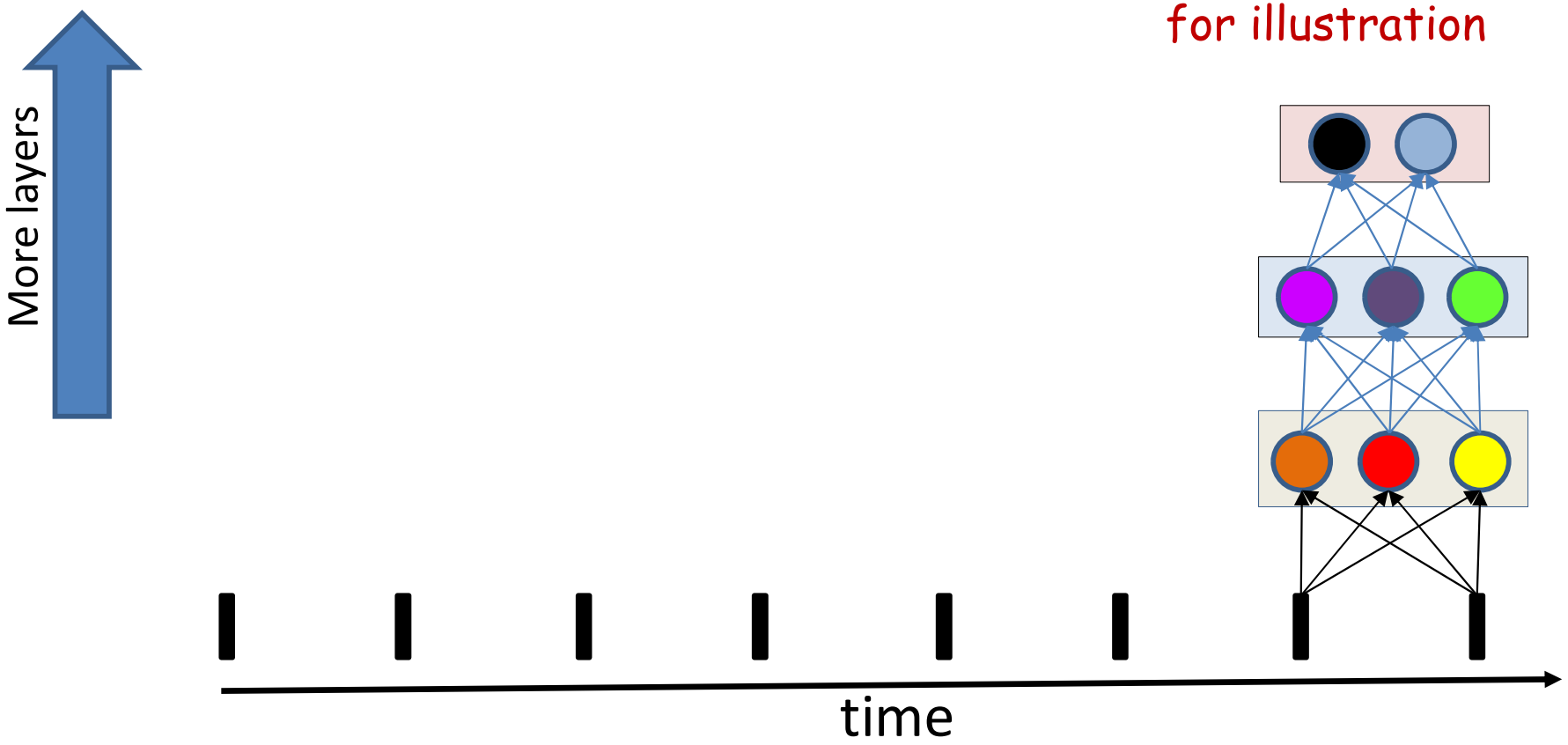
Scanning with 2-step "stride" for illustration



- Changing the illustration
  - Illustration of scanning with layers laid horizontally give erroneous impression of time (left to right) within a layer

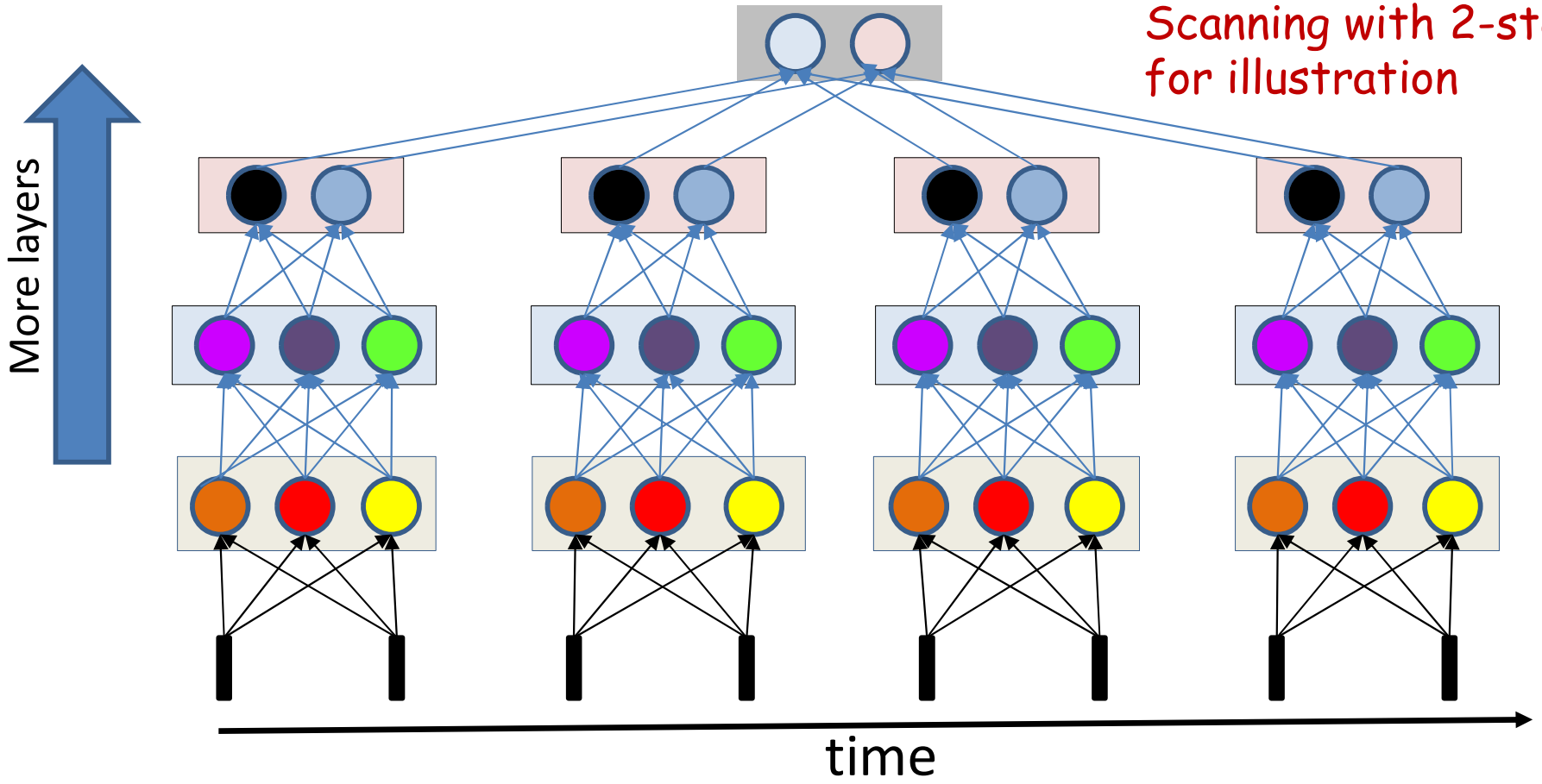
Scanning networks

Scanning with 2-step "stride" for illustration



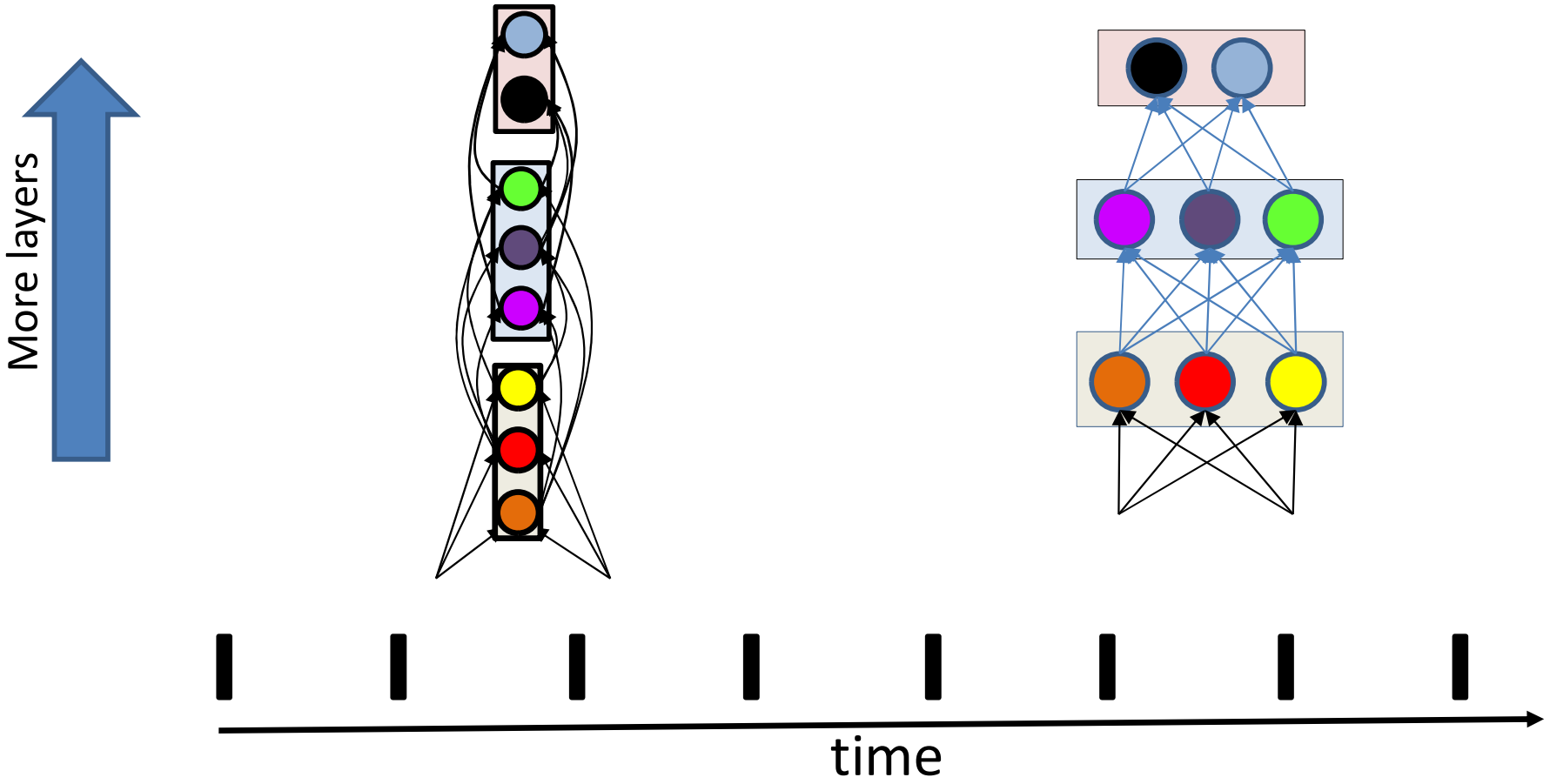
- Changing the illustration
  - Illustration of scanning with layers laid horizontally give erroneous impression of time (left to right) within a layer

Scanning networks



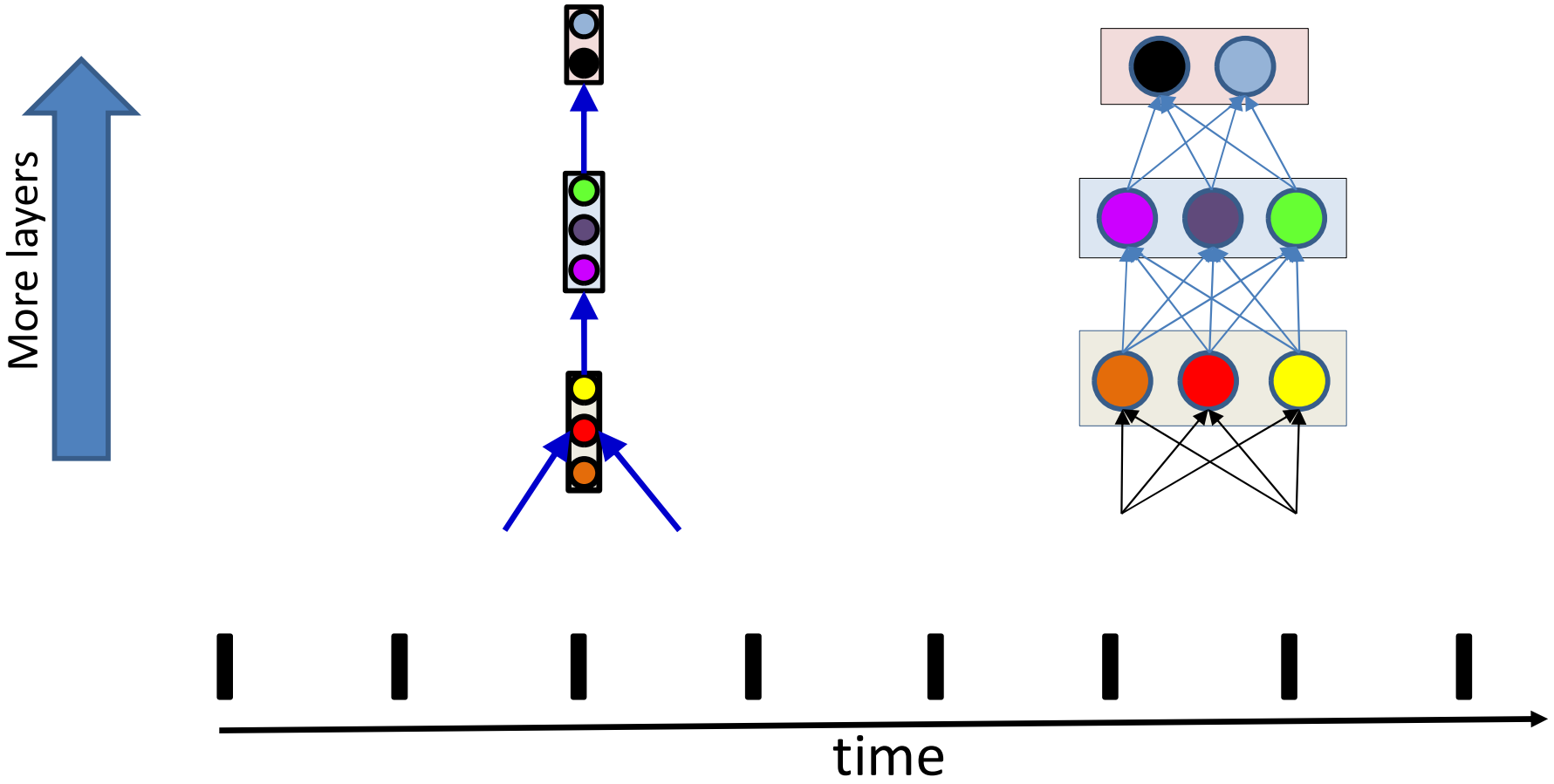
- Changing the illustration
  - Illustration of scanning with layers laid horizontally give erroneous impression of time (left to right) within a layer

Scanning networks



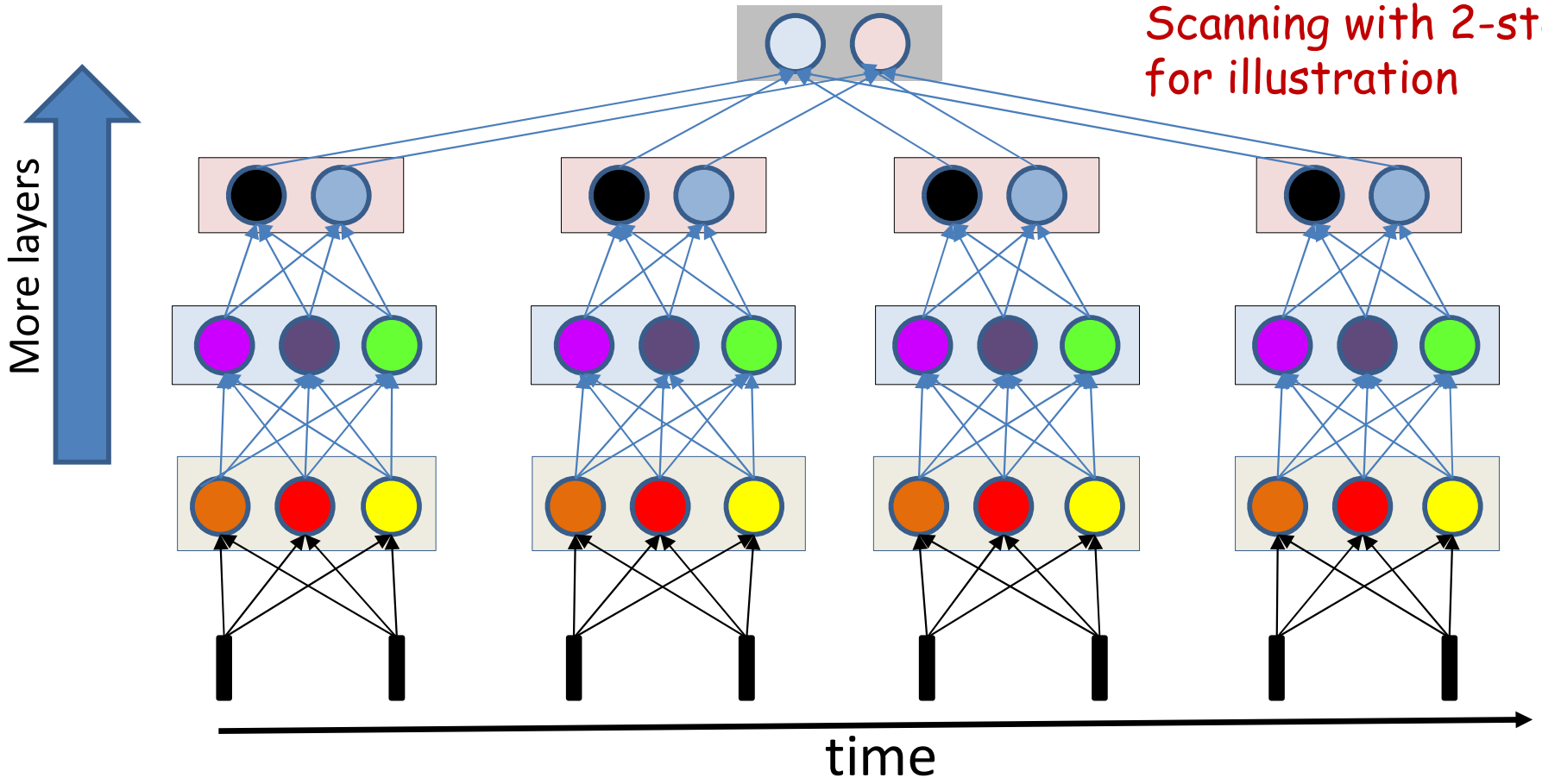
- Changing the illustration
  - Illustration of scanning with layers laid horizontally give erroneous impression of time (left to right) within a layer
- **A more appropriate illustration lays the layers vertically**

Scanning networks



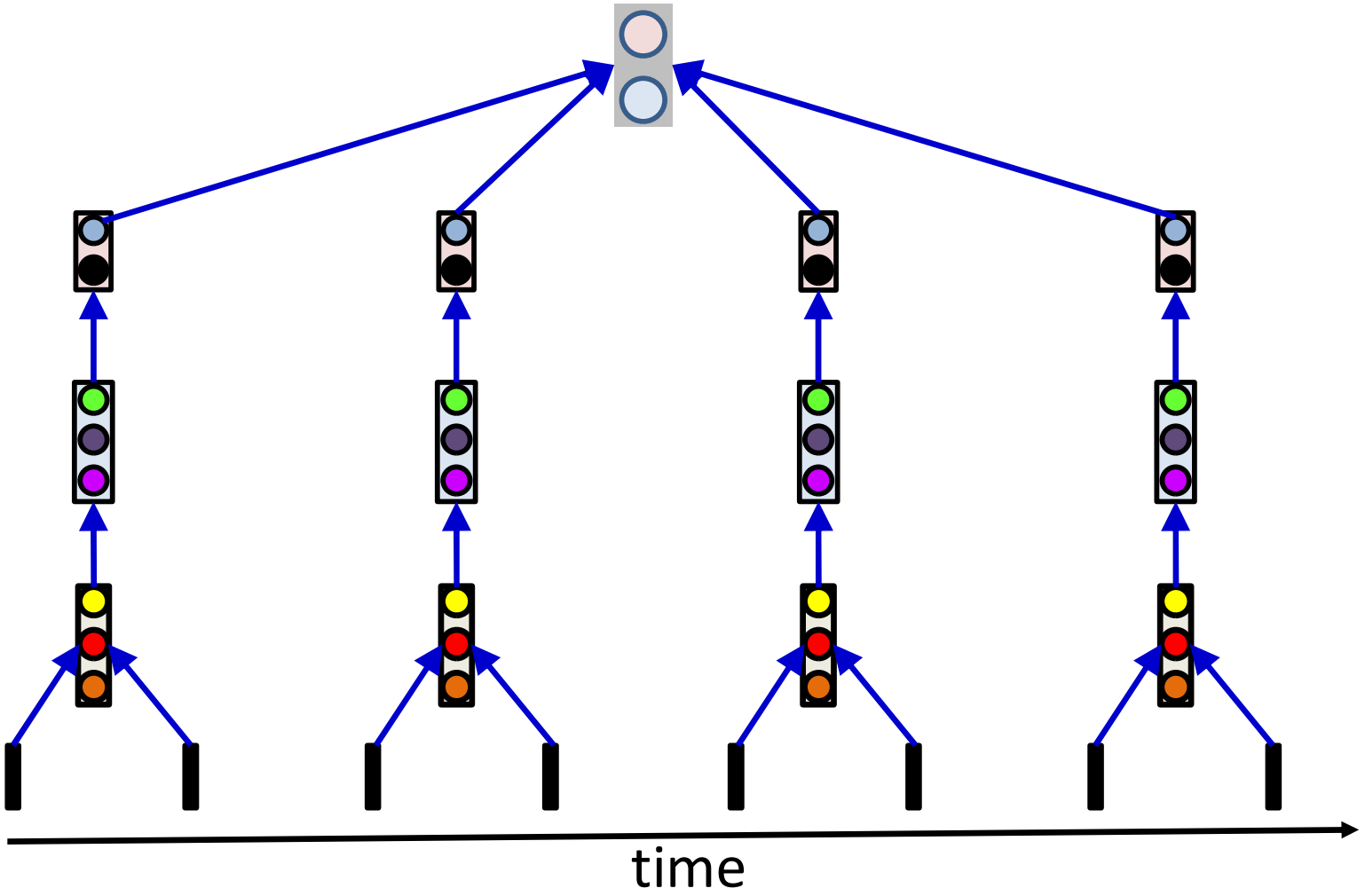
- Modifying the visualization for intuition..
  - Will still be the same network
- An arrow from one bar to another implies connections from *every* node in the source bar to *every* node in the destination bar
  - For N source-bar nodes and M destination-bar nodes, NxM connections

Scanning networks



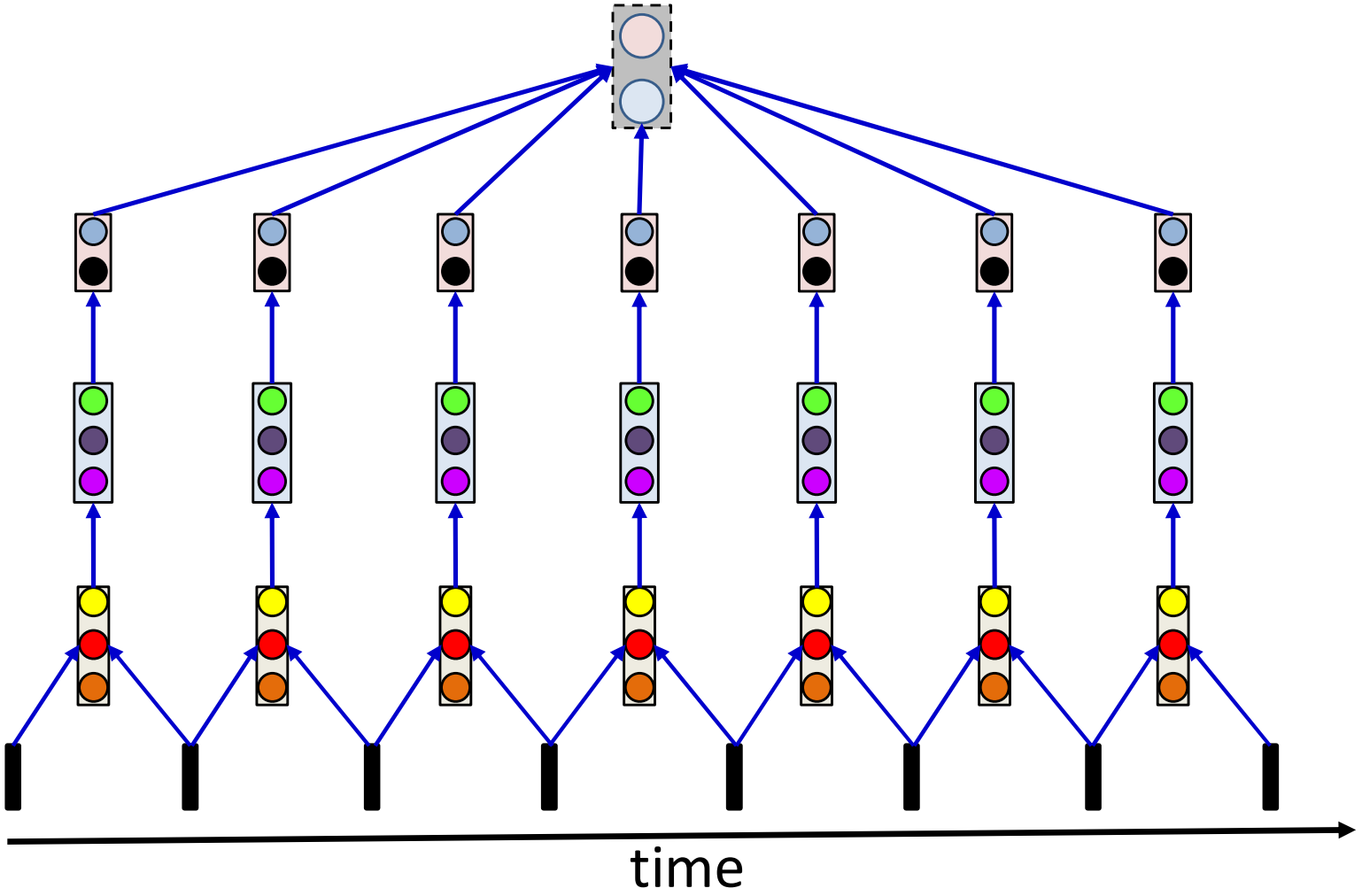
- Modifying the visualization for intuition..
  - Will still be the same network

Scanning networks



- A modified drawing
  - Indicates progression of time/space
    - The progression of “bars” of neurons is indicative of time
- More appropriate
  - Since vertical bars are vectors
  - A “bar” representing a layer represents computations occurring simultaneously, at that time

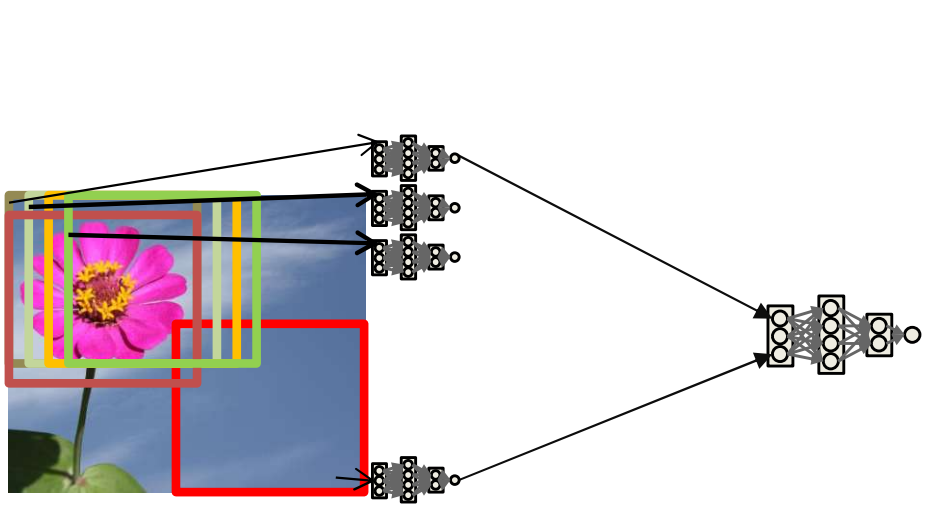
Scanning networks

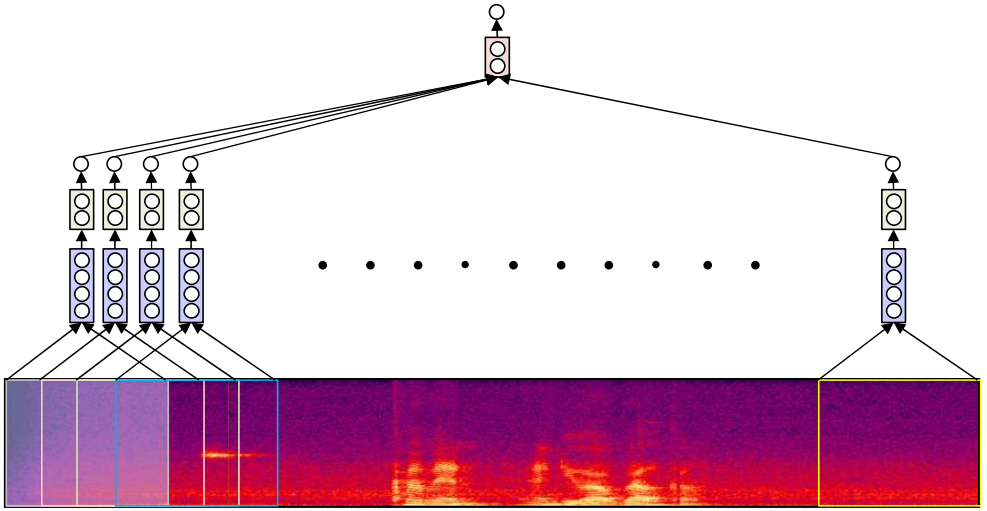


Visualizing scanning with a stride of 1

- A modified drawing
  - Indicates progression of time/space
    - The progression of "bars" of neurons is indicative of time
- More appropriate
  - Since vertical bars are vectors
  - A "bar" representing a layer represents computations occurring simultaneously, at that time

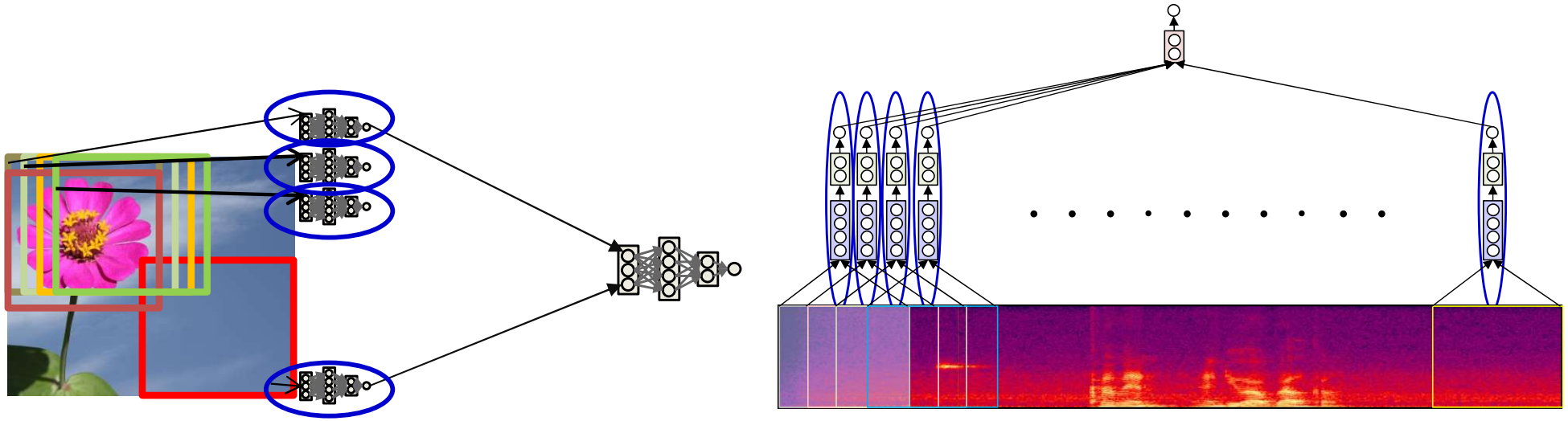
Training the network





- These are really just large networks
- Can just use conventional backpropagation to learn the parameters
  - Provide many training examples
    - Images with and without flowers
      - Target output 1 for flower images, 0 for non-flower images
    - Speech recordings with and without the word “welcome”
      - Target output 1 for “welcome” recordings, 0 for recordings without “welcome”
  - Gradient descent to minimize the total divergence between predicted and desired outputs
- Will actually learn the lower-level flower (or welcome) detector

Training the network: constraint



- These are *shared parameter* networks
  - All lower-level subnets are identical
    - Are all searching for the same pattern
  - Any update of the parameters of one copy of the subnet must equally update *all* copies

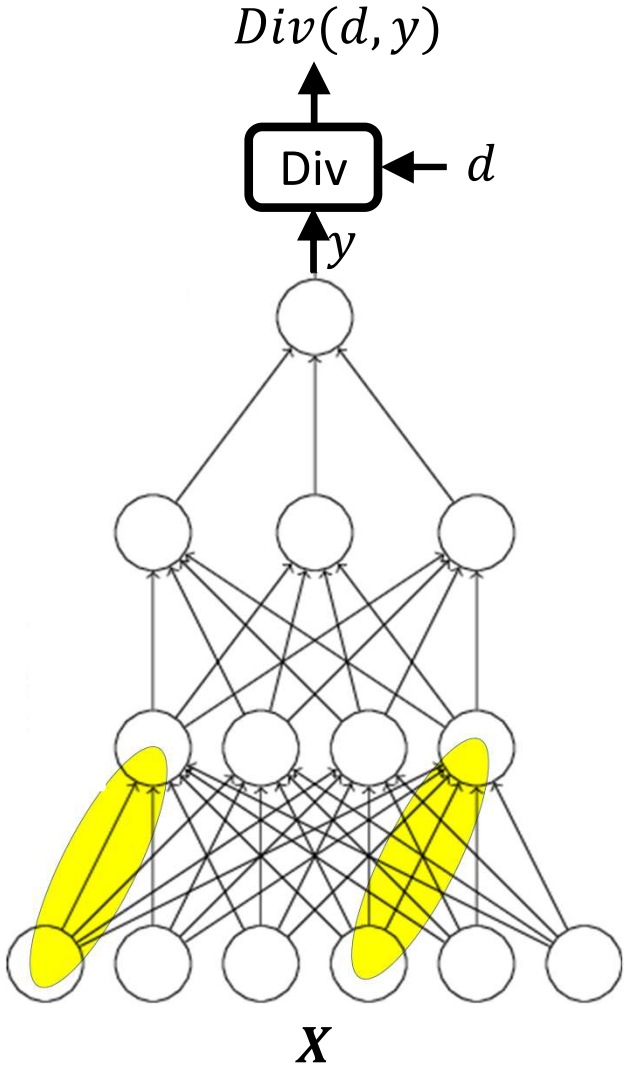
Learning in shared parameter networks

- Consider a simple network with shared weights

$$w_{ij}^{k} = w_{mn}^{l} = w^{\mathcal{S}}$$

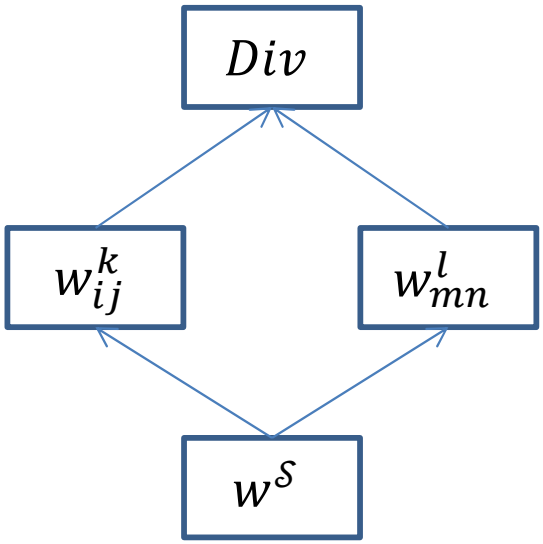
  - A weight w_{ij}^{k} is required to be identical to the weight w_{mn}^{l}

- For any training instance $\boldsymbol{X}$, a small perturbation of $w^{\mathcal{S}}$ perturbs both w_{ij}^{k} and w_{mn}^{l} identically
  - Each of these perturbations will individually influence the divergence $Div(d, y)$



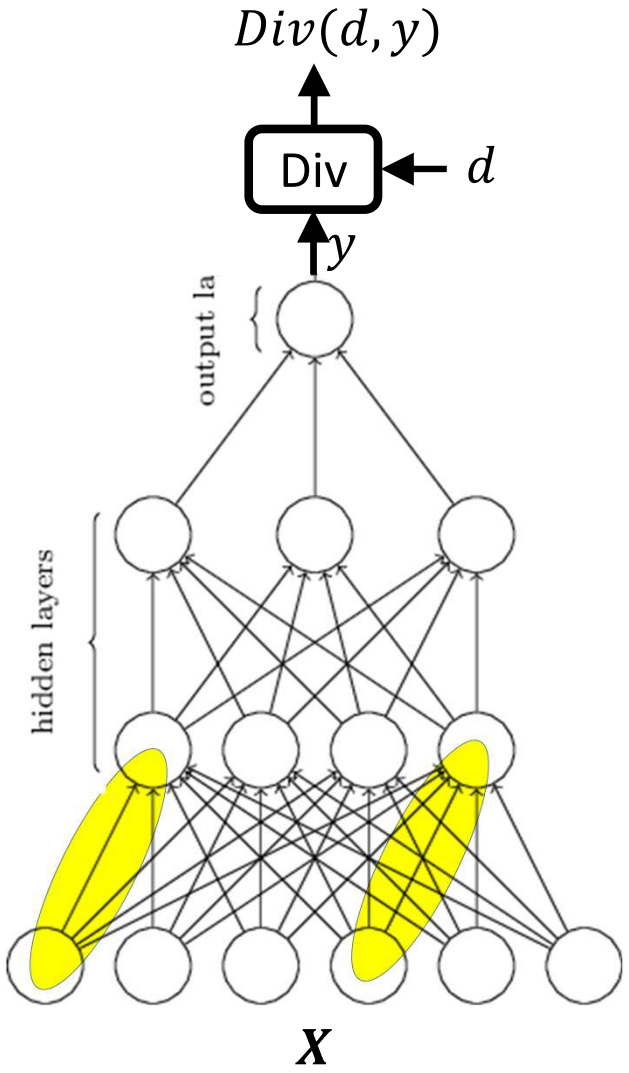
Computing the divergence of shared parameters

Influence diagram



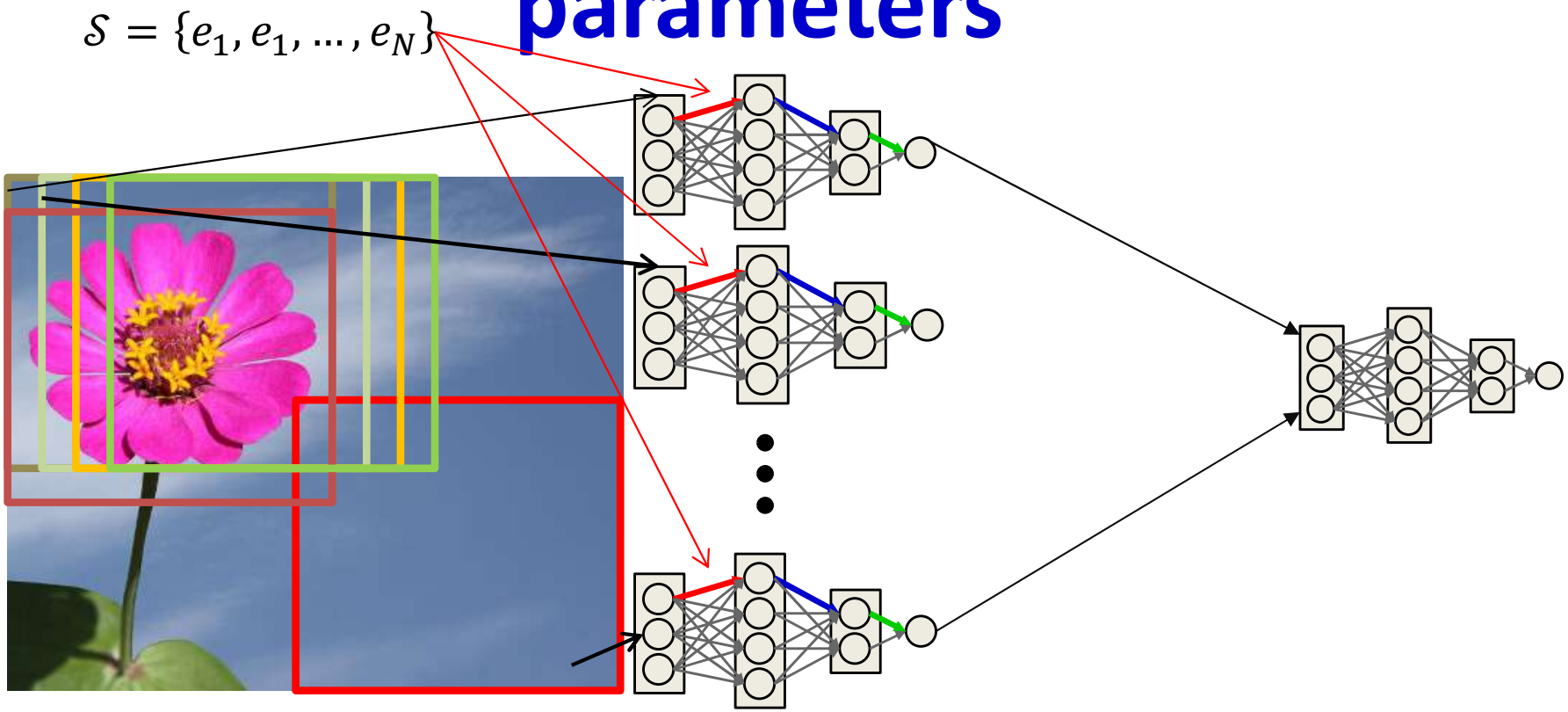
$$\frac{dDiv}{dw^s} = \frac{\partial Div}{\partial w_{ij}^k}\frac{dw_{ij}^k}{dw^s} + \frac{\partial Div}{\partial w_{mn}^l}\frac{dw_{mn}^l}{dw^s}$$

$$= \frac{\partial Div}{\partial w_{ij}^k} + \frac{\partial Div}{\partial w_{mn}^l}$$



- Each of the individual terms can be computed via backpropagation

Computing the divergence of shared parameters



- More generally, let S be any set of edges that have a common value, and w^S be the common weight of the set
  - E.g. the set of all red weights in the figure

$$\frac{dDiv}{dw^S} = \sum_{e \in S} \frac{\partial Div}{\partial w^e}$$

- The individual terms in the sum can be computed via backpropagation

Training networks with shared parameters

- Gradient descent algorithm:
- Initialize all weights $\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_K$
- Do:
  - For every set $\mathcal{S}$:
    - Compute:

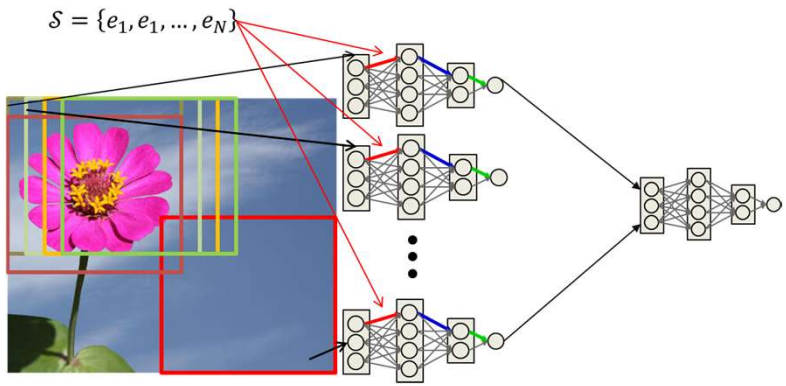
$$\nabla_{\mathcal{S}} Loss = \frac{dLoss}{dw^{\mathcal{S}}}$$

$$w^{\mathcal{S}} = w^{\mathcal{S}} - \eta \nabla_{\mathcal{S}} Loss^T$$

    - For every $(k, i, j) \in \mathcal{S}$ update:

$$w_{i,j}^{(k)} = w^{\mathcal{S}}$$

- Until $Loss$ has converged



Training networks with shared parameters

- Gradient descent algorithm:
- Initialize all weights $\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_K$
- Do:
  - For every set $\mathcal{S}$:
    - Compute:

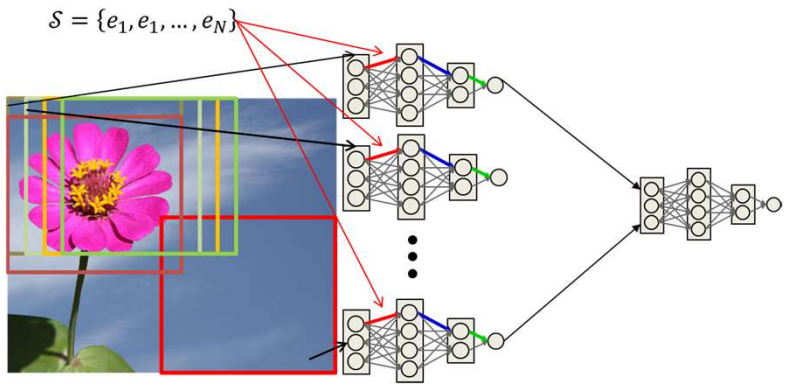
$$\nabla_{\mathcal{S}} Loss = \frac{dLoss}{dw^{\mathcal{S}}}$$

$$w^{\mathcal{S}} = w^{\mathcal{S}} - \eta \nabla_{\mathcal{S}} Loss^T$$

    - For every $(k, i, j) \in \mathcal{S}$ update:

$$w_{i,j}^{(k)} = w^{\mathcal{S}}$$

- Until $Loss$ has converged



Training networks with shared parameters

- For every training instance X
  - For every set $\mathcal{S}$:
    - For every $(k, i, j) \in \mathcal{S}$:

$$\frac{dLoss}{dw^{\mathcal{S}}} \mathrel{+}= \frac{\partial Div}{\partial w_{i,j}^{(k)}}$$

    - $\nabla_{\mathcal{S}} Loss = \frac{dLoss}{dw^{\mathcal{S}}}$
  - Compute:

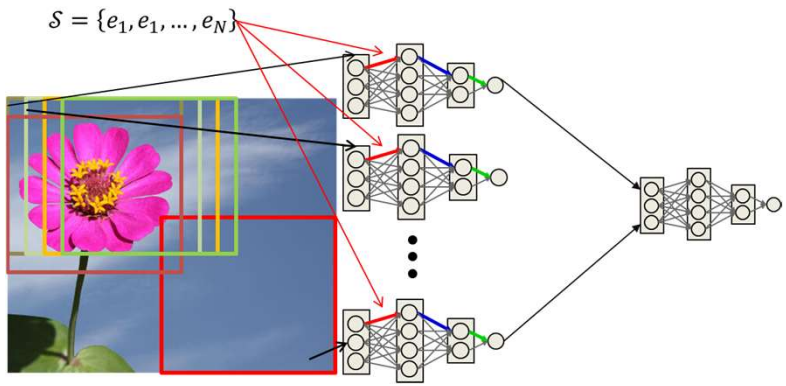
$$\nabla_{\mathcal{S}} Loss = \frac{dLoss}{dw^{\mathcal{S}}}$$

$$w^{\mathcal{S}} = w^{\mathcal{S}} - \eta \nabla_{\mathcal{S}} Loss^T$$

  - For every $(k, i, j) \in \mathcal{S}$ update:

$$w_{i,j}^{(k)} = w^{\mathcal{S}}$$

- Until $Loss$ has converged



Training networks with shared parameters

- For every training instance X
  - For every set $\mathcal{S}$:
    - For every $(k, i, j) \in \mathcal{S}$:

$$\frac{dLoss}{dw^{\mathcal{S}}} \mathrel{+}= \frac{\partial Div}{\partial w_{i,j}^{(k)}}$$

Computed by Backprop

    - $\nabla_{\mathcal{S}} Loss = \frac{dLoss}{dw^{\mathcal{S}}}$
  - Compute:

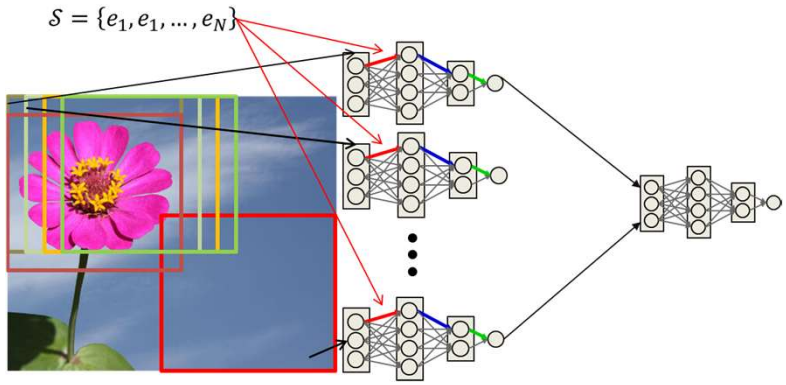
$$\nabla_{\mathcal{S}} Loss = \frac{dLoss}{dw^{\mathcal{S}}}$$

$$w^{\mathcal{S}} = w^{\mathcal{S}} - \eta \nabla_{\mathcal{S}} Loss^T$$

  - For every $(k, i, j) \in \mathcal{S}$ update:

$$w_{i,j}^{(k)} = w^{\mathcal{S}}$$

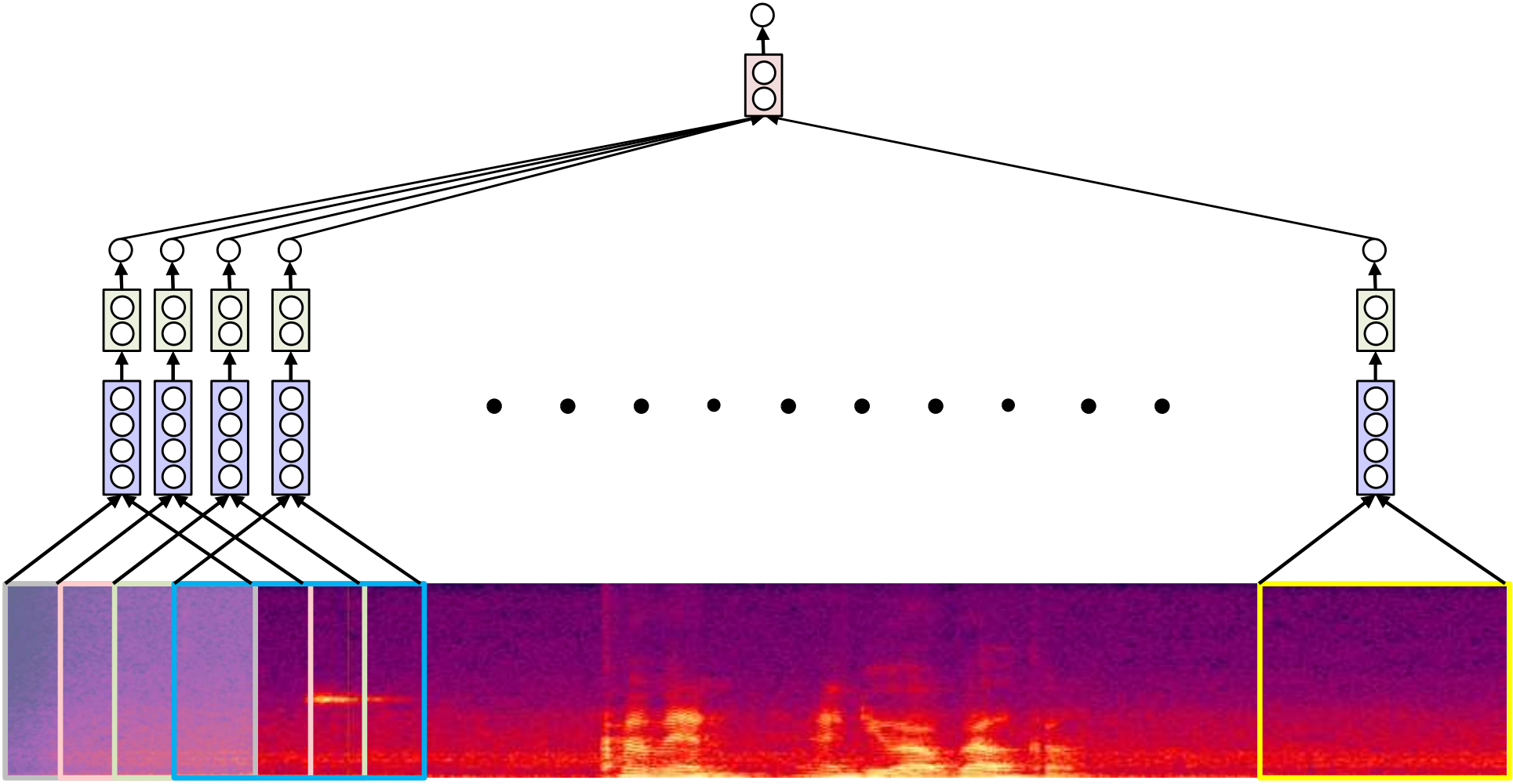
- Until $Loss$ has converged

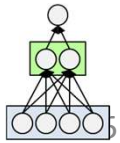


Story so far

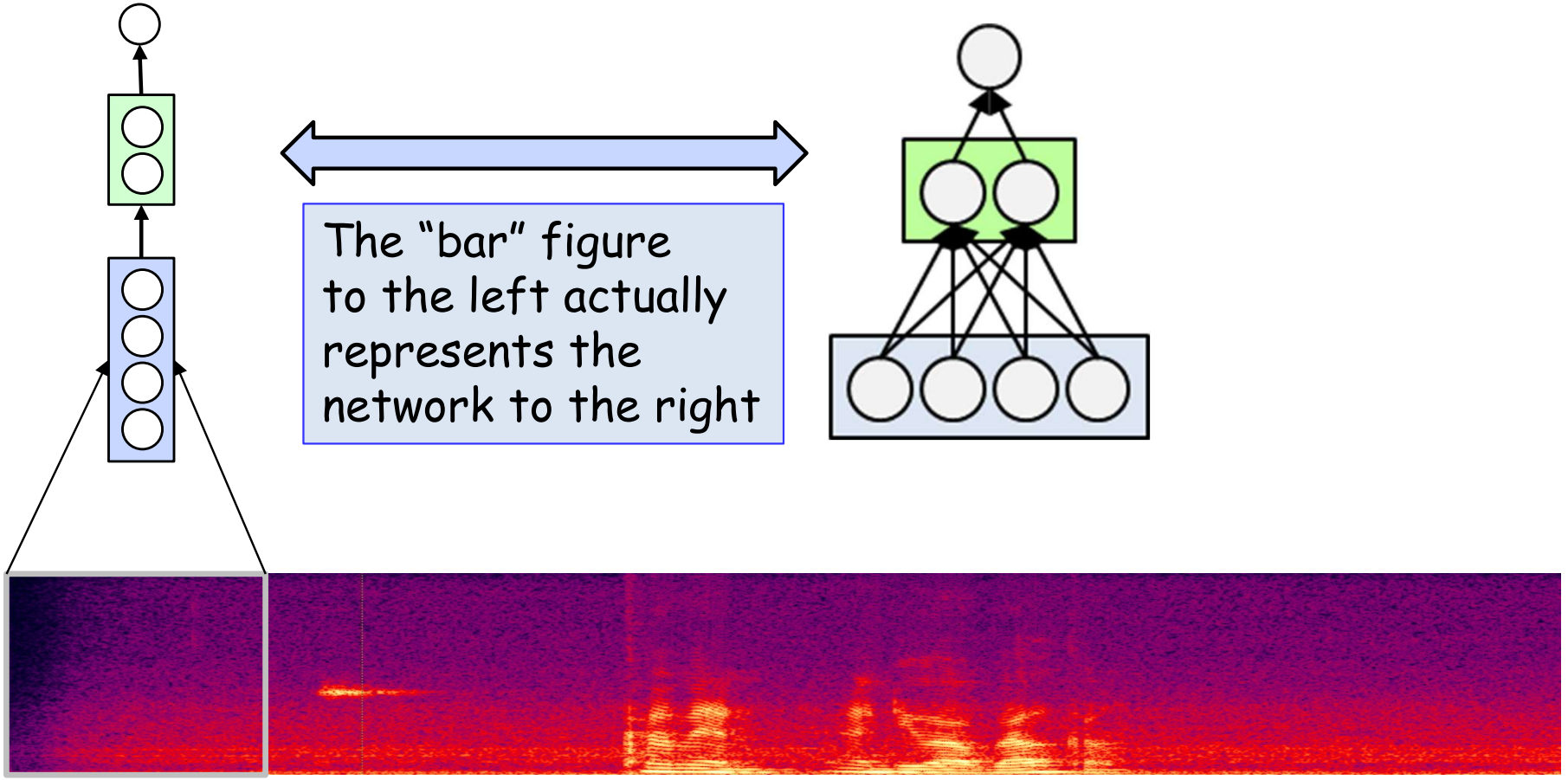
- Position-invariant pattern classification can be performed by scanning
  - 1-D scanning for sound
  - 2-D scanning for images
  - 3-D and higher-dimensional scans for higher dimensional data
- Scanning is equivalent to composing a large network with repeating subnets
  - The large network has shared subnets
- Learning in scanned networks: Backpropagation rules must be modified to combine gradients from parameters that share the same value
  - The principle applies in general for networks with shared parameters

Scanning: A closer look



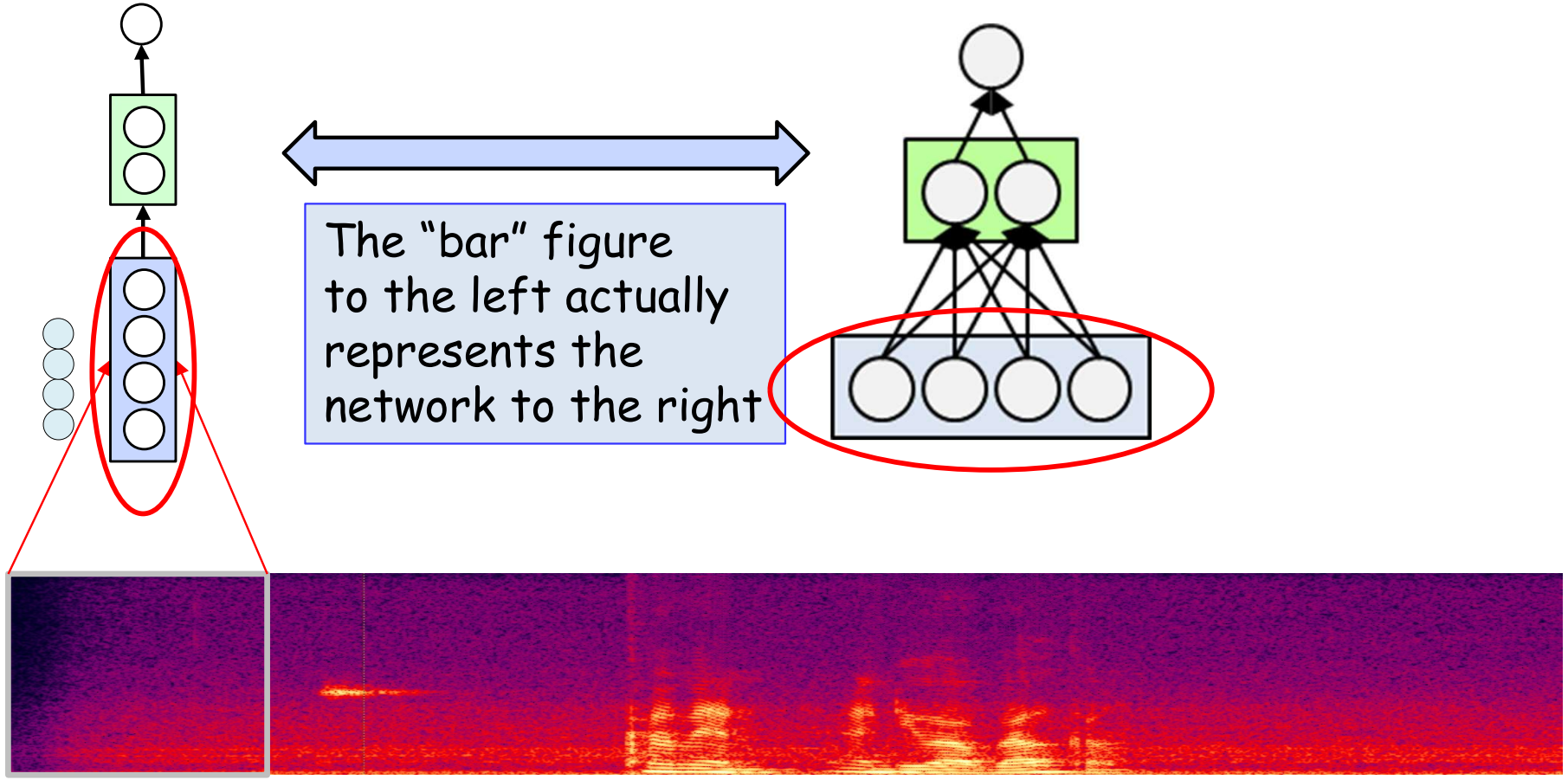
- The entire MLP operates on each “window” of the input
  - Using the “bar” visual  to represent the network 

Scanning : A closer look



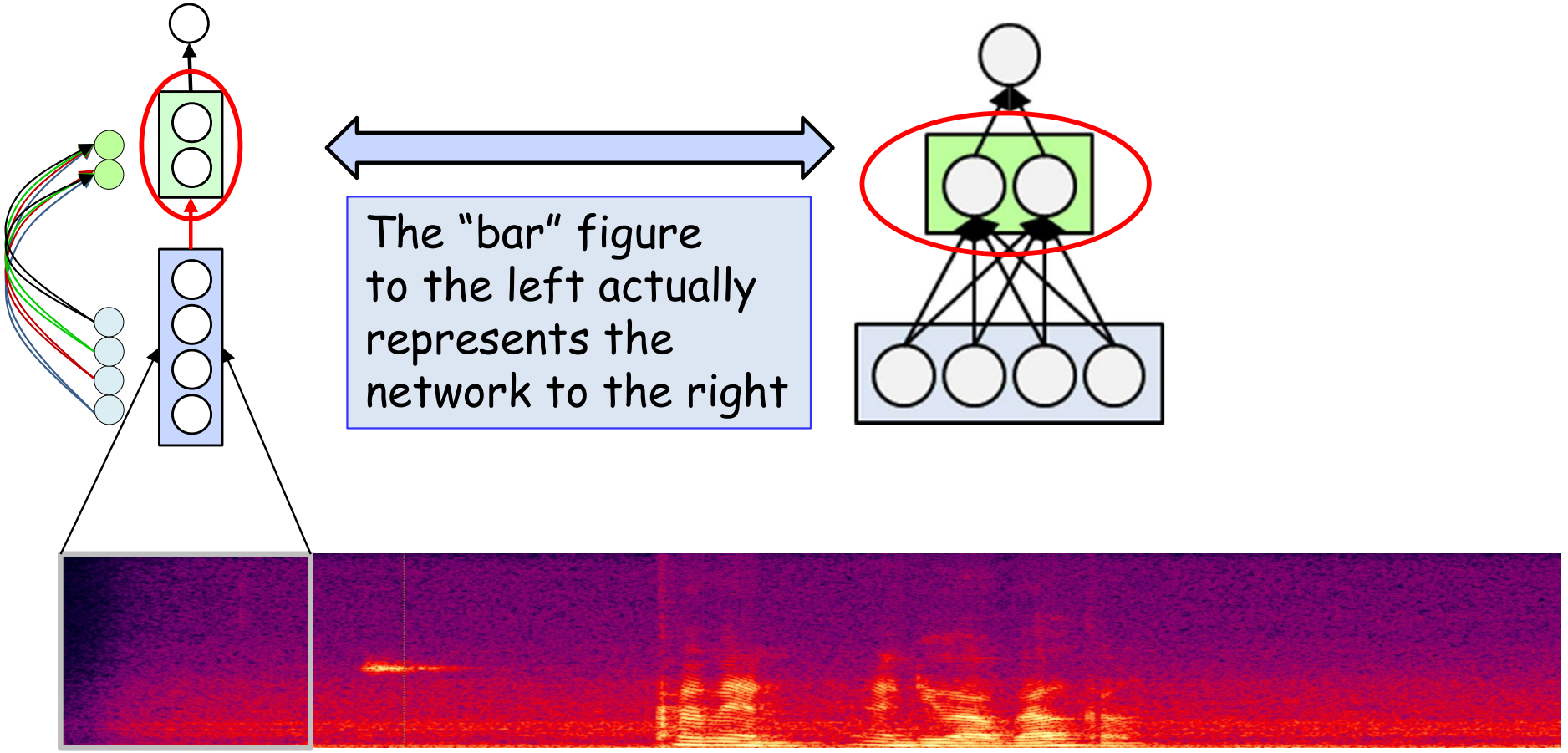
- Let's take a closer look at the scanning solution
- At each location, each neuron computes a value based on its inputs
  - Which may either be the input spectrogram or the outputs of the previous layer

Scanning



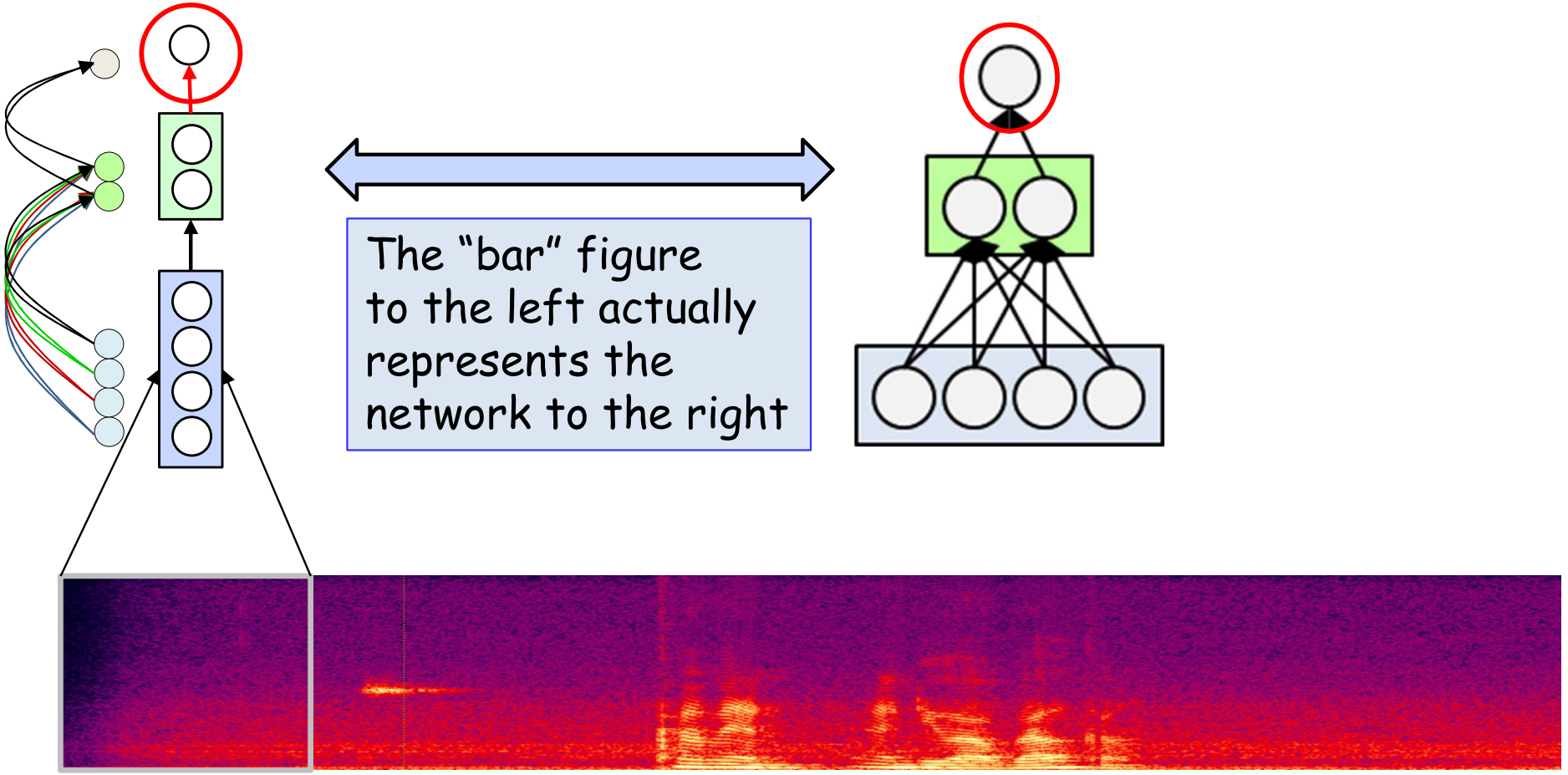
- At each location, each neuron computes a value based on its inputs
  - Which may either be the input spectrogram
  -

Scanning



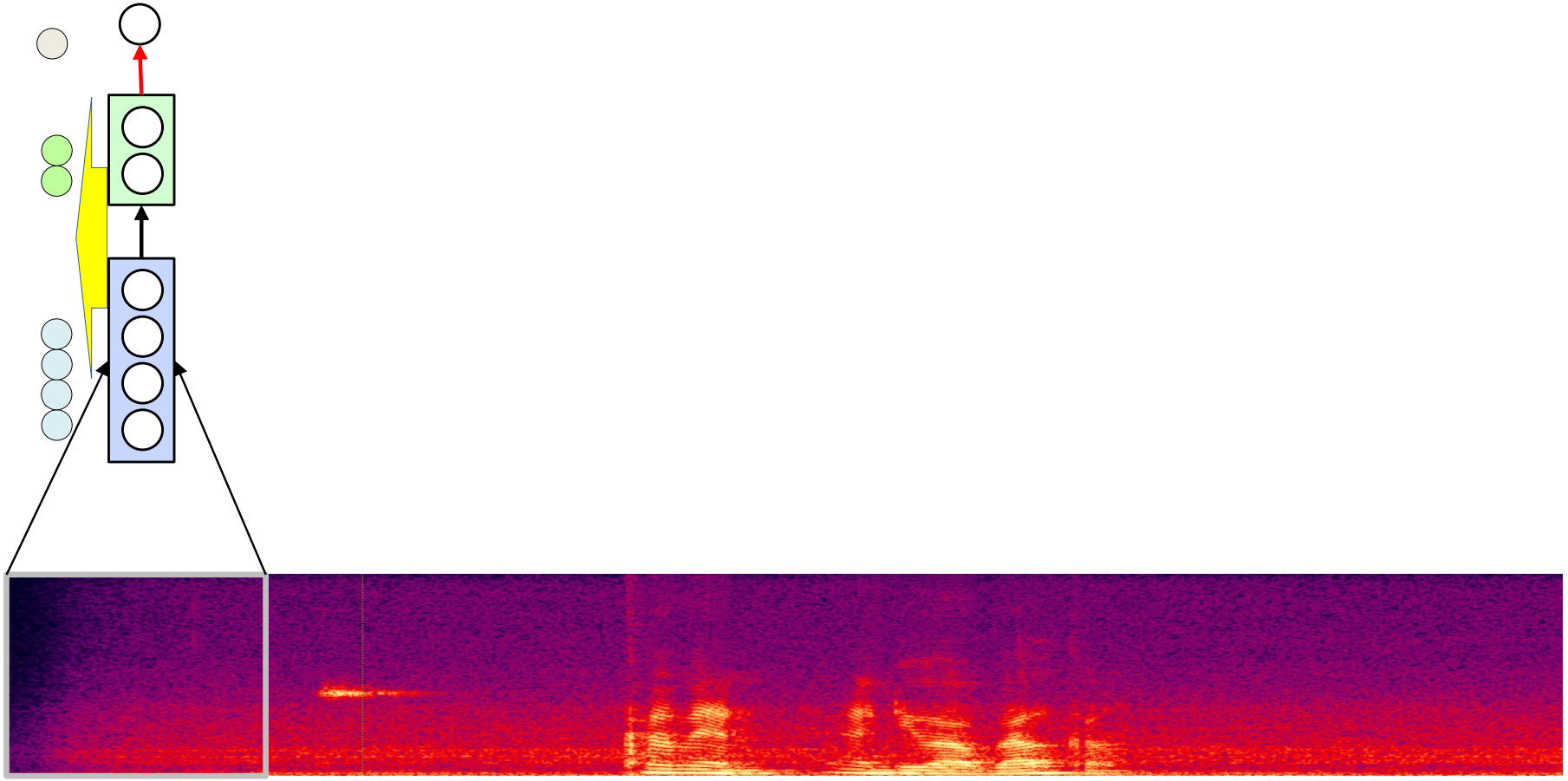
- At each location, each neuron computes a value based on its inputs
  - Which may either be the input spectrogram or the outputs of the previous layer

Scanning



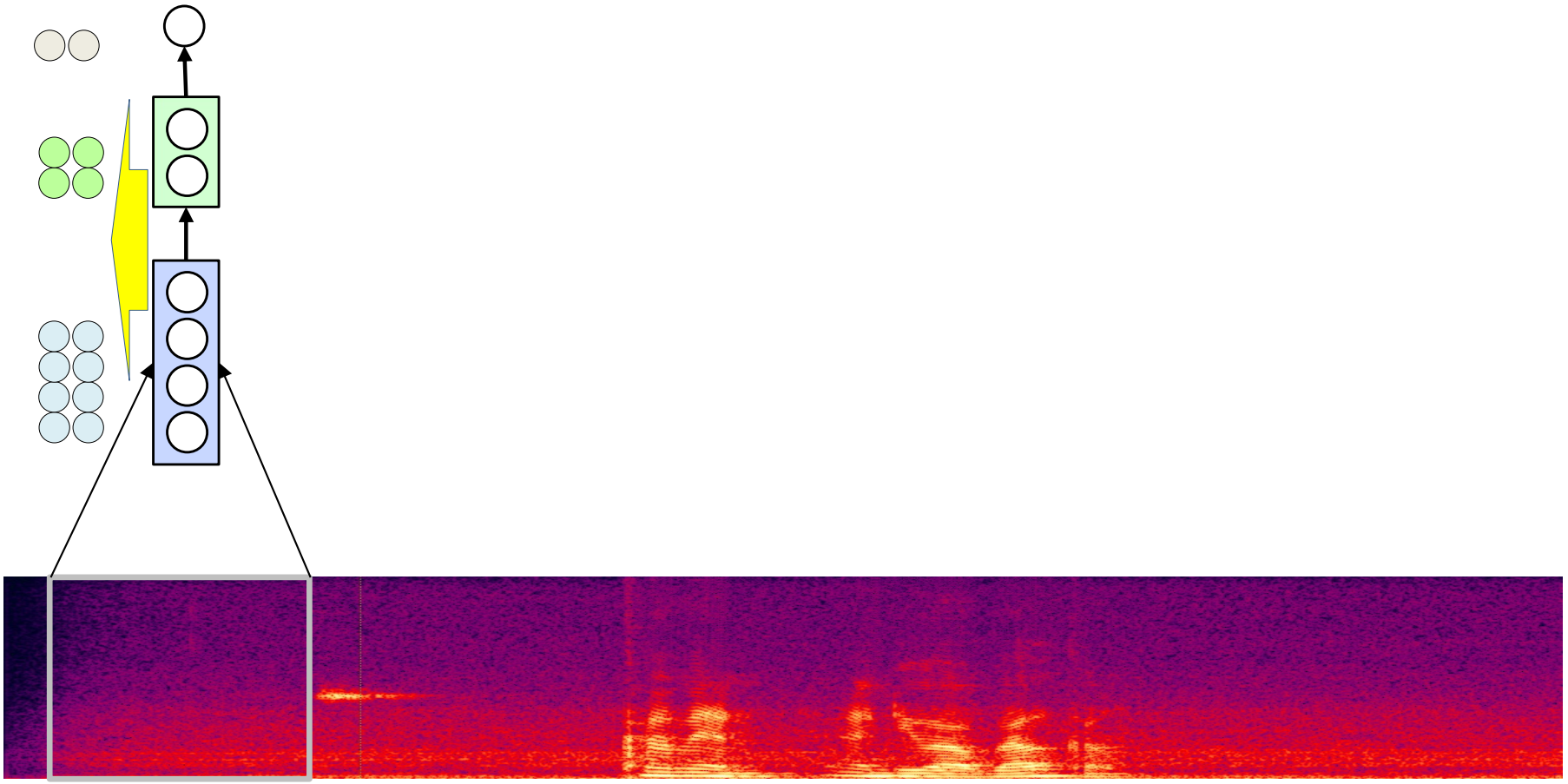
- At each location, each neuron computes a value based on its inputs
  - Which may either be the input spectrogram or the outputs of the previous layer

Scanning



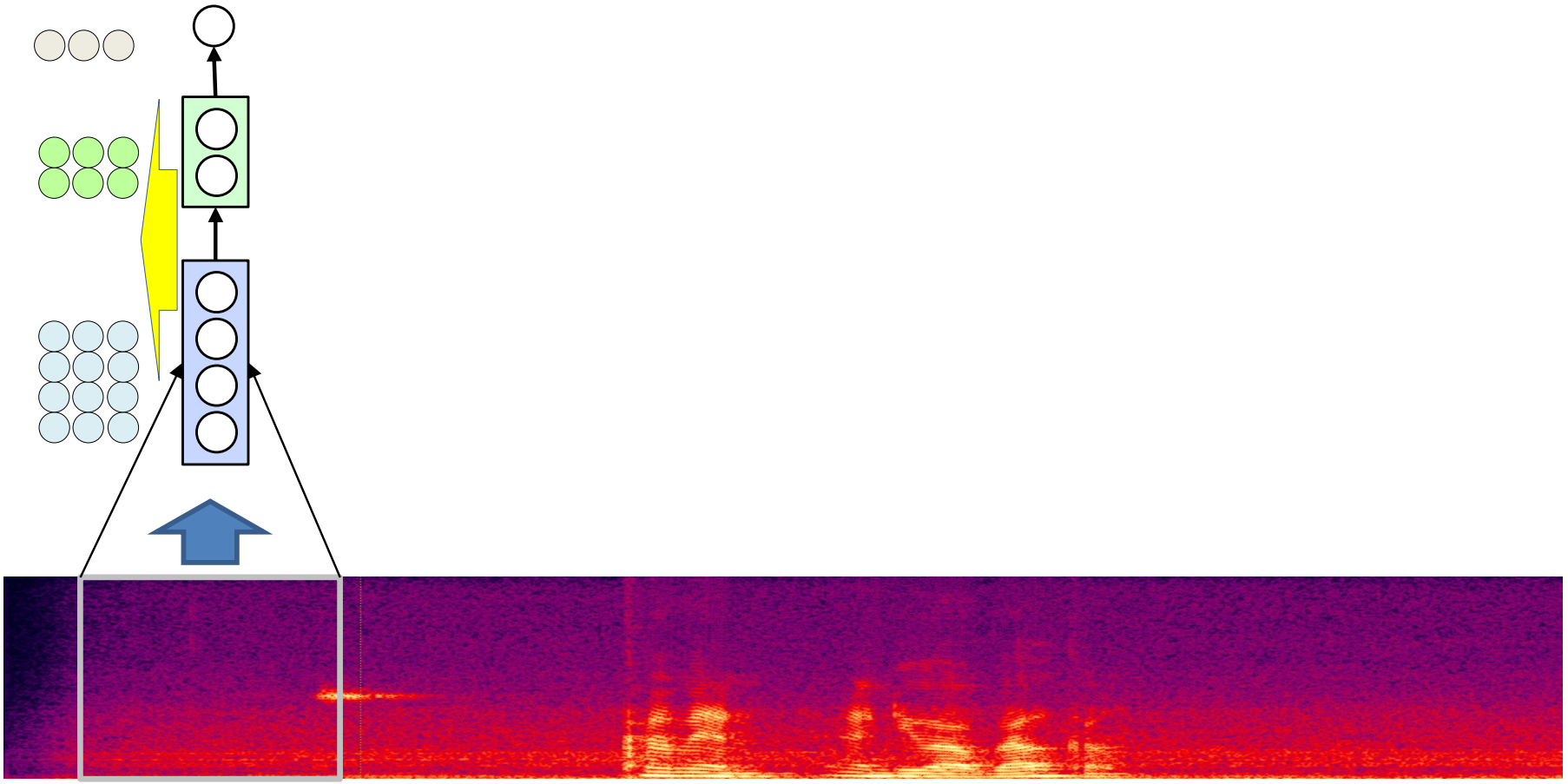
- At each location, each neuron computes a value based on its inputs
  - Which may either be the input spectrogram or the outputs of the previous layer

Scanning



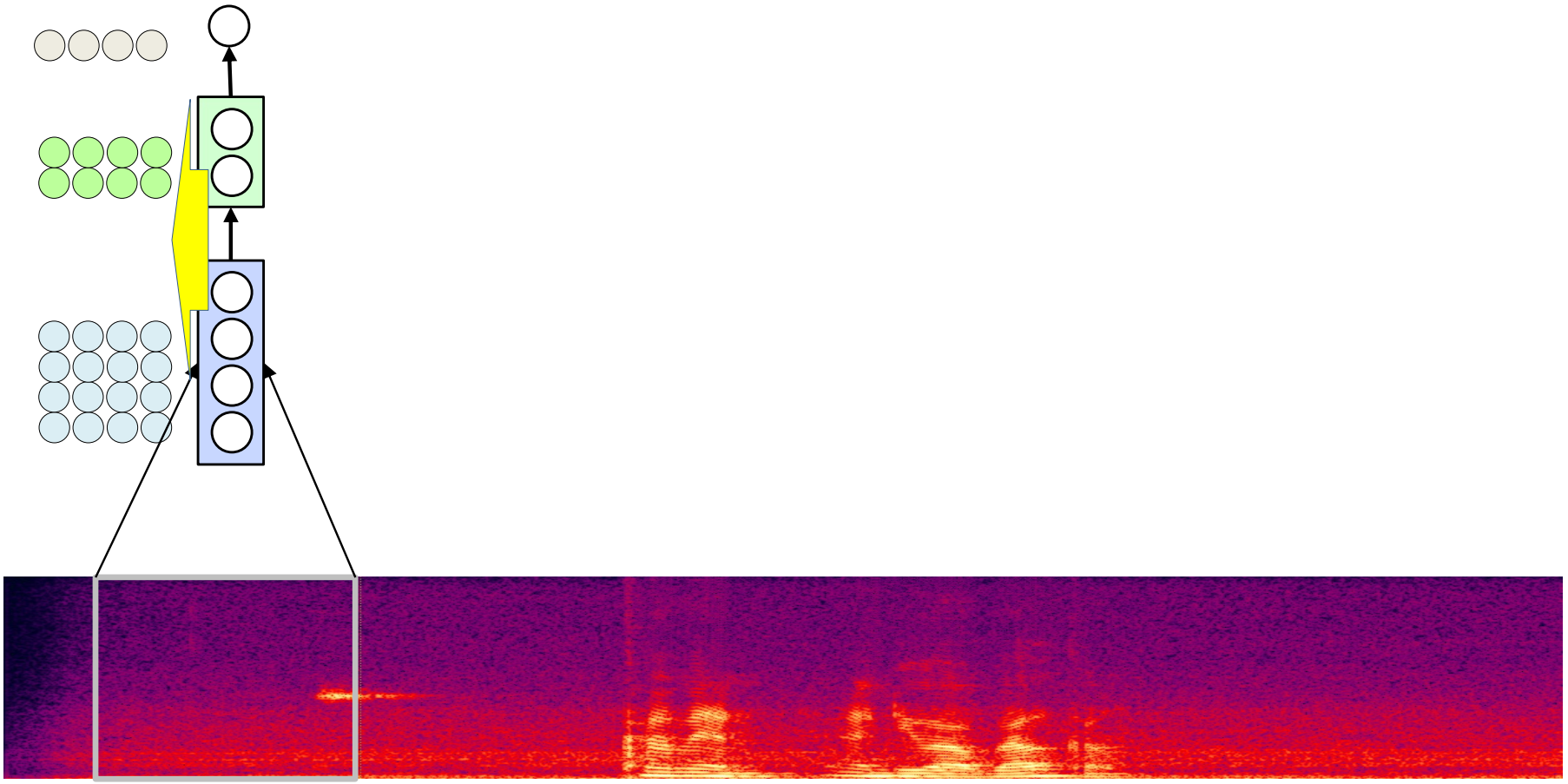
- The same sequence of computations is performed at each location
  - Producing similar sets of values
    - One value per neuron in each layer

Scanning



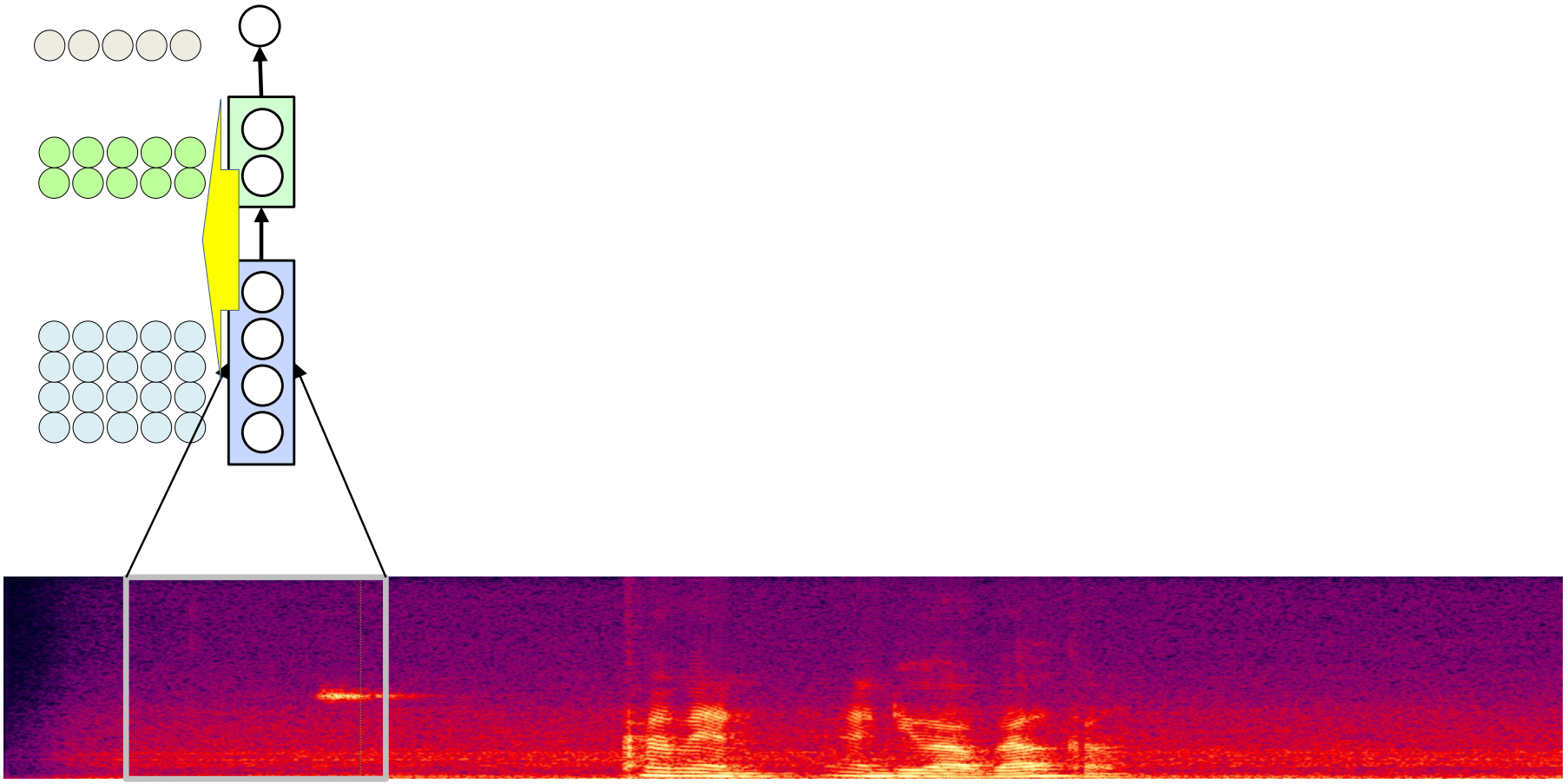
- The same sequence of computations is performed at each location
  - Producing similar sets of values
    - One value per neuron in each layer

Scanning



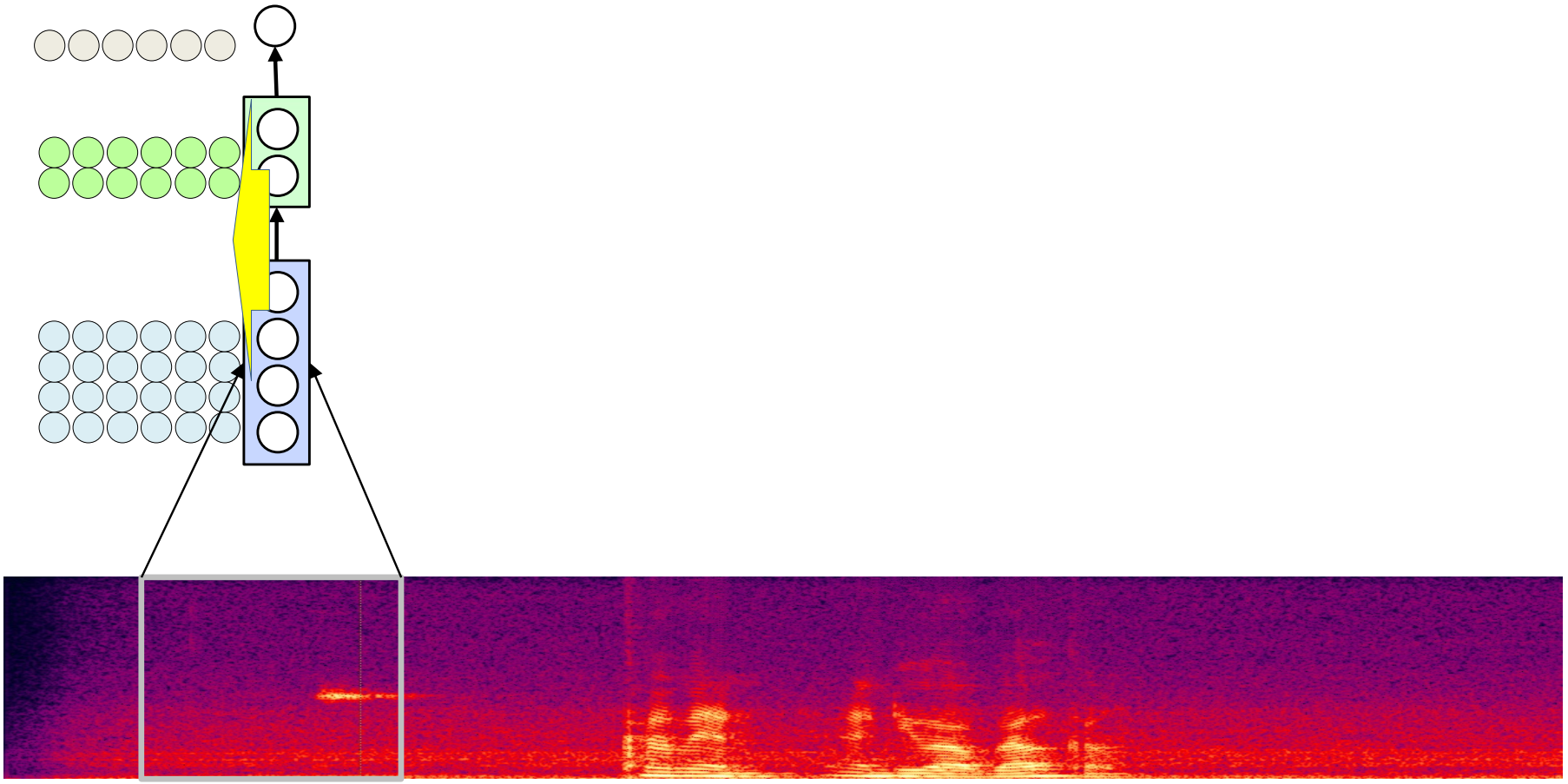
- The same sequence of computations is performed at each location
  - Producing similar sets of values
    - One value per neuron in each layer

Scanning



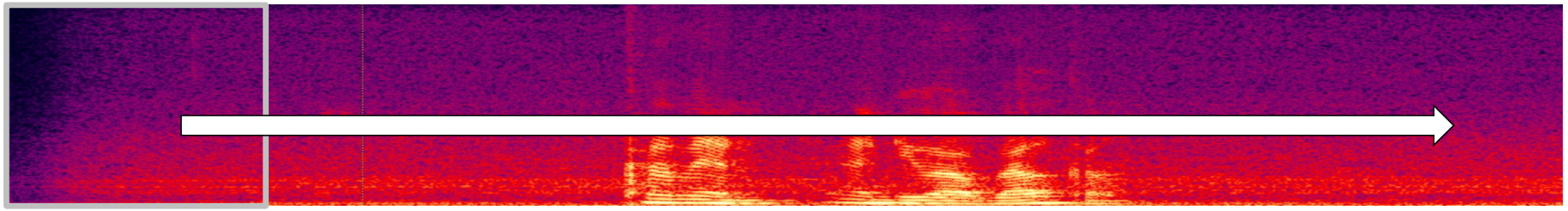
- The same sequence of computations is performed at each location
  - Producing similar sets of values
    - One value per neuron in each layer

Scanning



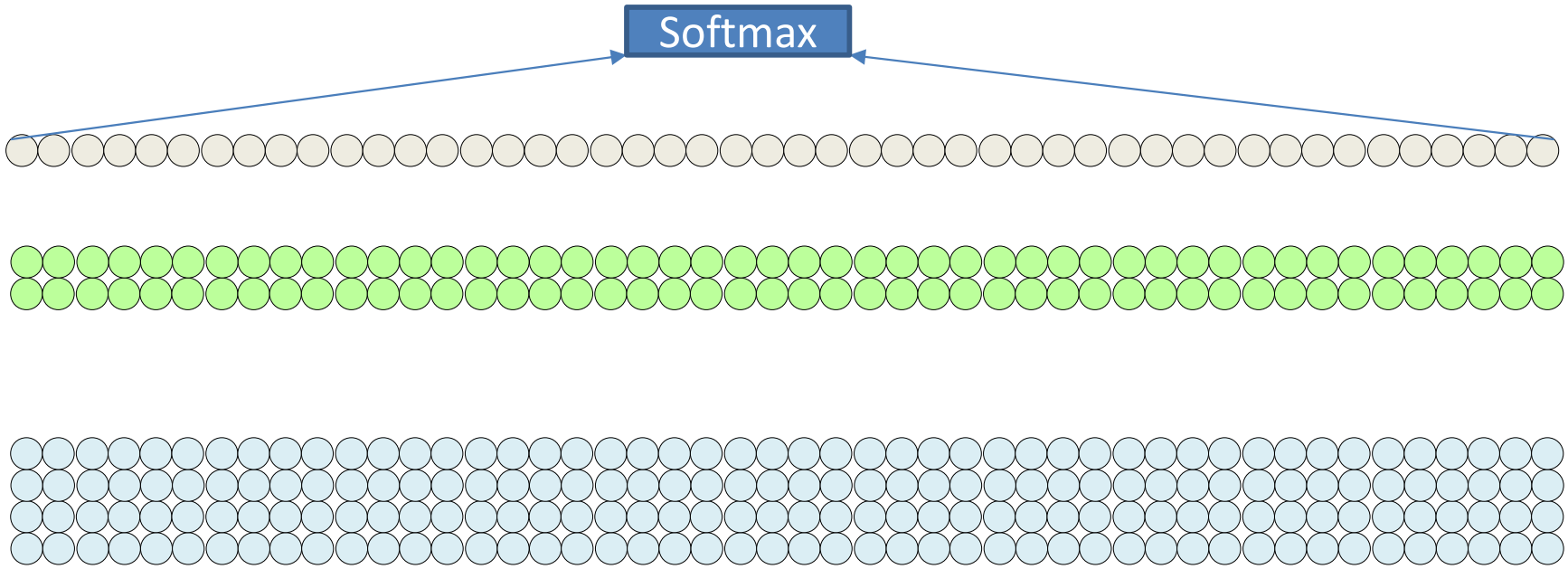
- The same sequence of computations is performed at each location
  - Producing similar sets of values
    - One value per neuron in each layer

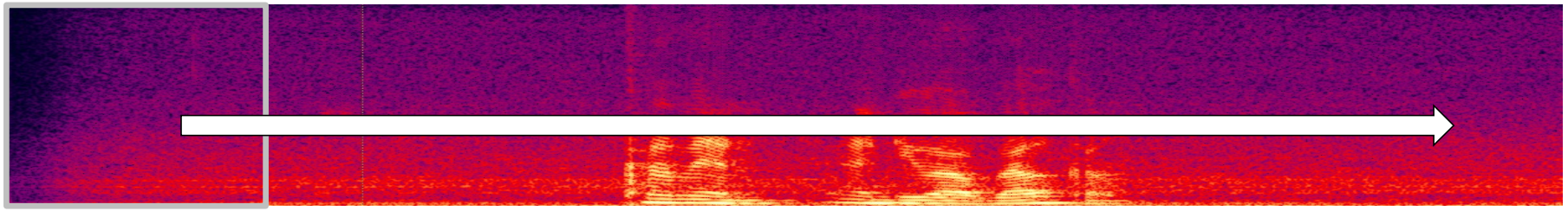
Scanning the input



- We get a complete set of values (represented as a column) at each location evaluated by the MLP during the scan

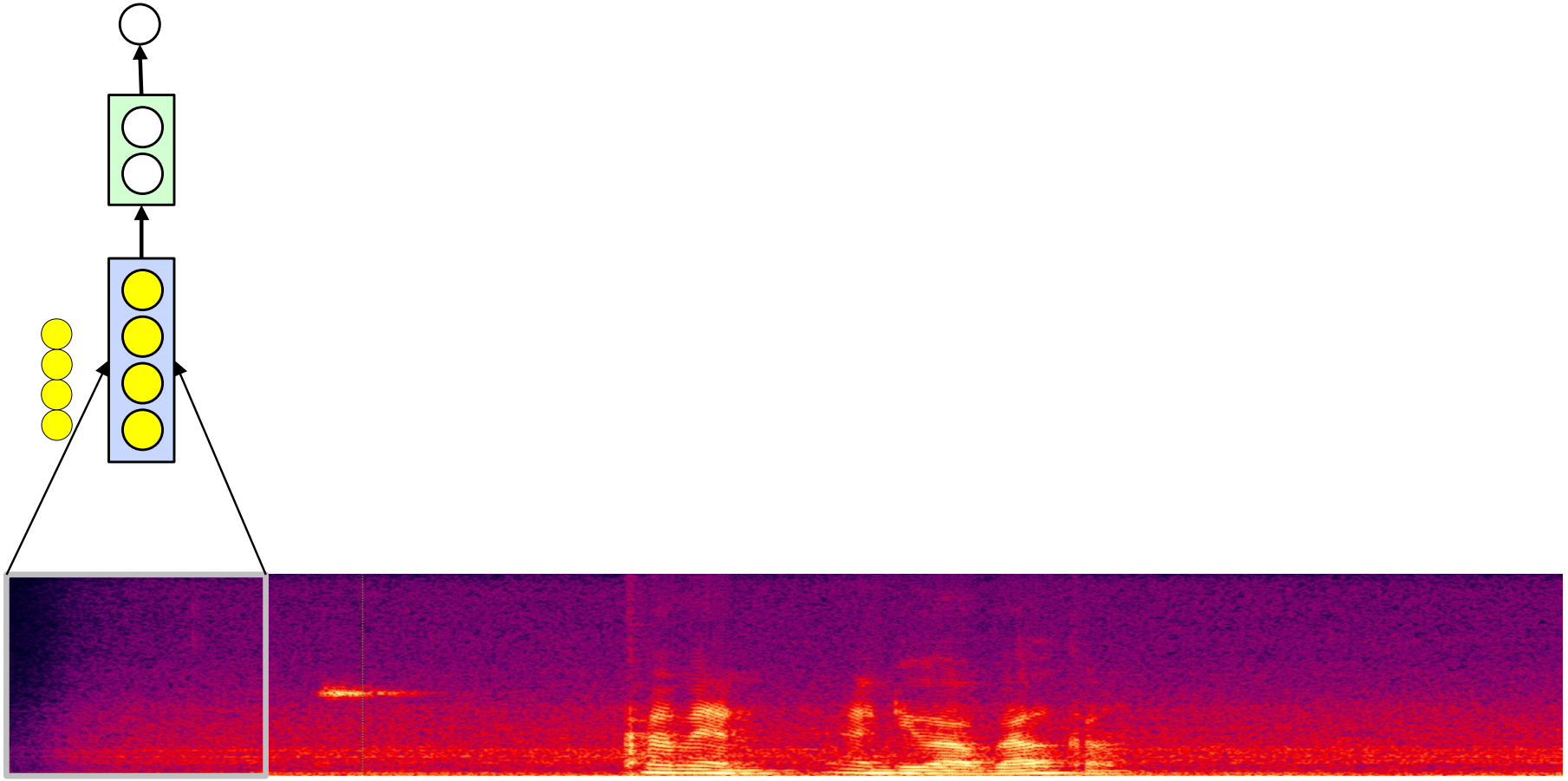
Scanning the input





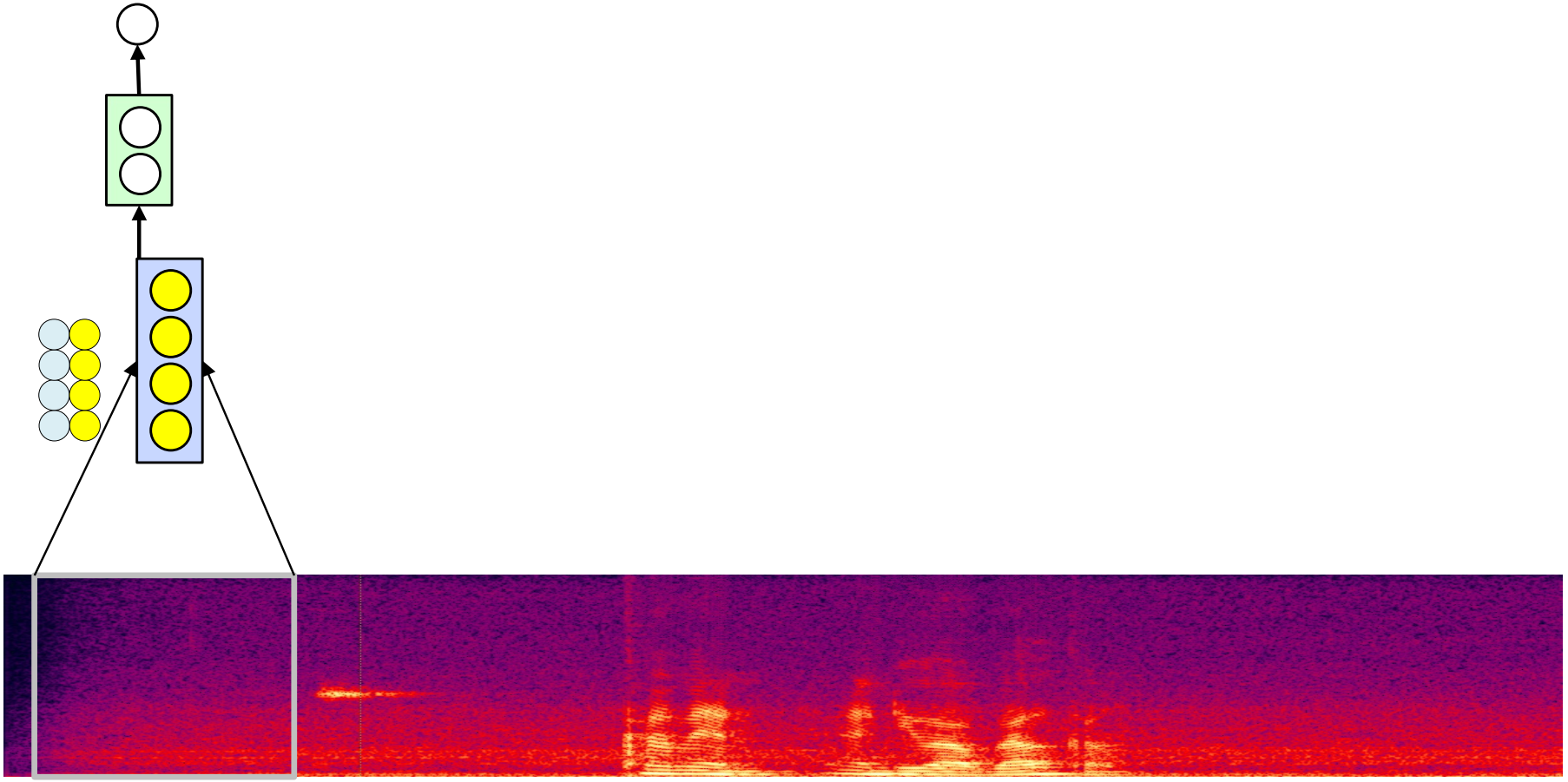
- We get a complete set of values (represented as a column) at each location evaluated by the MLP during the scan
  - Which we put through our final softmax to decide if the recording includes the word “Welcome”

Let's do it in a different order



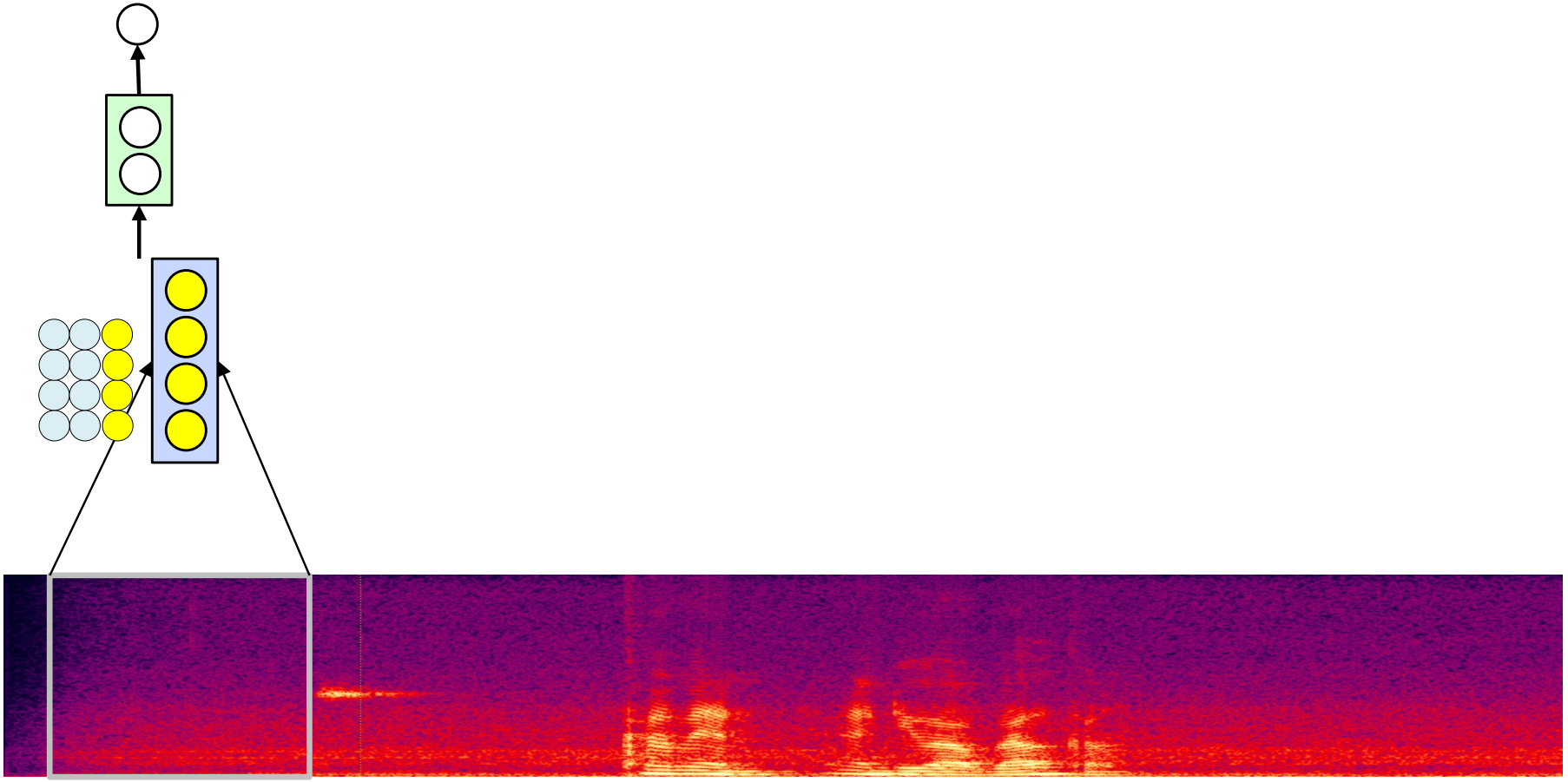
- Let us do the computation in a different order
- The first layer neurons evaluate each image first without waiting for the rest of the network
  - "Scan" the input

Let's do it in a different order



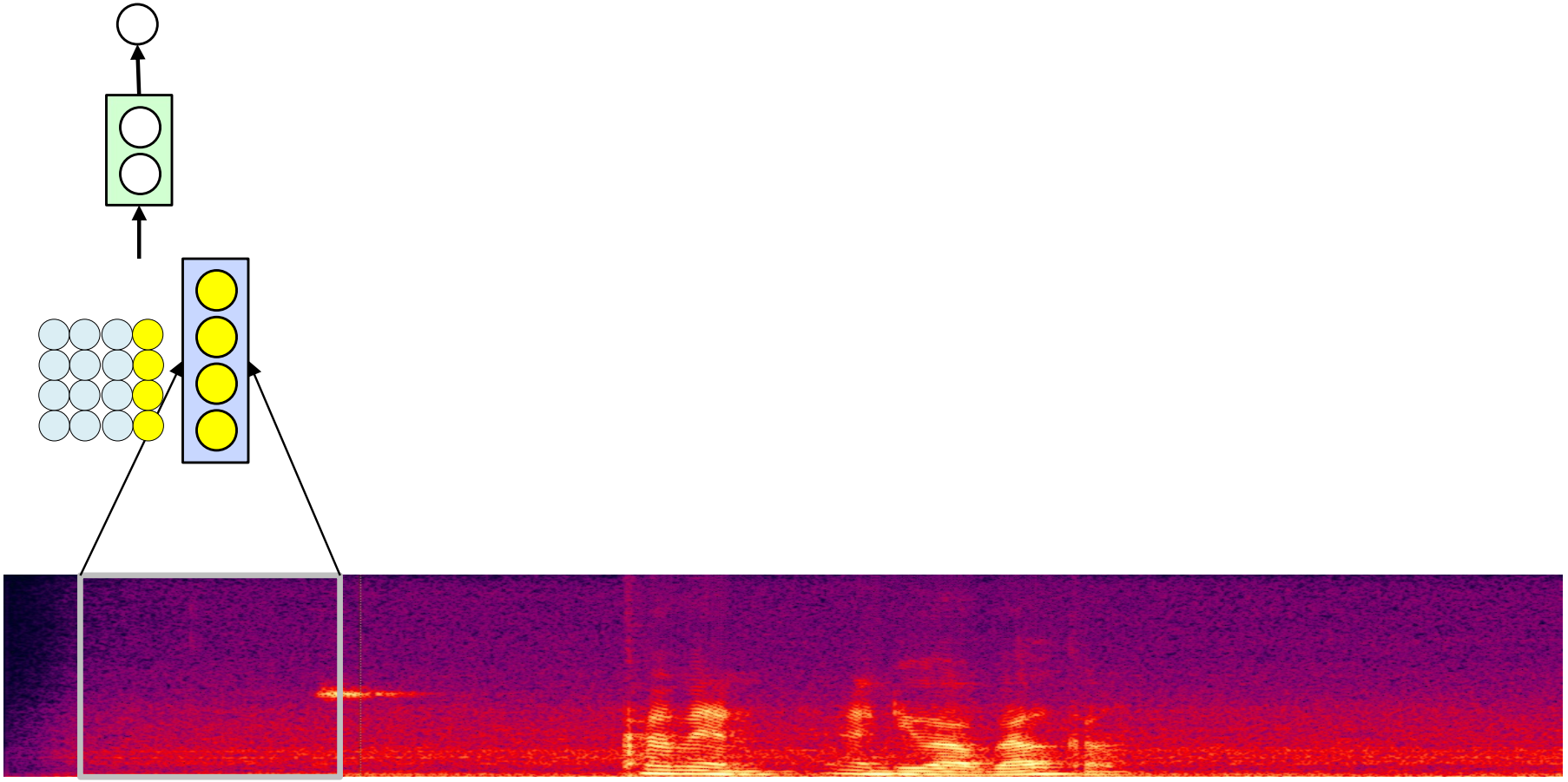
- Let us do the computation in a different order
- The first layer neurons evaluate each image first without waiting for the rest of the network
  - "Scan" the input

Let's do it in a different order



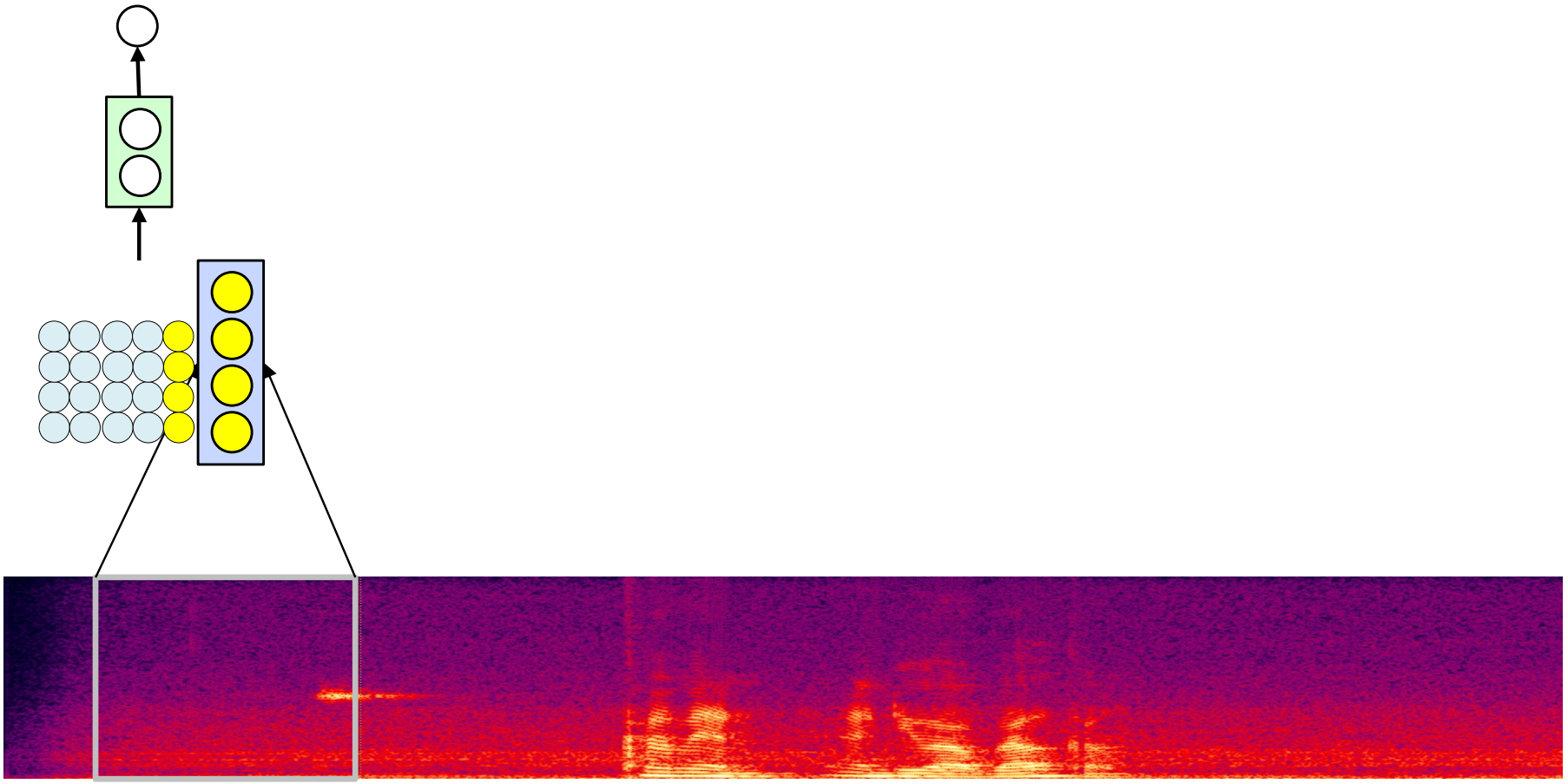
- Let us do the computation in a different order
- The first layer neurons evaluate each image first without waiting for the rest of the network
  - "Scan" the input

Let's do it in a different order



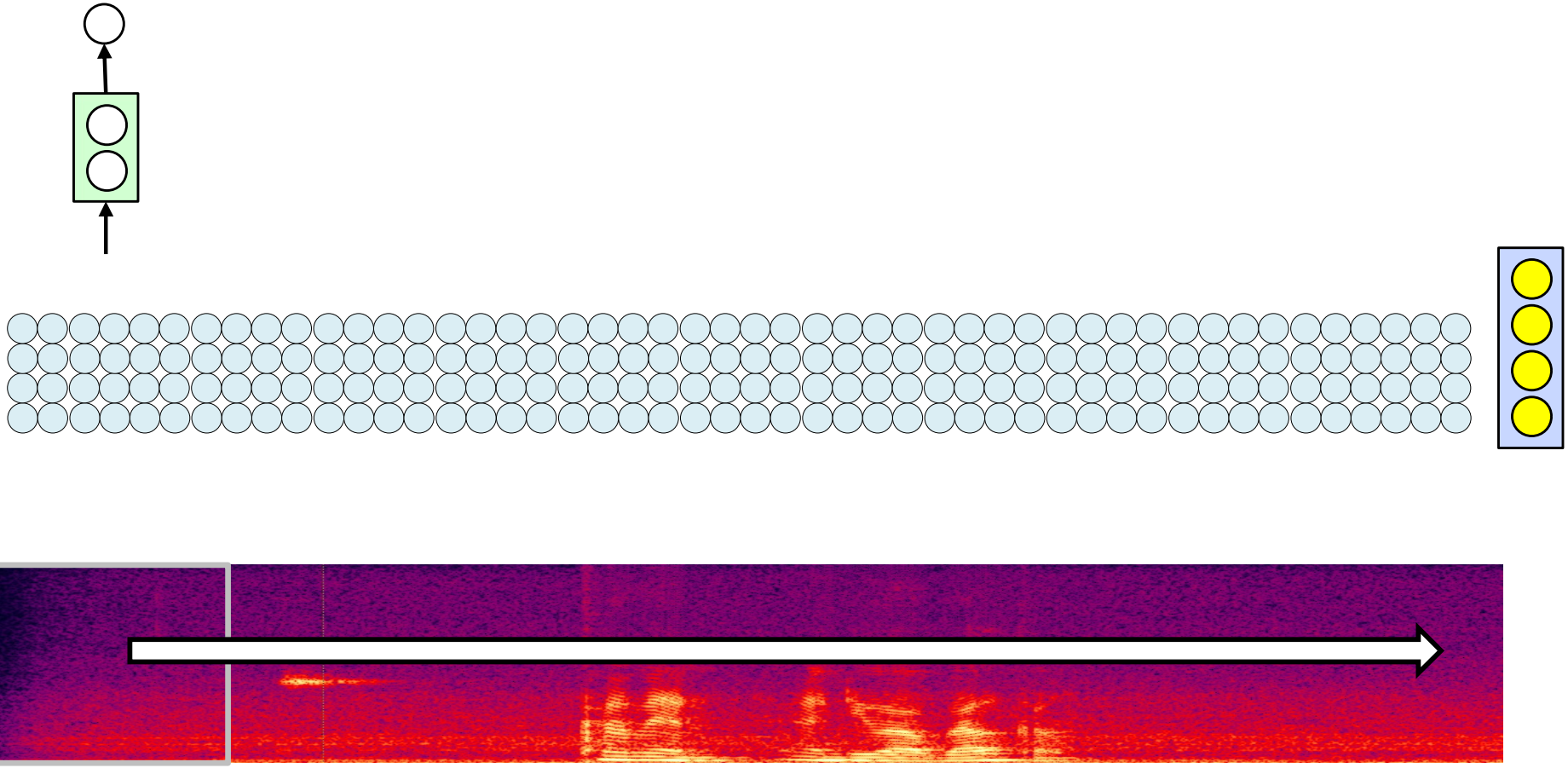
- Let us do the computation in a different order
- The first layer neurons evaluate each image first without waiting for the rest of the network
  - "Scan" the input

Let's do it in a different order



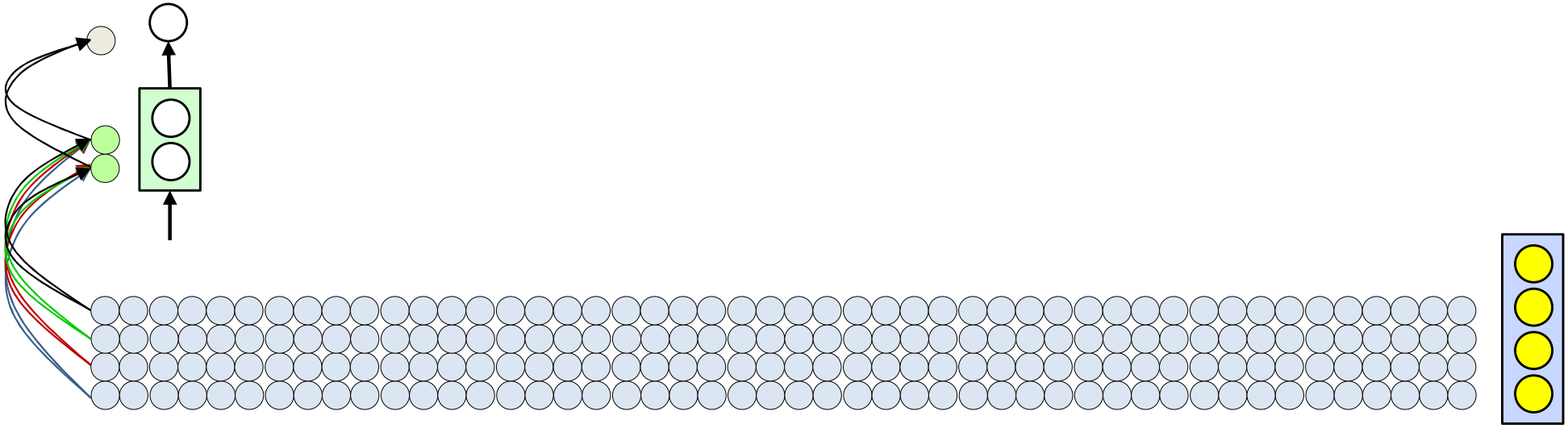
- Let us do the computation in a different order
- The first layer neurons evaluate each image first without waiting for the rest of the network
  - "Scan" the input

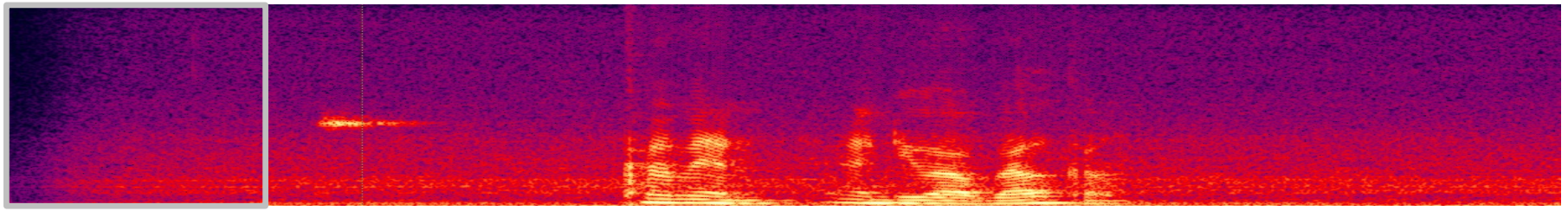
Let's do it in a different order



- Let us do the computation in a different order
- The first layer neurons evaluate each image first without waiting for the rest of the network
  - "Scan" the input

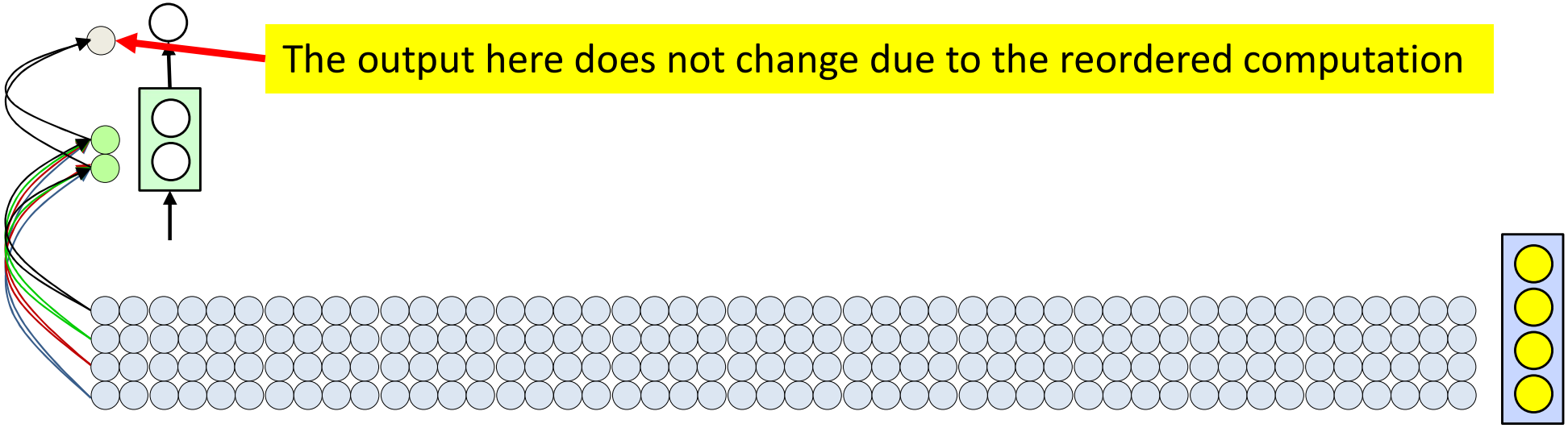
Let's do it in a different order

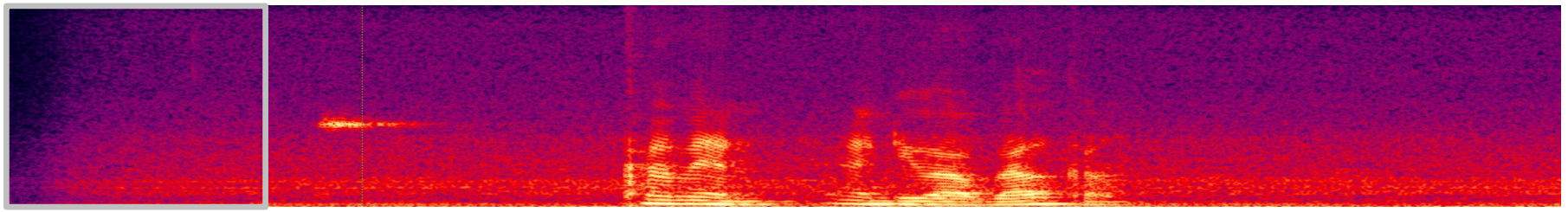




- Subsequently the rest of the layers operate on the first block
  - From the values computed by the impatient first layer
- Would the output of the MLP at the first block be different?

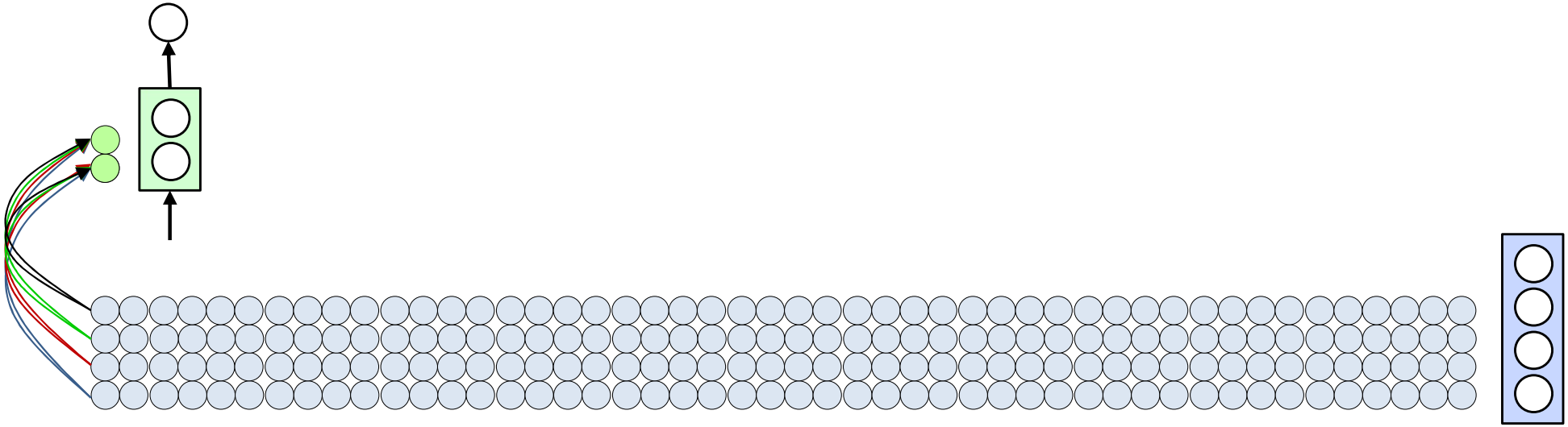
Let's do it in a different order

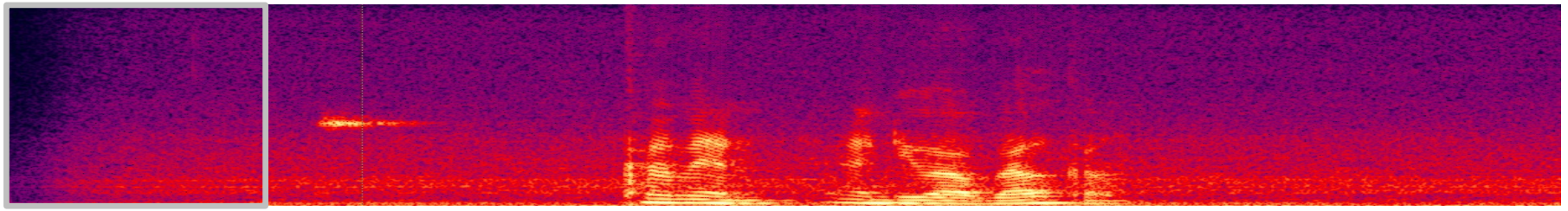




- Subsequently the rest of the layers operate on the first block
  - From the values computed by the impatient first layer
- Would the output of the MLP at the first block be different?

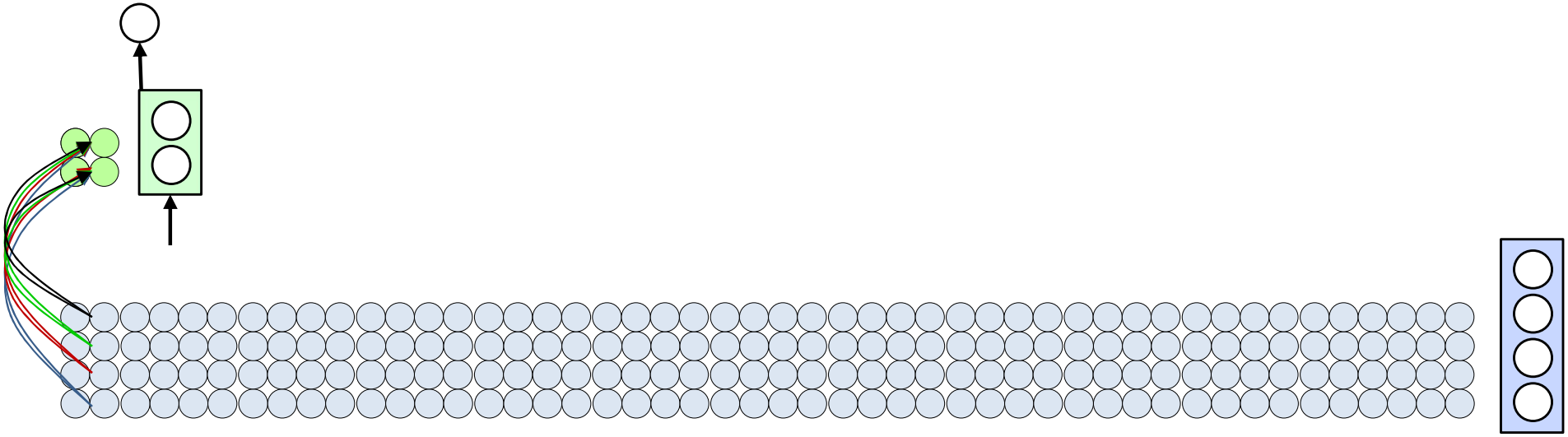
Let's do it in a different order

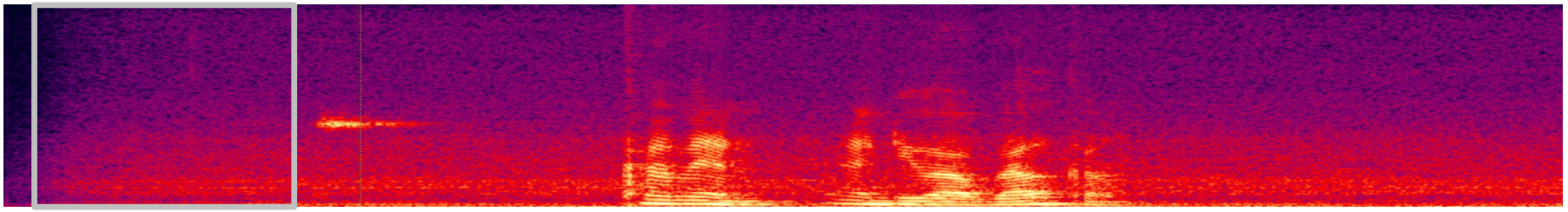




- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

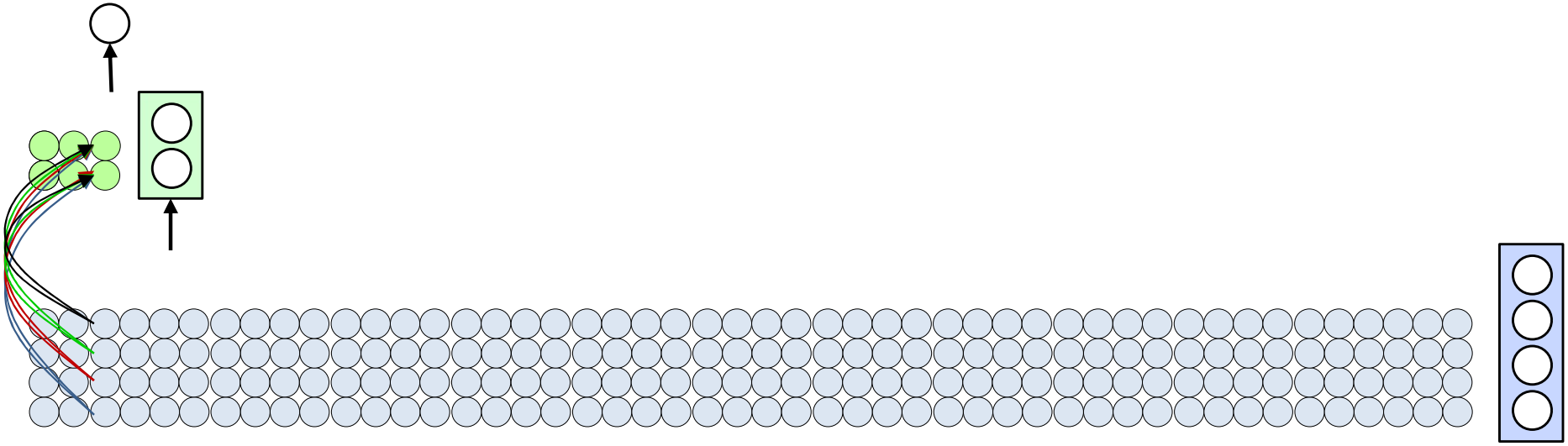
Let's do it in a different order

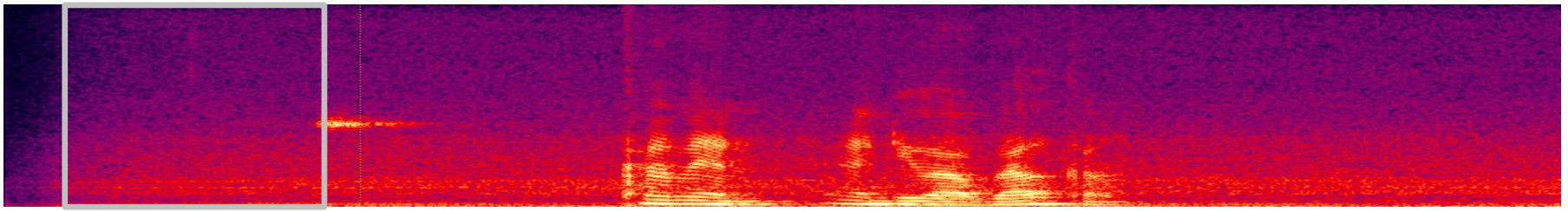




- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

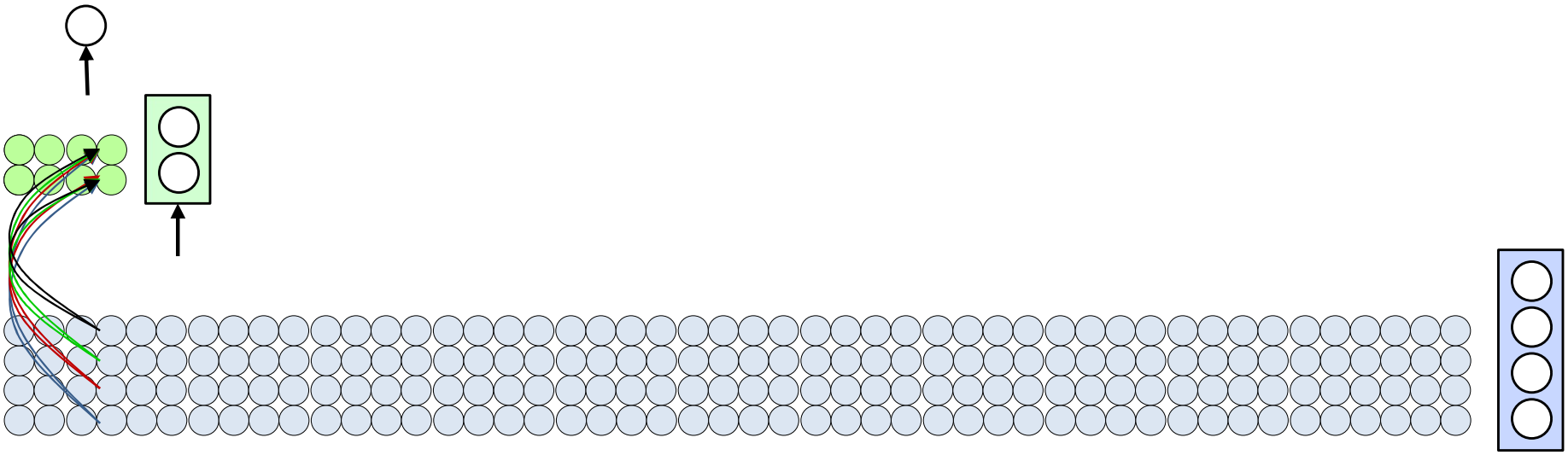
Let's do it in a different order

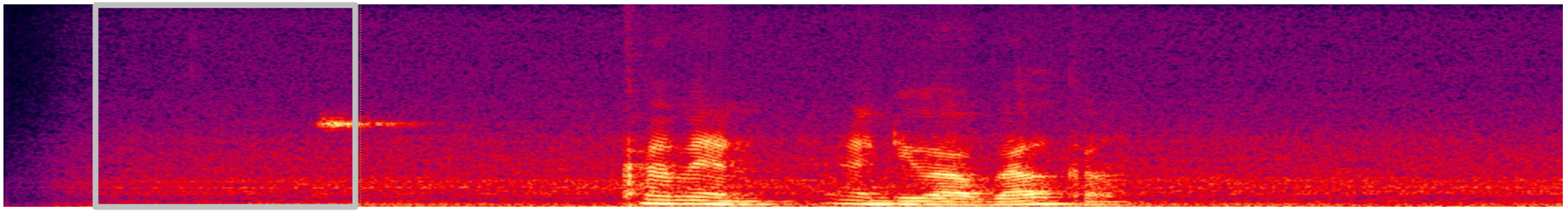




- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

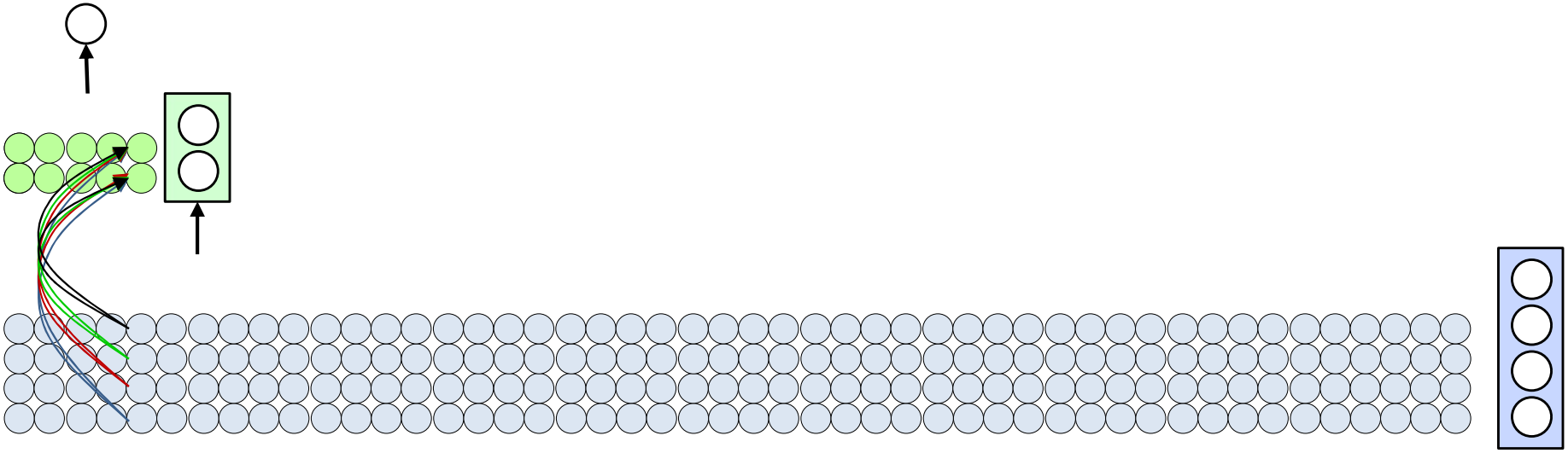
Let's do it in a different order

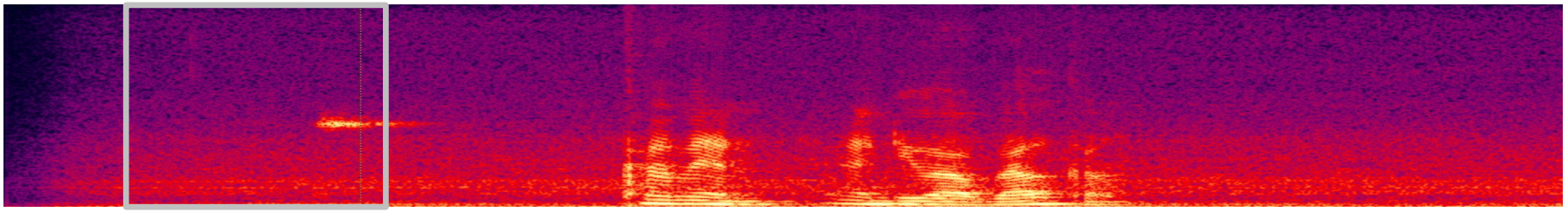




- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

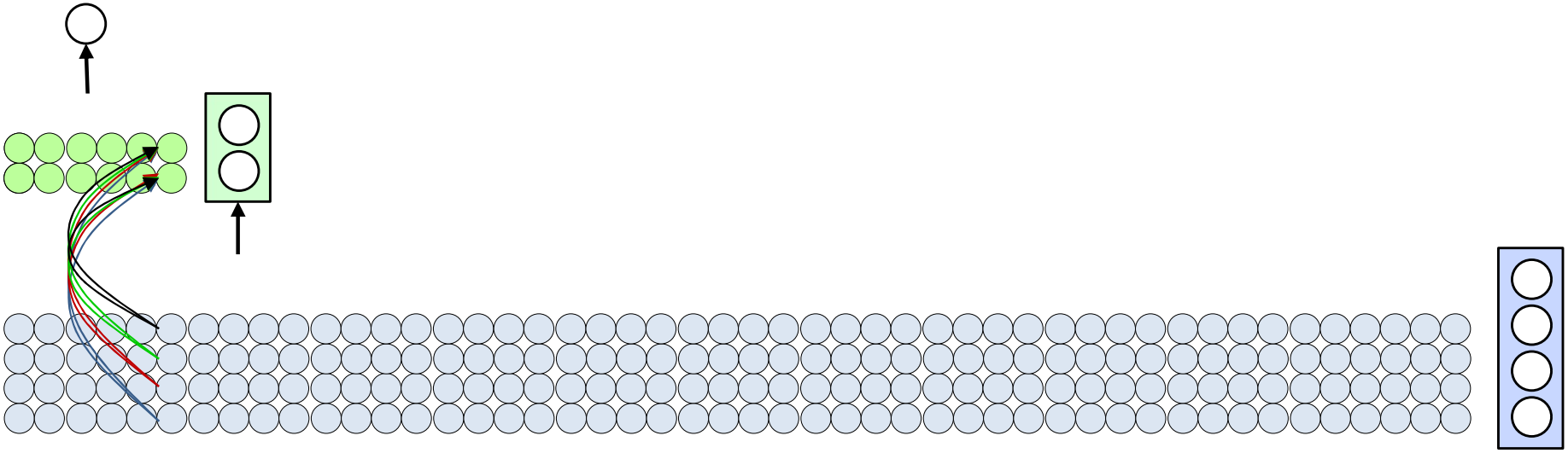
Let's do it in a different order

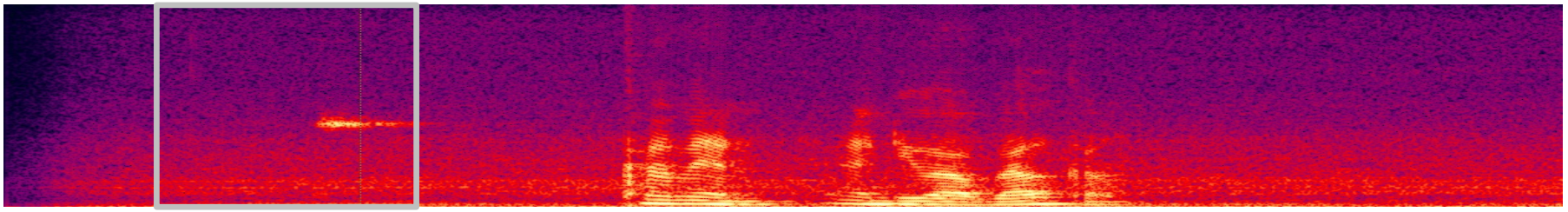




- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

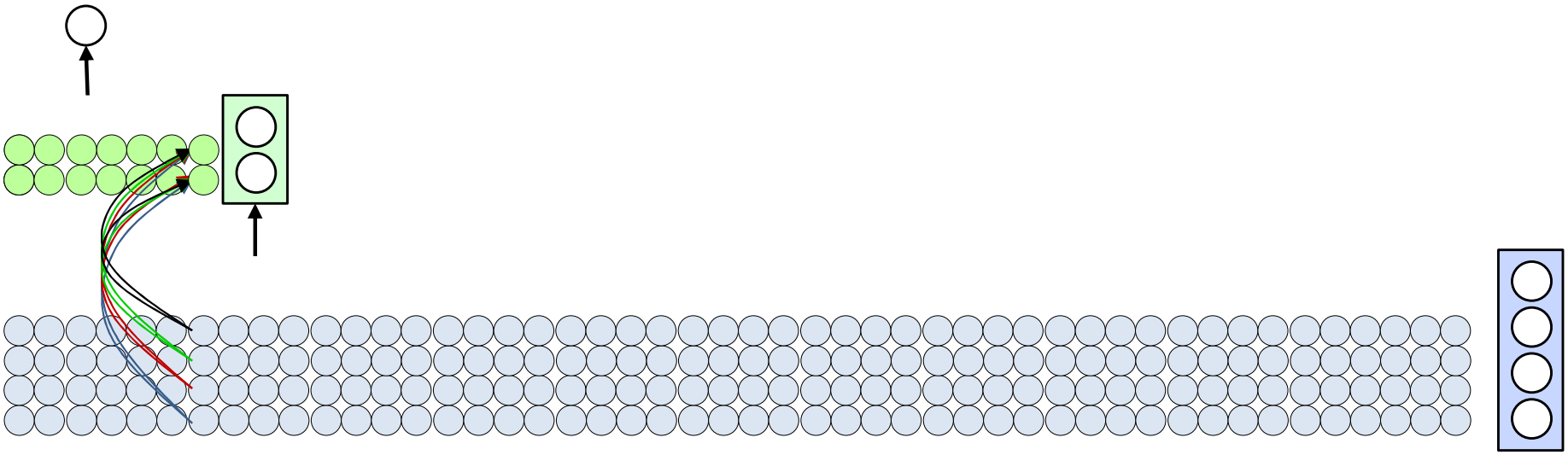
Let's do it in a different order

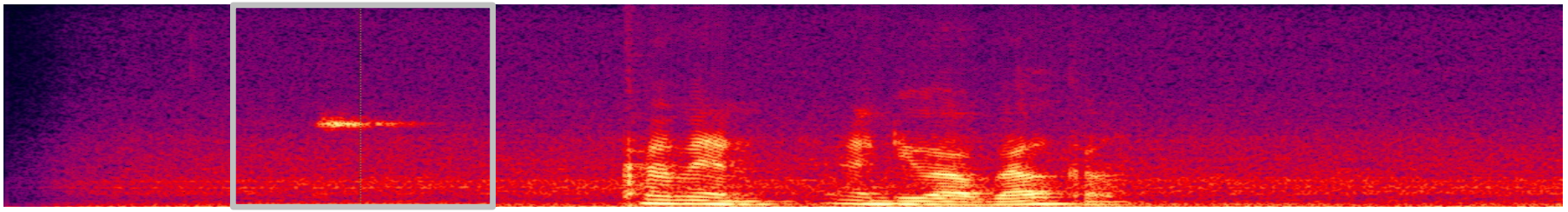




- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

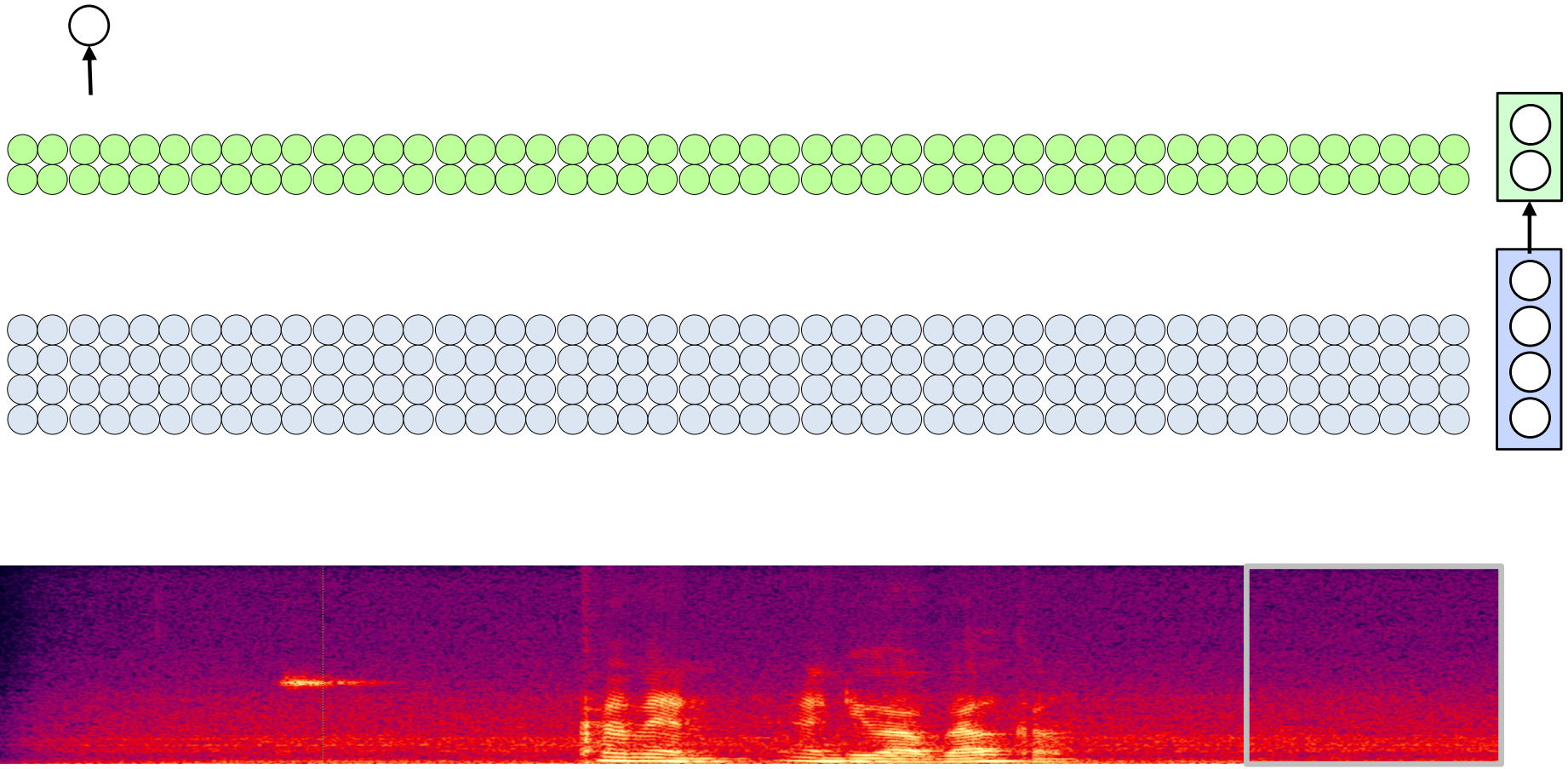
Let's do it in a different order





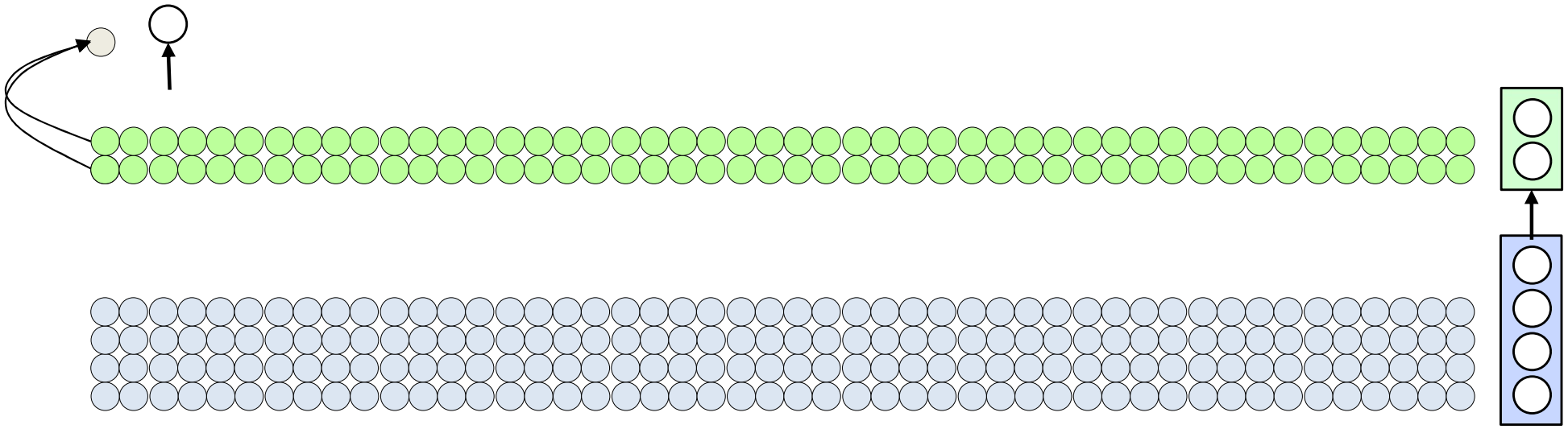
- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

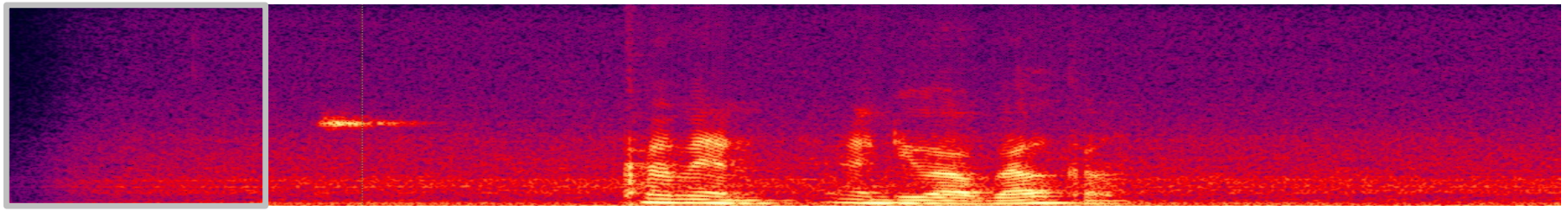
Let's do it in a different order



- But now, since the first layer neurons have already produced outputs for every location, the second layer neurons can go ahead and produce outputs for every position without waiting for the rest of the net
  - "Scan" the outputs of the first layer neurons

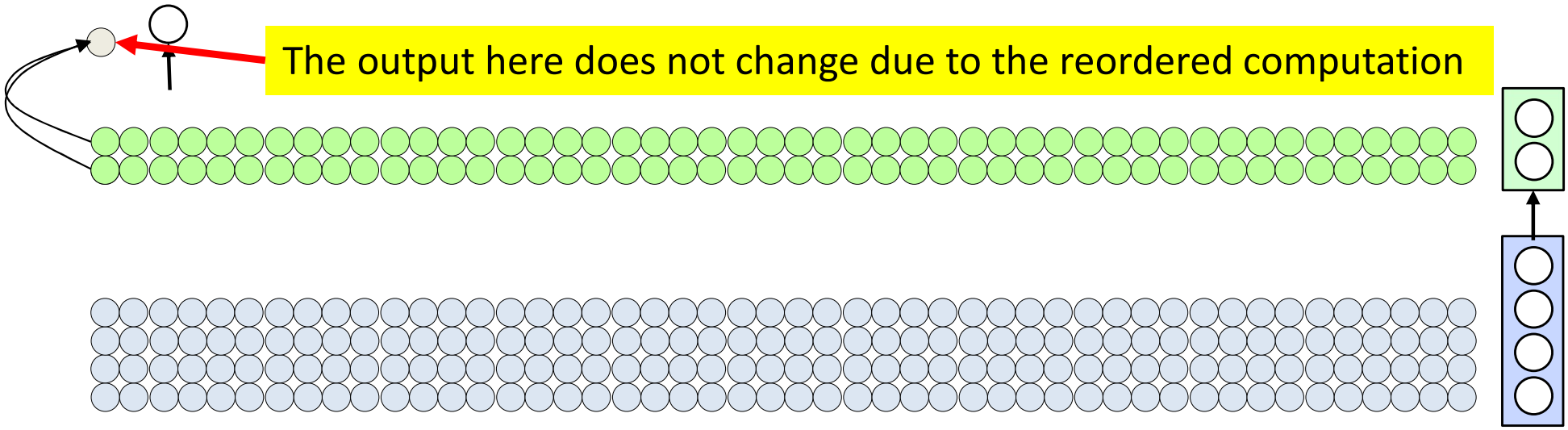
Let's do it in a different order

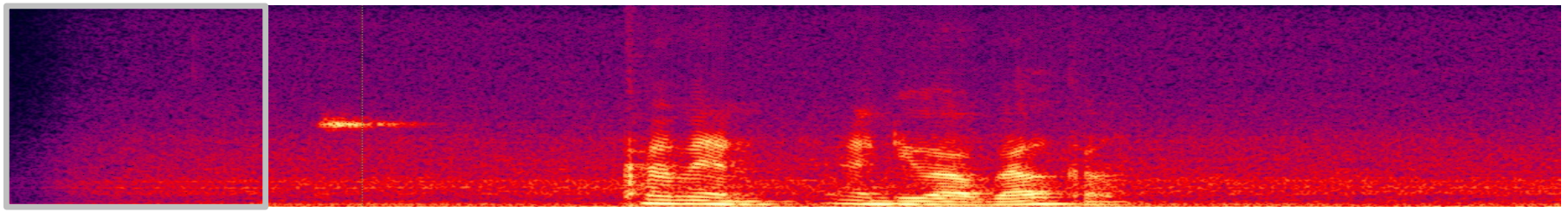




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

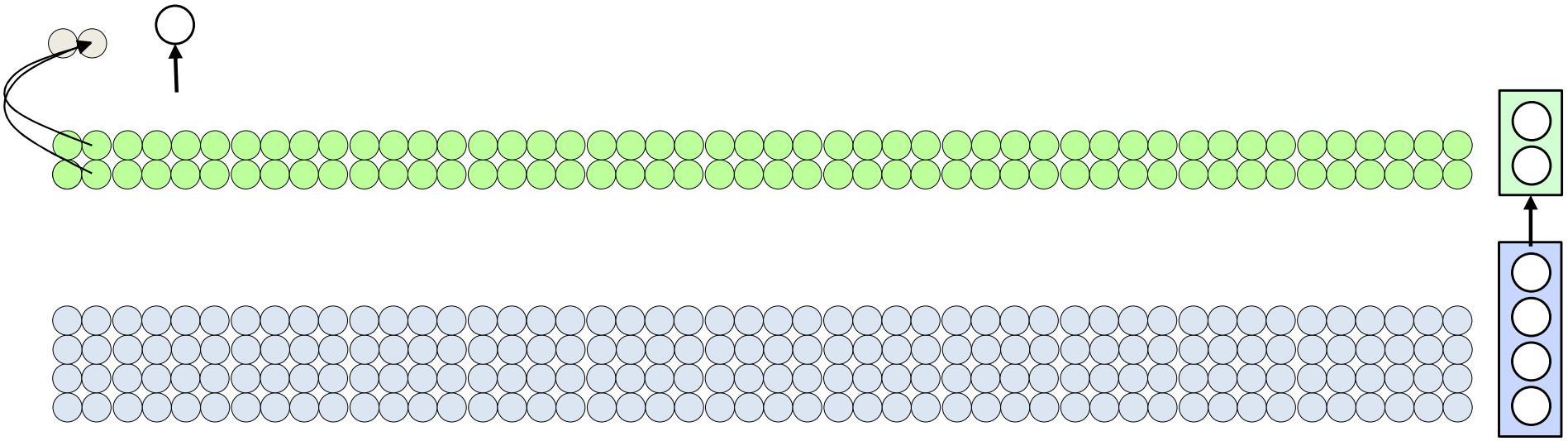
Let's do it in a different order

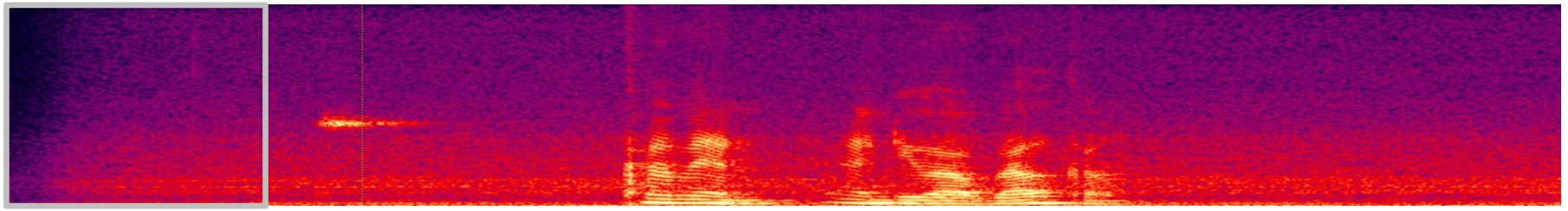




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

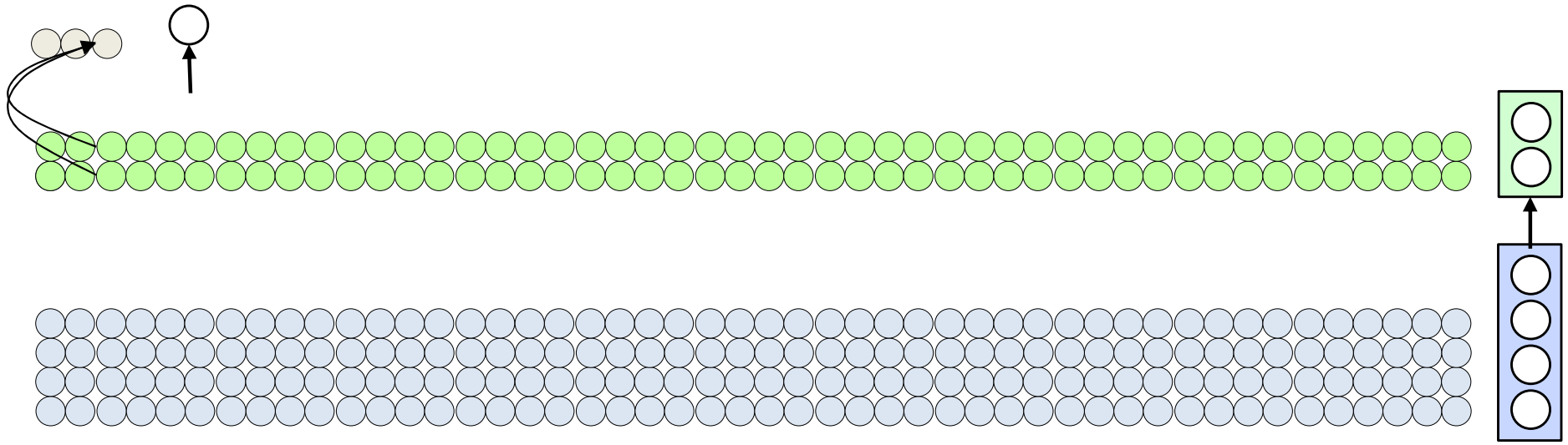
Let's do it in a different order

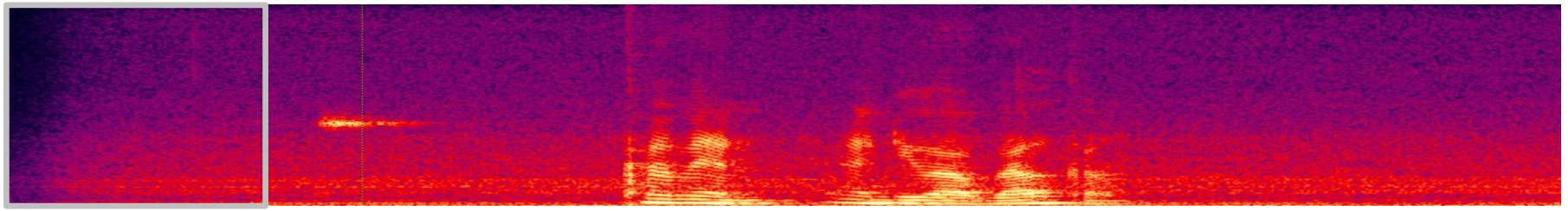




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

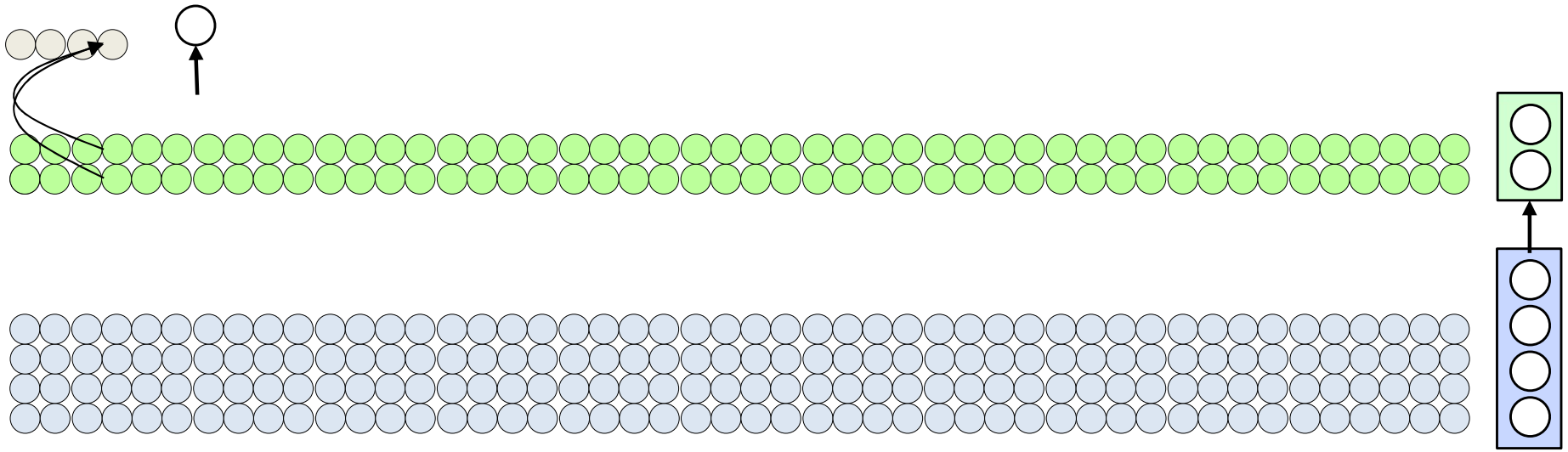
Let's do it in a different order

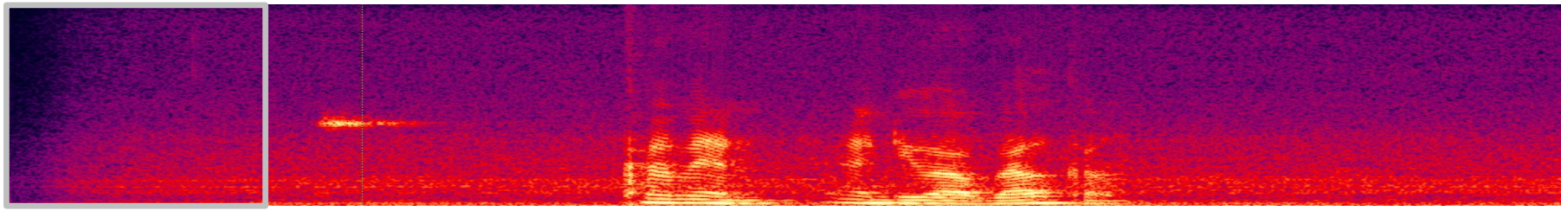




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

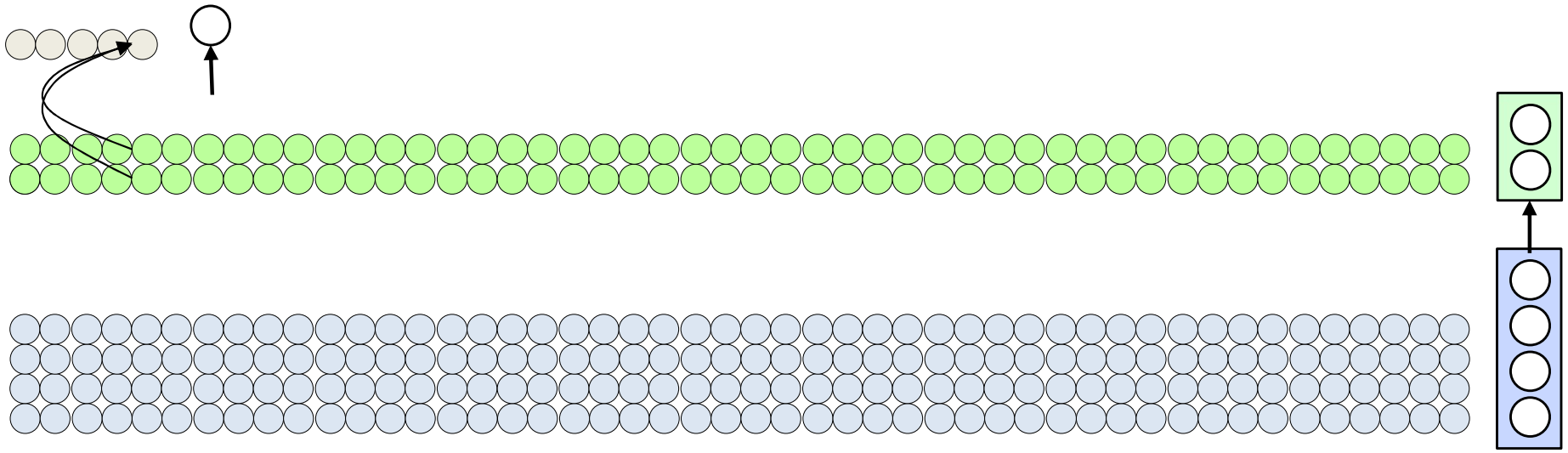
Let's do it in a different order

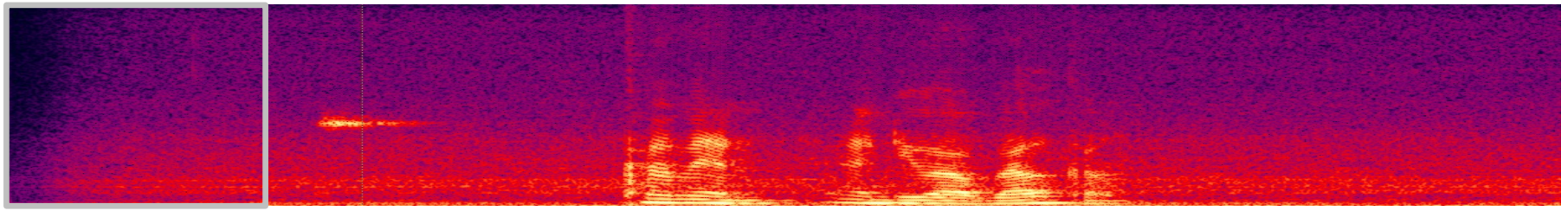




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

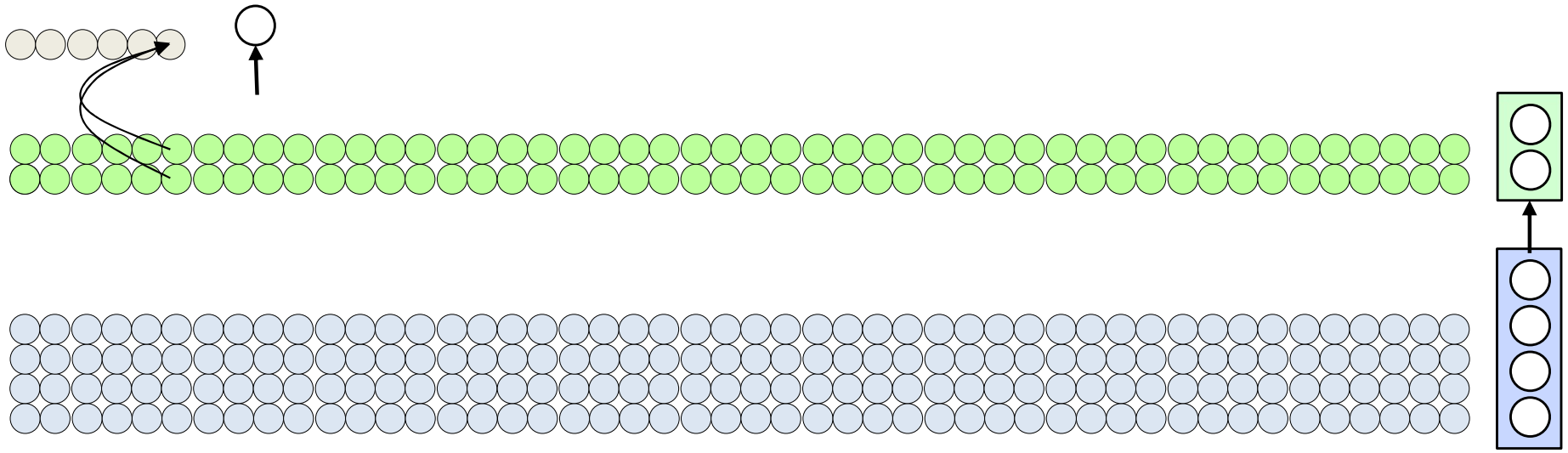
Let's do it in a different order

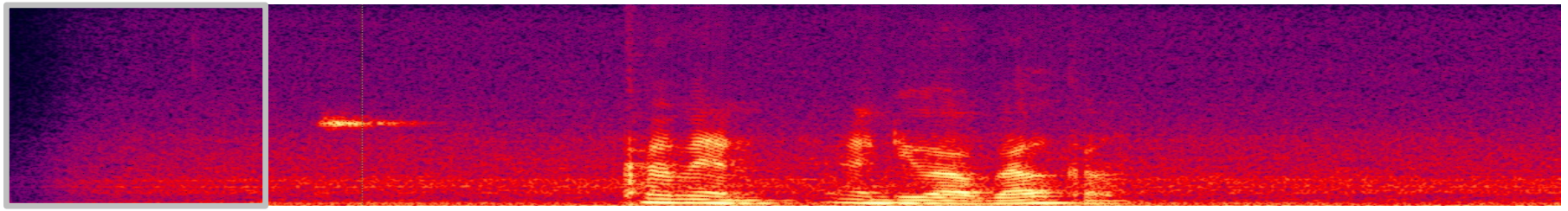




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

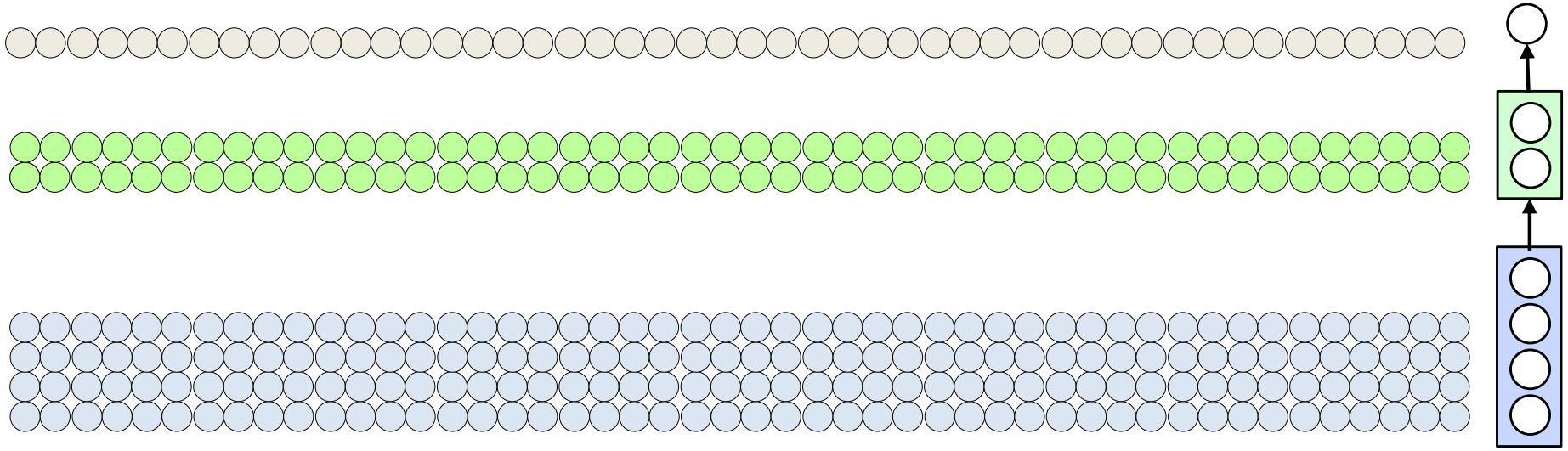
Let's do it in a different order

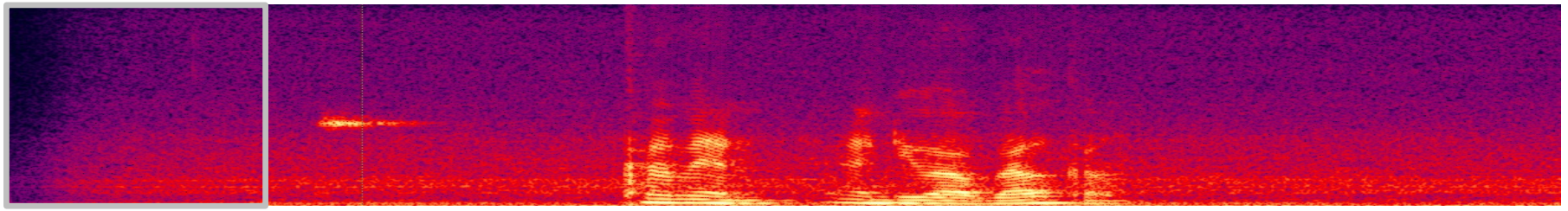




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

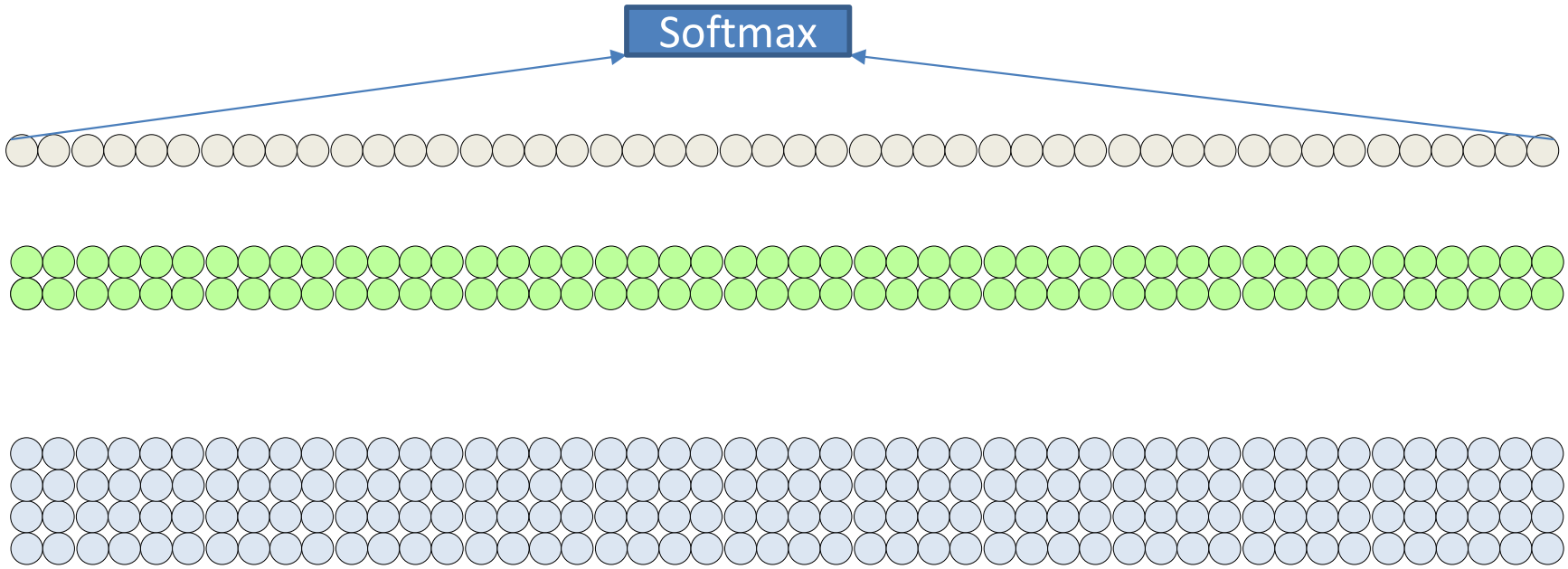
Let's do it in a different order

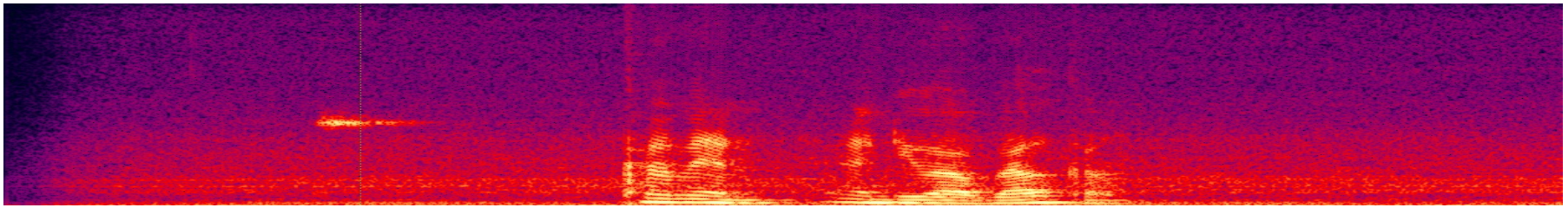




- At each position the output layer neurons can now operate on the outputs of the penultimate layer and produce the correct classification for the corresponding block!
  - Scan the outputs of the second layer neurons

Let's do it in a different order





- The final softmax will give us the correct answer for the entire input

Scanning with an MLP

- K = width of “patch” evaluated by MLP

```
For t = 1:T-K+1
   XSegment = x(:, t:t+K-1)
   y(t) = MLP(XSegment)

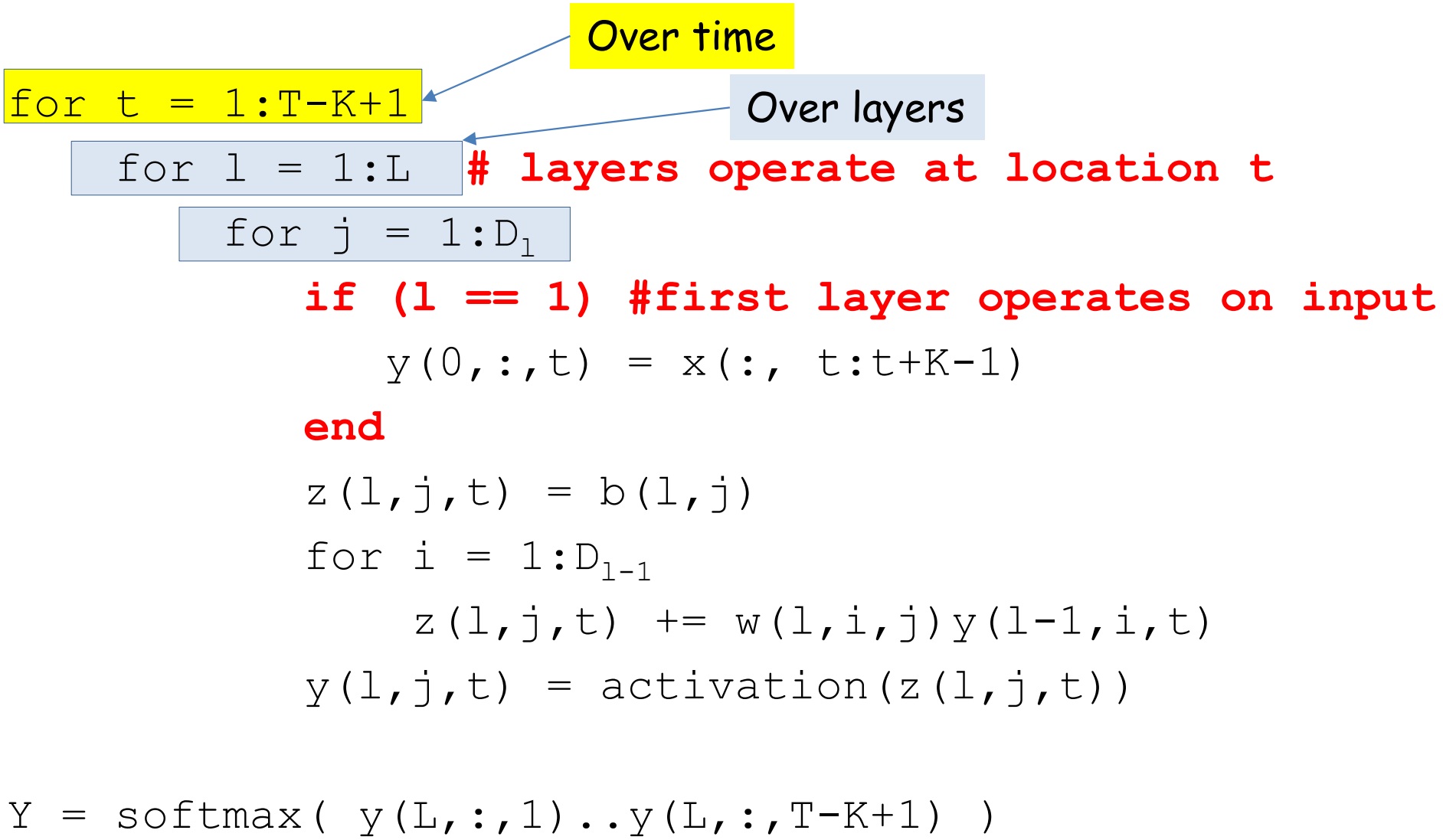
Y = softmax(y(1)..y(T-K+1))
```

Scanning with an MLP

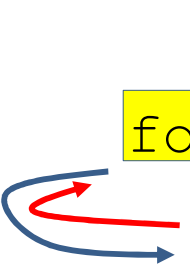
```
for t = 1:T-K+1
    for l = 1:L  # layers operate at location t
        for j = 1:D_l
            if (l == 1) #first layer operates on input
                y(0,:,t) = x(:, t:t+K-1)
            end
            z(l,j,t) = b(l,j)  # bias
            for i = 1:D_{l-1}
                z(l,j,t) += w(l,i,j)y(l-1,i,t)
            y(l,j,t) = activation(z(l,j,t))
```

```
Y = softmax( y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP



Scanning with an MLP



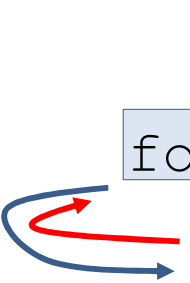
Over time

Over layers

```
for t = 1:T-K+1
   for l = 1:L   # layers operate at location t
      for j = 1:D_l
         if (l == 1) #first layer operates on input
            y(0,:,t) = x(:, t:t+K-1)
         end
         z(l,j,t) = b(l,j)
         for i = 1:D_{l-1}
            z(l,j,t) += w(l,i,j)y(l-1,i,t)
         y(l,j,t) = activation(z(l,j,t))

Y = softmax(y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP



Over layers

Over time

```
for l = 1:L    # layers operate at location t
    for t = 1:T-K+1
        for j = 1:D_l
            if (l == 1) #first layer operates on input
                y(0,:,t) = x(:, t:t+K-1)
            end
            z(l,j,t) = b(l,j)
            for i = 1:D_l-1
                z(l,j,t) += w(l,i,j)y(l-1,i,t)
            y(l,j,t) = activation(z(l,j,t))

Y = softmax(y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP

```
for l = 1:L    # layers operate at location t
    for t = 1:T-K+1
        for j = 1:D_l
            if (l == 1) #first layer operates on input
                y(0,:,t) = x(:, t:t+K-1)
            end
            z(l,j,t) = b(l,j)
            for i = 1:D_l-1
                z(l,j,t) += w(l,i,j)y(l-1,i,t)
            y(l,j,t) = activation(z(l,j,t))

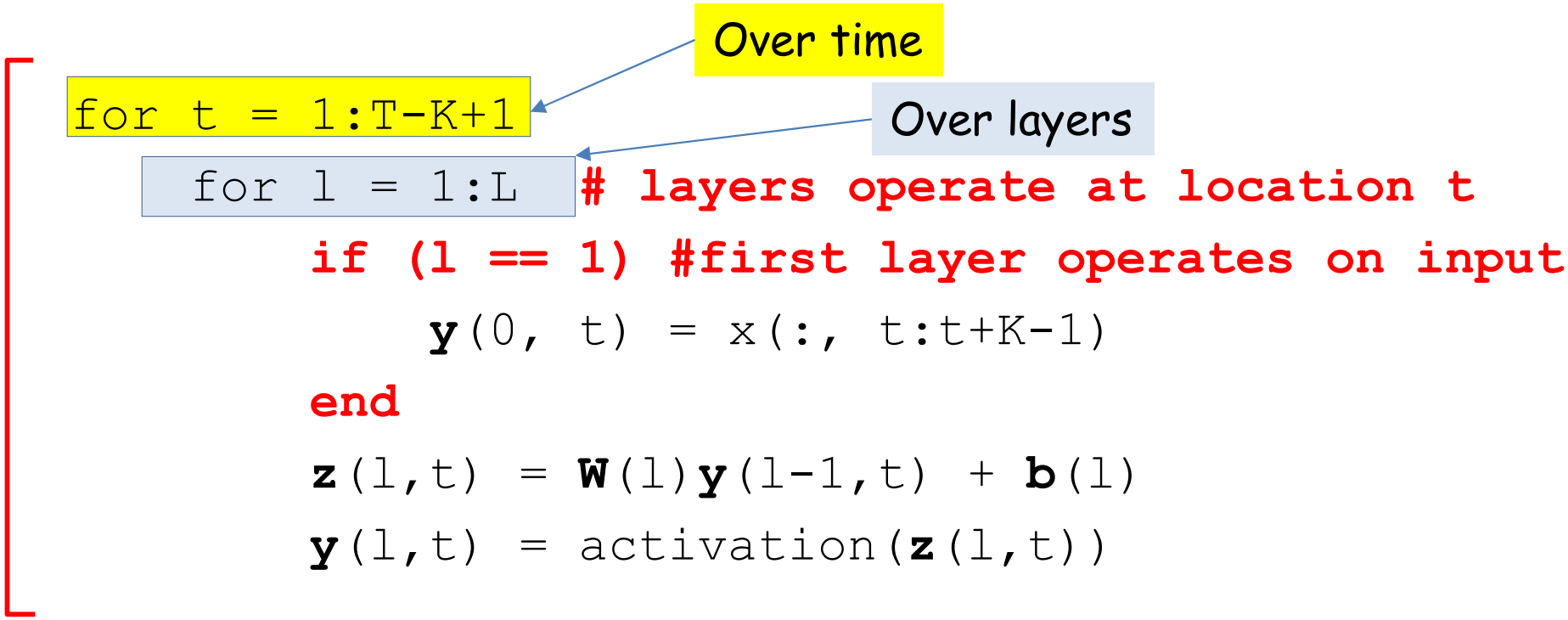
Y = softmax(y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP

```
for t = 1:T-K+1
    for l = 1:L   # layers operate at location t
        if (l == 1) #first layer operates on input
            y(0, t) = x(:, t:t+K-1)
        end
        z(l,t) = W(l)y(l-1,t) + b(l)
        y(l,t) = activation(z(l,t))

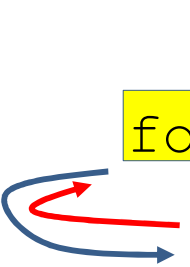
Y = softmax( y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP




```
Y = softmax( y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP



Over time

Over layers

```
for t = 1:T-K+1
    for l = 1:L   # layers operate at location t
        if (l == 1) #first layer operates on input
            y(0, t) = x(:, t:t+K-1)
        end
        z(l,t) = W(l)y(l-1,t) + b(l)
        y(l,t) = activation(z(l,t))

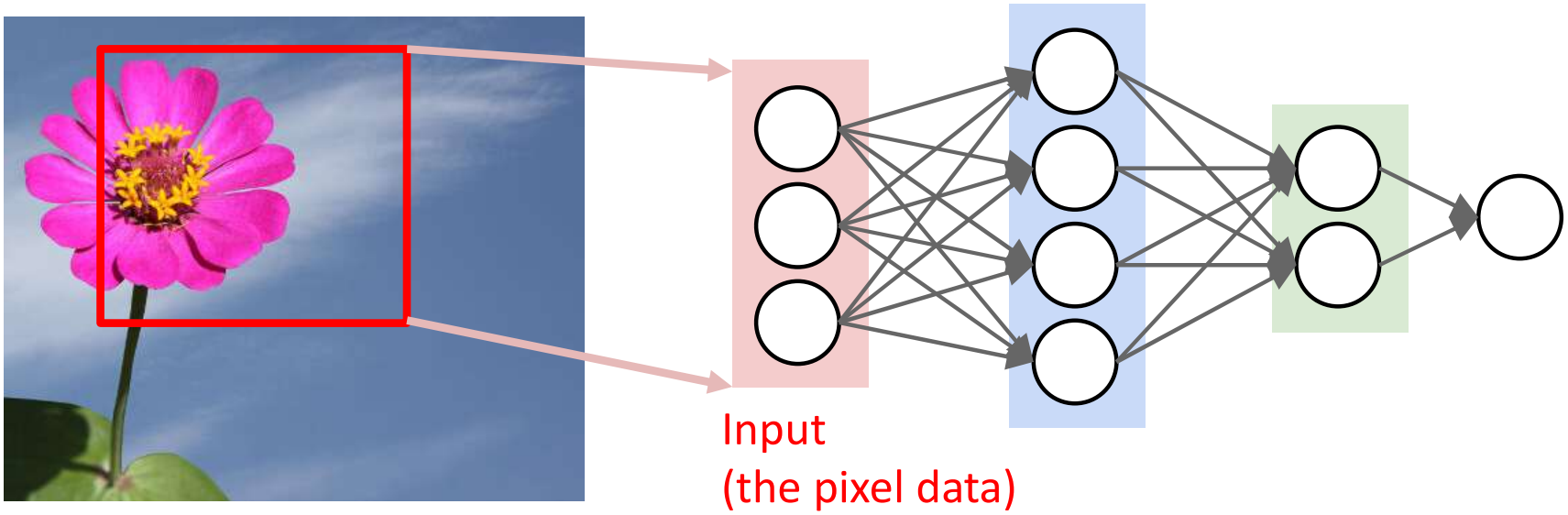
Y = softmax( y(L,:,1)..y(L,:,T-K+1) )
```

Scanning with an MLP

```
for l = 1:L   # layers operate at location t
    for t = 1:T-K+1
        if (l == 1) #first layer operates on input
            y(0, t) = x(:, t:t+K-1)
        end
        z(l,t) = W(l)y(l-1,t) + b(l)
        y(l,t) = activation(z(l,t))

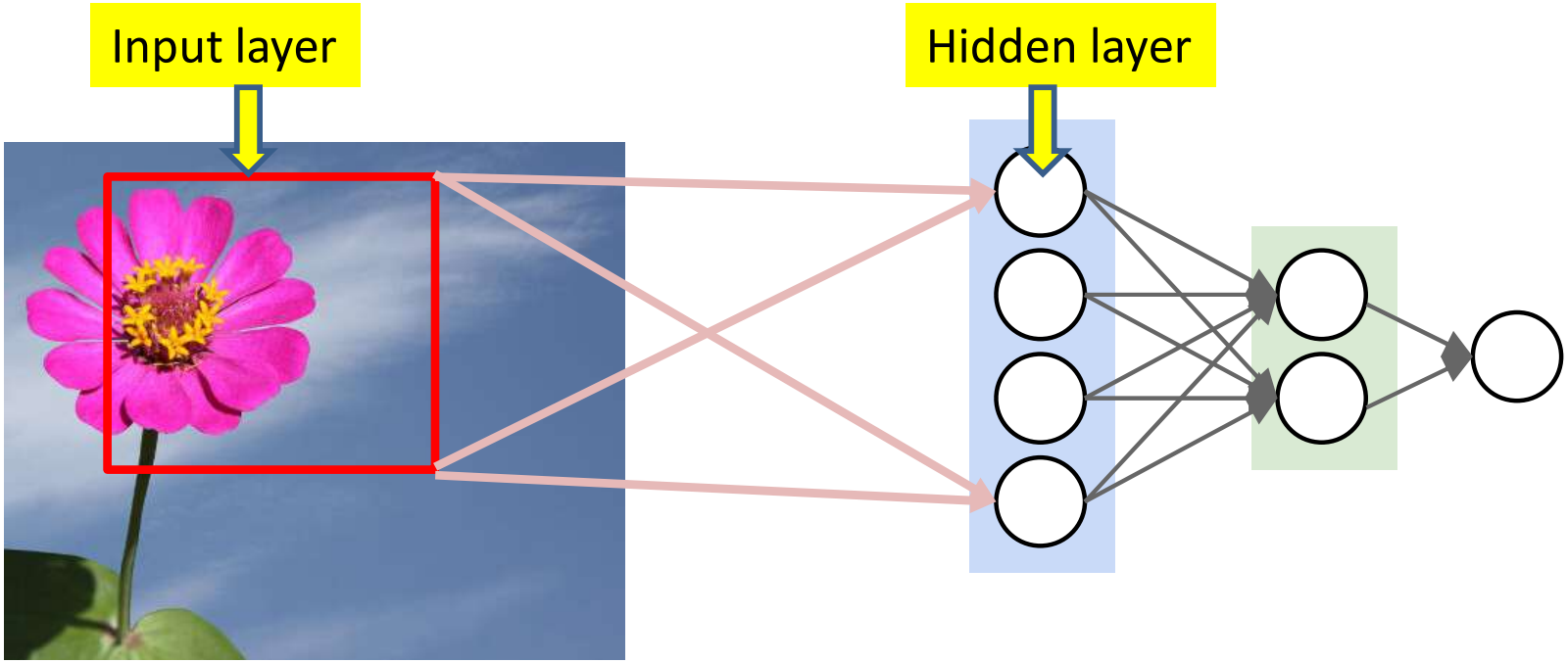
Y = softmax( y(L,:,1)..y(L,:,T-K+1) )
```

Scanning in 2D: A closer look



- *Scan* for the desired object
- At each location, the entire region is sent through an MLP

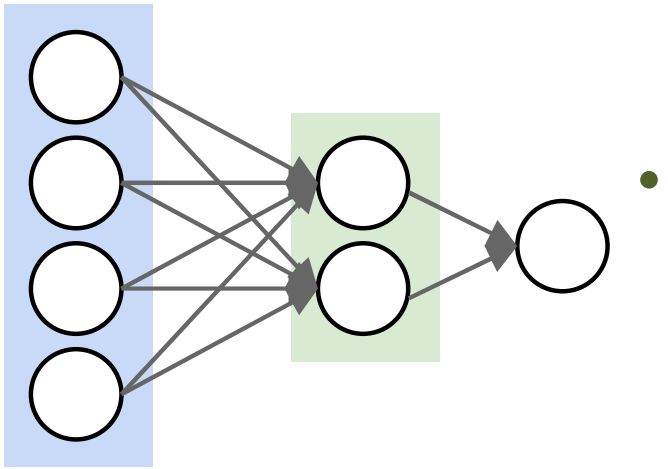
Scanning: A closer look



- The “input layer” is just the pixels in the image connecting to the hidden layer

Scanning: A closer look

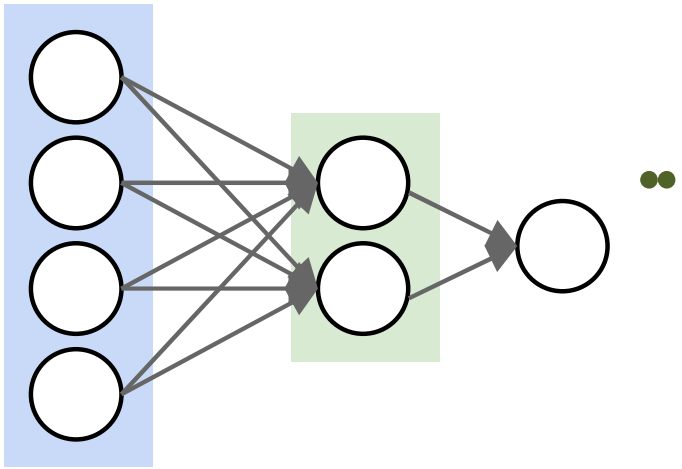




- Scanning: Analyze windows of pixels starting from top left, until the bottom right of the image
  - Produce an output for every window analyzed
  - Pass collection of outputs through a softmax

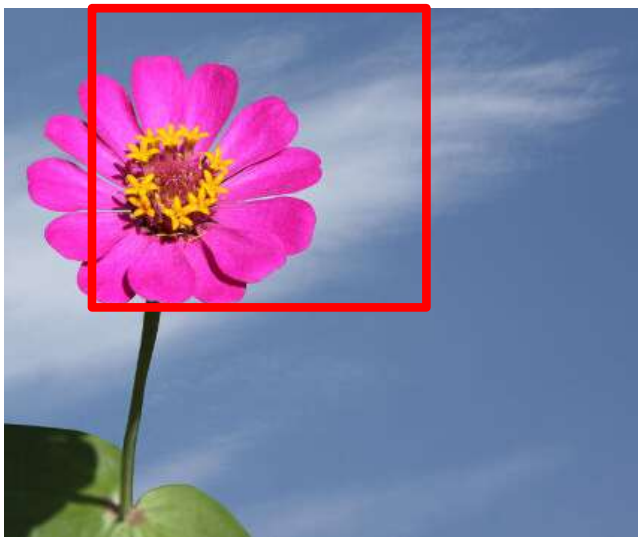
Scanning: A closer look

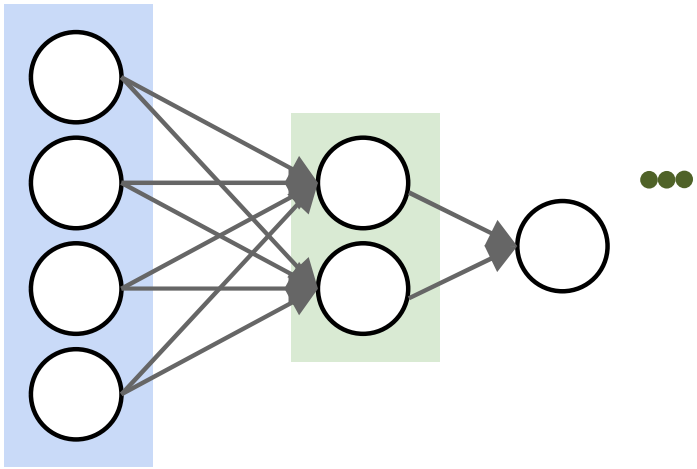




- Scanning: Analyze windows of pixels starting from top left, until the bottom right of the image
  - Produce an output for every window analyzed
  - Pass collection of outputs through a softmax

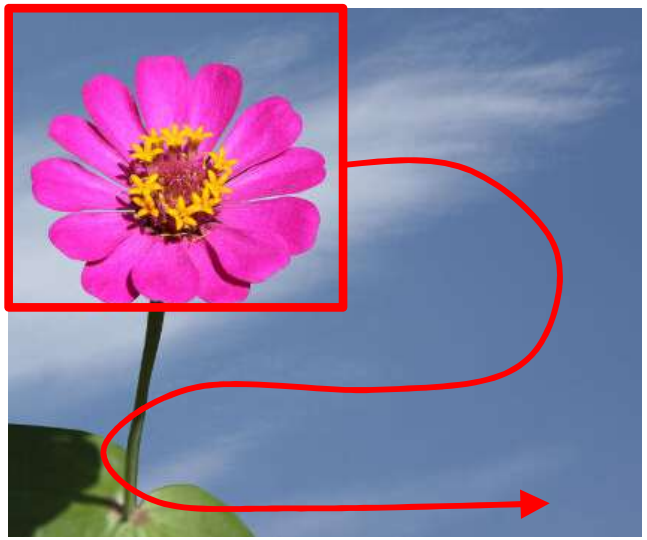
Scanning: A closer look

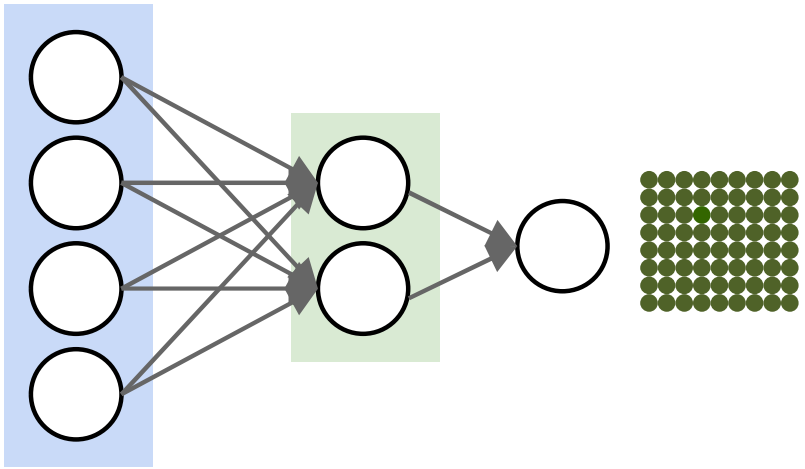




- Scanning: Analyze windows of pixels starting from top left, until the bottom right of the image
  - Produce an output for every window analyzed
  - Pass collection of outputs through a softmax

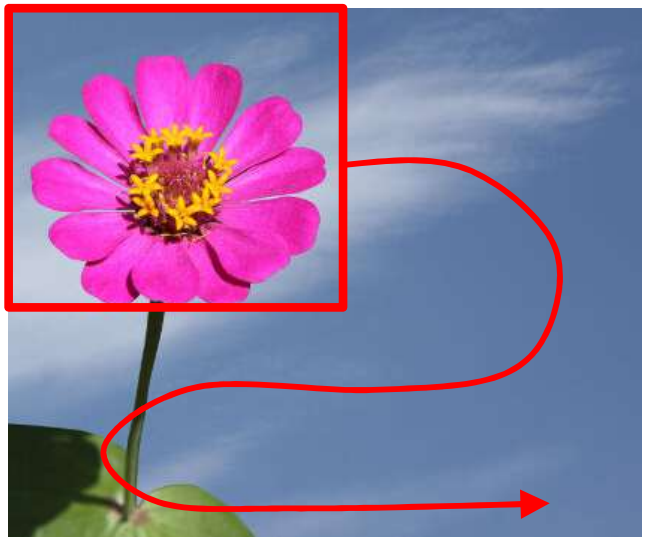
Scanning: A closer look

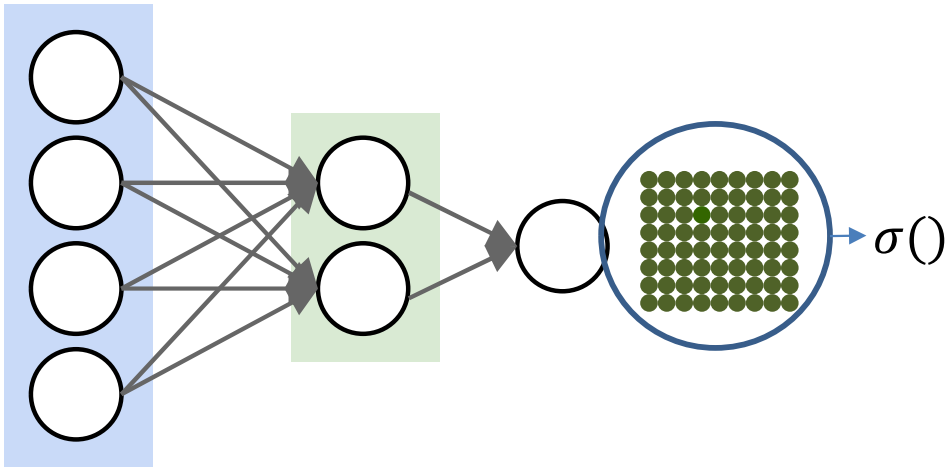




- Scanning: Analyze windows of pixels starting from top left, until the bottom right of the image
  - Produce an output for every window analyzed
  - Pass collection of outputs through a softmax

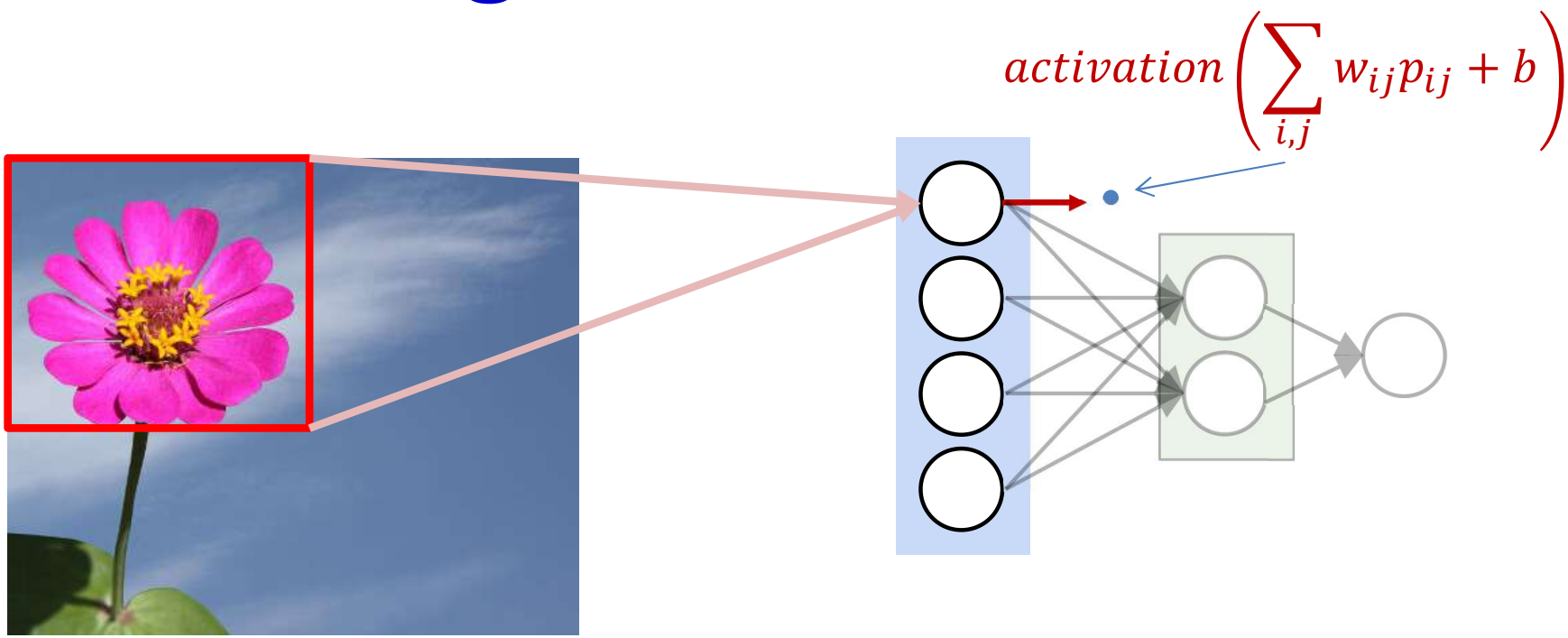
Scanning: A closer look





- Scanning: Analyze windows of pixels starting from top left, until the bottom right of the image
  - Produce an output for every window analyzed
  - Pass collection of outputs through a softmax

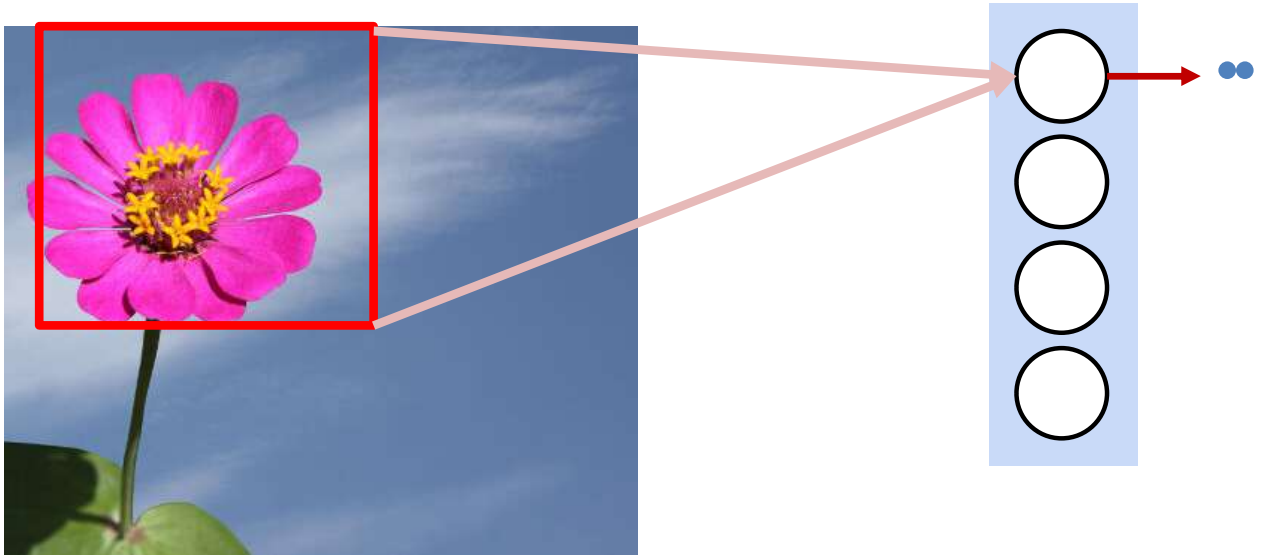
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a "window" of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - "Scanning" the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

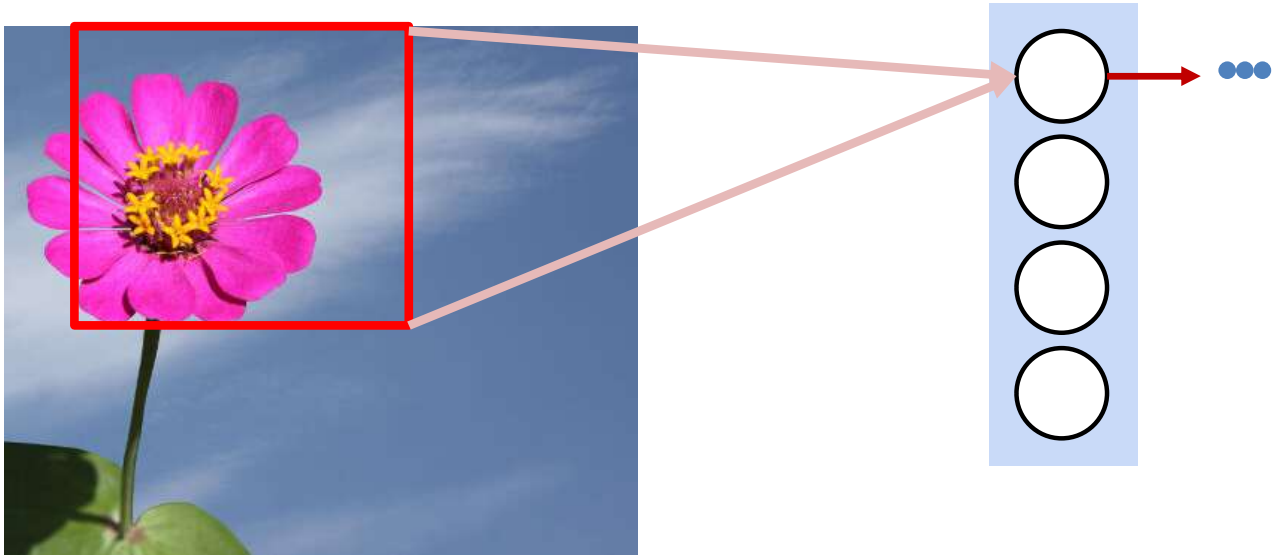
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

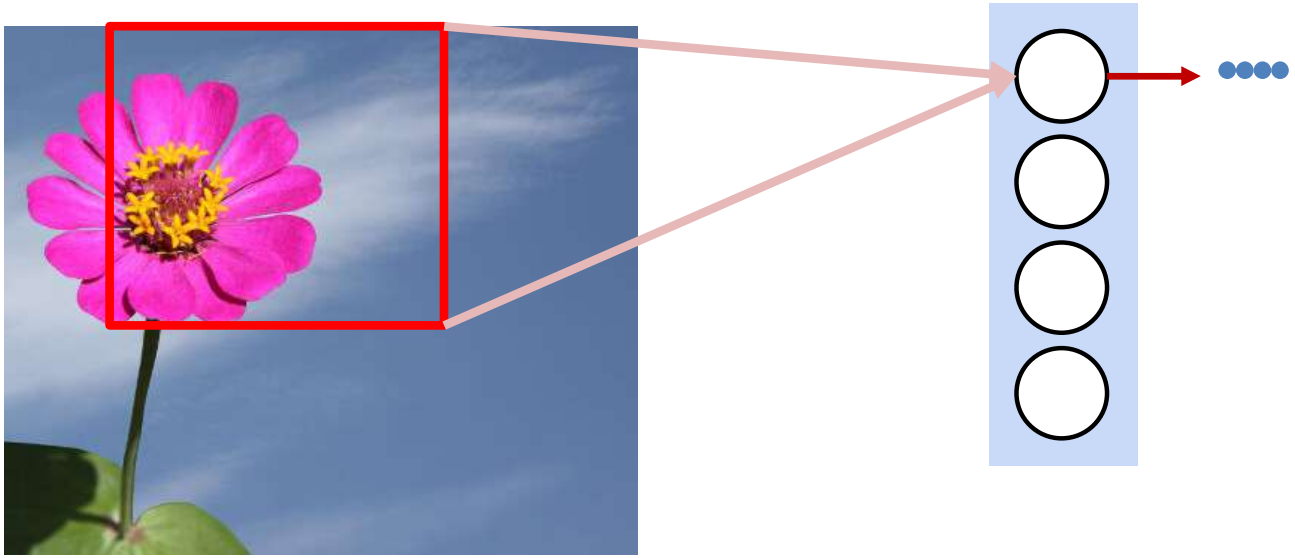
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

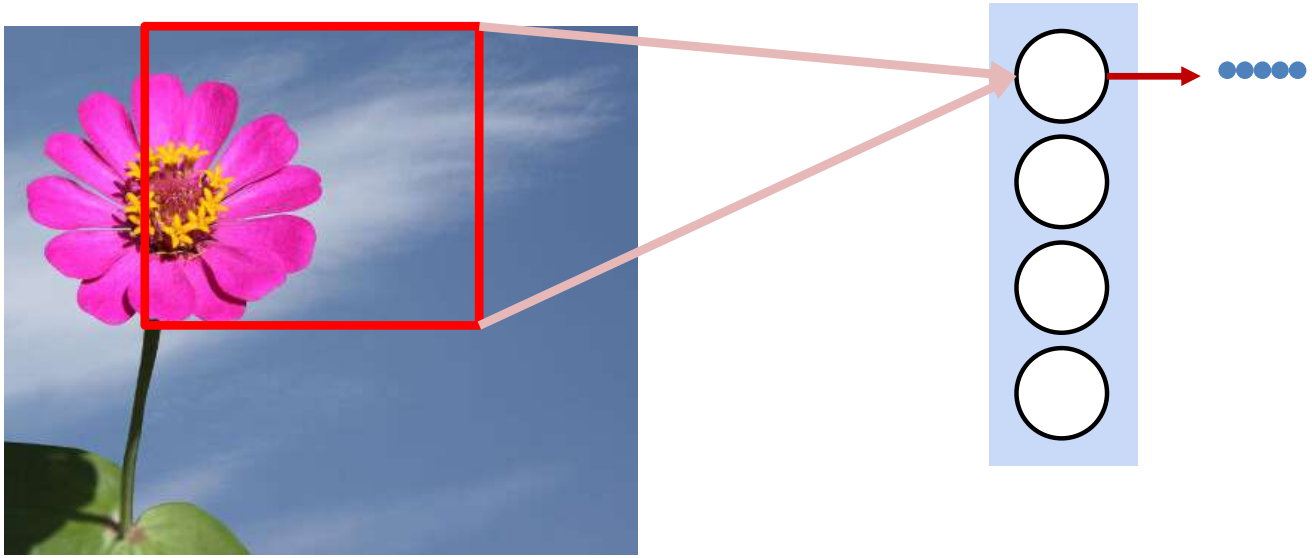
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

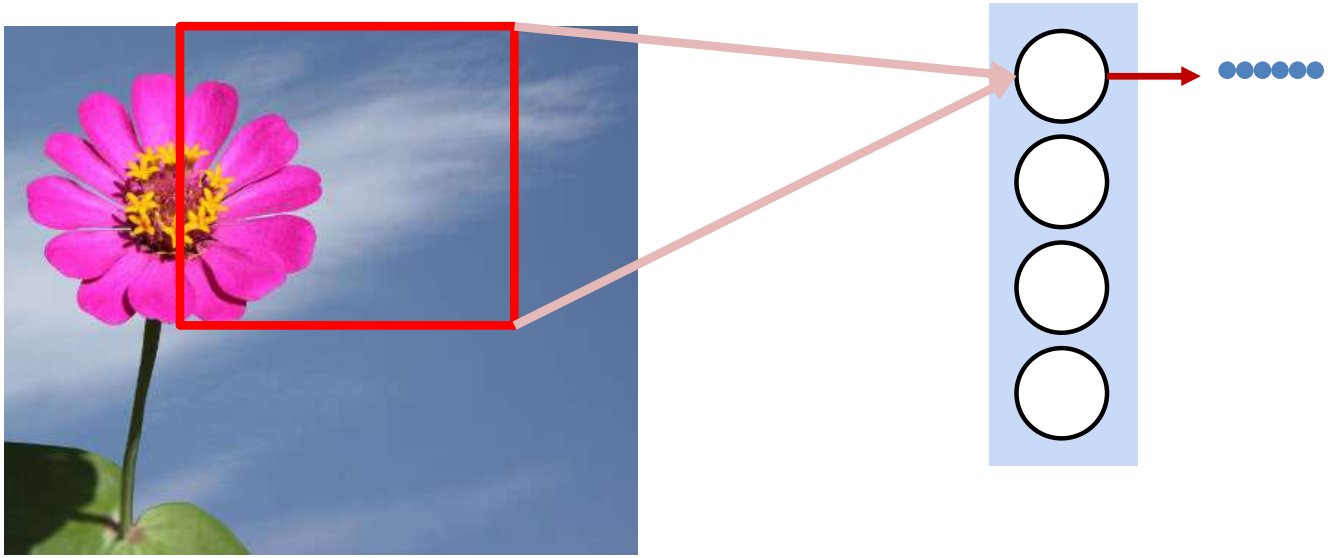
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

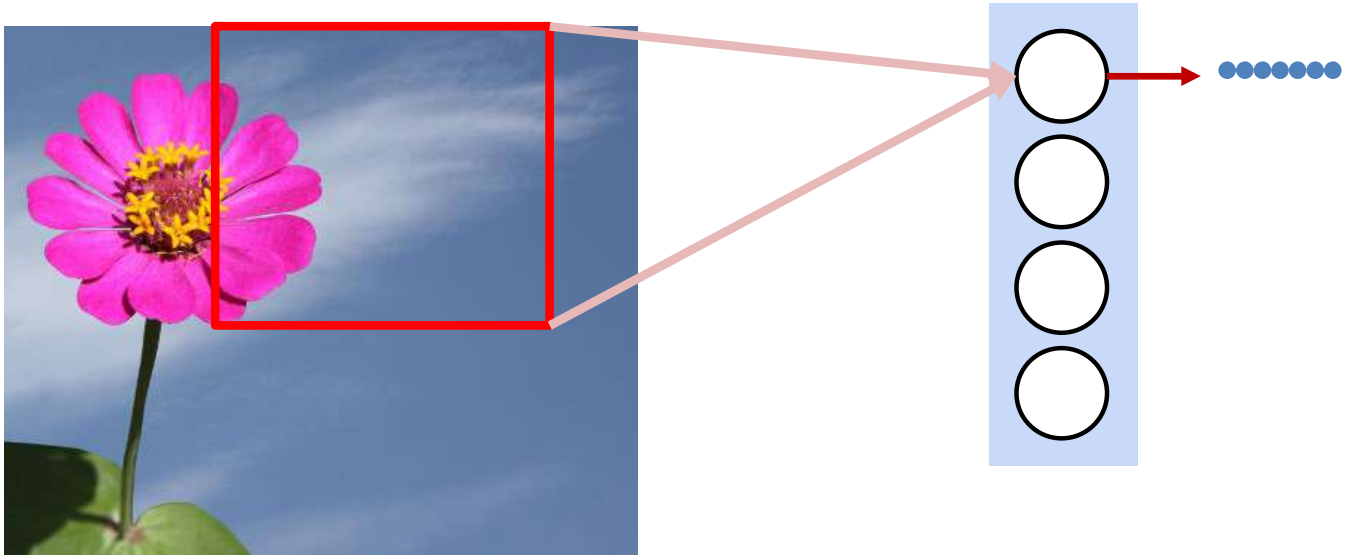
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

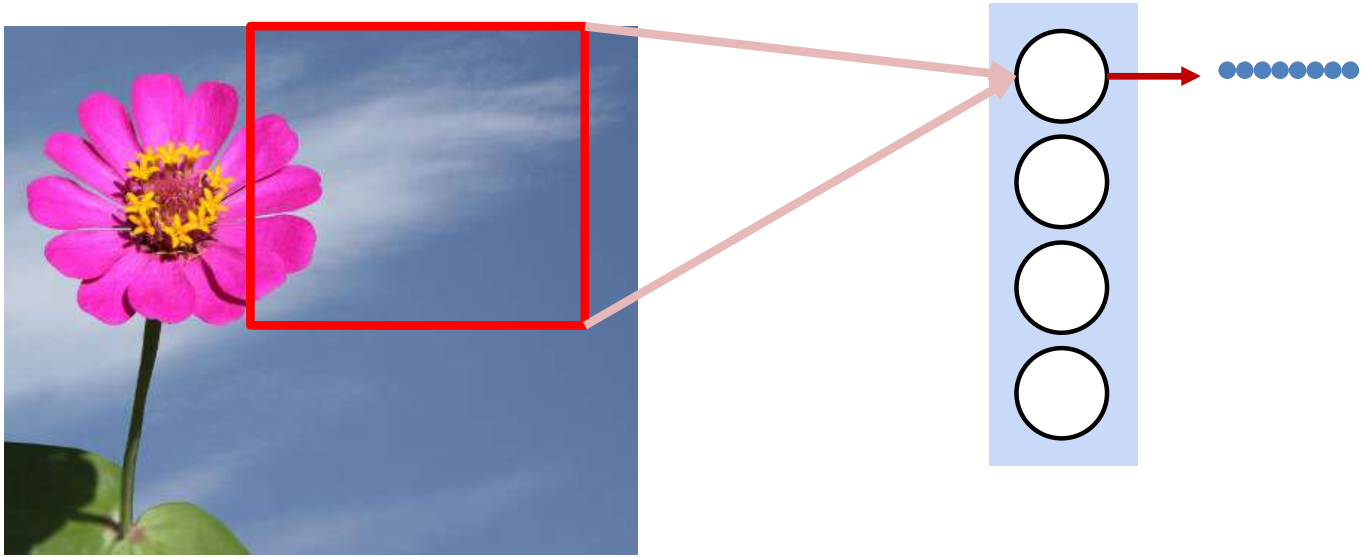
- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region
- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

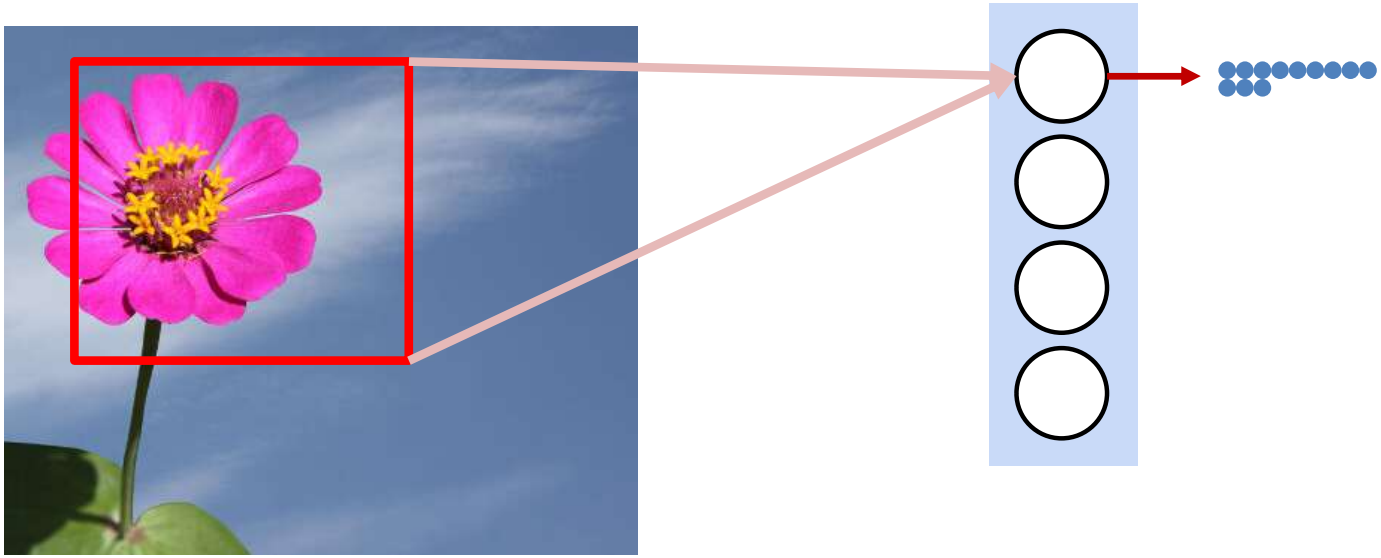
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

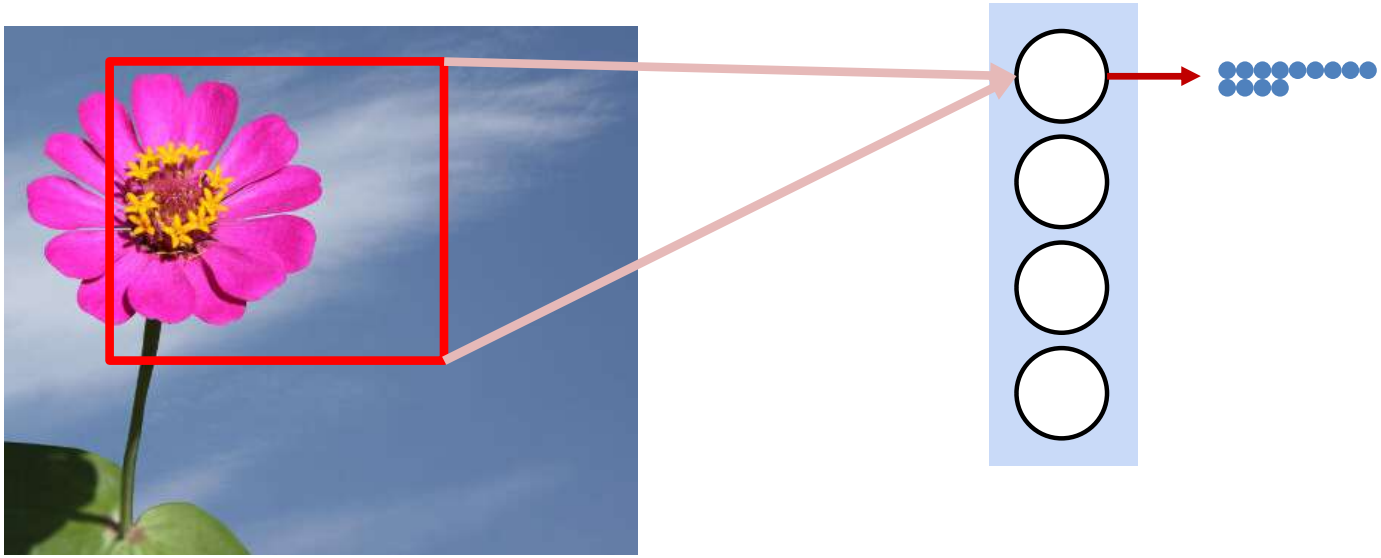
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

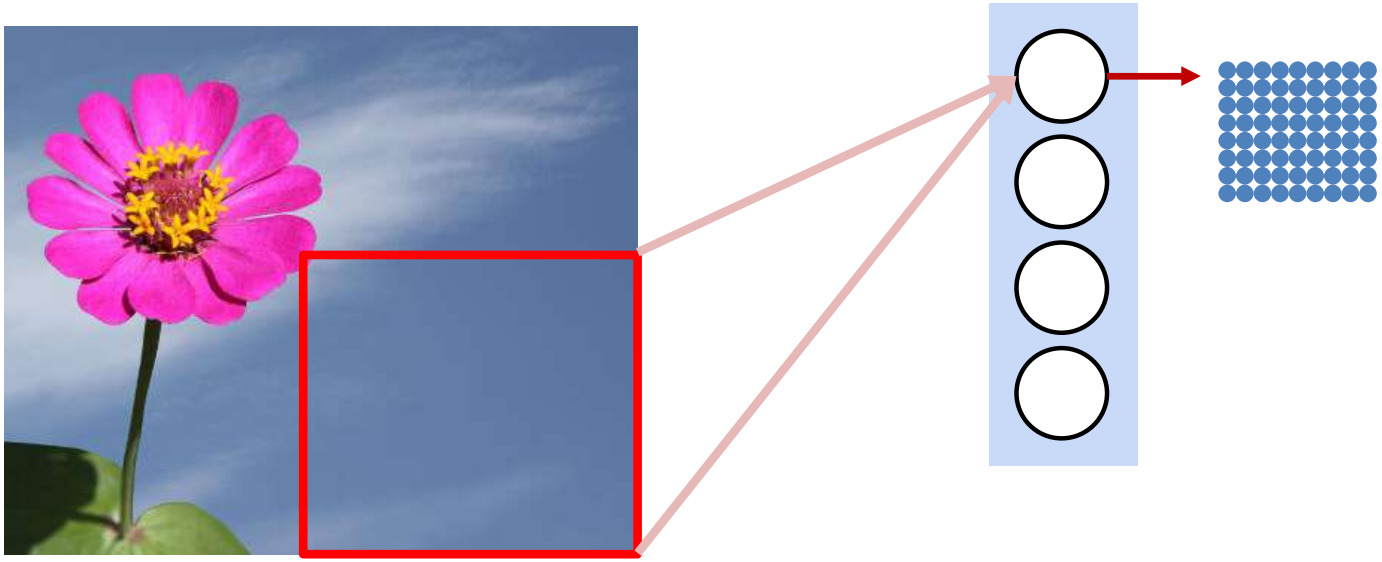
Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

Scanning: A closer look



- Consider a single neuron in the first layer
  - At each position of the box, the neuron is evaluating a “window” of the picture at that location, as part of the classification for *that* region

- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons
  - “Scanning” the image with just the neuron
  - We could arrange the outputs in correspondence to the original picture

BR2

Scanning: A closer look



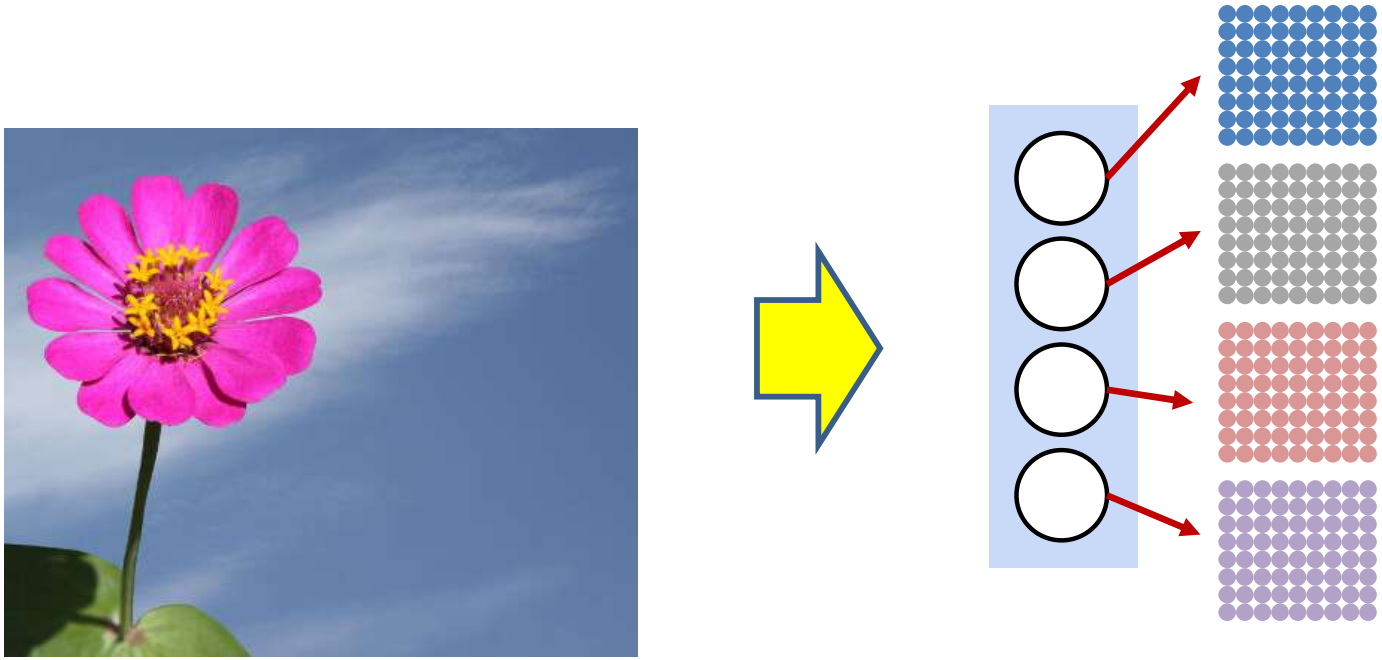
- Let us compute the output of the first neuron for *all* the windows in the picture before computing the rest of the neurons

- Eventually, we can arrange the outputs from the response at the scanned positions into a rectangle that's proportional in size to the original picture

Slide 144

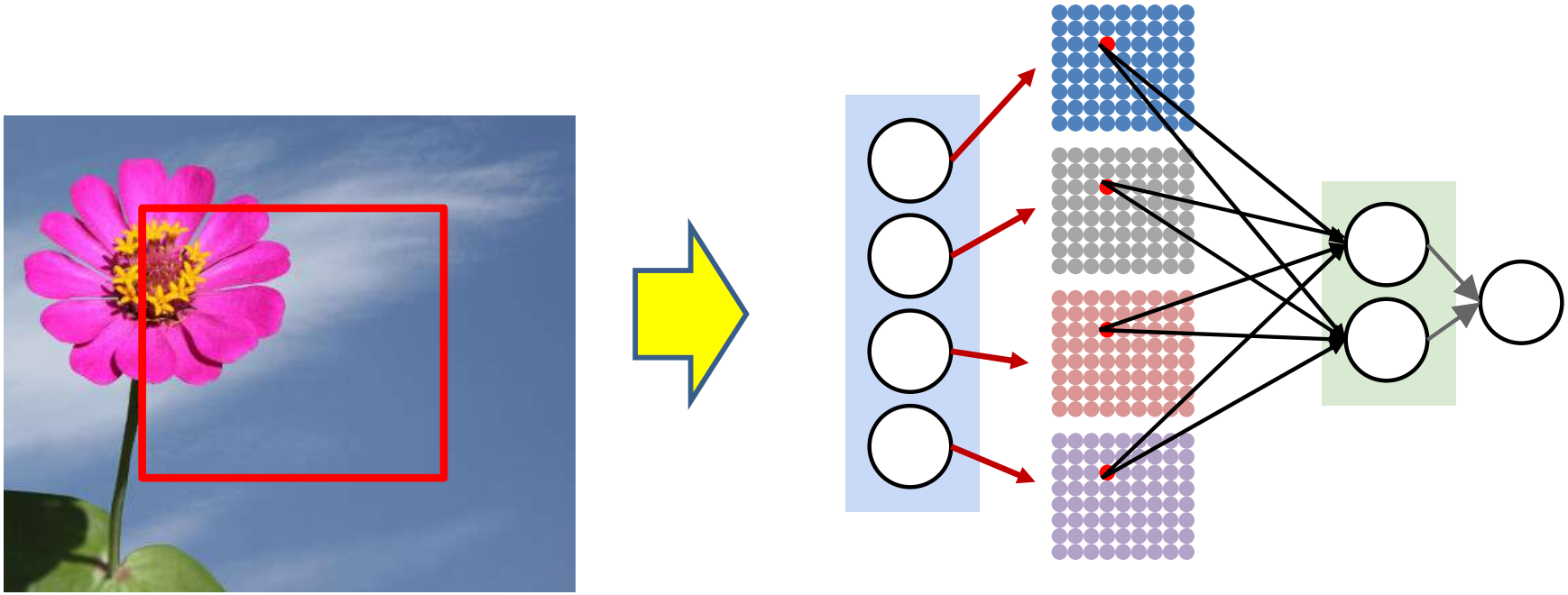
BR2 Bhiksha Raj, 9/28/2021

Scanning: A closer look



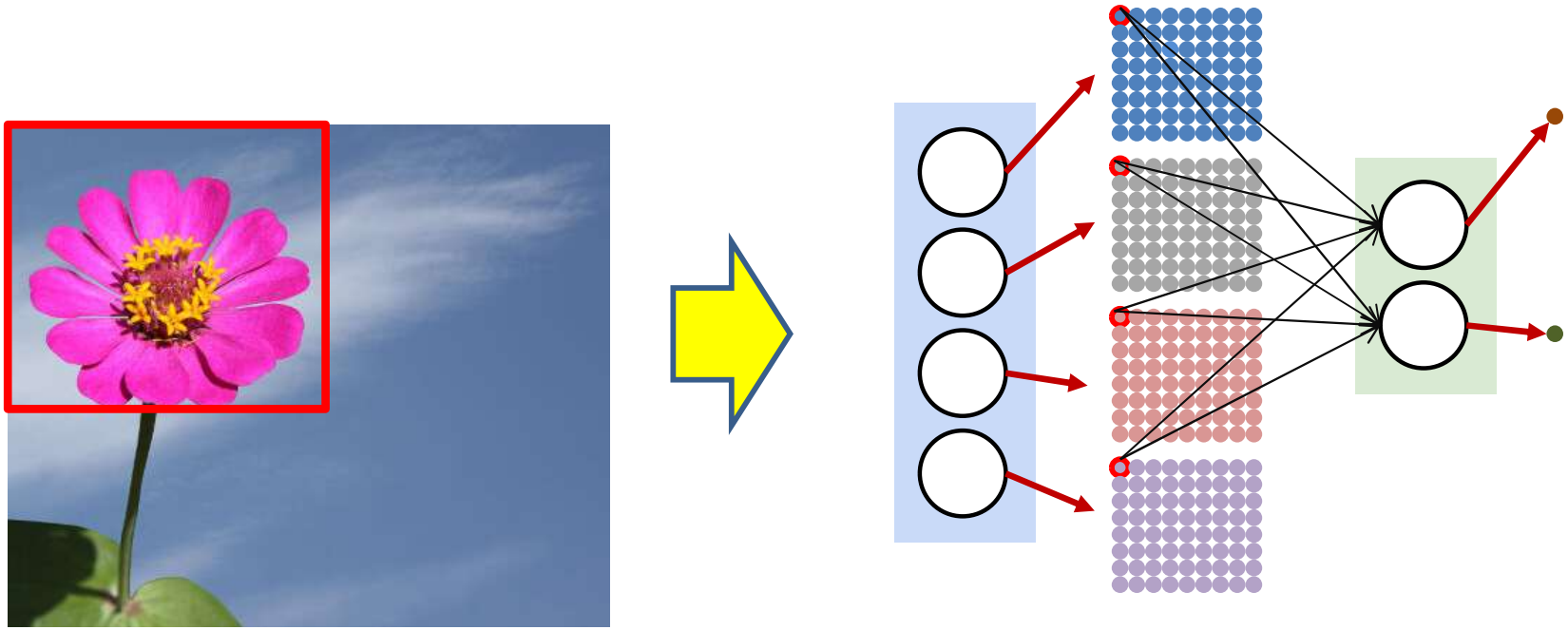
- We can repeat the process for each of the first-layer neurons
  - "Scan" the input with the neuron
  - Arrange the neuron's outputs from the scanned positions according to their positions in the original image

Scanning: A closer look



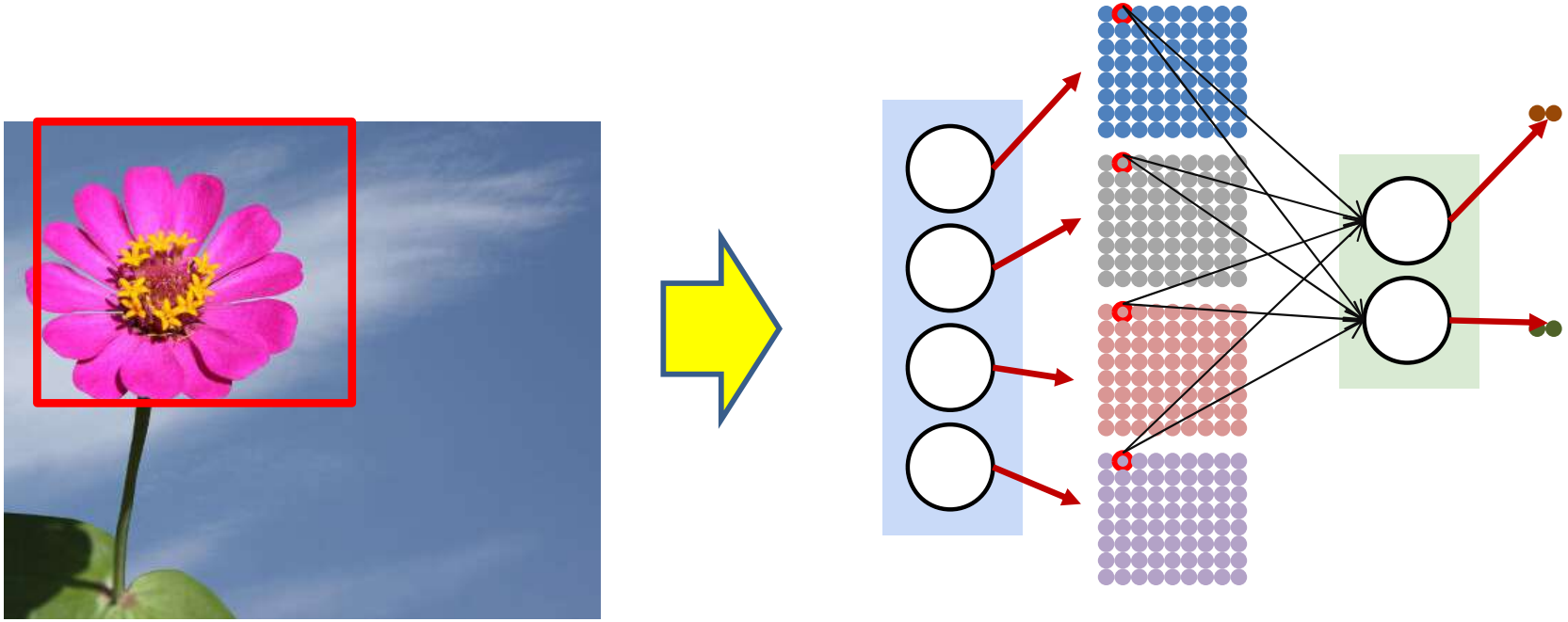
- To classify a specific “window” in the image, we send the first level activations from the positions corresponding to that position to the next layer

Scanning: A closer look



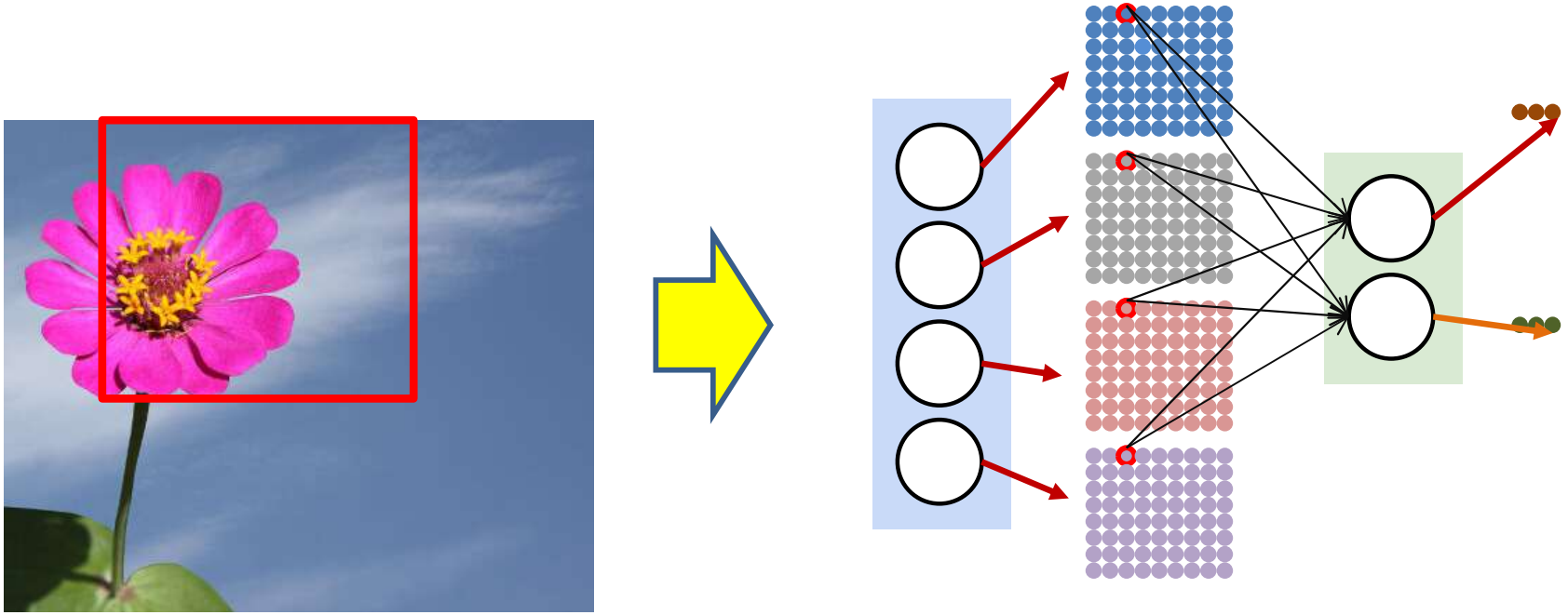
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



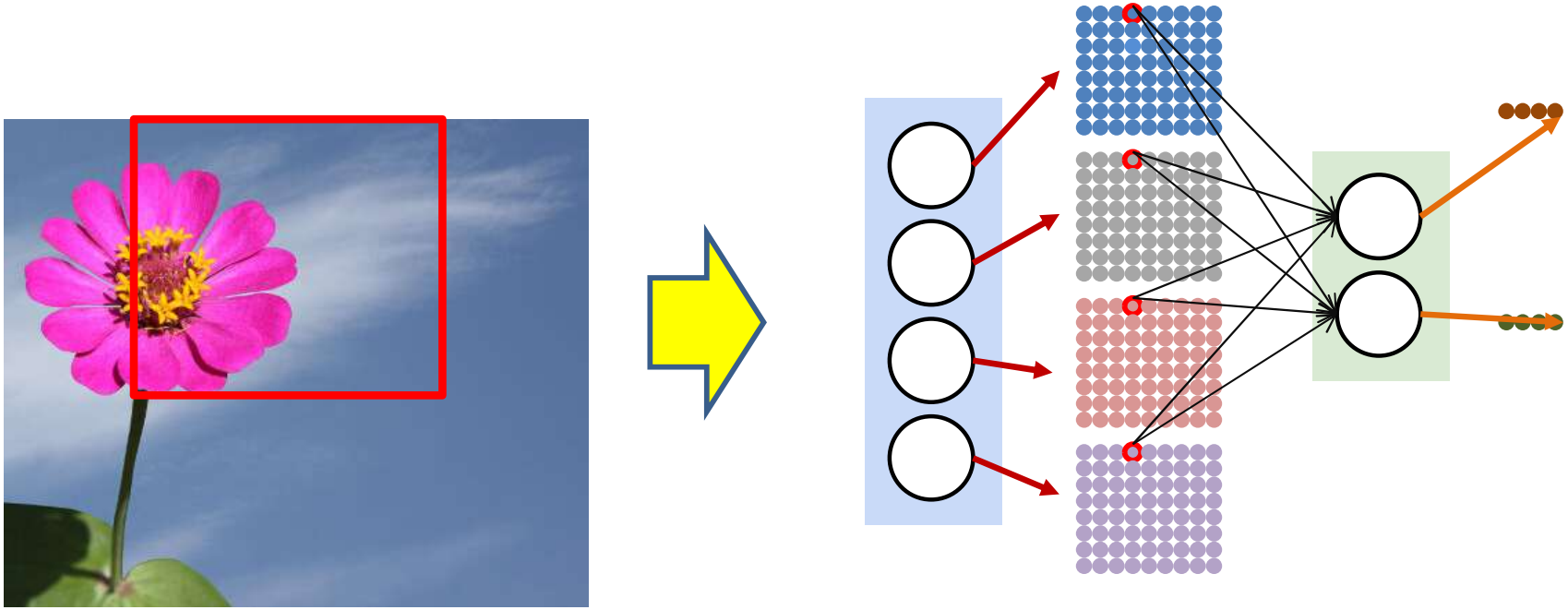
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



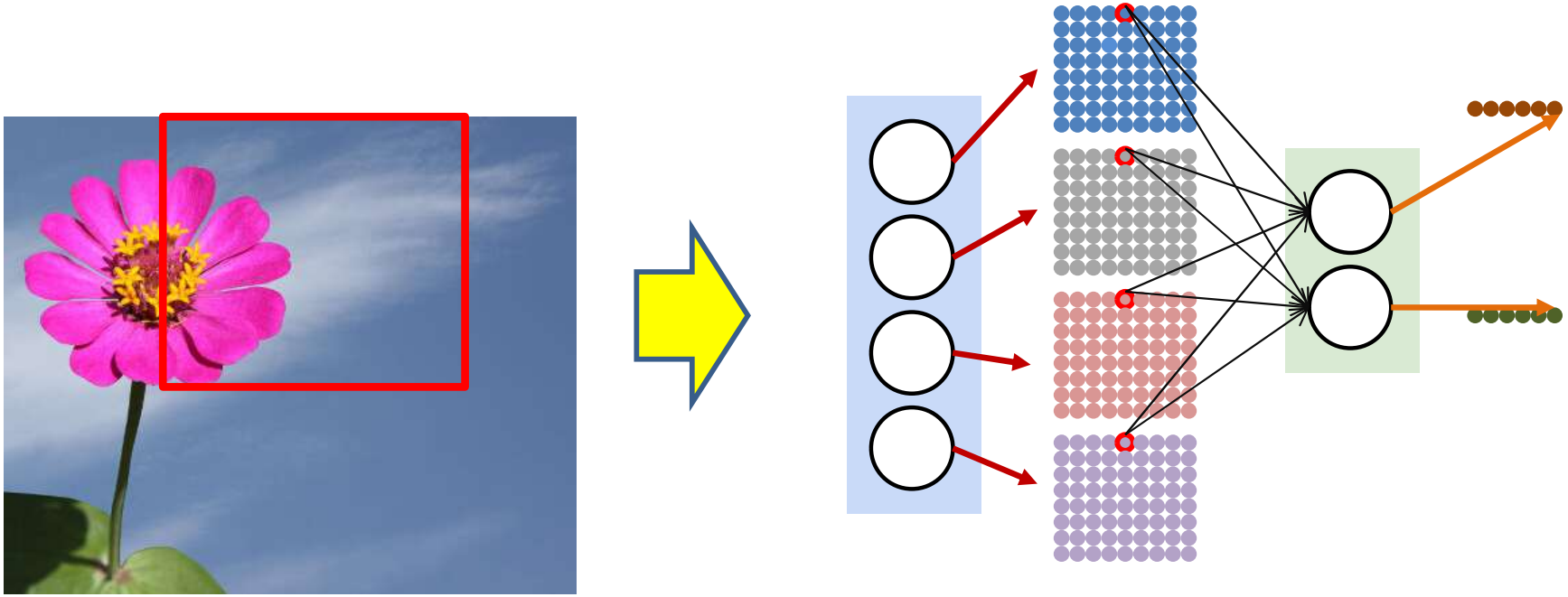
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



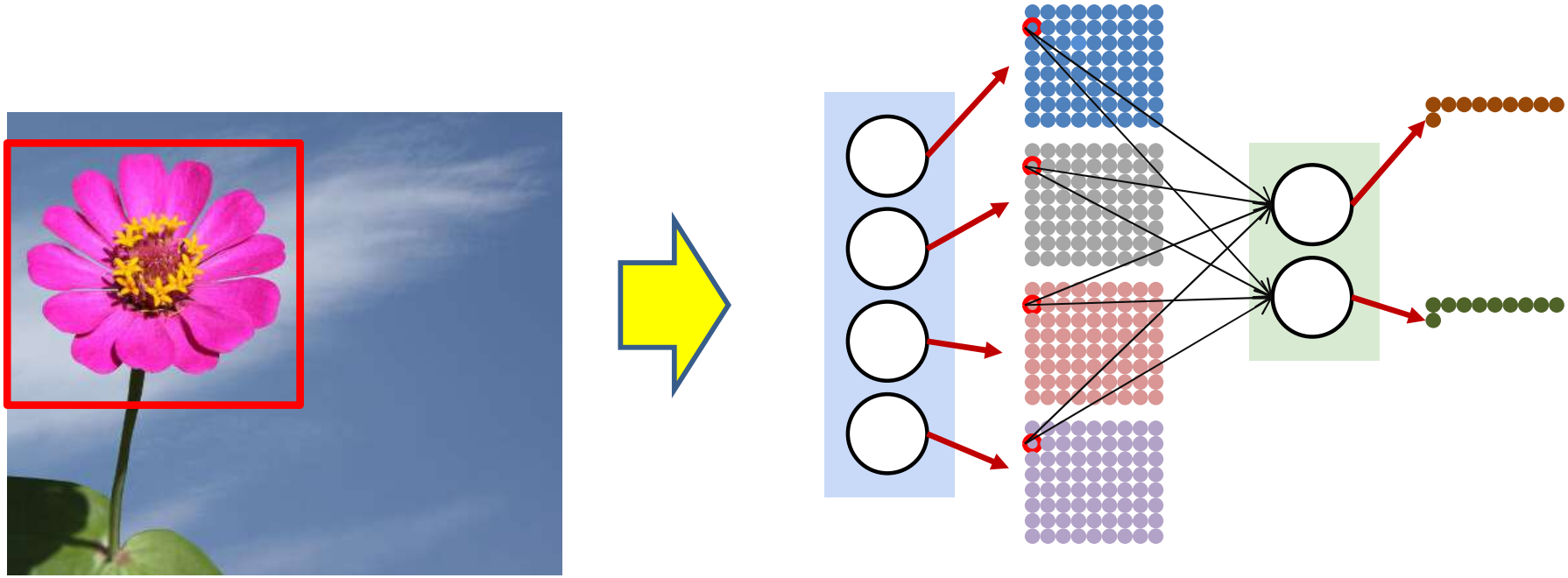
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



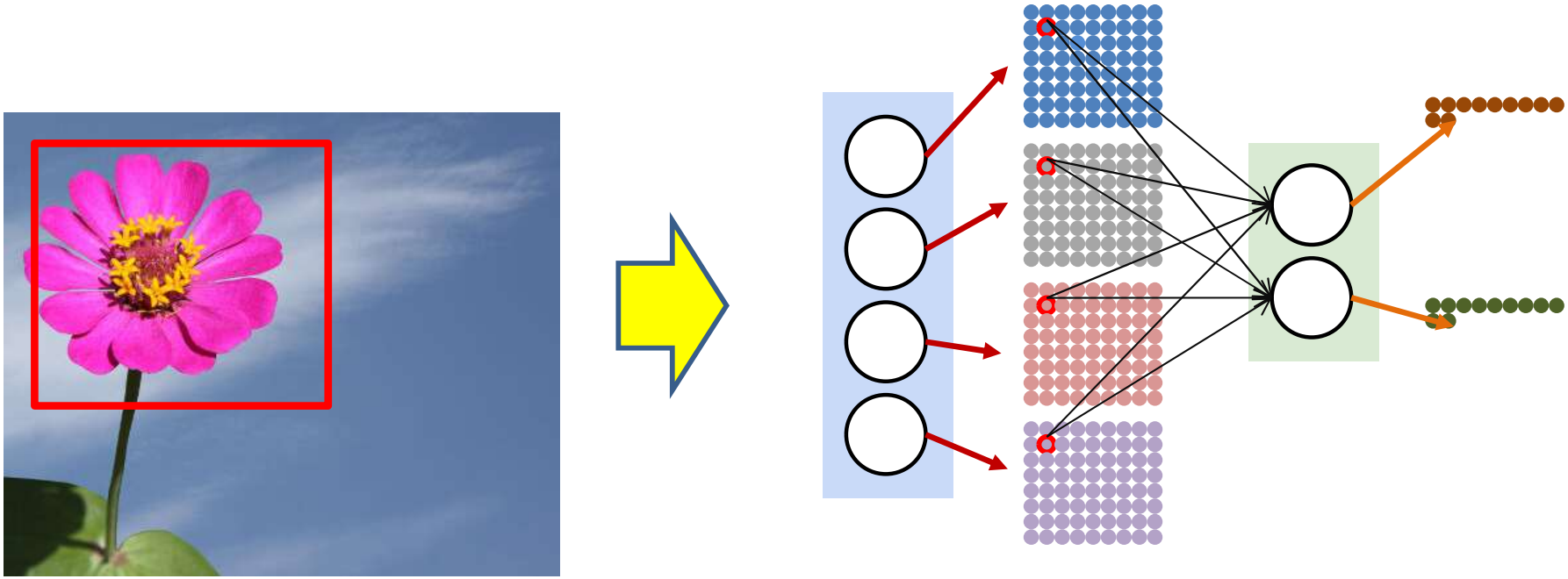
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



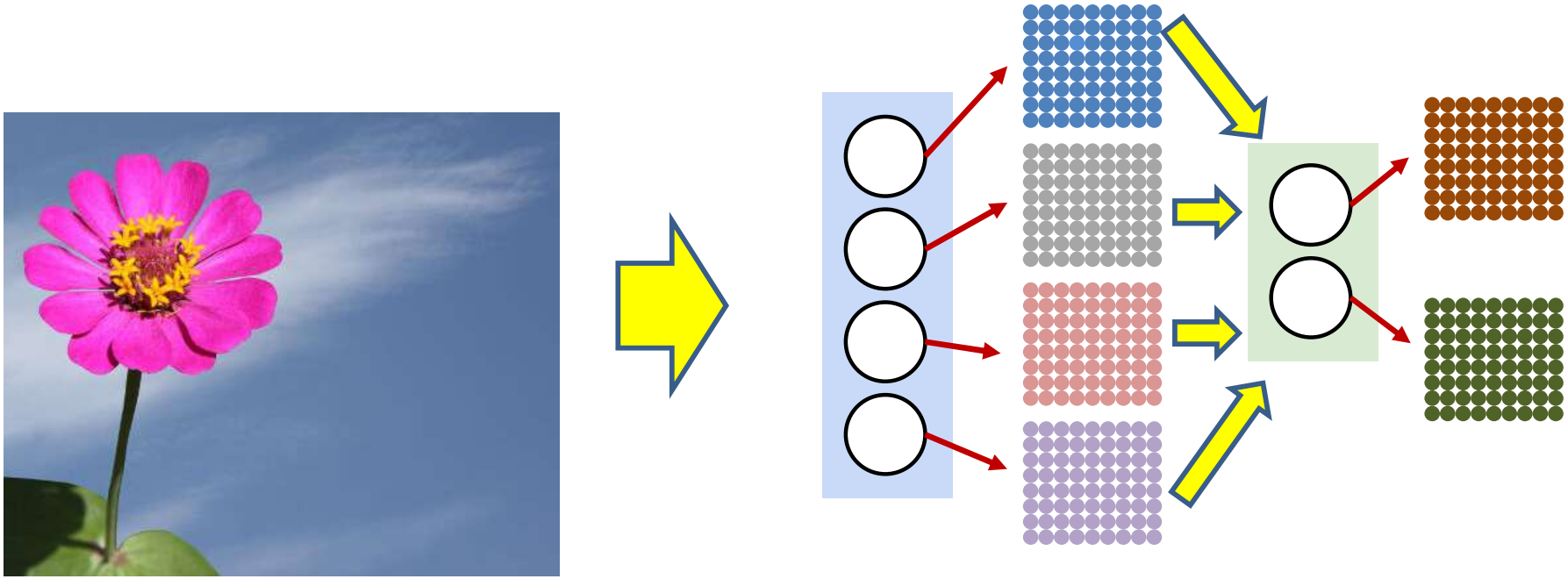
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



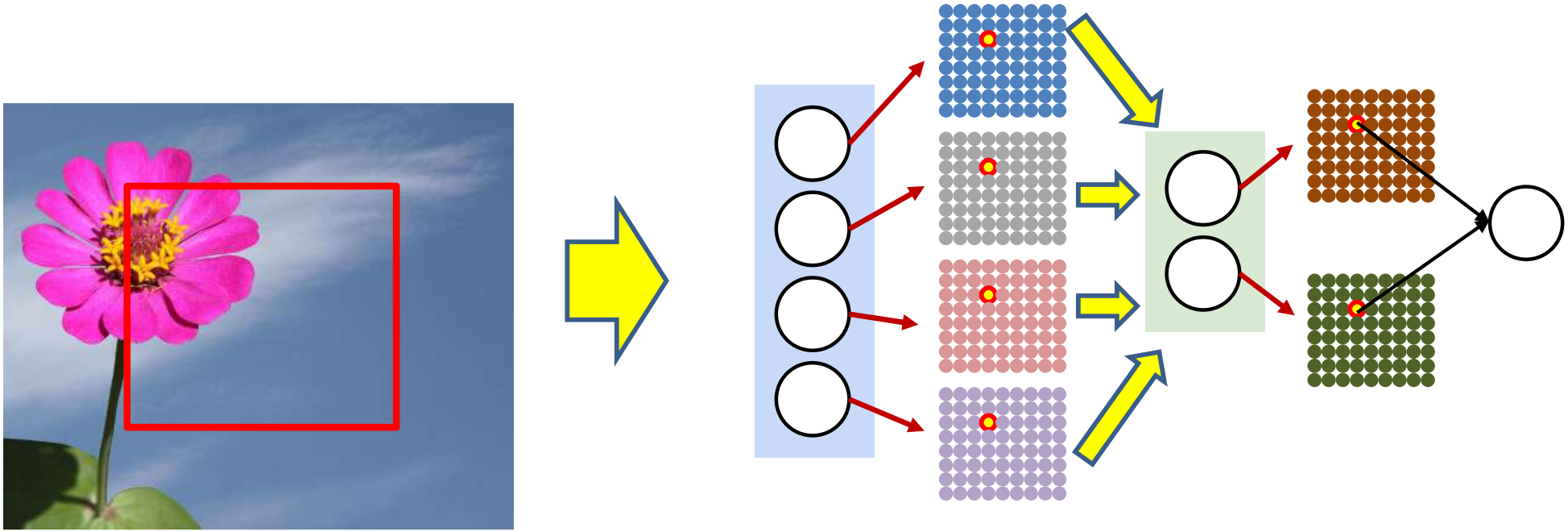
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



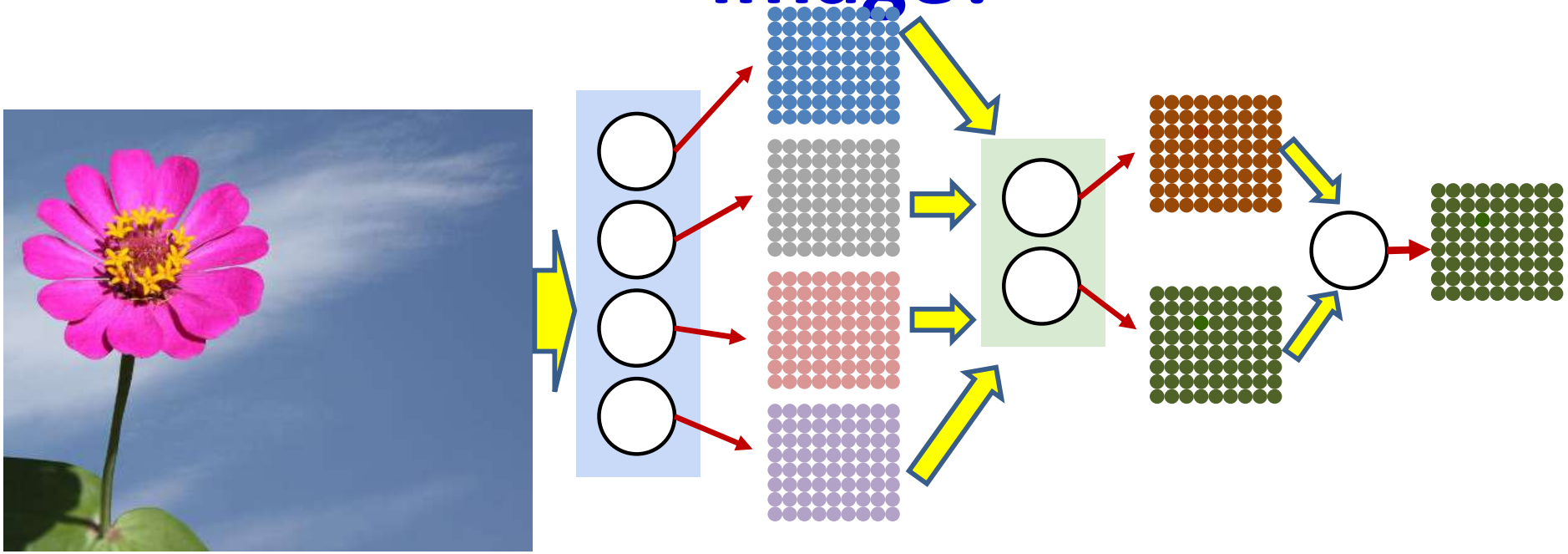
- We can recurse the logic
  - The second level neurons too can "**scan**" the rectangular outputs of the first-level neurons before computing subsequent layers
  - (Un)like the first level, they must jointly scan *multiple* "maps"
    - Each location in the output of the second level neuron considers the corresponding locations from the output maps of all the first-level neurons

Scanning: A closer look



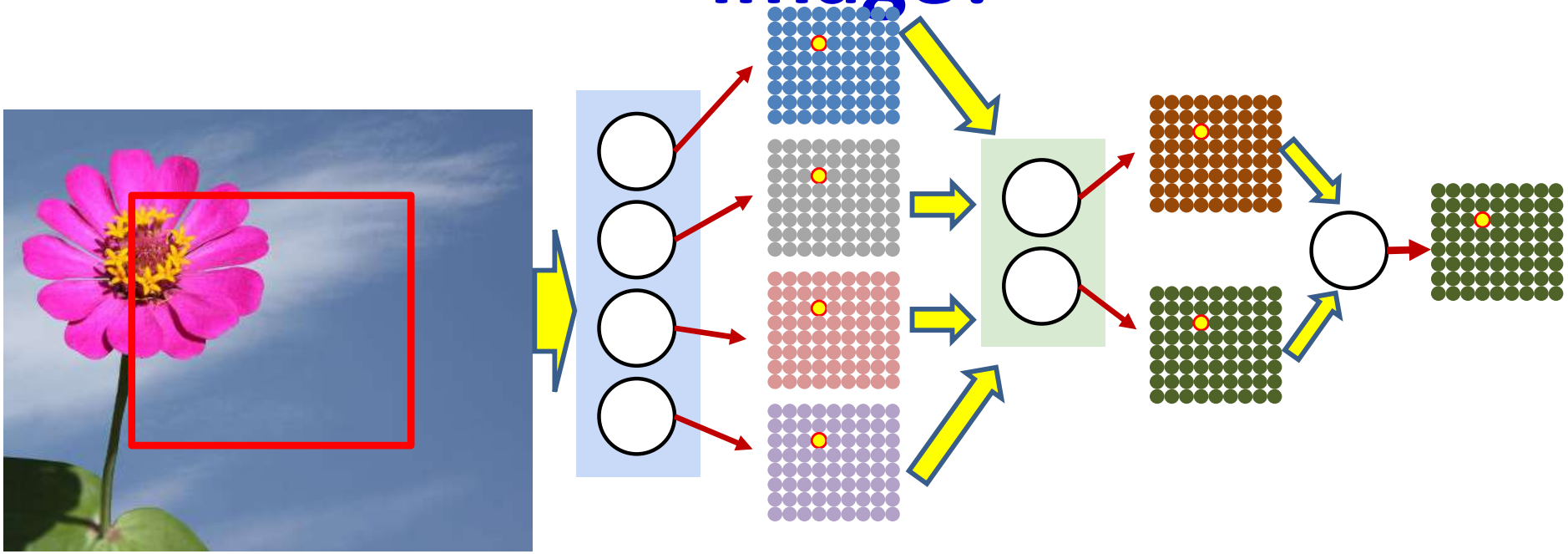
- To detect a picture ***at any location*** in the original image, the output layer must consider the corresponding outputs of the last hidden layer

Detecting a picture anywhere in the image?



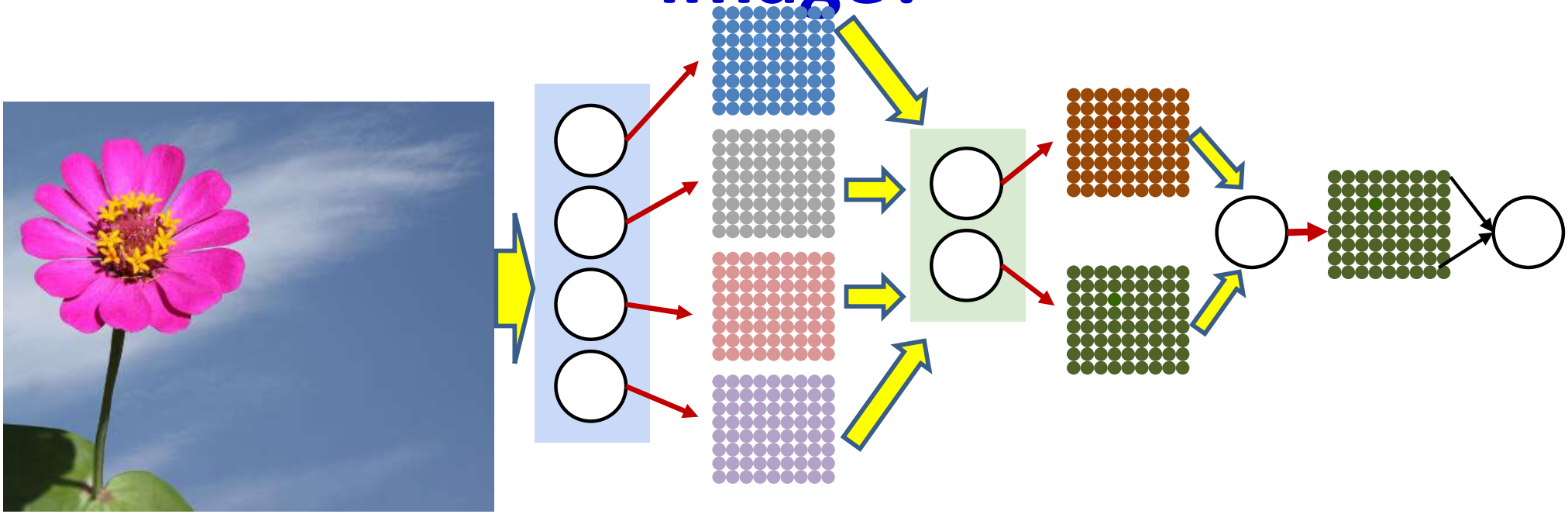
- Recursing the logic, we can create a map for the neurons in the next layer as well
  - The map is a flower detector for each location of the original image

Detecting a picture anywhere in the image?



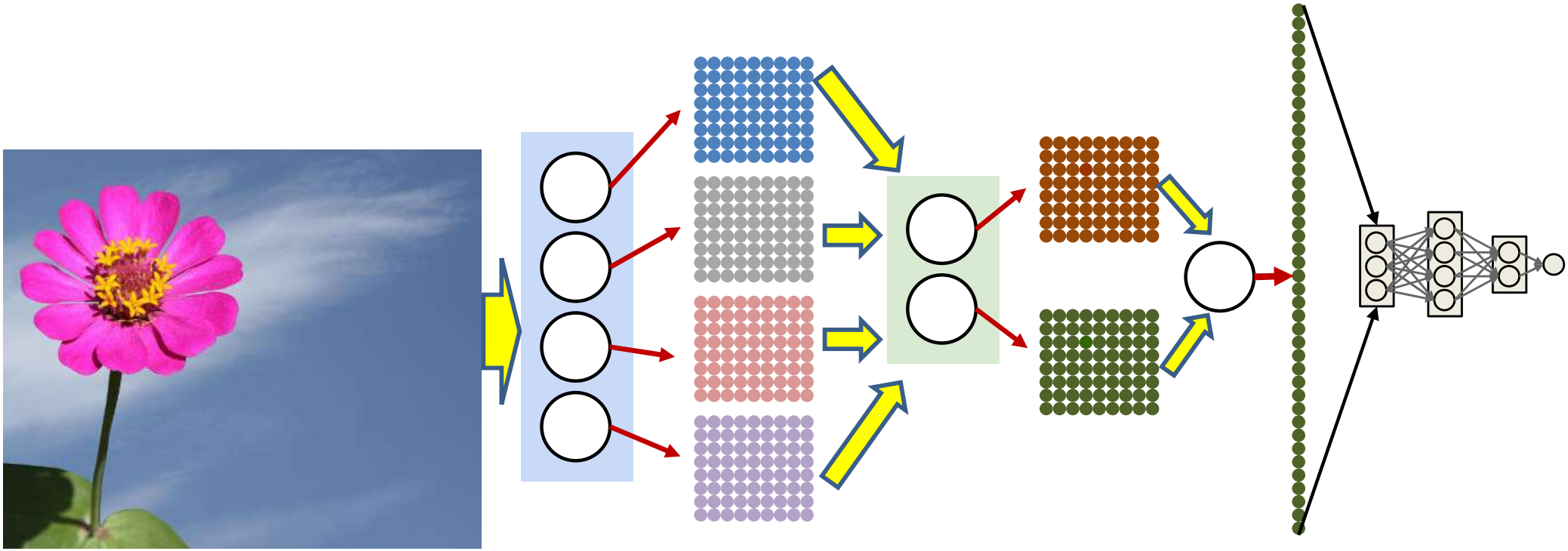
- To detect a picture ***at any location*** in the original image, only need to consider the corresponding location of the output map

Detecting a picture anywhere in the image?



- Actual question: Is there a flower in the picture?
- The entire output map can be sent into a final "max" to detect a flower in the full picture
  - Or a softmax, or a full MLP…

Detecting a picture in the image



- Redrawing the final layer
  - "Flatten" the output of the neurons into a single block, since the arrangement is no longer important
  - Pass that through a max/softmax/MLP

Scanning with an MLP

- `KxK` = size of "patch" evaluated by MLP
- `W` is width of image
- `H` is height of image

```
for x = 1:W-K+1
   for y = 1:H-K+1
      ImgSegment = Img(*, x:x+K-1, y:y+K-1)
      Y(x,y) = MLP(ImgSegment)

Y = softmax( Y(1,1)..Y(W-K+1,H-K+1) )
```

Scanning with an MLP

```
for x = 1:W-K+1
  for y = 1:H-K+1
    for l = 1:L  # layers operate on vector at (x,y)
      for j = 1:D_l
        if (l == 1) #first layer operates on input
          Y(0,:,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
        end
        z(l,j,x,y) = b(l,j)
        for i = 1:D_{l-1}
          z(l,j,x,y) += w(l,i,j)Y(l-1,i,x,y)
        Y(l,j,x,y) = activation(z(l,j,x,y))
```

```
Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

Scanning with an MLP

```
for x = 1:W-K+1
    for y = 1:H-K+1
        for l = 1:L          # layers operate on vector at (x,y)
            for j = 1:D_l
                if (l == 1) #first layer operates on input
                    Y(0,:,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
                end
                z(l,j,x,y) = b(l,j)
                for i = 1:D_{l-1}
                    z(l,j,x,y) += w(l,i,j)Y(l-1,i,x,y)
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

Scanning with an MLP

```
for l = 1:L
    for x = 1:W-K+1
        for y = 1:H-K+1
            for j = 1:D_l
                if (l == 1) #first layer operates on input
                    Y(0,:,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
                end
                z(l,j,x,y) = b(l,j)
                for i = 1:D_{l-1}
                    z(l,j,x,y) += w(l,i,j)Y(l-1,i,x,y)
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

Scanning with an MLP

```
for l = 1:L
  for x = 1:W-K+1
    for y = 1:H-K+1
      for j = 1:D_l
        if (l == 1) #first layer operates on input
          Y(0,:,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
        end
        z(l,j,x,y) = b(l,j)
        for i = 1:D_l-1
          z(l,j,x,y) += w(l,i,j)Y(l-1,i,x,y)
        Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

Scanning with an MLP

```
for x = 1:W-K+1
  for y = 1:H-K+1
    for l = 1:L   # layers operate on vector at (x,y)
      if (l == 1) #first layer operates on input
        Y(0,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
      end
      z(l,x,y) = W(l)Y(l-1,x,y) + b(l)
      Y(l,x,y) = activation(z(l,x,y))

Y = softmax( Y(L,1,1)..Y(L,W-K+1,H-K+1) )
```

Scanning with an MLP

```
for x = 1:W-K+1
    for y = 1:H-K+1
        for l = 1:L   # layers operate on vector at (x,y)
            if (l == 1) #first layer operates on input
                Y(0,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
            end
            z(l,x,y) = W(l)Y(l-1,x,y) + b(l)
            Y(l,x,y) = activation(z(l,x,y))

Y = softmax( Y(L,1,1)..Y(L,W-K+1,H-K+1) )
```

Scanning with an MLP

```
for l = 1:L          # layers
   for x = 1:W-K+1
      for y = 1:H-K+1
         if (l == 1) #first layer operates on input
            Y(0,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
         end
         z(l,x,y) = W(l)Y(l-1,x,y) + b(l)
         Y(l,x,y) = activation(z(l,x,y))

Y = softmax( Y(L,1,1)..Y(L,W-K+1,H-K+1) )
```

Reordering the computation

```
for l = 1:L  # layers operate on vector at (x,y)
    for x = 1:W-K+1
        for y = 1:H-K+1
            if (l == 1) #first layer operates on input
                Y(0,x,y) = Img(1:C, x:x+K-1, y:y+K-1)
            end
            z(l,x,y) = W(l)Y(l-1,x,y) + b(l)
            Y(l,x,y) = activation(z(l,x,y))

Y = softmax( Y(L,1,1)..Y(L,W-K+1,H-K+1) )
```

Story so far

- Position-invariant pattern classification can be performed by scanning the input for a target pattern
  - Scanning is equivalent to composing a large network with shared subnets
- The operations in scanning the input with a full network can be equivalently reordered as
  - scanning the input with individual neurons in the first layer to produce scanned “maps” of the input
  - Jointly scanning the “map” of outputs by all neurons in the previous layers by neurons in subsequent layers

Poll 2 (@626, @627)

- Scanning an input image with an MLP is mathematically equivalent to first scanning it with the individual neurons in the first hidden layer, and then scanning the output maps of the first layer neurons with rest of the network
  - True
  - False

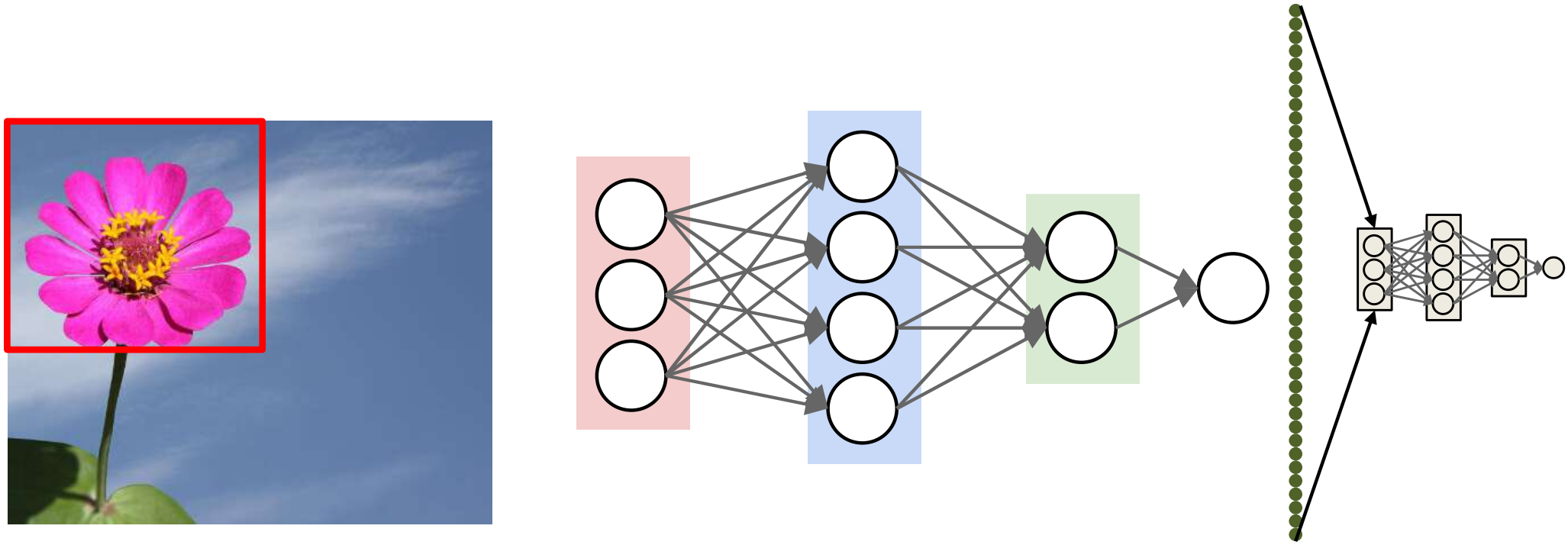
- This operation can be recursed: Scanning the output maps of the first layer with the rest of the network is equivalent to scanning the first layer maps with the second layer, and then scanning the second layer maps with the remaining network
  - True
  - False

Poll 2

- Scanning an input image with an MLP is mathematically equivalent to first scanning it with the individual neurons in the first hidden layer, and then scanning the output maps of the first layer neurons with rest of the network
  - **True**
  - False

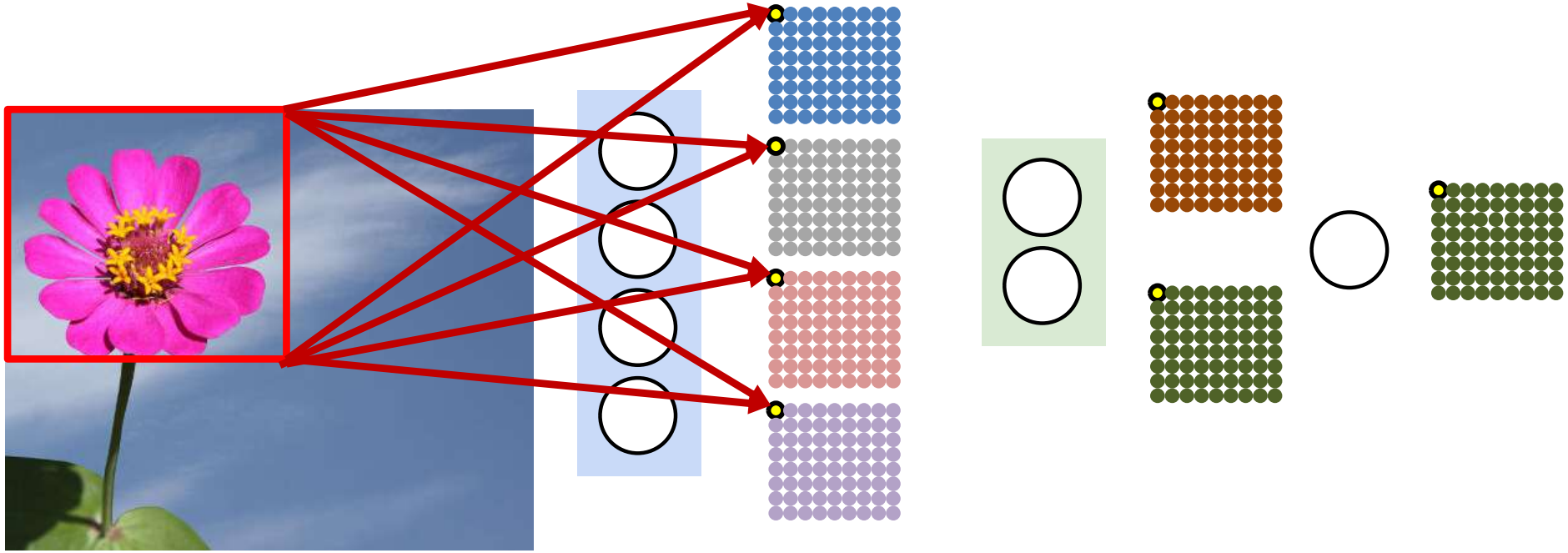
- This operation can be recursed: Scanning the output maps of the first layer with the rest of the network is equivalent to scanning the first layer maps with the second layer, and then scanning the second layer maps with the remaining network
  - **True**
  - False

What the neurons in the layers do…



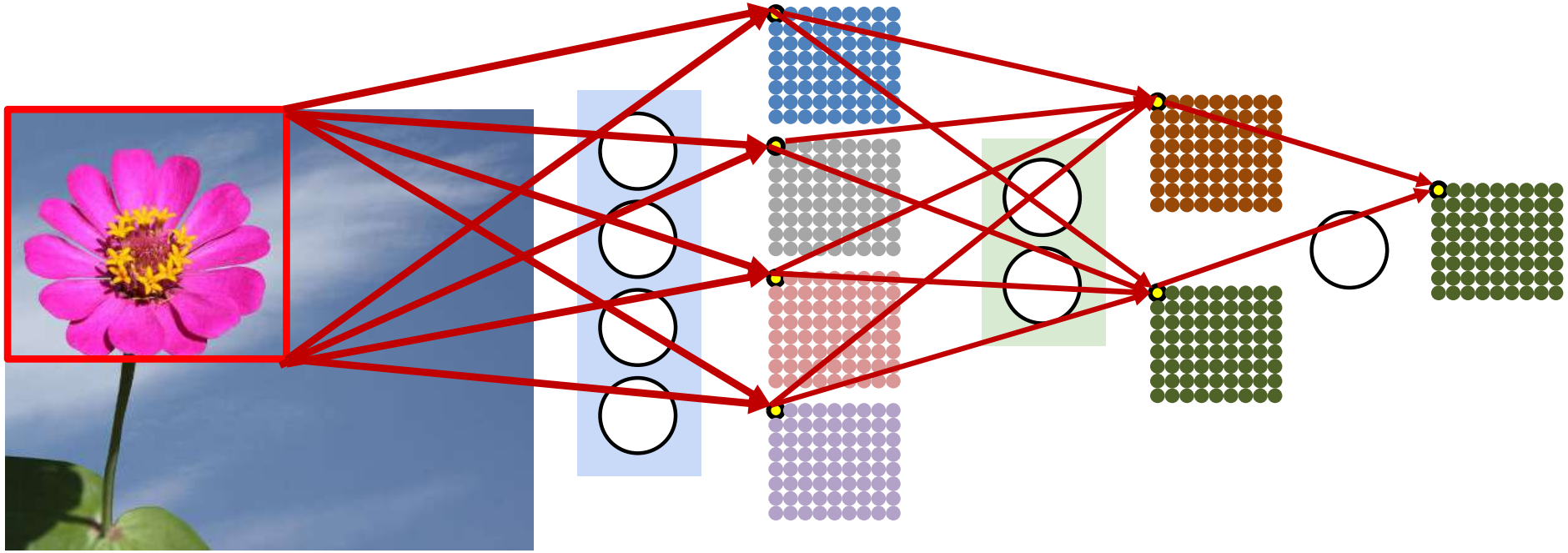
- The entire MLP looks for a flower-like pattern at each location

The behavior of the layers



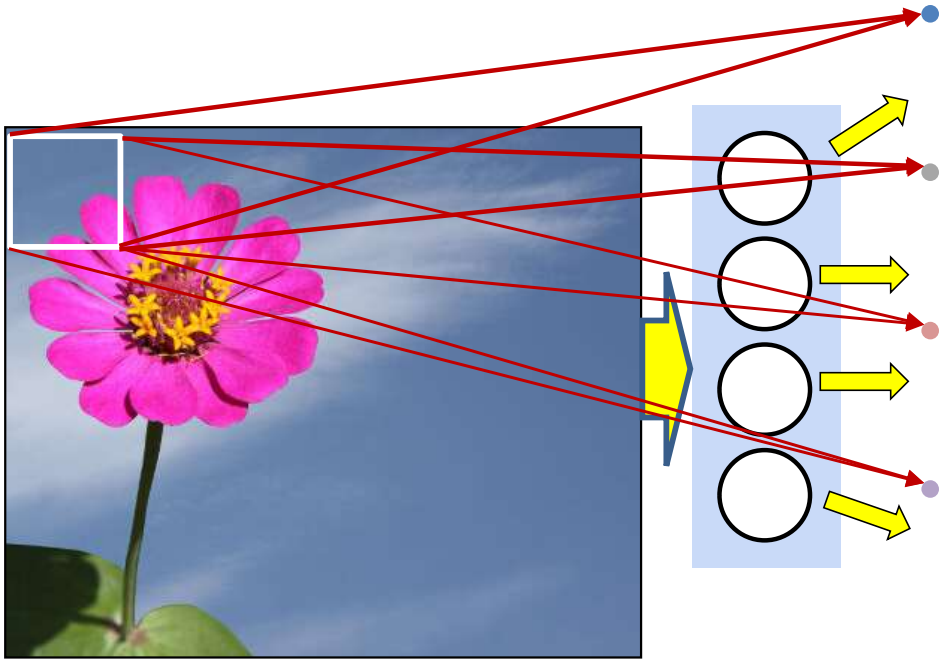
- The first layer neurons “look” at the entire “window” to extract window-level features

The behavior of the layers



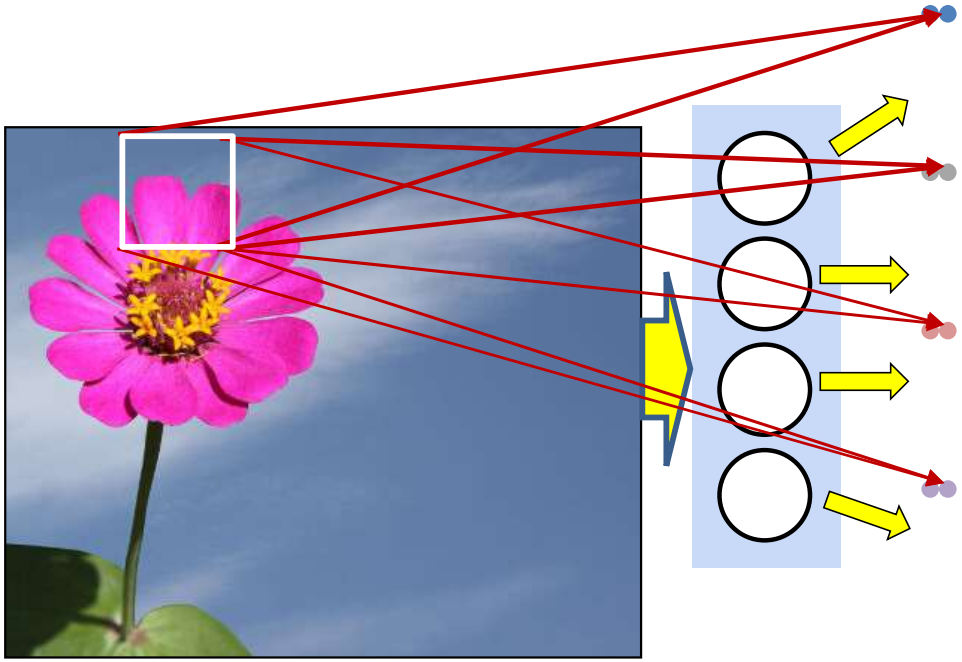
- The first layer neurons “look” at the entire “window” to extract window-level features
  - Subsequent layers only perform classification over these window-level features
- **The first layer neurons is responsible for evaluating the entire *window* of pixels**
  - **Subsequent layers only look at a *single* pixel in their input maps**

Distributing the scan



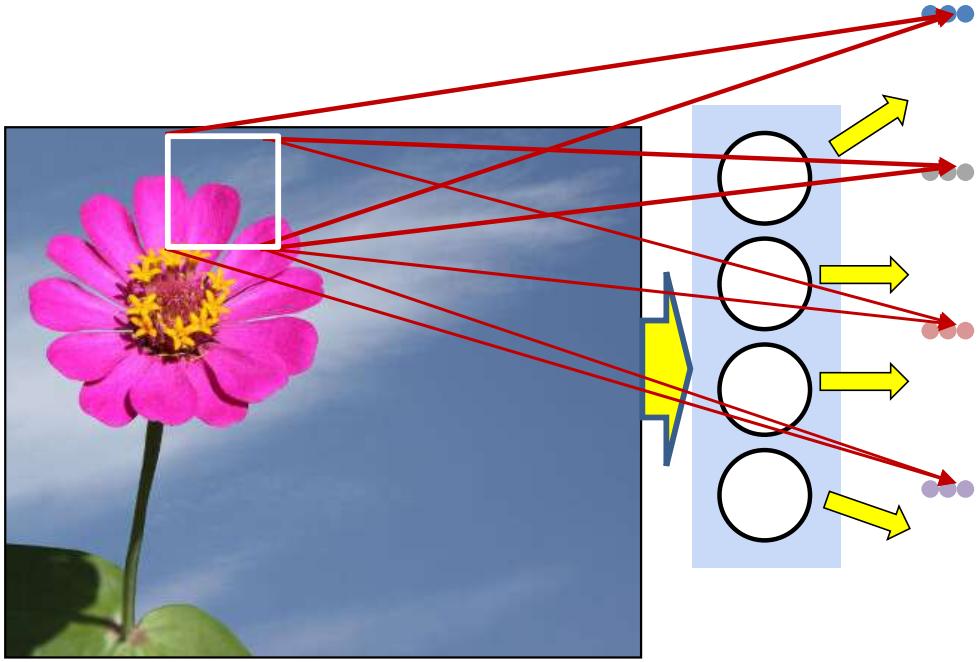
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels

Distributing the scan



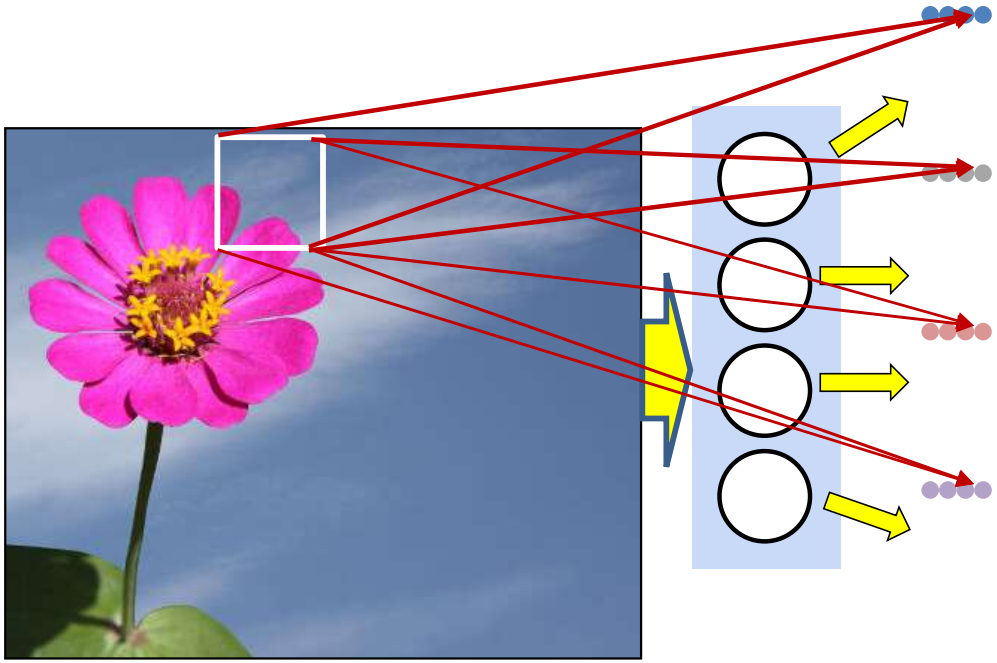
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels

Distributing the scan



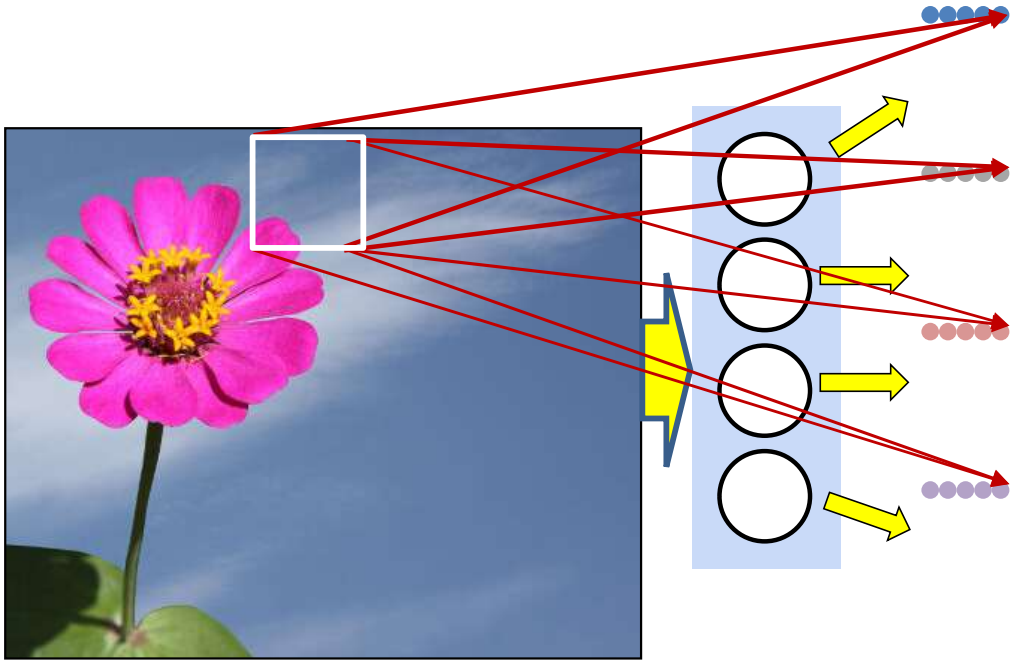
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels

Distributing the scan



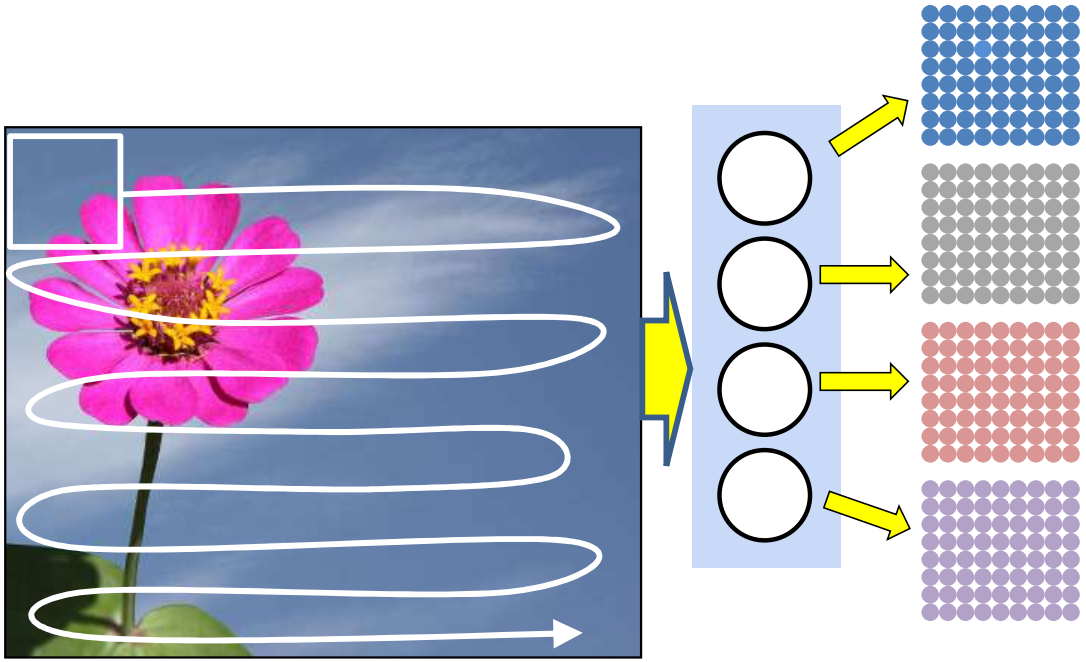
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels

Distributing the scan



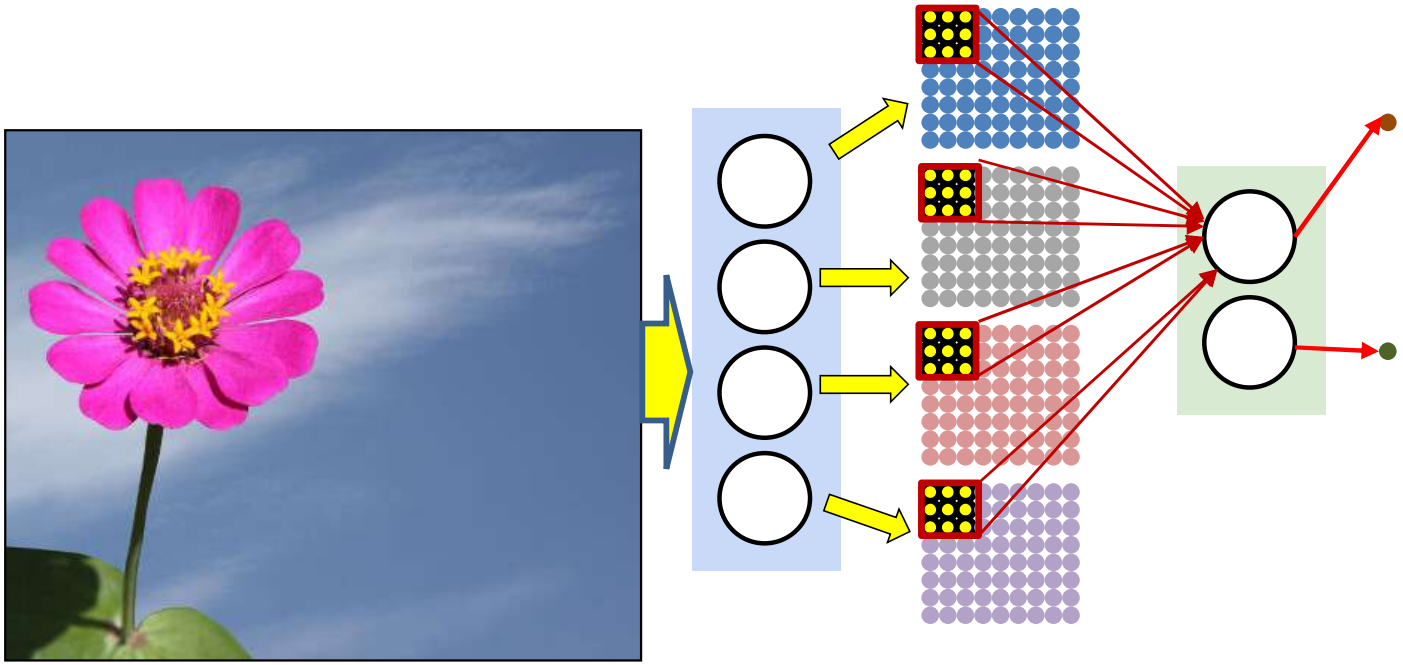
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels

Distributing the scan



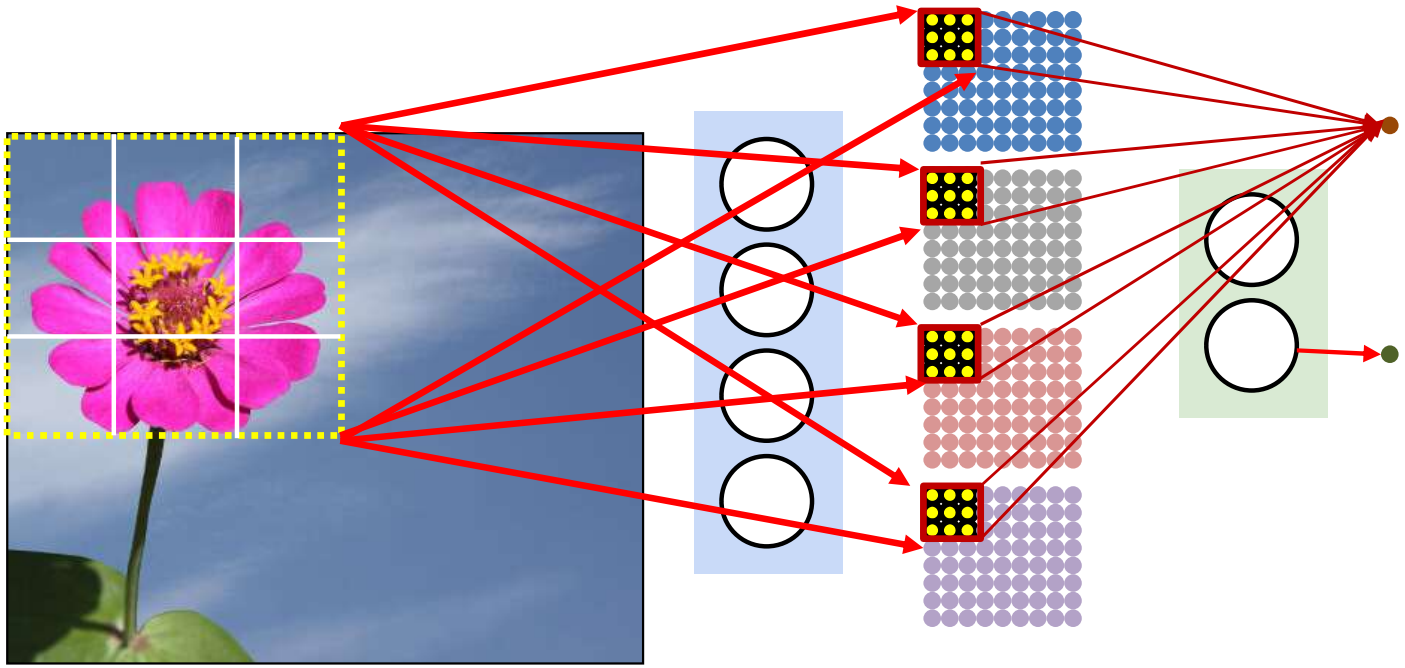
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels

Distributing the scan



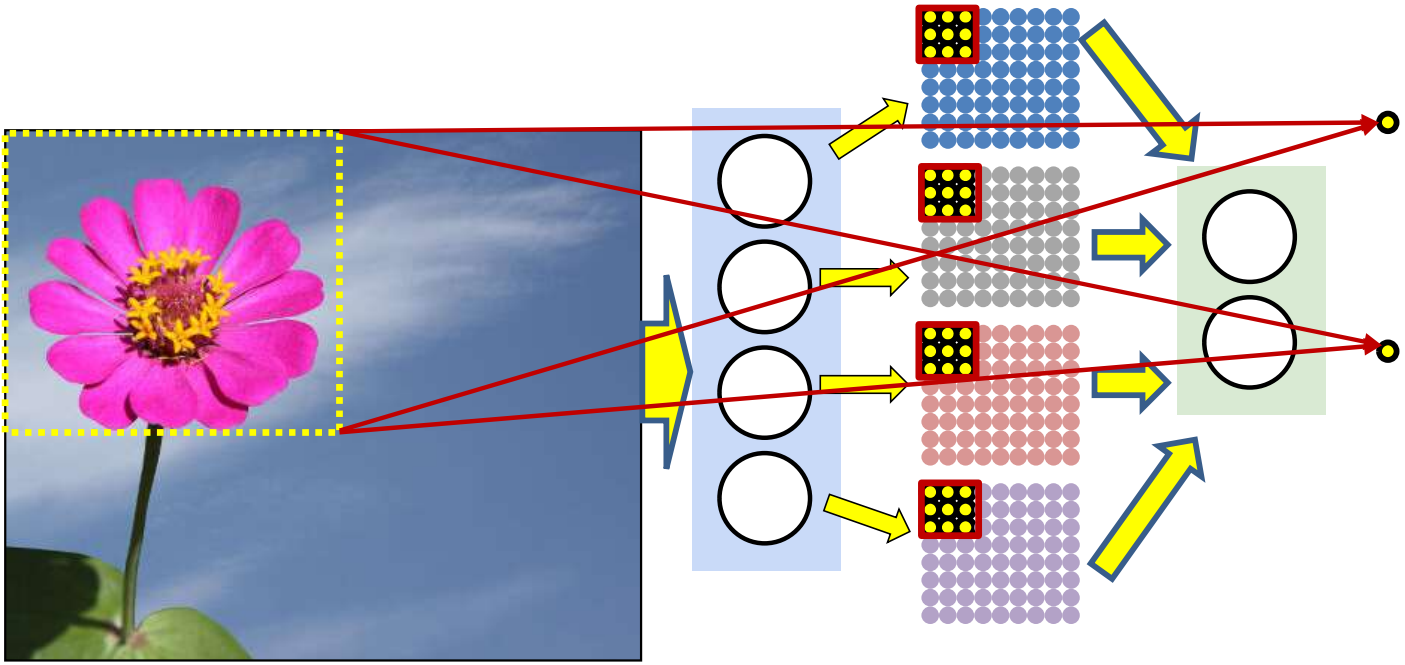
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels
  - The next layer evaluates blocks of outputs from the first layer

Distributing the scan



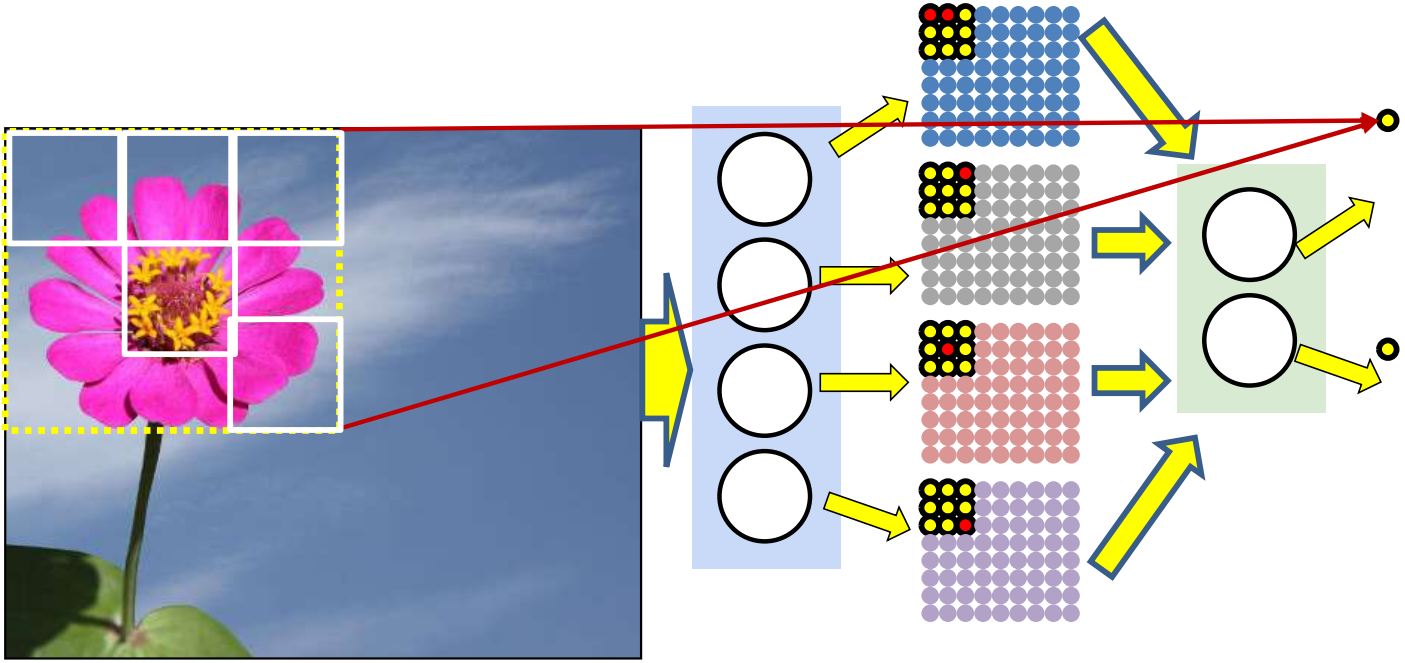
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels
  - The next layer evaluates the windows of outputs from the first layer
  - This effectively evaluates the larger window of the original image

Distributing the scan



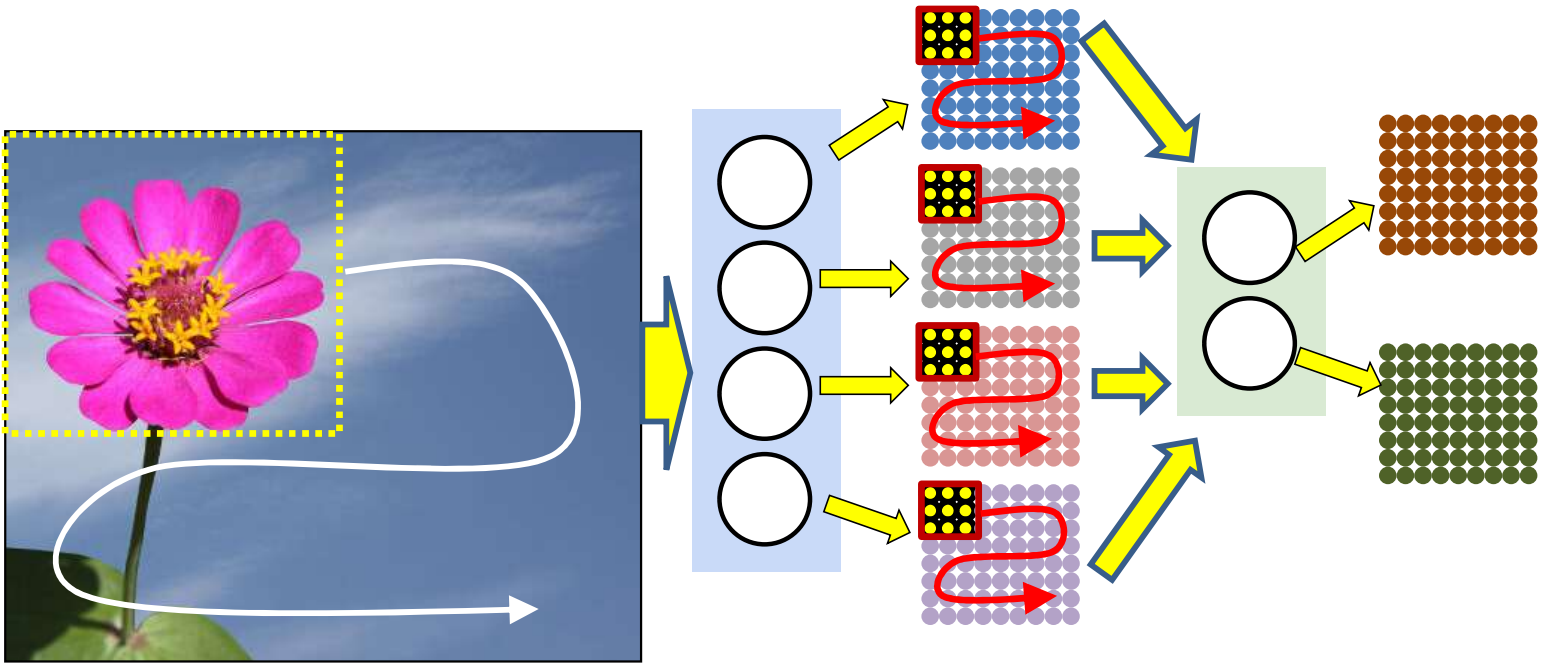
- We can distribute the pattern matching over two layers and still achieve the same block analysis at the second layer
  - The first layer evaluates smaller blocks of pixels
  - The next layer evaluates windows of outputs from the first layer
  - This effectively evaluates the larger window of the original image

Distributing the scan



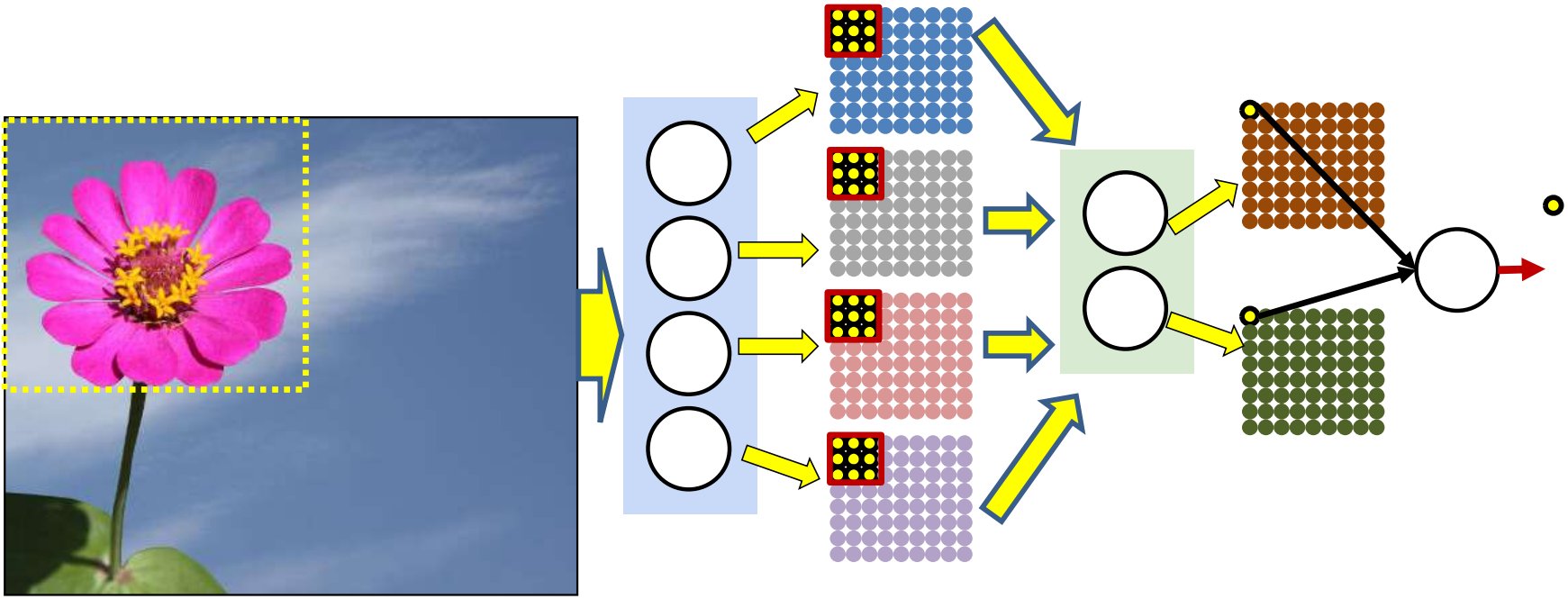
- The window has been *distributed* over two layers
- The higher layer implicitly learns the *arrangement* of sub patterns that represents the larger pattern (the flower in this case)

Distributing the scan



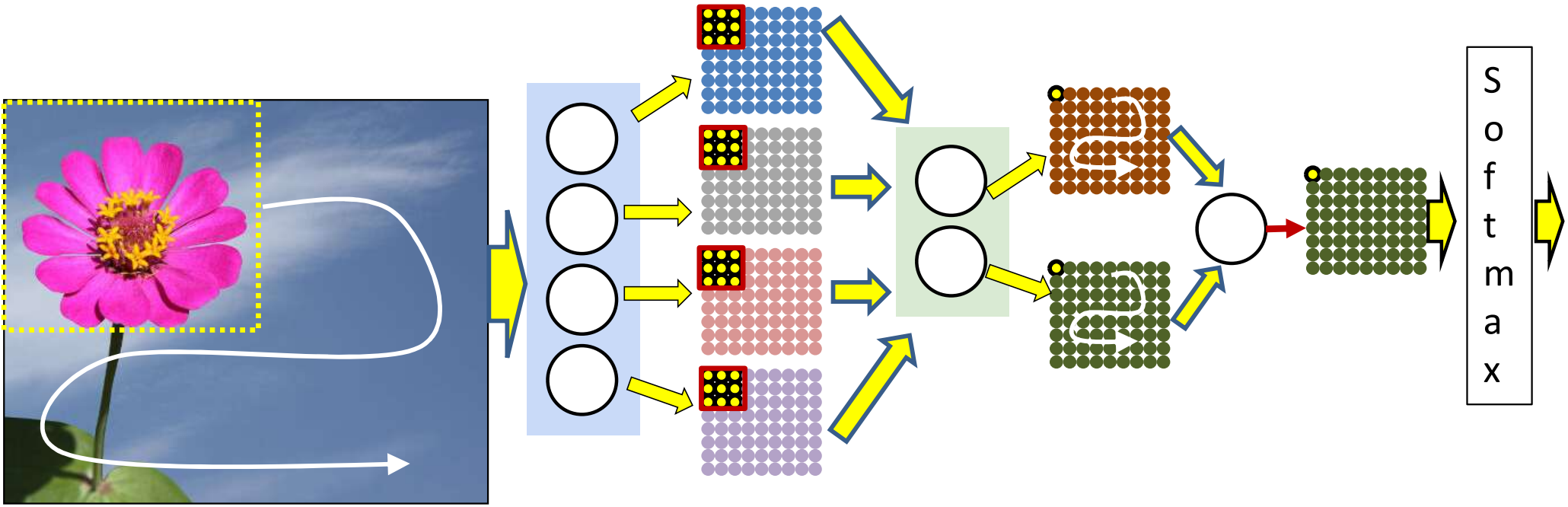
- If second layer neurons scan the maps output by first-layer neurons, they effectively scan the input with the full-sized window
  - *Jointly* scan all the first-layer maps
  - Each output of the second-layer neuron represents the output for *one* full-sized input window

Distributing the scan



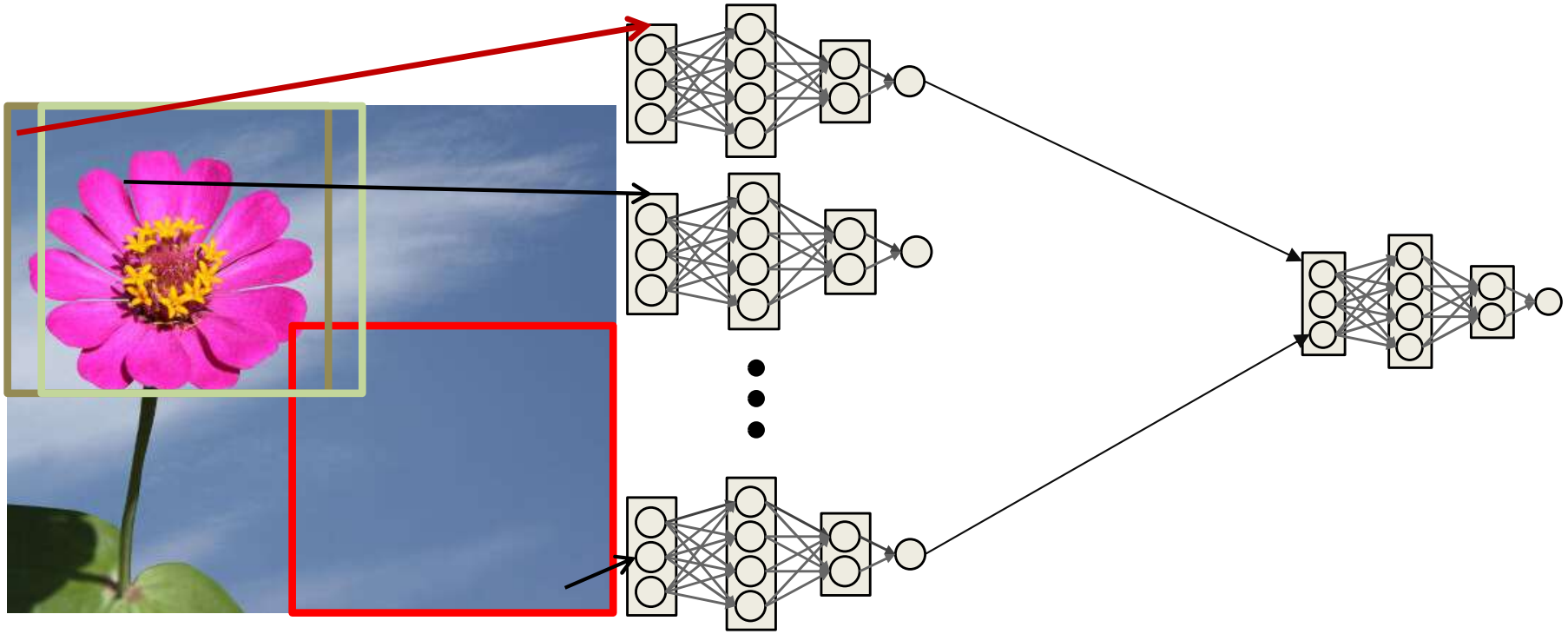
- If second layer neurons (jointly) scan the maps output by first-layer neurons, they effectively scan the input with the full-sized window
  - Each output of the second-layer neuron represents the output for *one* full-sized input window
- To compute the MLP output for a window of input, the output neuron only needs to consider the corresponding outputs of second-layer maps

Distributing the scan



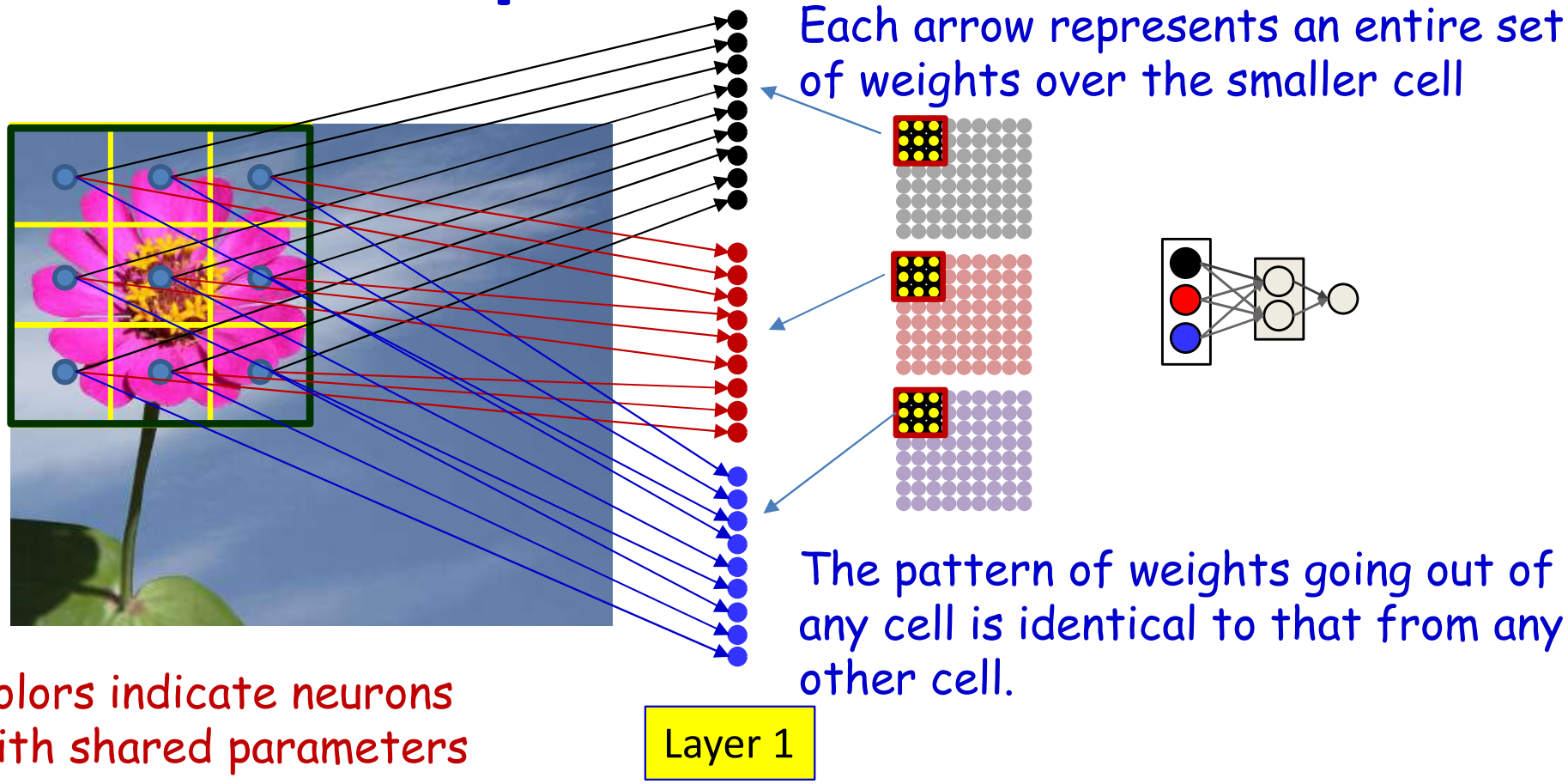
- If second layer neurons (jointly) scan the maps output by first-layer neurons, they effectively scan the input with the full-sized window
  - Each output of the second-layer neuron represents the output for *one* full-sized input window
- To compute the MLP output for a window of input, the output neuron only needs to consider the corresponding outputs of second-layer maps
- The output neuron can compute its outputs for every window in the input from the values of the second layer maps (and send it to a subsequent softmax)

This is *still* just scanning, but with a shared parameter network



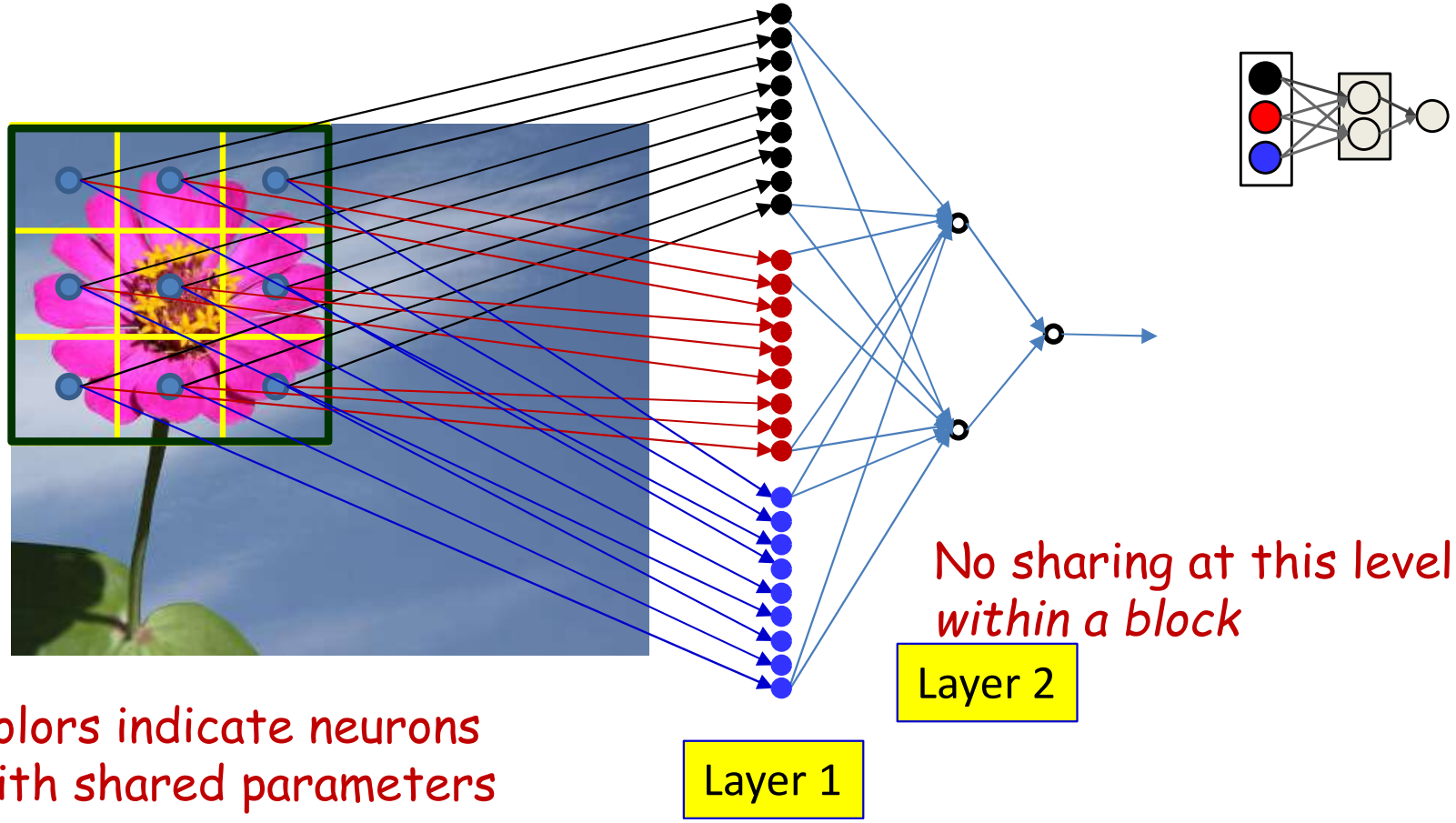
- With a minor modification…

This is *still* just scanning, but with a shared parameter network



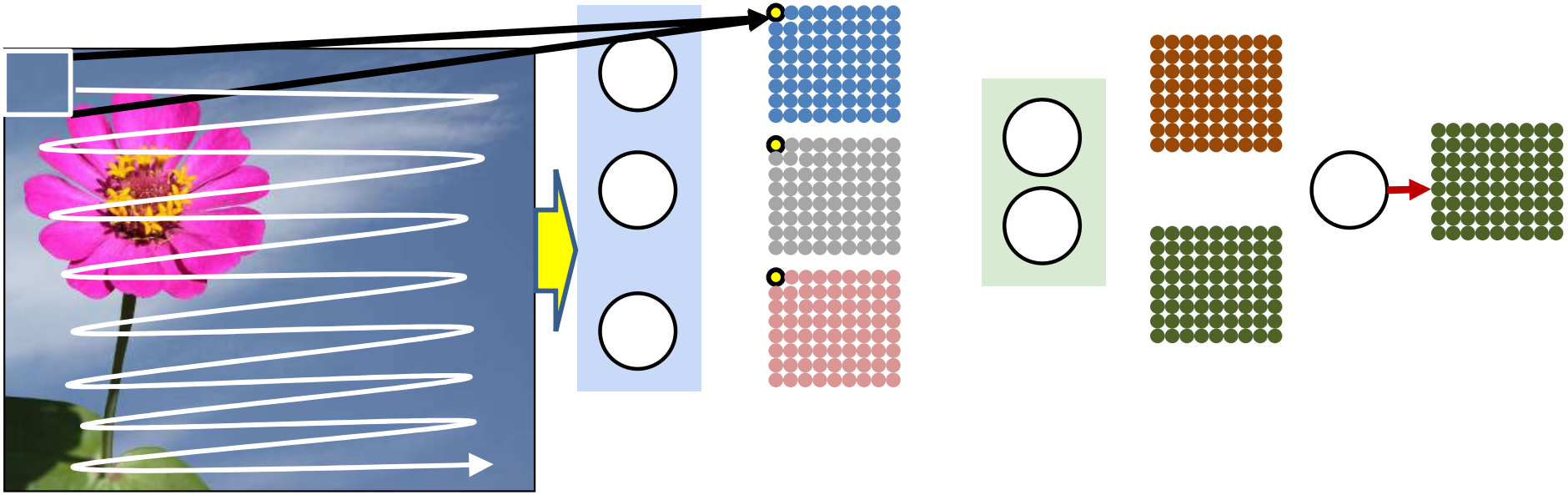
- The network that analyzes individual blocks is now itself a shared parameter network..

This is *still* just scanning with a shared parameter network



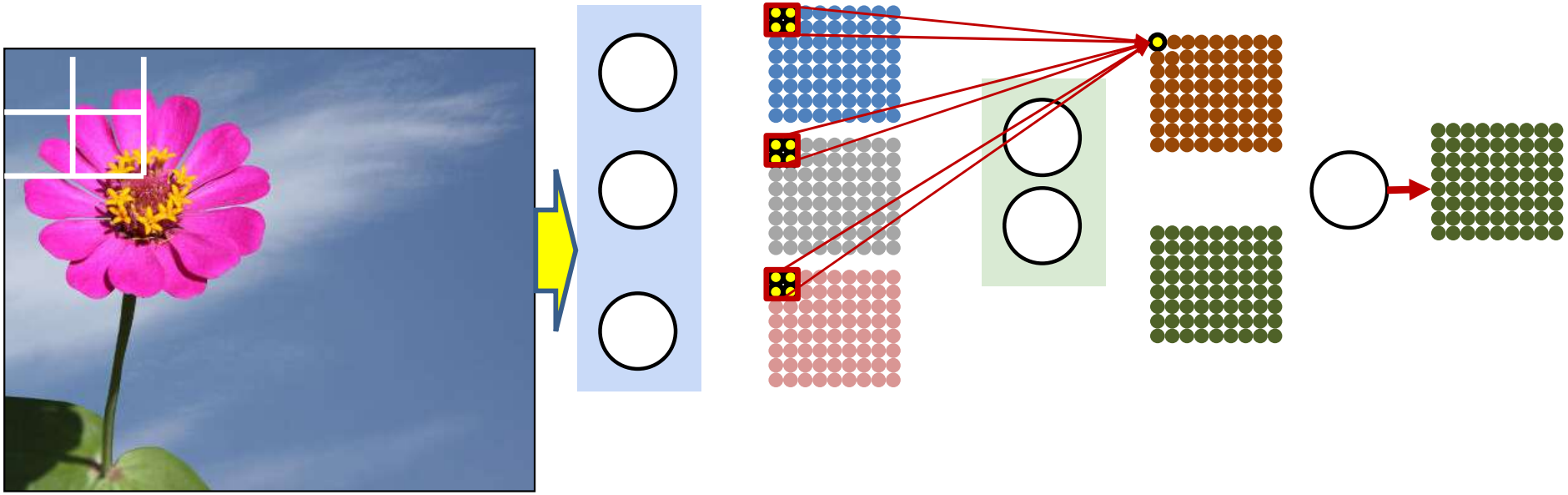
- The network that analyzes individual blocks is now itself a shared parameter network..

This logic can be recursed



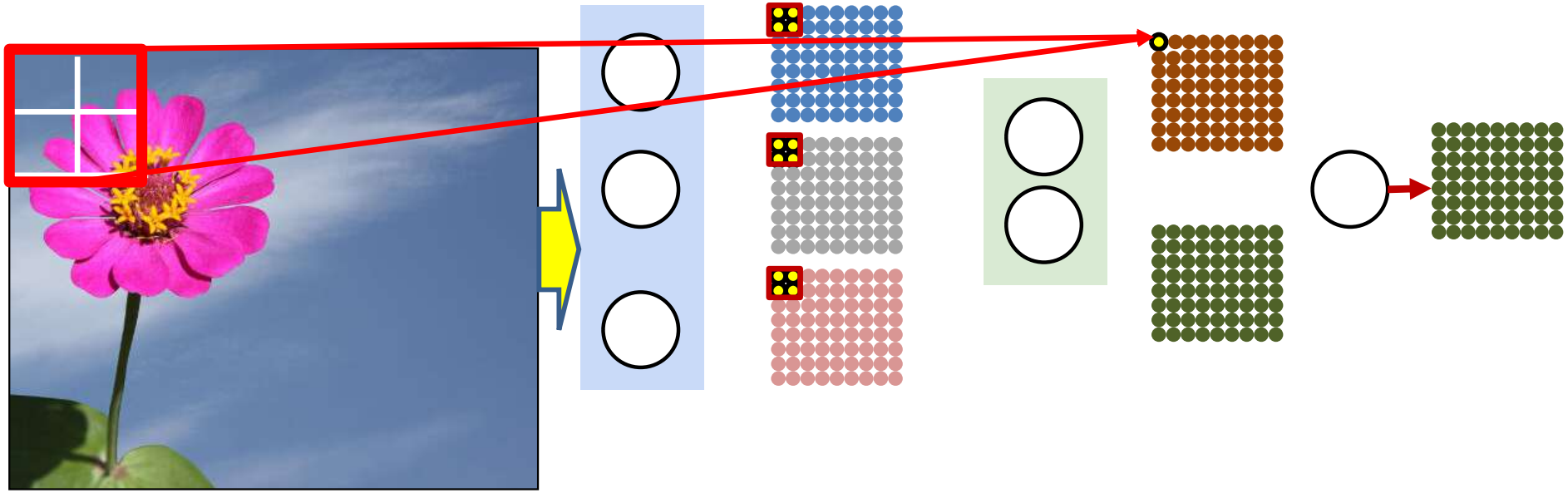
- Building the pattern over 3 layers

This logic can be recursed



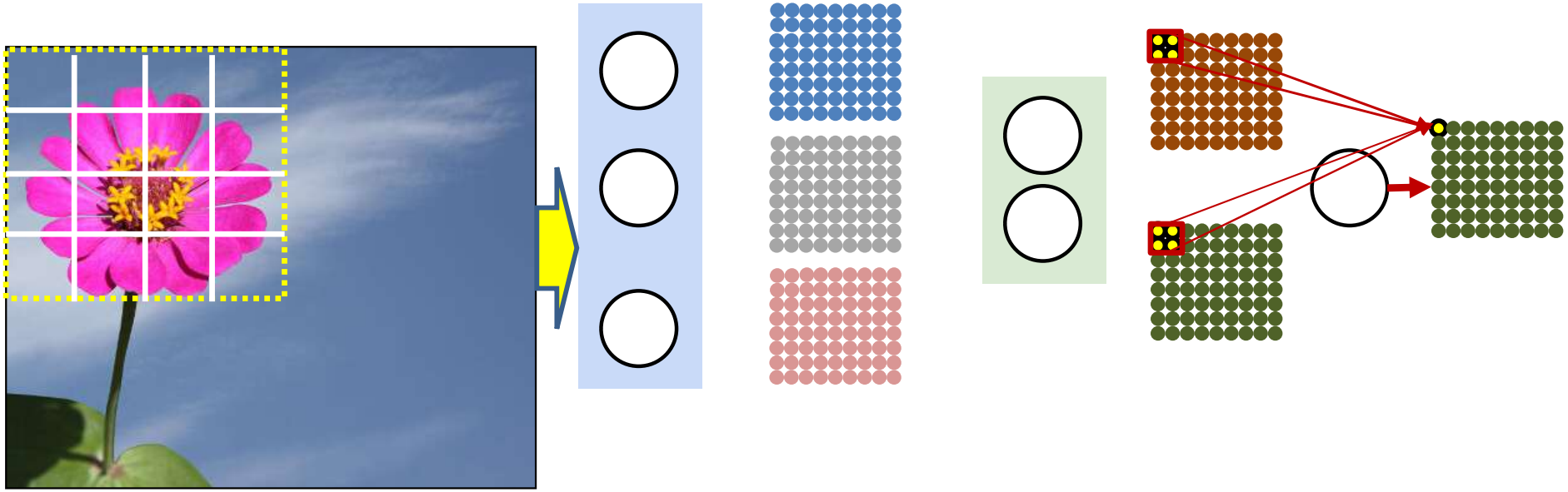
- Building the pattern over 3 layers

This logic can be recursed



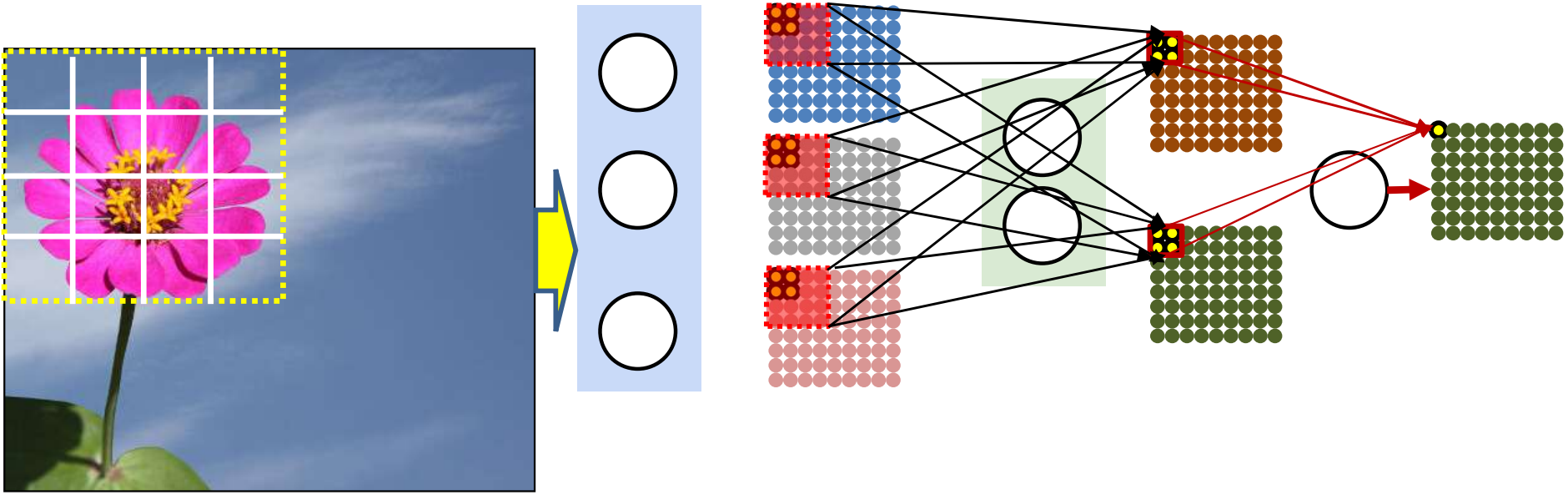
- Building the pattern over 3 layers

This logic can be recursed



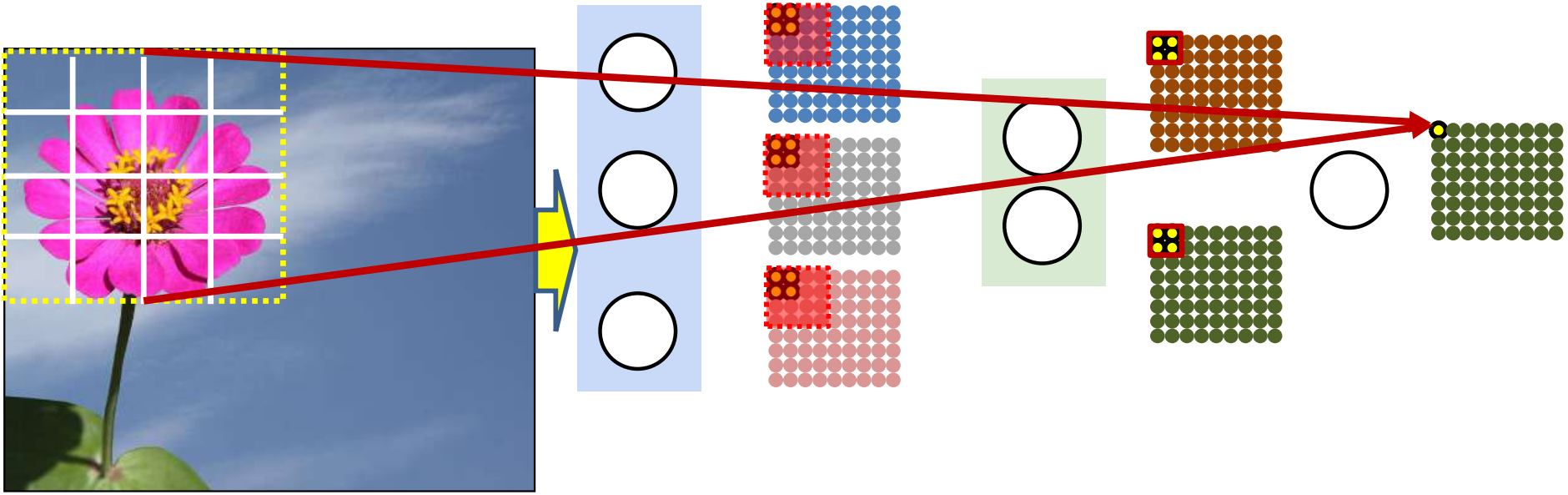
- Building the pattern over 3 layers

This logic can be recursed



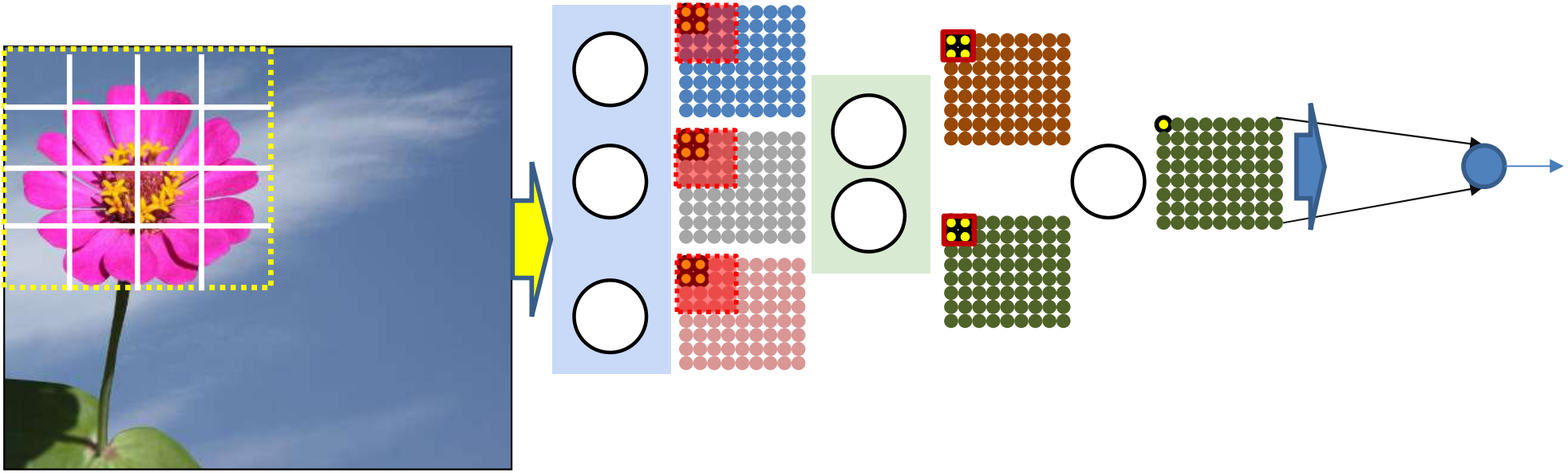
- Building the pattern over 3 layers

This logic can be recursed



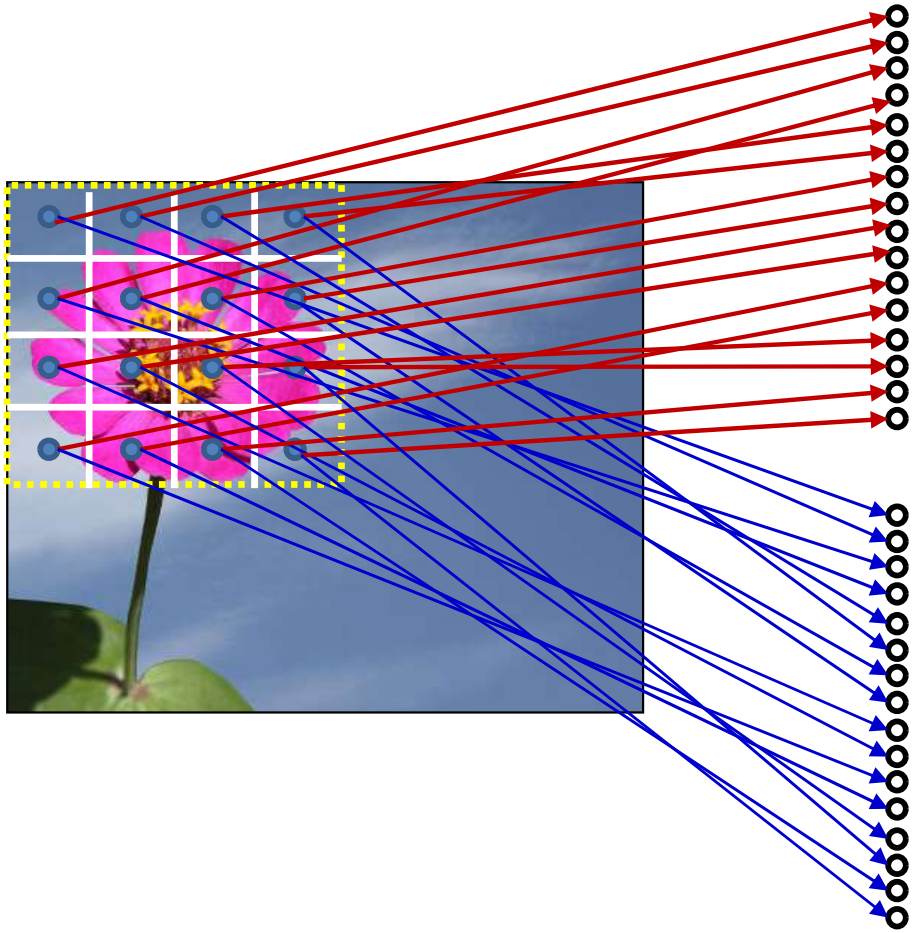
- Building the pattern over 3 layers

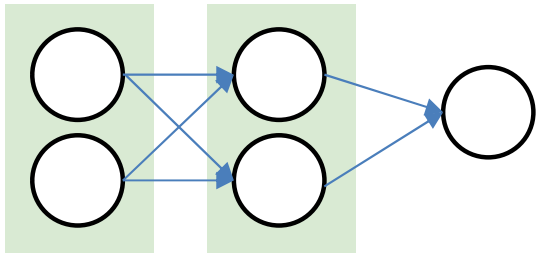
Does the picture have a flower



- Building the pattern over 3 layers
- The final classification for the entire image views the outputs from all locations, as seen in the final map

The 3-layer shared parameter net

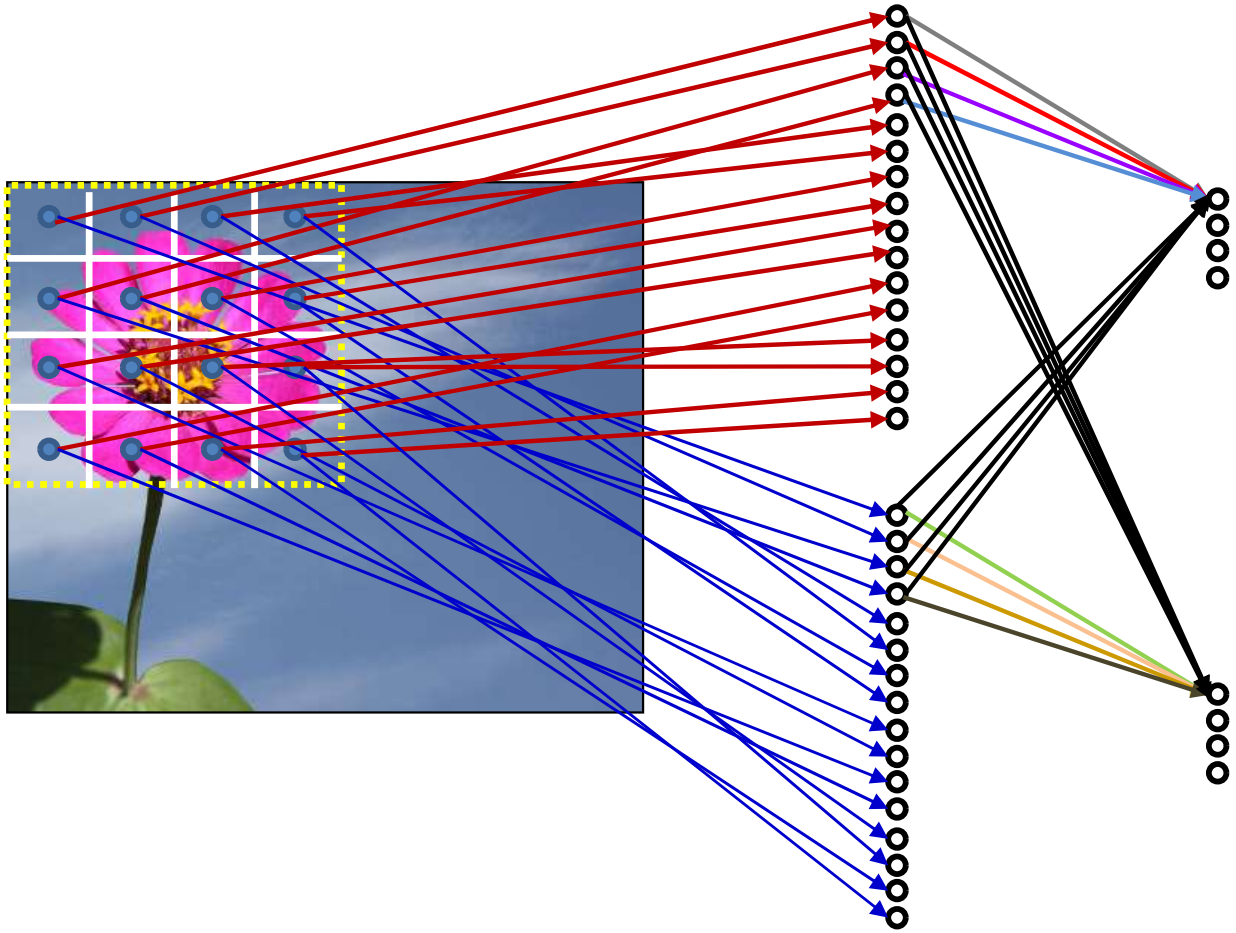




Showing a simpler 2x2x1 network to fit on the slide

- Building the pattern over 3 layers

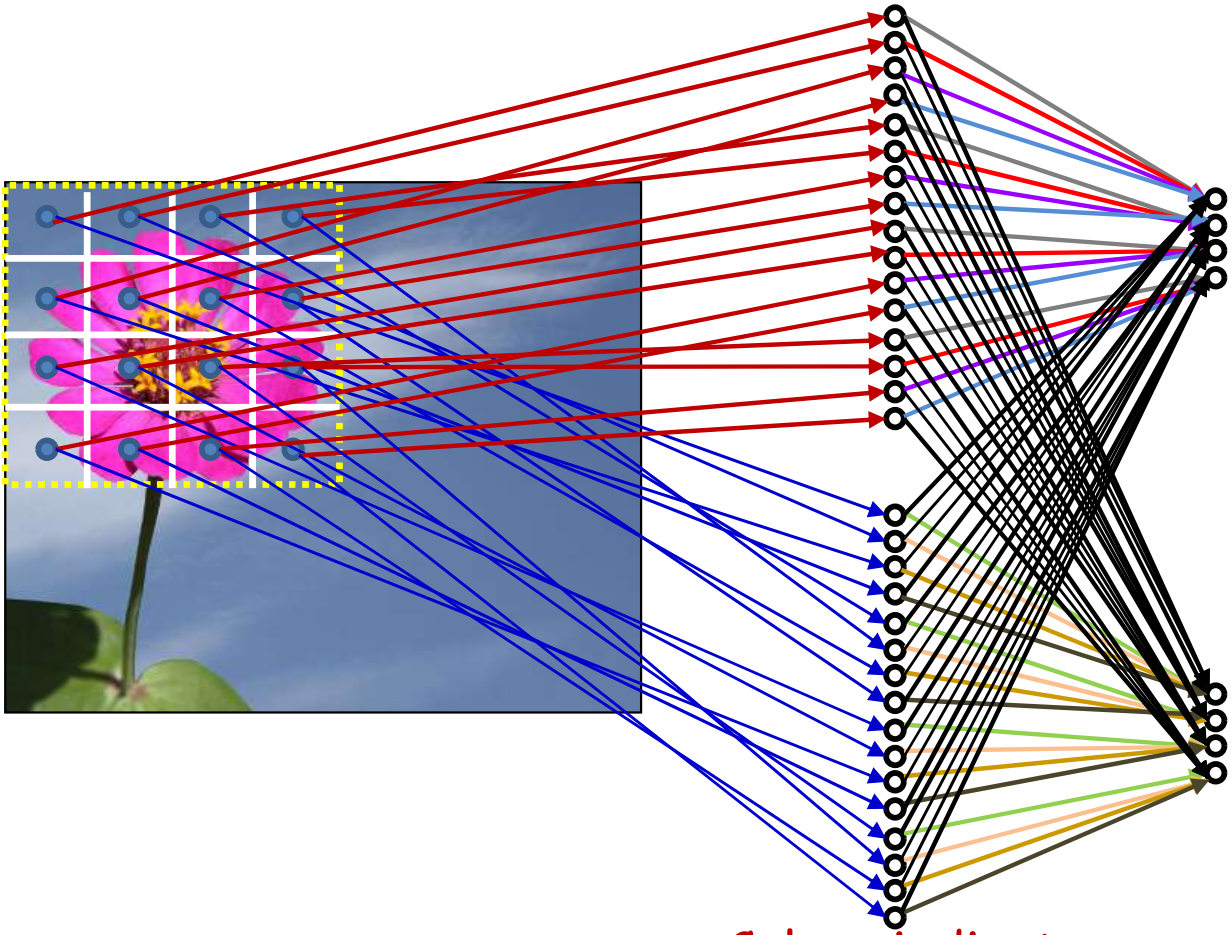
The 3-layer shared parameter net



All weights shown are unique

- Building the pattern over 3 layers

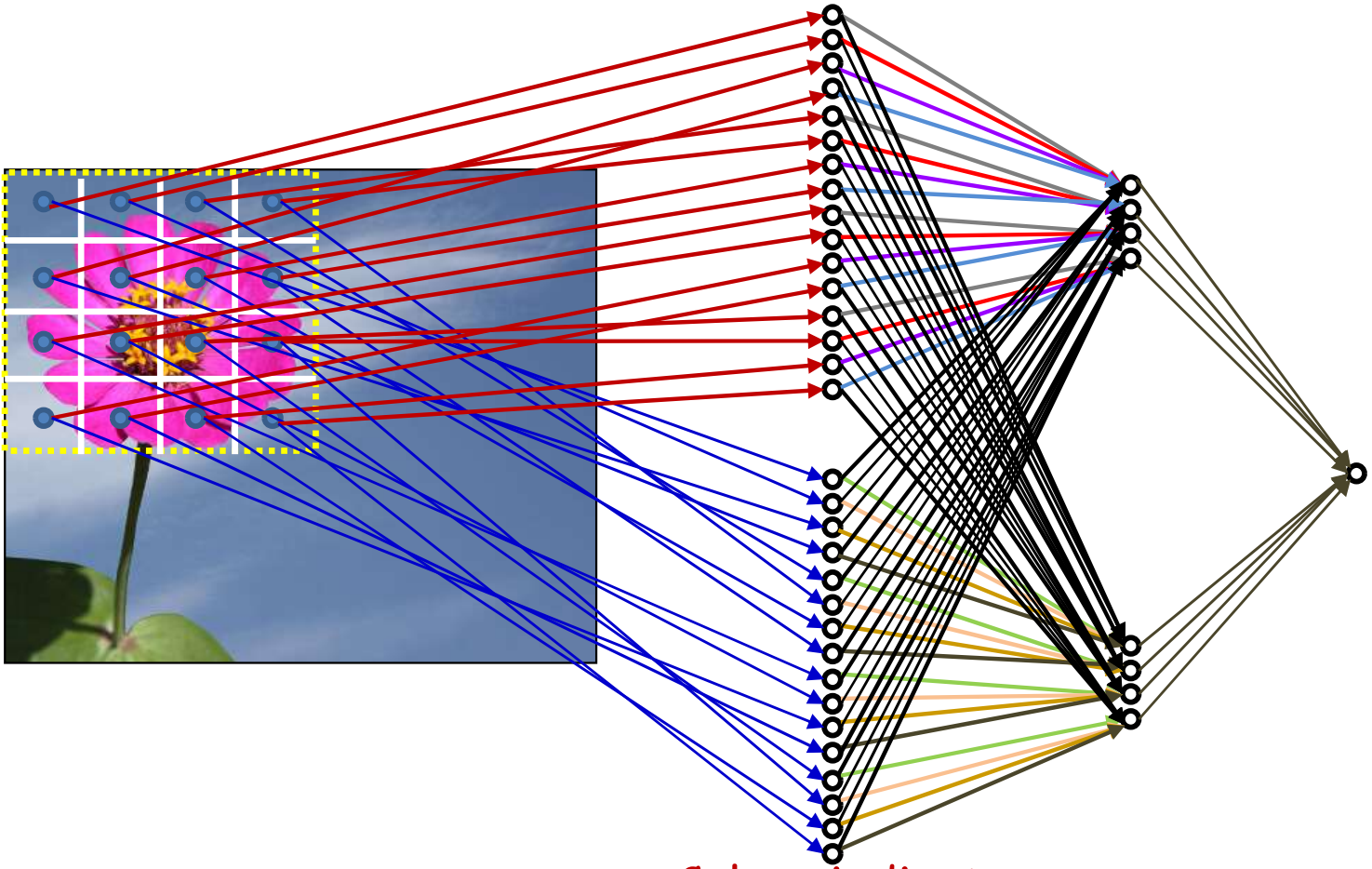
The 3-layer shared parameter net



Colors indicate shared parameters

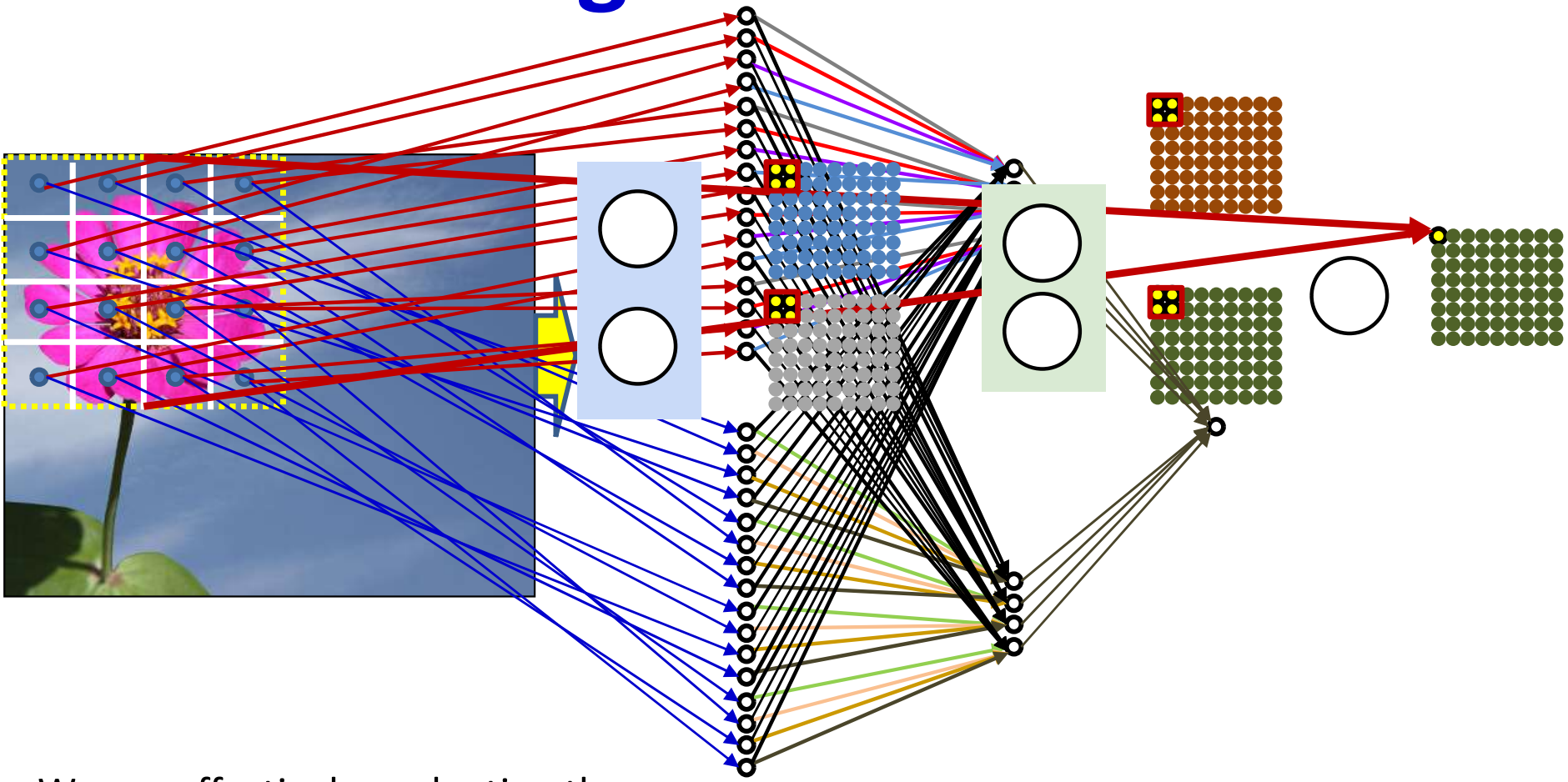
- Building the pattern over 3 layers

The 3-layer shared parameter net



- Building the pattern over 3 layers

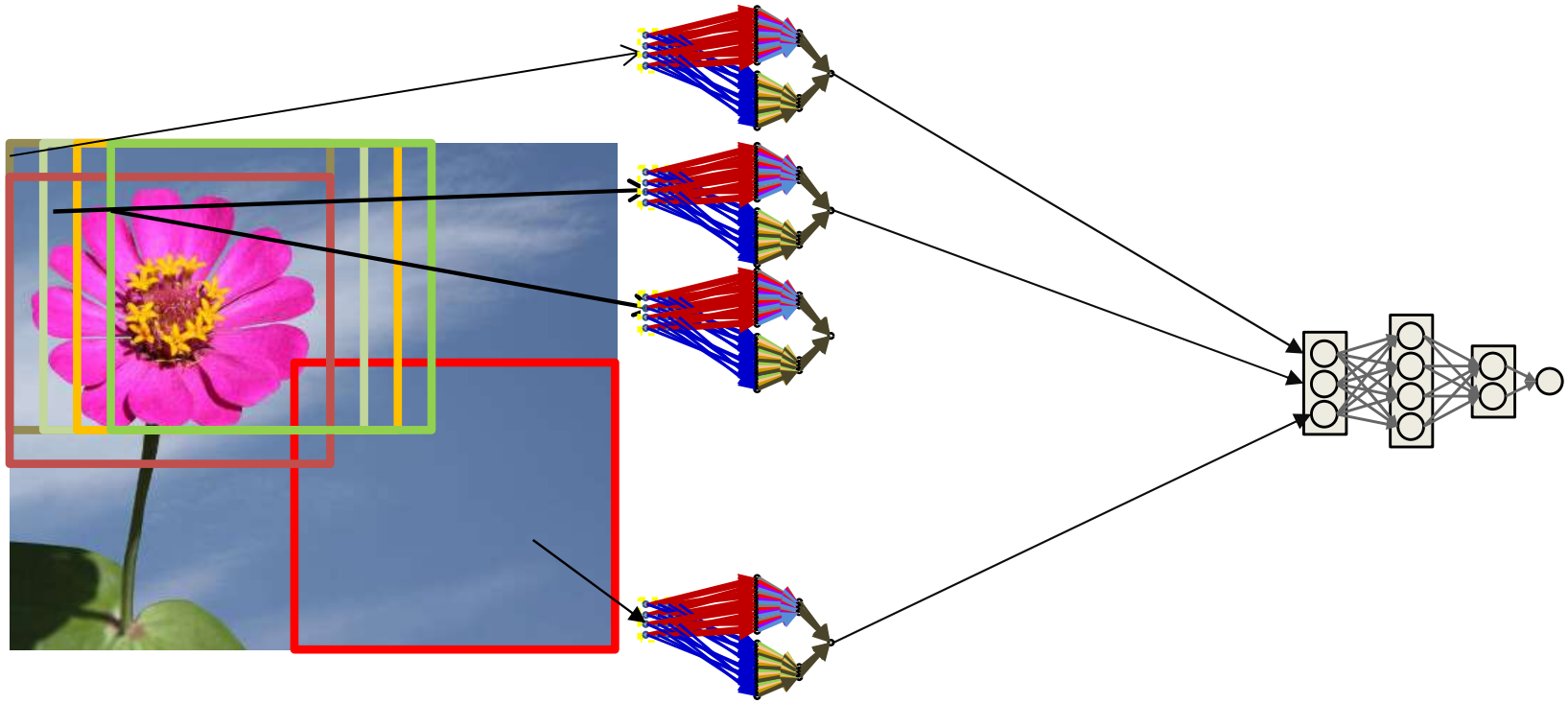
This logic can be recursed



We are effectively evaluating the yellow block with the shared parameter net to the right

Every block is evaluated using the same net in the overall computation

Using hierarchical build-up of features



- The individual blocks are now themselves shared-parameter networks
- We scan the figure using the shared parameter network
- The entire operation can be viewed as a single giant network
  - Where individual subnets are themselves shared-parameter nets

Scanning with an MLP (2D) (without distribution)

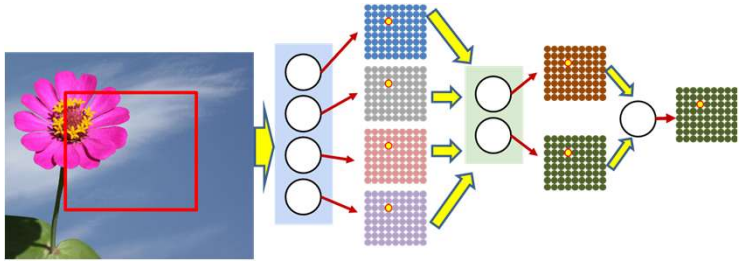
- `KxK` = size of "window" evaluated by MLP
- `W` is width of image
- `H` is height of image



```
for x = 1:W-K+1
   for y = 1:H-K+1
      ImgSegment = Img(*, x:x+K-1, y:y+K-1)
      Y(x,y) = MLP(ImgSegment)

Y = softmax( Y(1,1)..Y(W-K+1,H-K+1) )
```

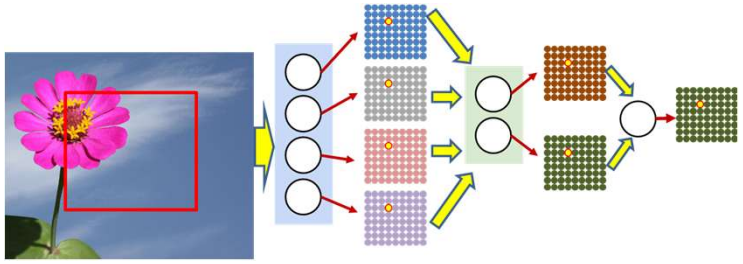
Scanning with an MLP (2D) (without distribution)



```
for x = 1:W-K+1
    for y = 1:H-K+1
        for l = 1:L  # layers
            for (l==1)
                Segment = Y(0,1:C,x:x+K-1,y:y+K-1)
            else
                Segment = Y(l-1,1:D_{l-1},x,y)
            for j = 1:D_l
                Compute z(l,j,x,y)from Segment
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

Scanning with an MLP (2D) (without distribution)



```
for l = 1:L  # layers
    for x = 1:W-K+1
        for y = 1:H-K+1
            for (l==1)
                Segment = Y(0,1:C,x:x+K-1,y:y+K-1)
            else
                Segment = Y(l-1,1:D_{l-1},x,y)
            for j = 1:D_l
                Compute z(l,j,x,y) [from Segment]
                Y(l,j,x,y) = activation(z(l,j,x,y))

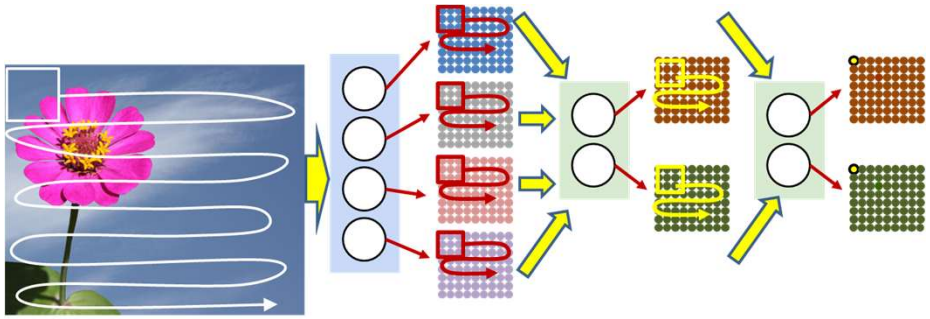
Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

Scanning with an MLP (2D) (without distribution)

Each layer now scans the output maps from the previous layer in windows of K_lxK_l

```
for l = 1:L  # layers
    for x = 1:W_{l-1}-K_l+1
        for y = 1:H_{l-1}-K_l+1
            Segment = Y(l-1,1:D_{l-1},x:x+K_l-1,y:y+K_l-1)
            for j = 1:D_l
                Compute z(l,j,x,y) from Segment
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

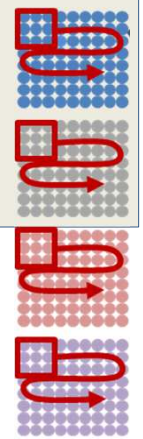


Convolution

```
for l = 1:L  # layers
    for x = 1:W_{l-1}-K_l+1
        for y = 1:H_{l-1}-K_l+1
            Segment = Y(l-1,1:D_{l-1},x:x+K_l-1,y:y+K_l-1)
            for j = 1:D_l
                Compute z(l,j,x,y) from Segment
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

This operation is a "convolution"

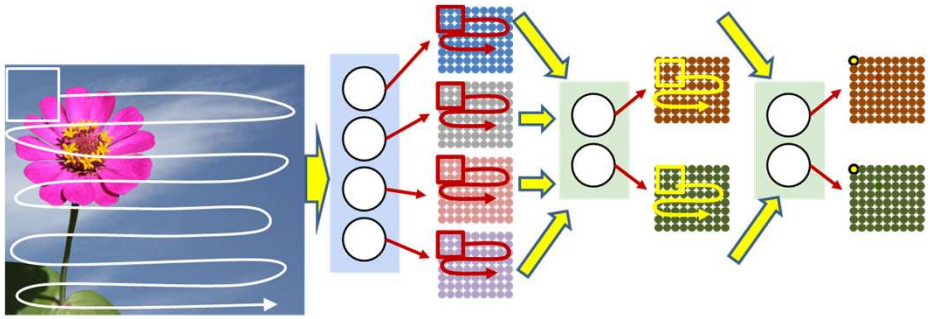


Convolutional neural network

This is a "convolutional neural net"

```
for l = 1:L  # layers
    for x = 1:W_{l-1}-K_l+1
        for y = 1:H_{l-1}-K_l+1
            Segment = Y(l-1,1:D_{l-1},x:x+K_l-1,y:y+K_l-1)
            for j = 1:D_l
                Compute z(l,j,x,y) from Segment
                Y(l,j,x,y) = activation(z(l,j,x,y))

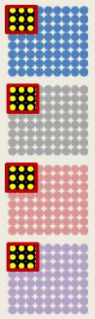
Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```



Reordered scanning with distribution

```
Y(0,:,:,:) = Image
for l = 1:L  # layers operate on vector at (x,y)
    for x = 1:W_{l-1}-K_l+1
        for y = 1:H_{l-1}-K_l+1
            for j = 1:D_l
                z(l,j,x,y) = b(l,j)
                for i = 1:D_{l-1}
                    for x' = 1:K_l
                        for y' = 1:K_l
                            z(l,j,x,y) += w(l,i,j,x',y')
                                          Y(l-1,i,x+x'-1,y+y'-1)
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

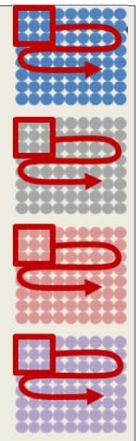


Reordered scanning with distribution

```
Y(0,:,:,:) = Image
for l = 1:L  # layers operate on vector at (x,y)
    for x = 1:W_{l-1}-K_l+1
        for y = 1:H_{l-1}-K_l+1
            for j = 1:D_l
                z(l,j,x,y) = b(l,j)
                for i = 1:D_{l-1}
                    for x' = 1:K_l
                        for y' = 1:K_l
                            z(l,j,x,y) += w(l,i,j,x',y')
                                          Y(l-1,i,x+x'-1,y+y'-1)
            Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```

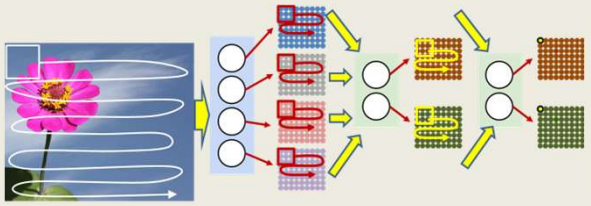
This operation is a "convolution"



"Convolutional Neural Network" (aka scanning with an MLP)

```
Y(0,:,:,:) = Image
for l = 1:L  # layers operate on vector at (x,y)
    for x = 1:W_{l-1}-K_l+1
        for y = 1:H_{l-1}-K_l+1
            for j = 1:D_l
                z(l,j,x,y) = 0
                for i = 1:D_{l-1}
                    for x' = 1:K_l
                        for y' = 1:K_l
                            z(l,j,x,y) += w(l,i,j,x',y')
                                          Y(l-1,i,x+x'-1,y+y'-1)
                Y(l,j,x,y) = activation(z(l,j,x,y))

Y = softmax( Y(L,:,1,1)..Y(L,:,W-K+1,H-K+1) )
```



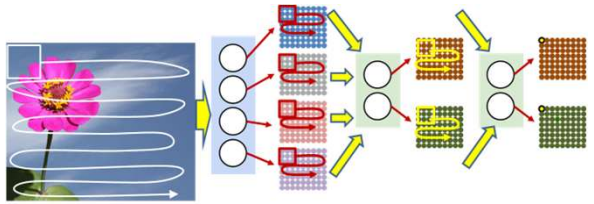
Convolutional neural net: Vector notation

The weight W(l) is now a 4D $D_l \times D_{l-1} \times K_l \times K_l$ tensor (assuming square receptive fields)

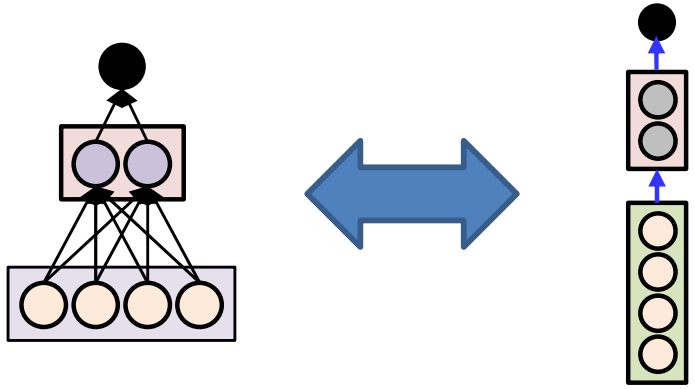
The product in blue is a tensor inner product with a scalar output

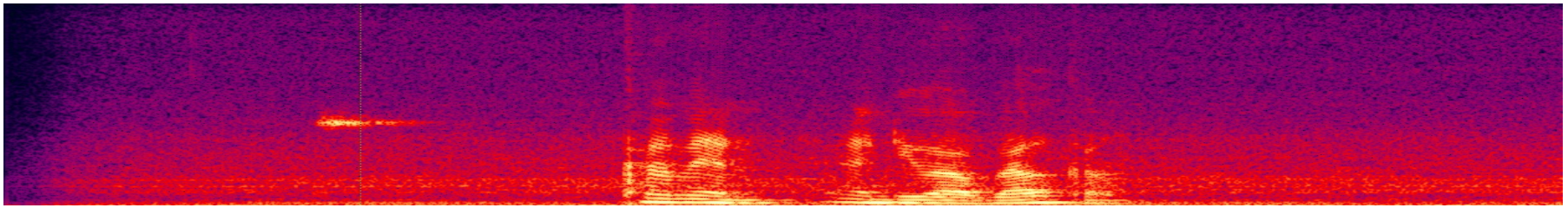
```
Y(0) = Image
for l = 1:L  # layers operate on vector at (x,y)
  for x = 1:W_{l-1}-K_l+1
    for y = 1:H_{l-1}-K_l+1
      segment = Y(l-1,:,x:x+K_l-1,y:y+K_l-1) #3D tensor
      z(l,x,y) = W(l).segment + b(l)
                 #tensor inner prod.
      Y(l,x,y) = activation(z(l,x,y))

Y = softmax( Y(L) )
```



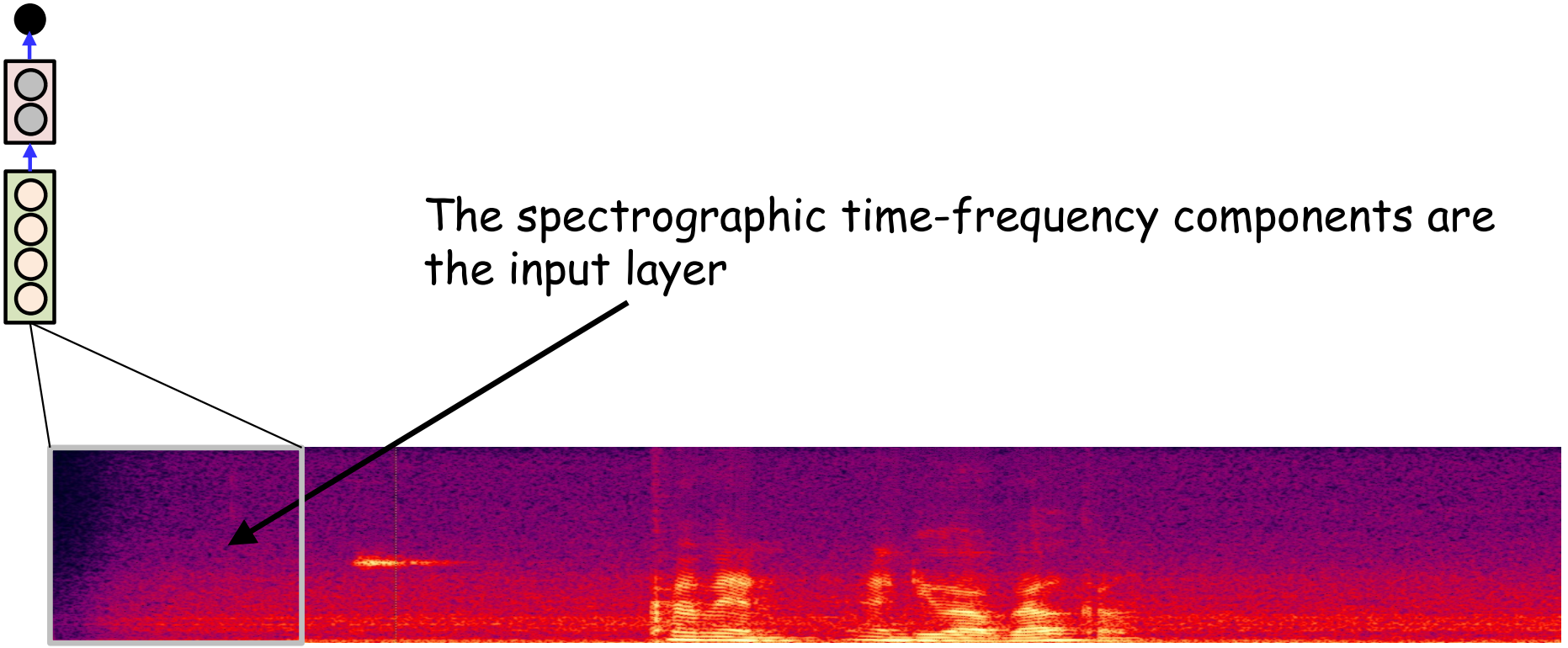
1-D convolution





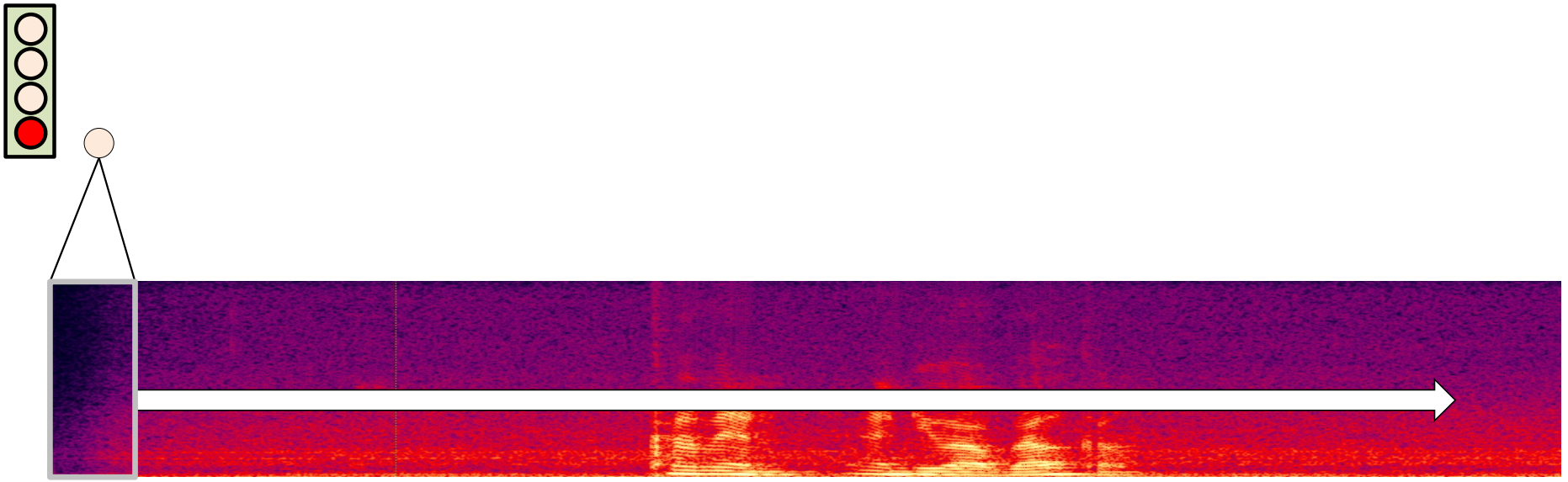
- The 1-D scan version of the convolutional neural network is the *time-delay neural network*
  - Used primarily for speech recognition

1-D scan version



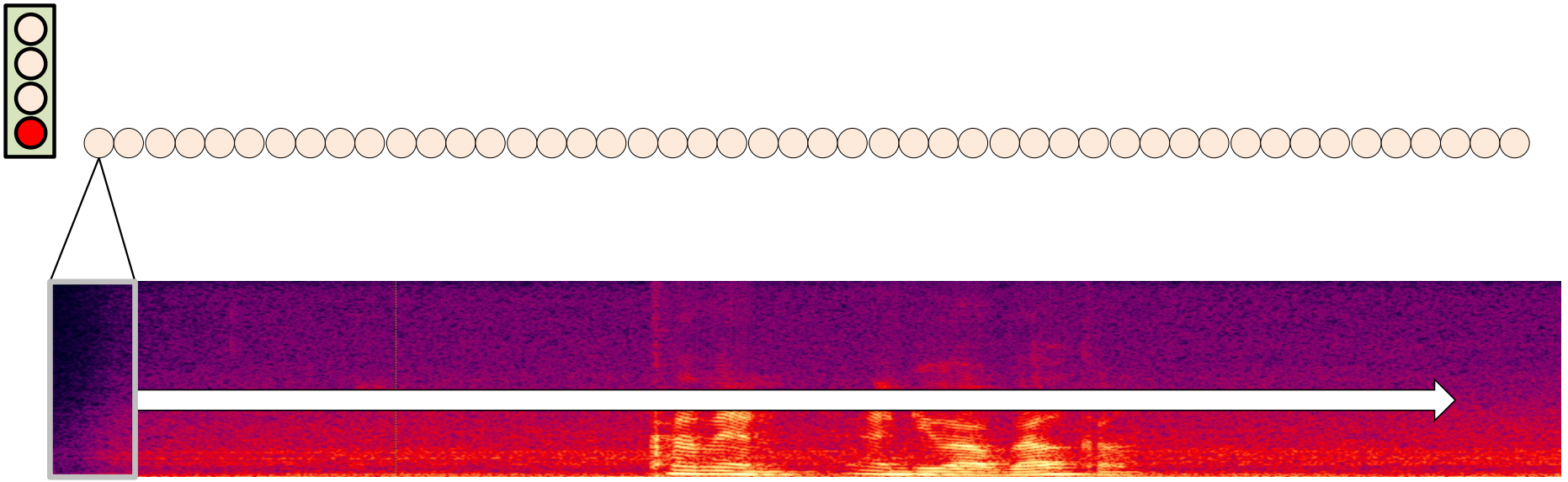
- The 1-D scan version of the convolutional neural network
  - We will scan the entire width heirarchially

1-D scan version



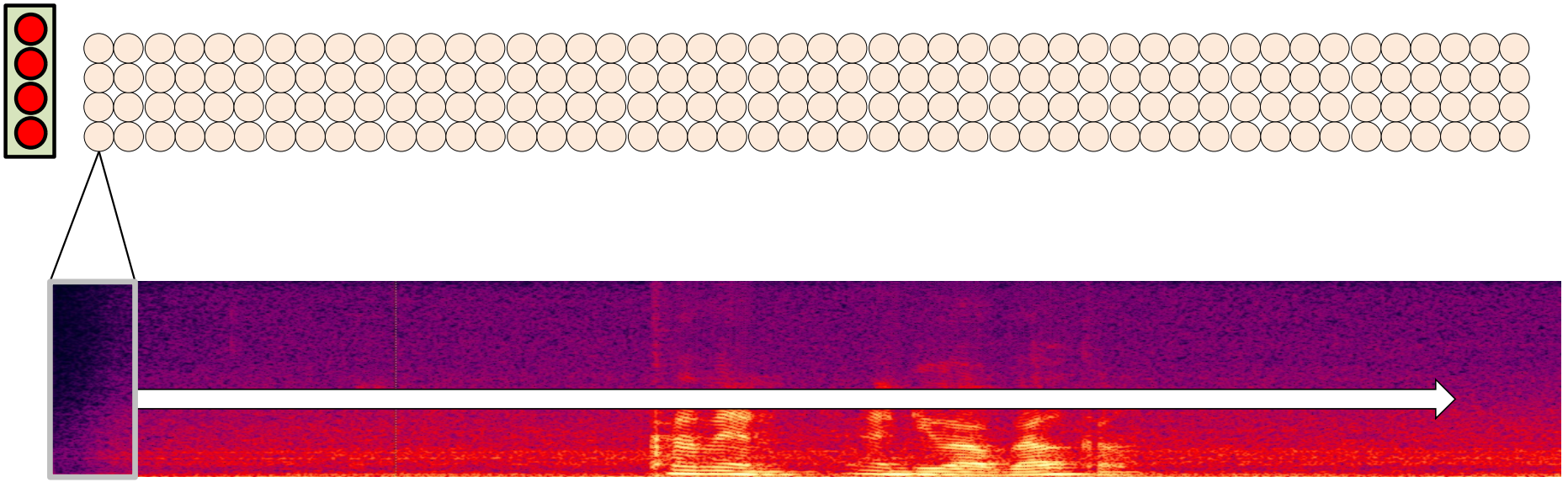
- The 1-D scan version of the convolutional neural network

1-D scan version



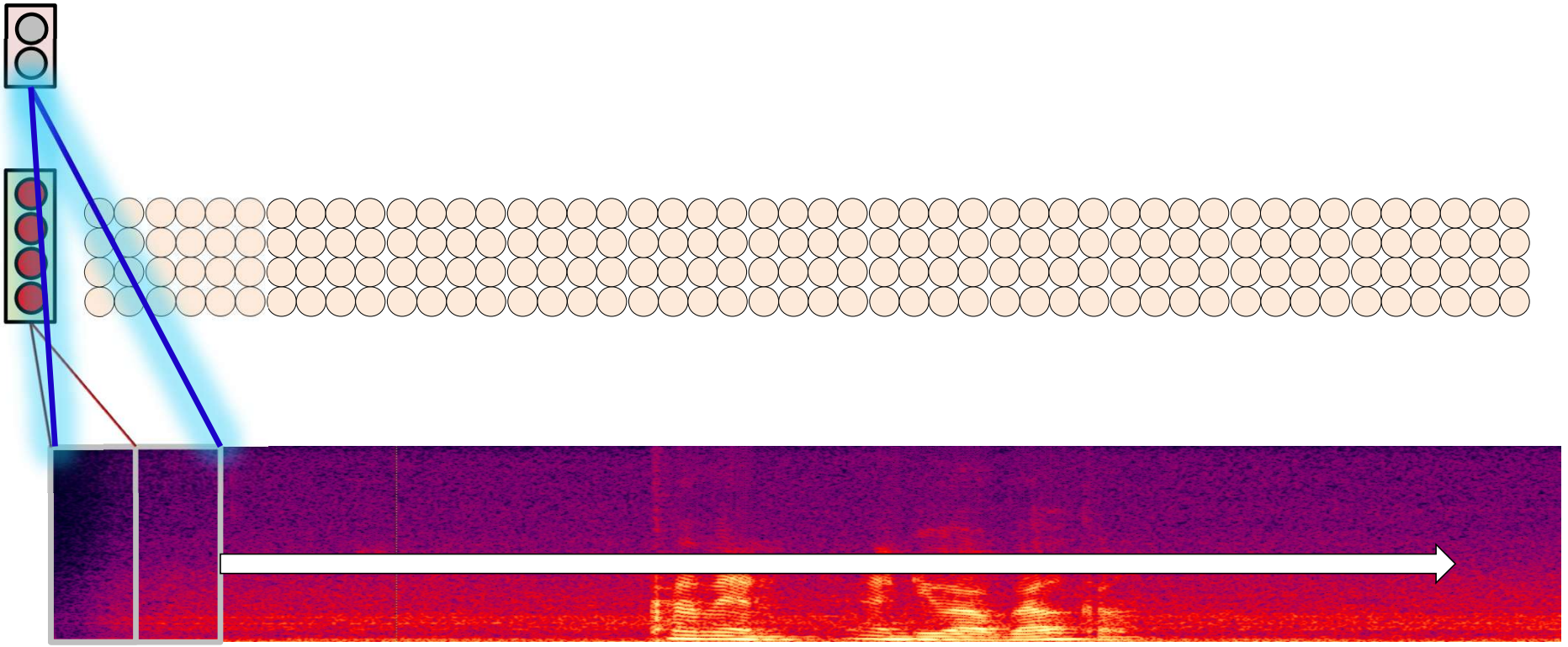
- The 1-D scan version of the convolutional neural network

1-D scan version



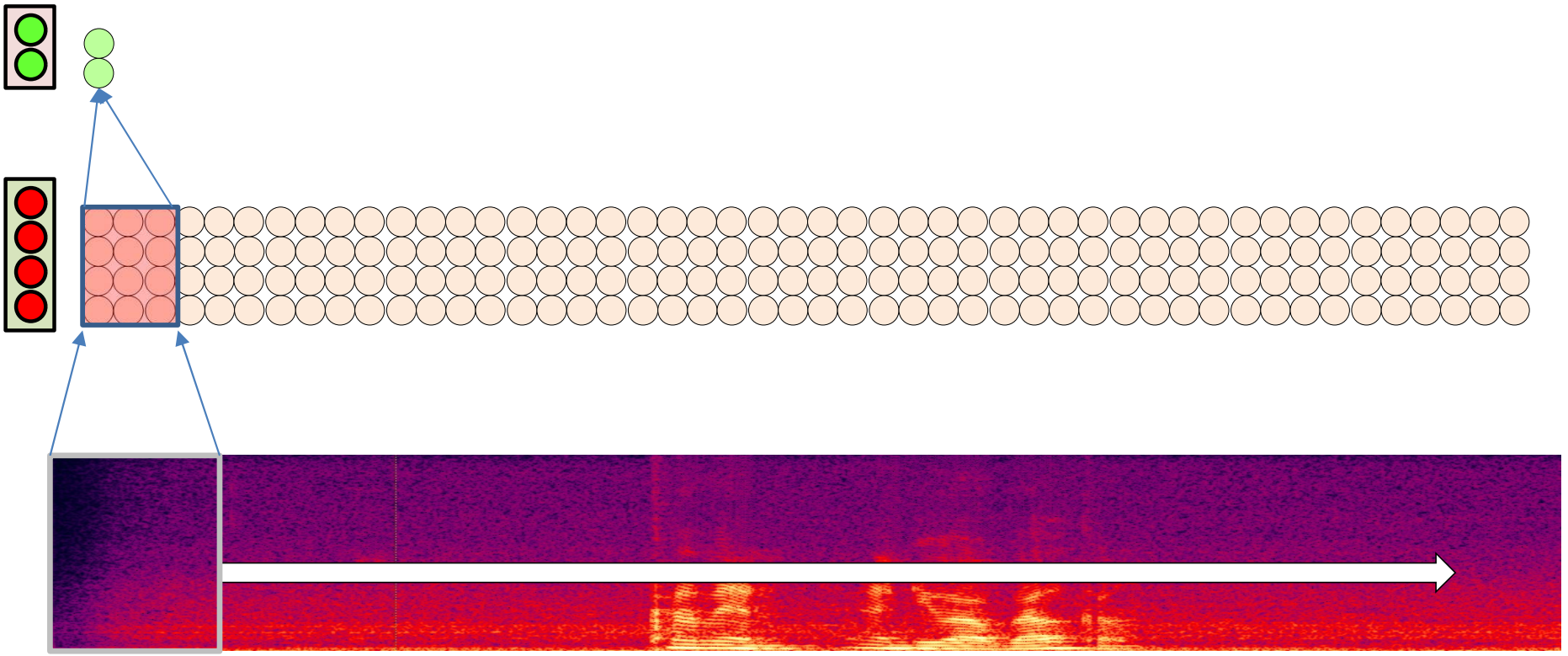
- The 1-D scan version of the convolutional neural network

1-D scan version



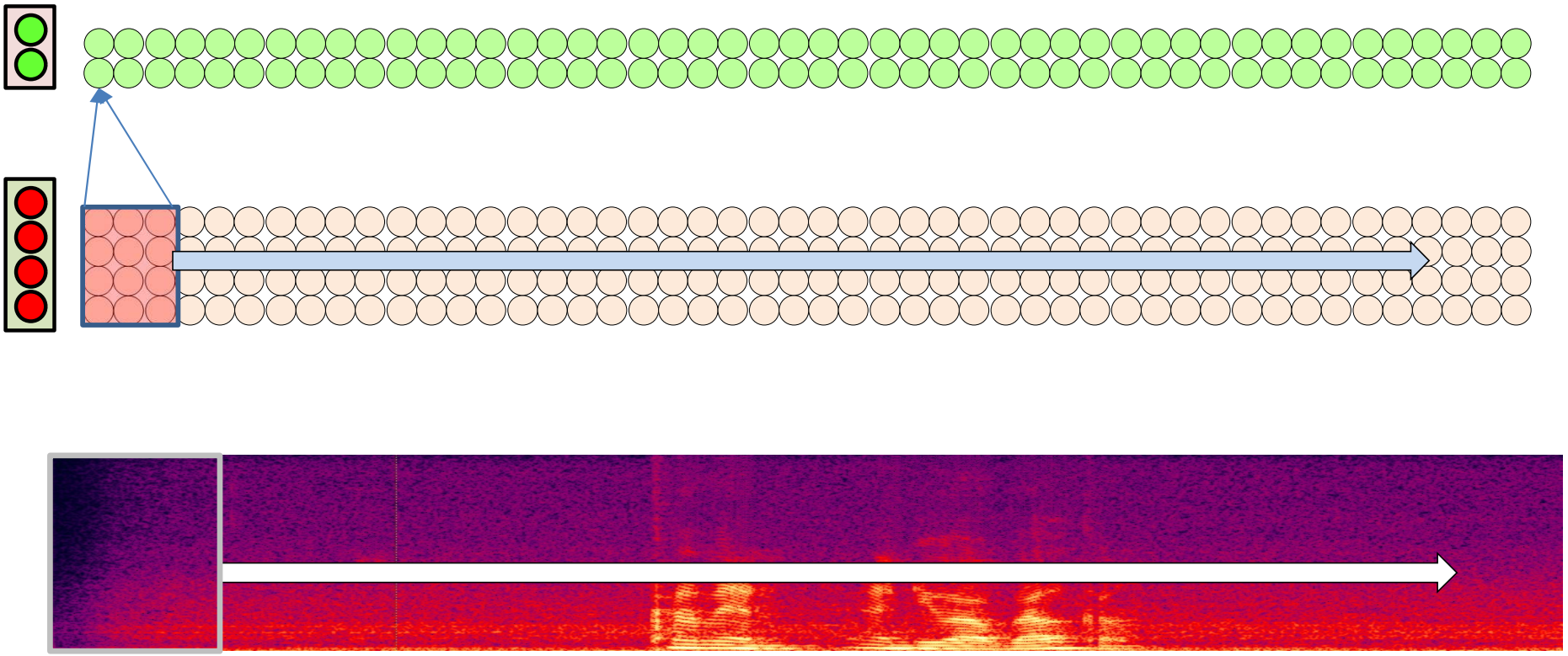
- The 1-D scan version of the convolutional neural network

1-D scan version



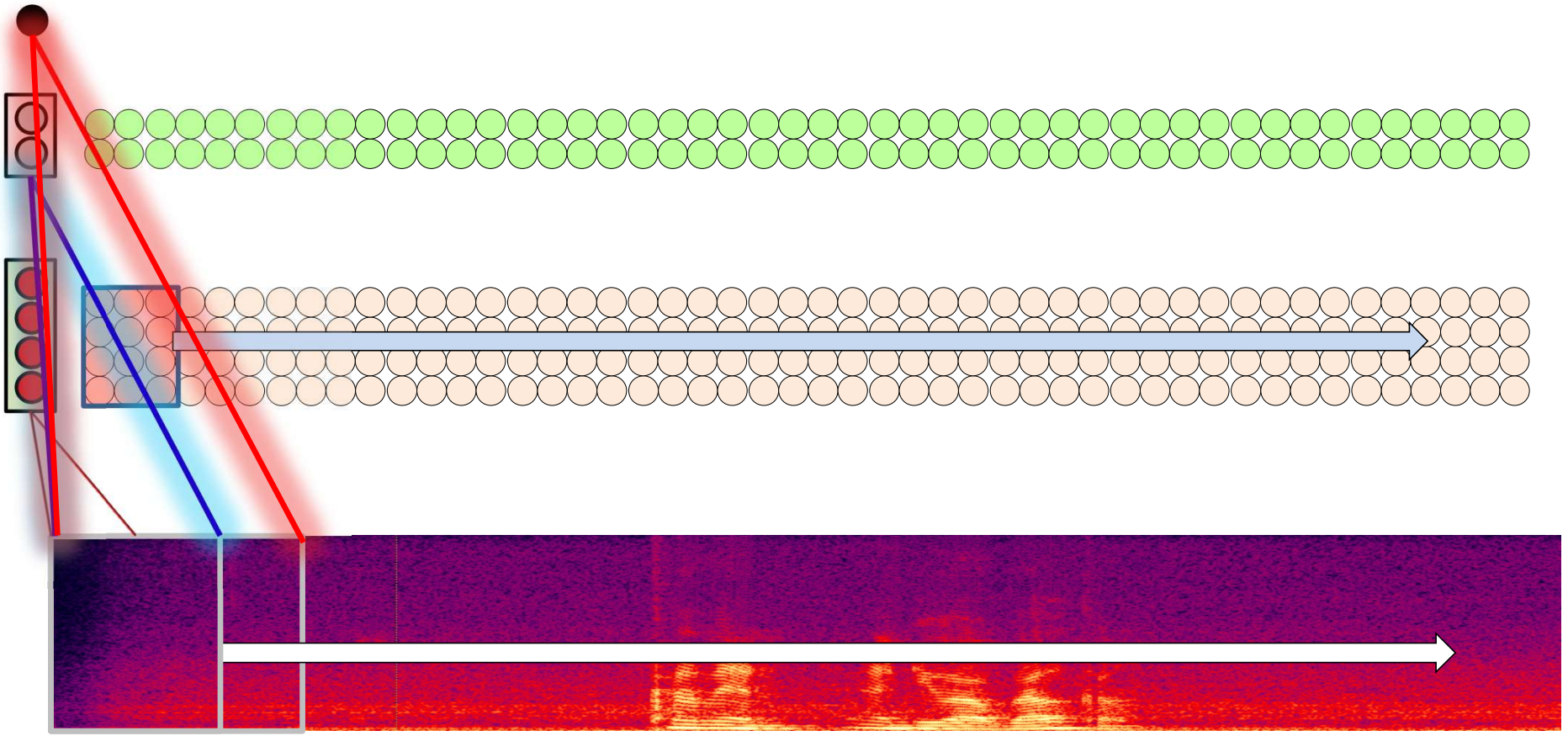
- The 1-D scan version of the convolutional neural network

1-D scan version



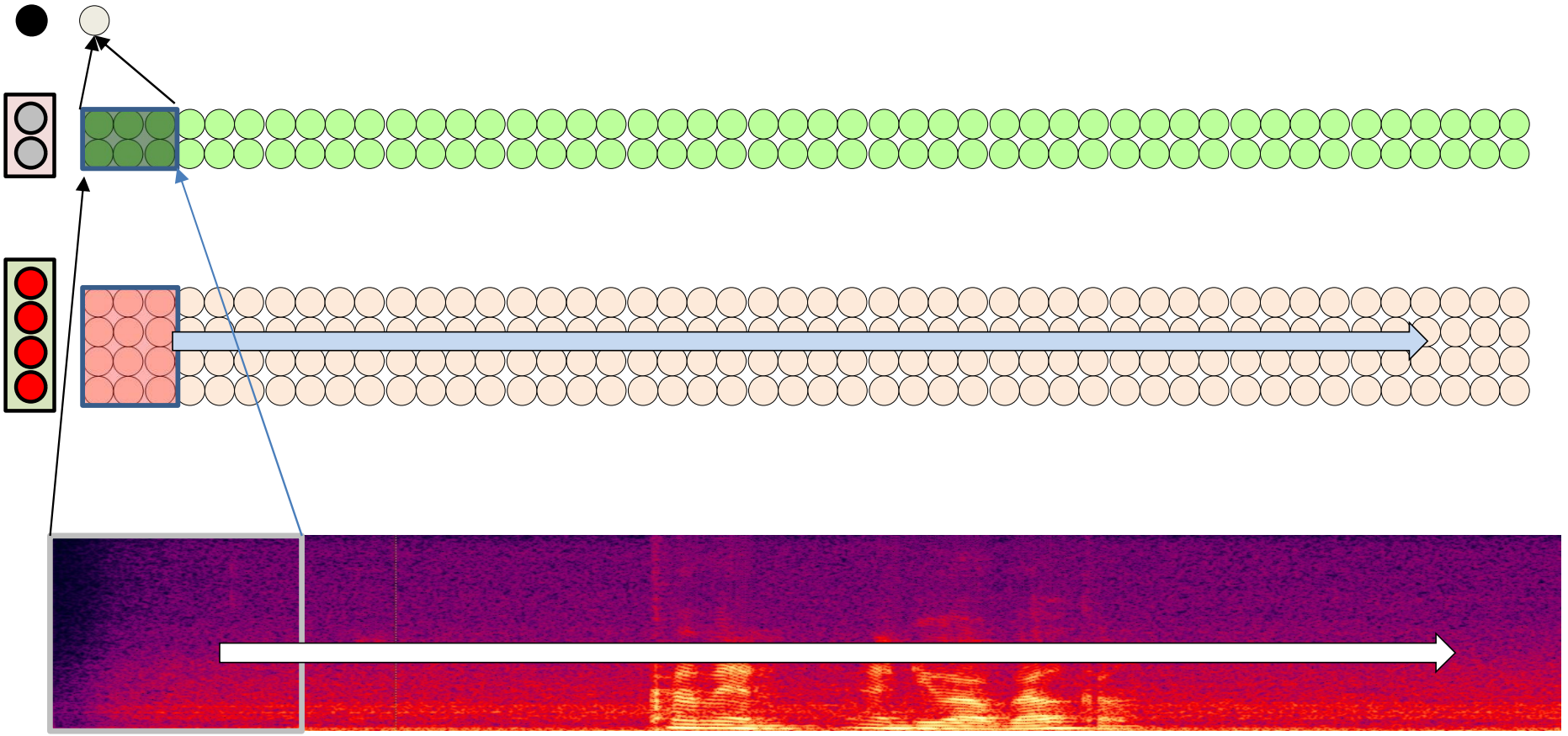
- The 1-D scan version of the convolutional neural network

1-D scan version



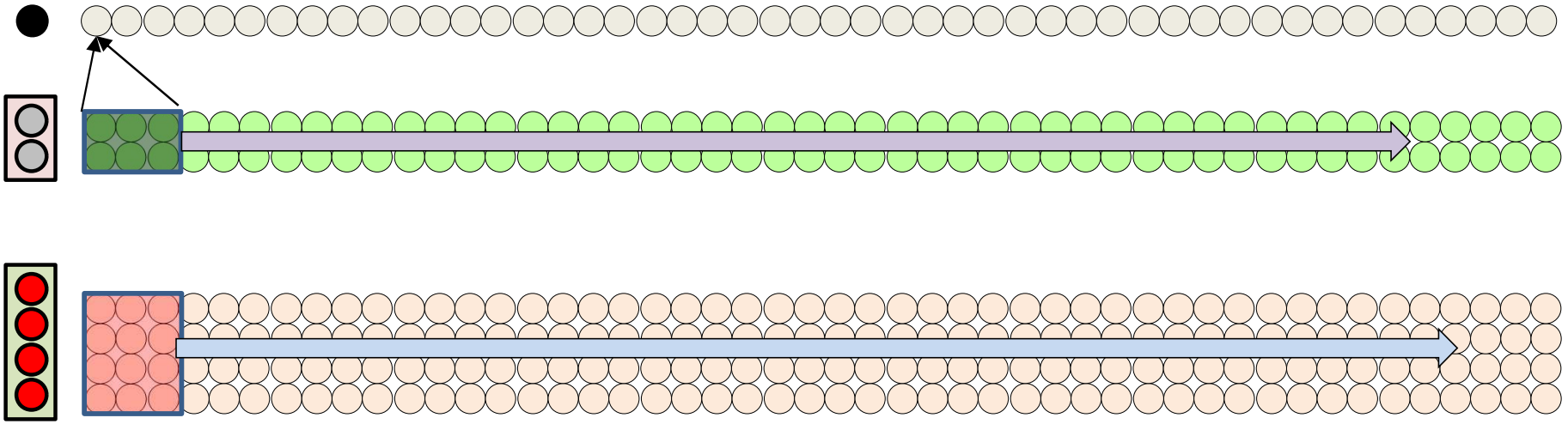
- The 1-D scan version of the convolutional neural network

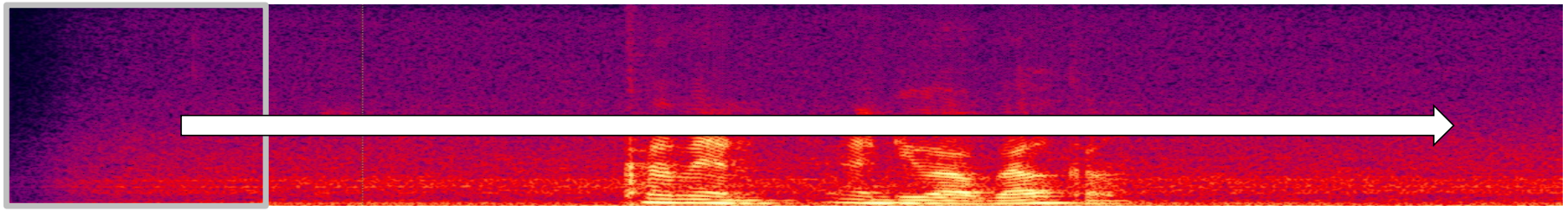
1-D scan version



- The 1-D scan version of the convolutional neural network

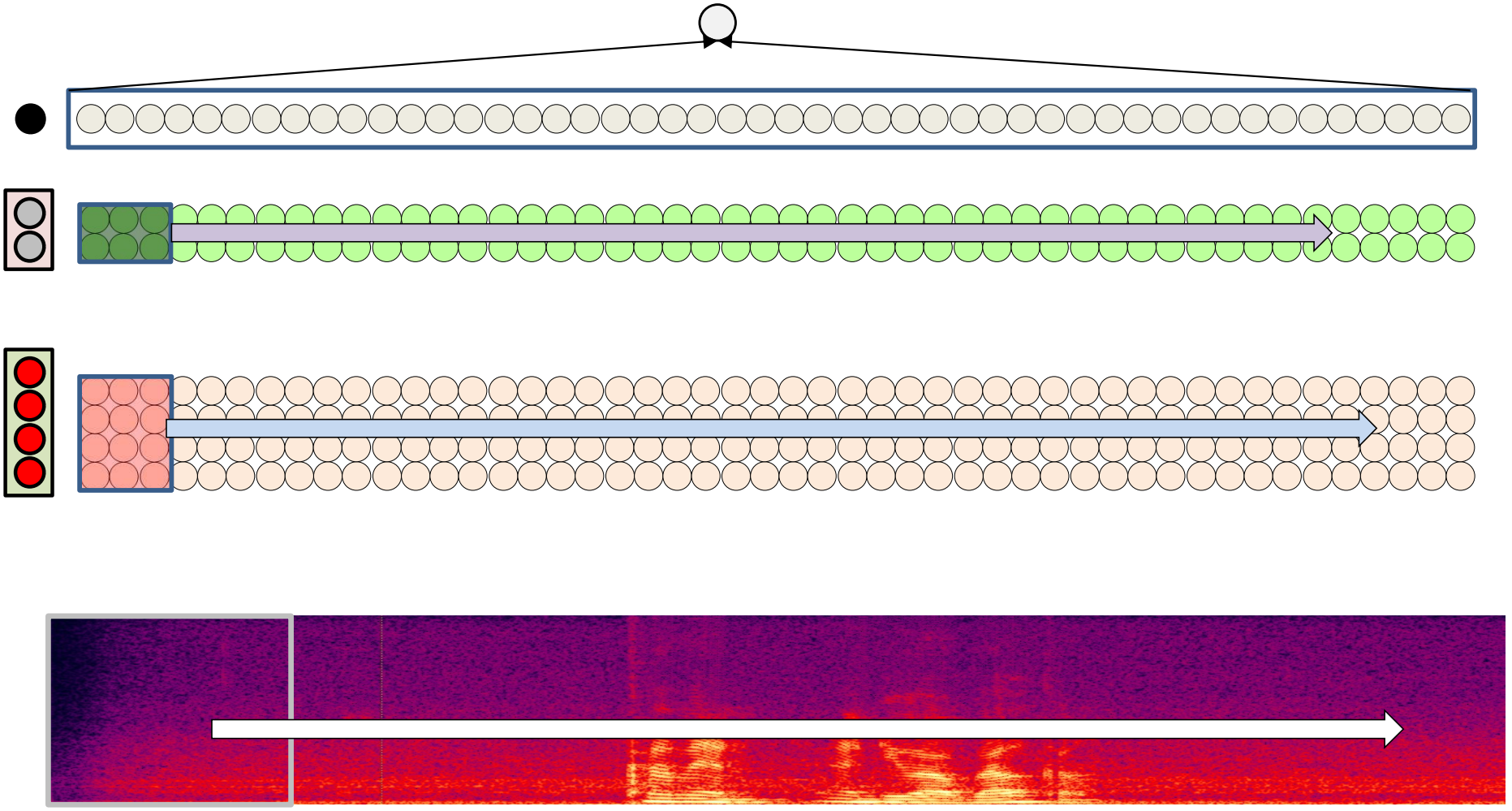
1-D scan version





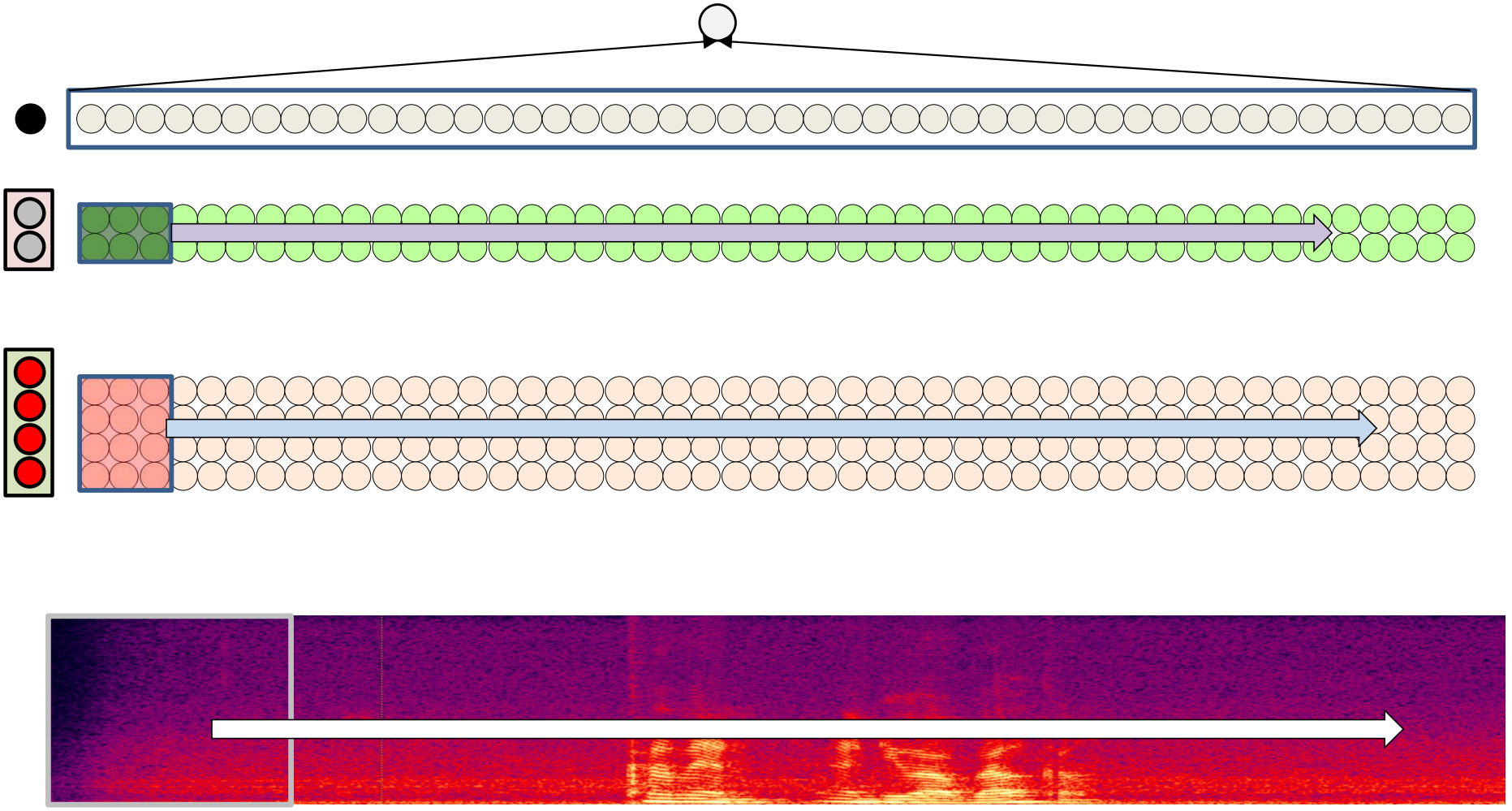
- The 1-D scan version of the convolutional neural network

1-D scan version



- The 1-D scan version of the convolutional neural network
- A final perceptron (or MLP) to aggregate evidence
  - "Does this recording have the target word"

Time-Delay Neural Network



- This structure is called a **time-delay neural network**

Poll 3 (@628)

- Mark all that are true
  - Non-distributed scanning requires the output maps of the neurons to be arranged in the same shape as the input
  - Non-distributed scanning does not require the output maps of neurons to be arranged in the same shape as the input
  - Distributed scanning requires the output maps of the neurons to be arranged in the same shape as the input
  - Distributed scanning does not require the output maps of neurons to be arranged in the same shape as the input

Poll 3

- Mark all that are true
  - Non-distributed scanning requires the output maps of the neurons to be arranged in the same shape as the input
  - **Non-distributed scanning does not require the output maps of neurons to be arranged in the same shape as the input**
  - **Distributed scanning requires the output maps of the neurons to be arranged in the same shape as the input**
  - Distributed scanning does not require the output maps of neurons to be arranged in the same shape as the input

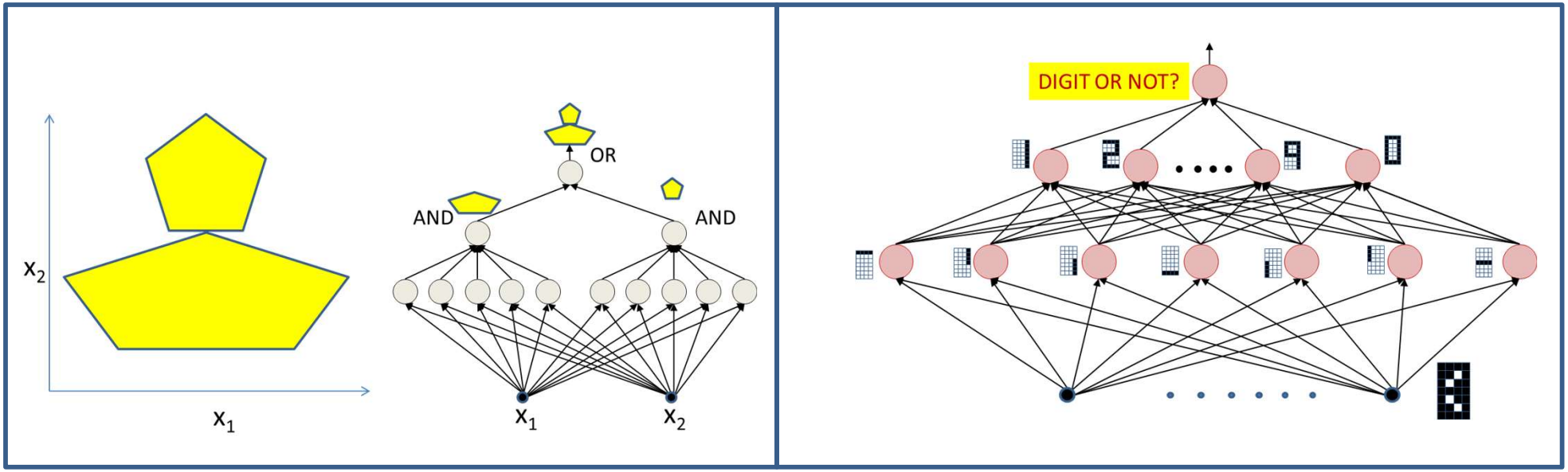
Why distribute?

- Distribution forces hierarchical representations with *localized* patterns in lower layers
  - More generalizable
- Fewer computations
  - Reusable computations from lower layers
- Far fewer number of parameters

Why distribute?

- **Distribution forces hierarchical representations with *localized* patterns in lower layers**
  - **More generalizable**
- Fewer computations
  - Reusable computations from lower layers
- Far fewer number of parameters

Hierarchies of local patterns are better

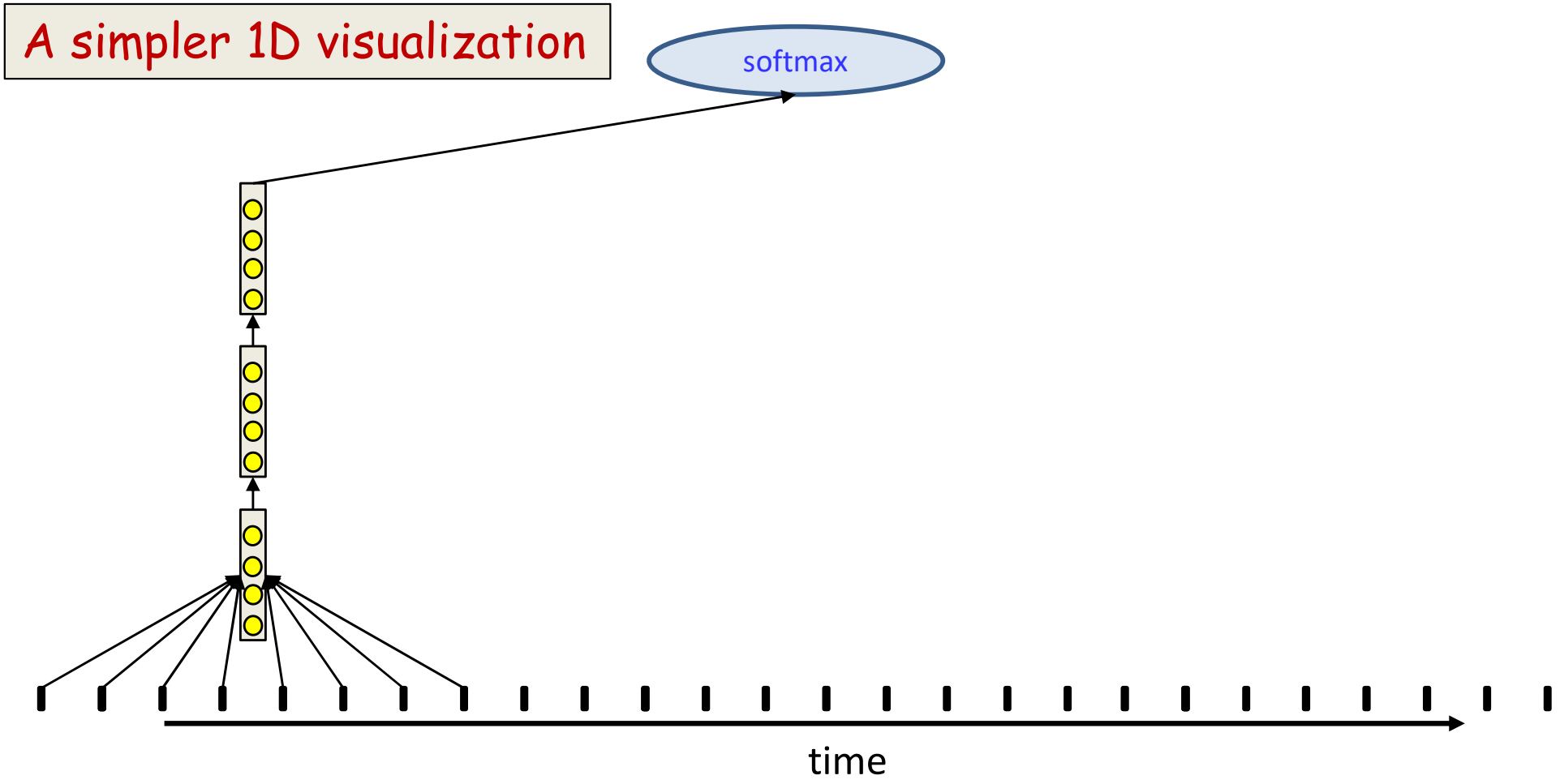


- **The neurons in an MLP *build up* complex patterns from simple pattern hierarchically**
  - Each layer learns to "detect" simple combinations of the patterns detected by earlier layers
- Ideally must encourage such hierarchical learning
  - More data/parameter efficient

Why distribute?

- Distribution forces hierarchical representations with *localized* patterns in lower layers
  - More generalizable
- **Fewer computations**
  - **Reusable computations from lower layers**
- **Far fewer number of parameters**

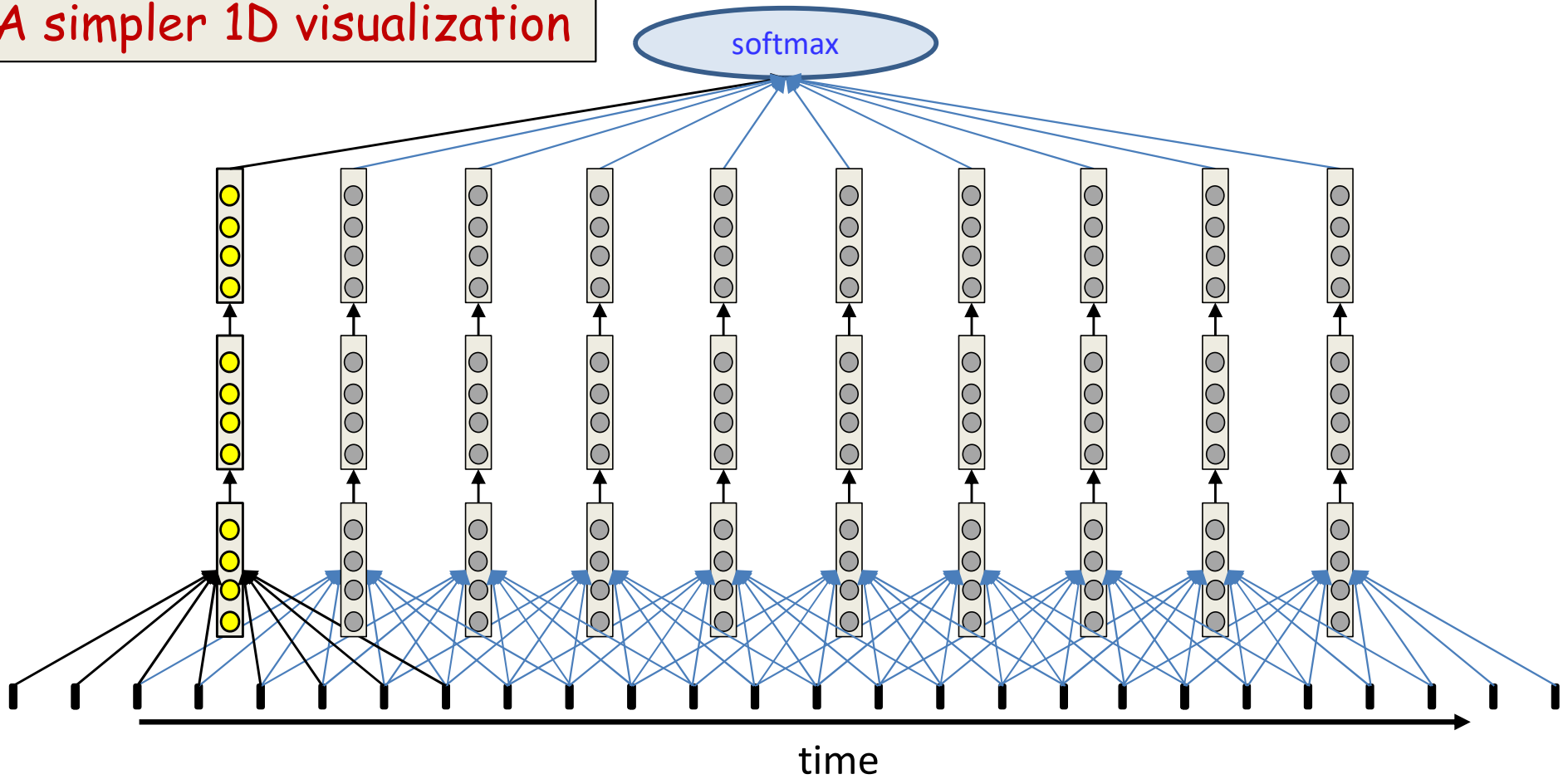
Scanning without distribution



- ***Non-distributed*** scan of 8-time-step wide patterns with a stride of two time steps

Scanning without distribution

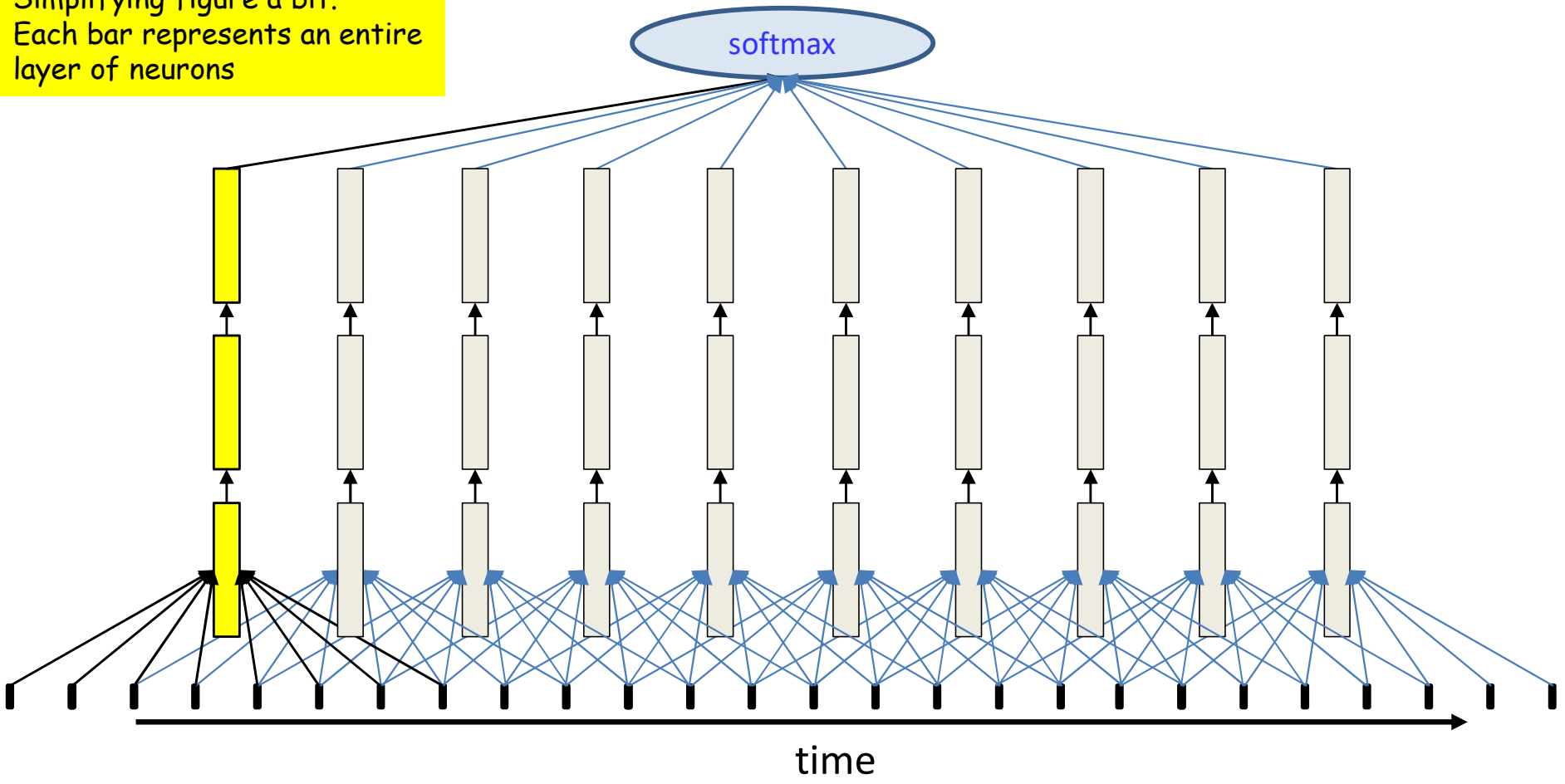
A simpler 1D visualization



- ***Non-distributed*** scan of 8-time-step wide patterns with a stride of two time steps
- Each column (scanning net) operates independently of every other column
  - No computation is shared across columns

Scanning without distribution

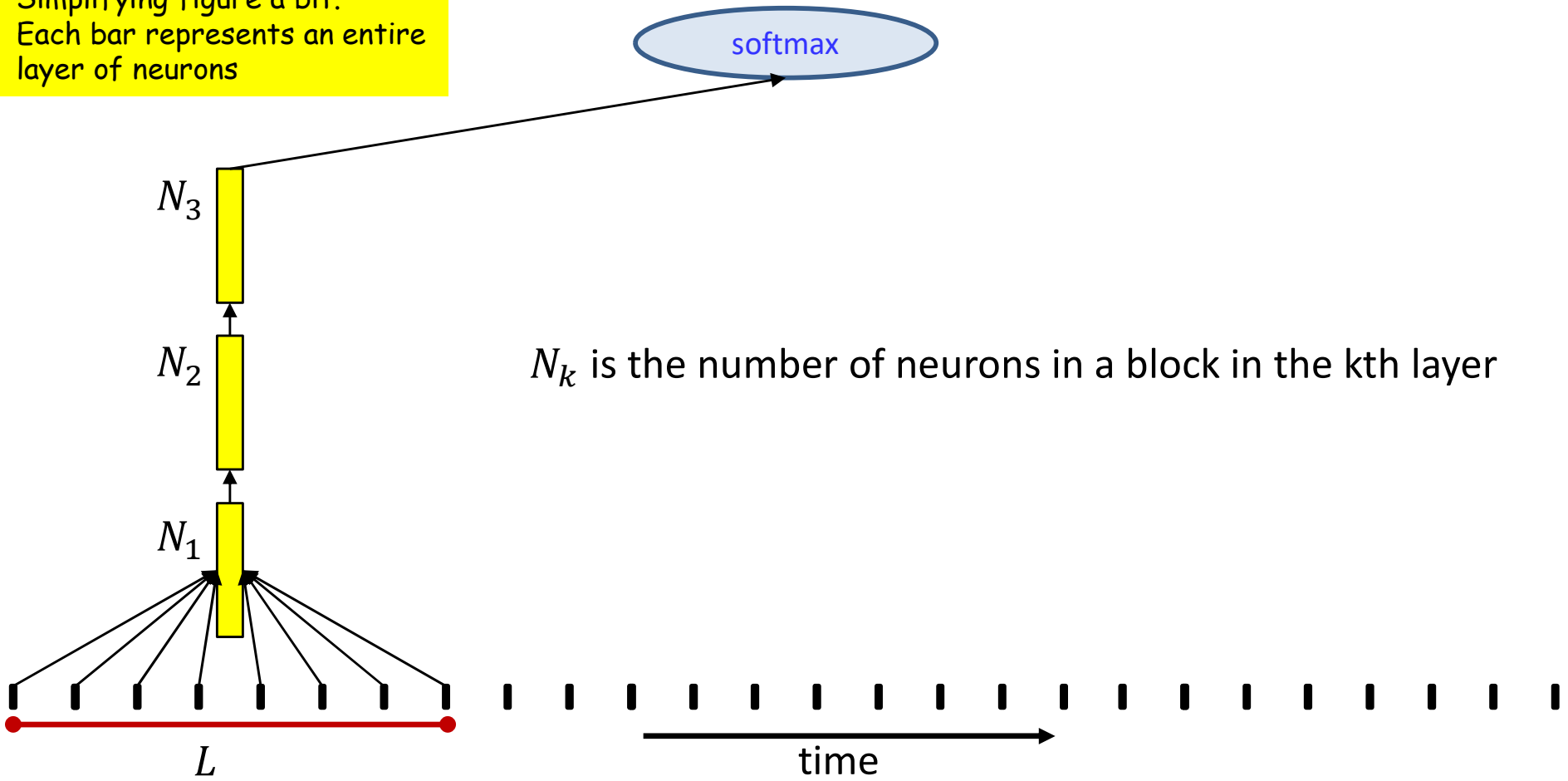
Simplifying figure a bit.
Each bar represents an entire layer of neurons



- ***Non-distributed*** scan of 8-time-step wide patterns with a stride of two time steps

Scanning without distribution

Simplifying figure a bit.
Each bar represents an entire layer of neurons

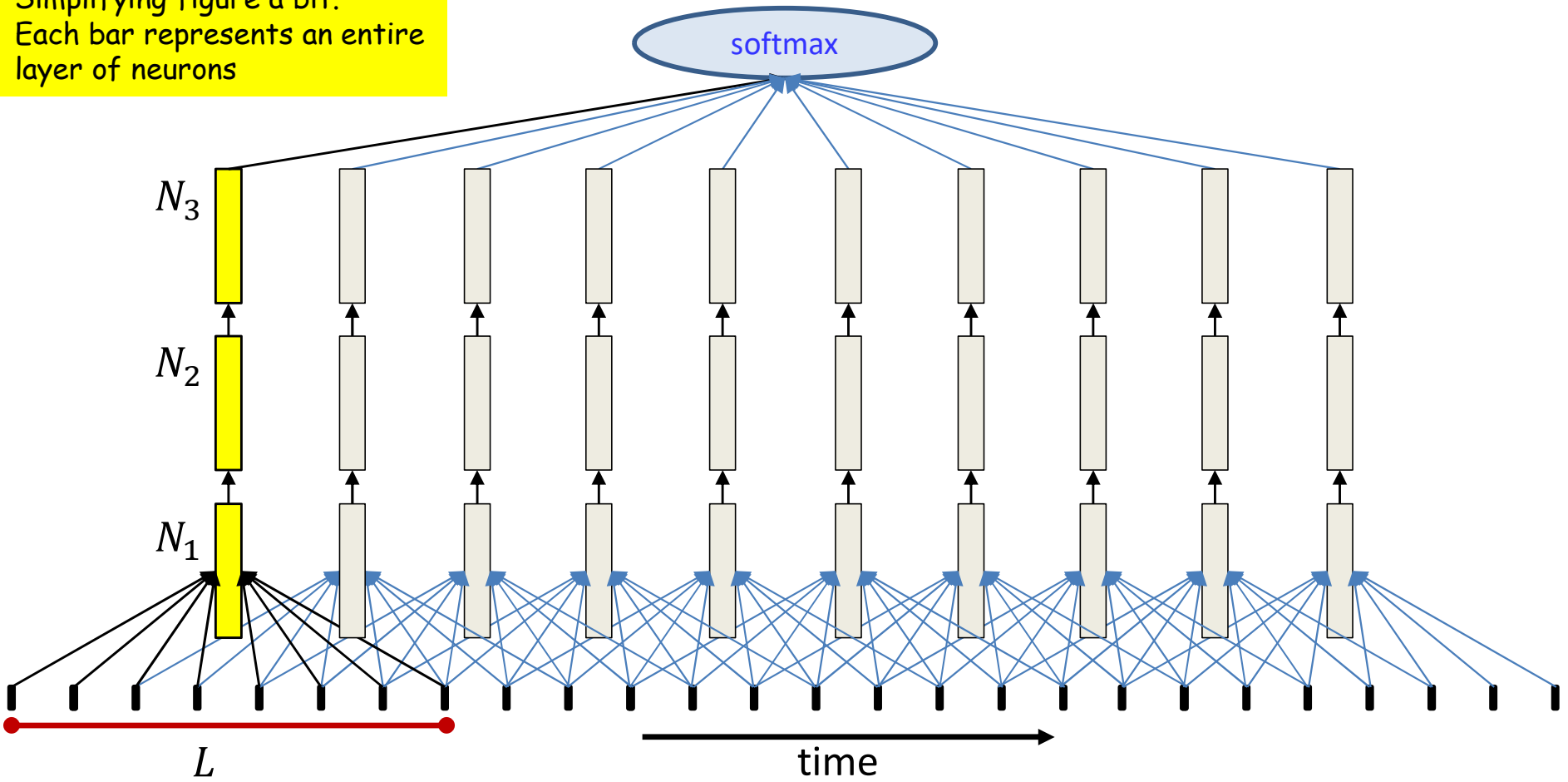


N_k is the number of neurons in a block in the kth layer

- Total parameters: $8DN_1 + N_1N_2 + N_2N_3$
  - D is dimensionality of input
  - More generally: $LDN_1 + N_1N_2 + N_2N_3$
  - Ignoring bias terms in computation
- Only need to count parameters for *one* column, since other columns are identical

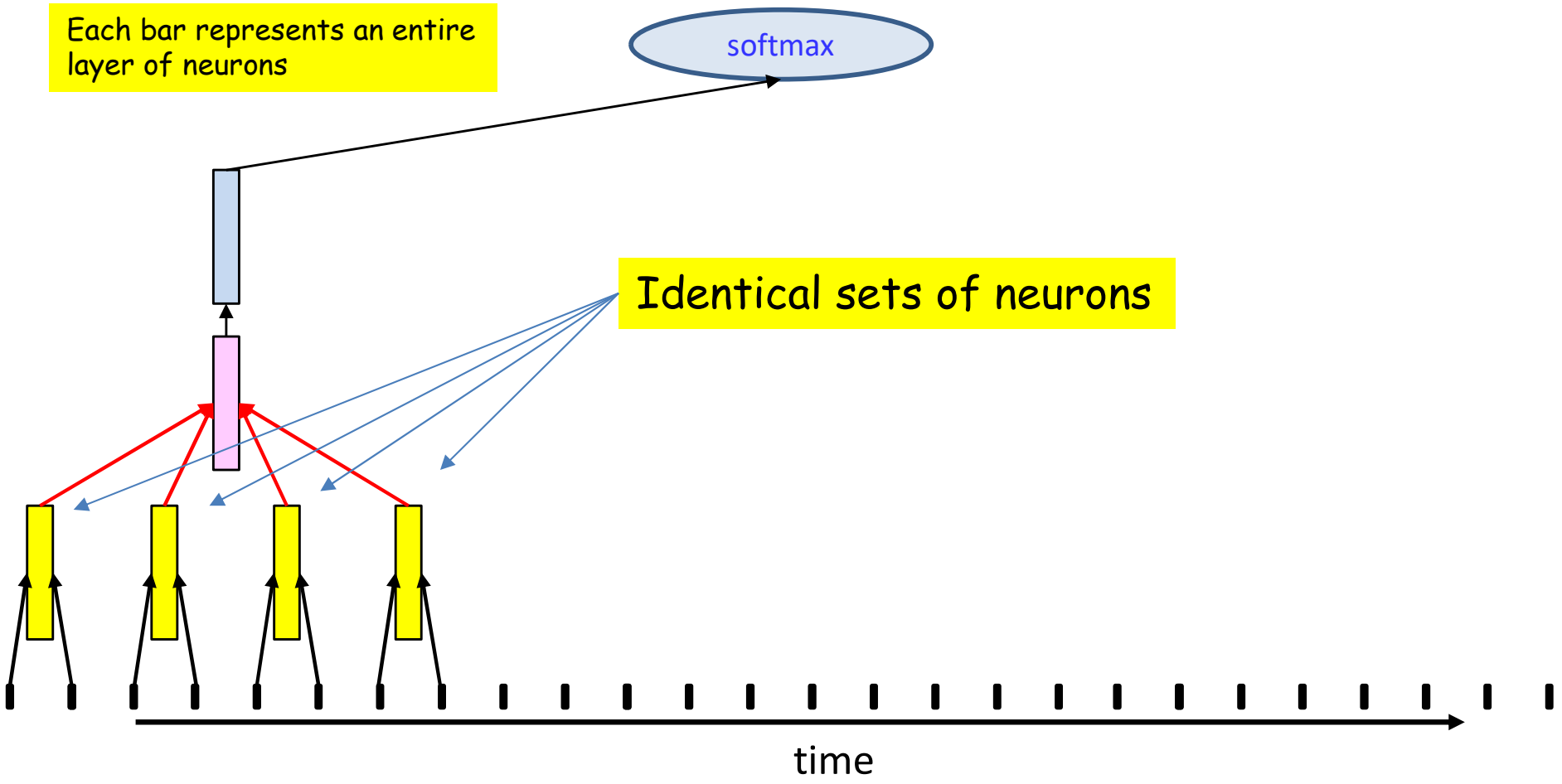
Scanning without distribution

Simplifying figure a bit.
Each bar represents an entire layer of neurons



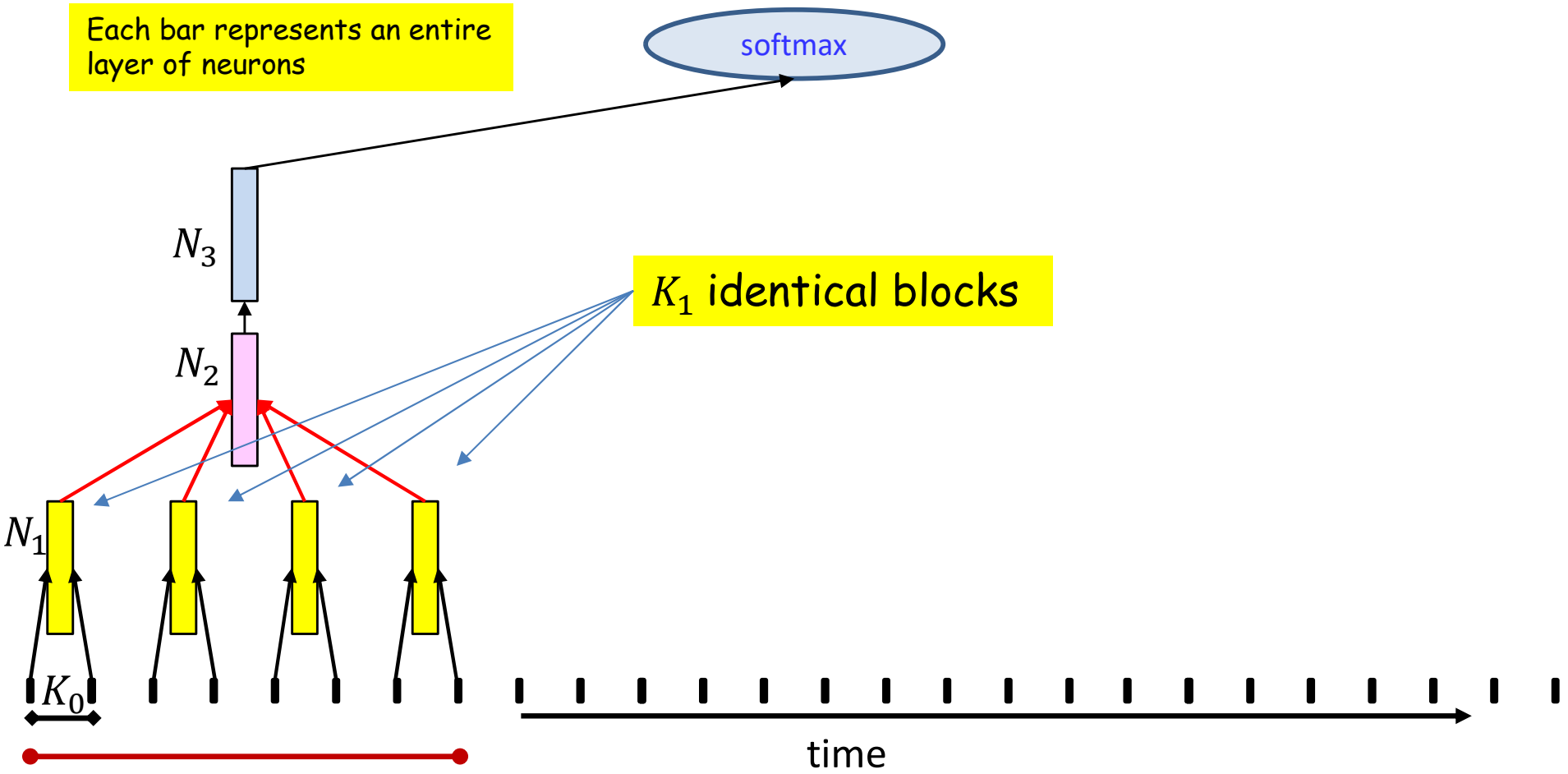
- Total parameters: $8DN_1 + N_1N_2 + N_2N_3$
  - D is dimensionality of input
  - More generally: $LDN_1 + N_1N_2 + N_2N_3$
  - Ignoring bias terms in computation
- Only need to count parameters for *one* column, since other columns are identical

Distributed scanning



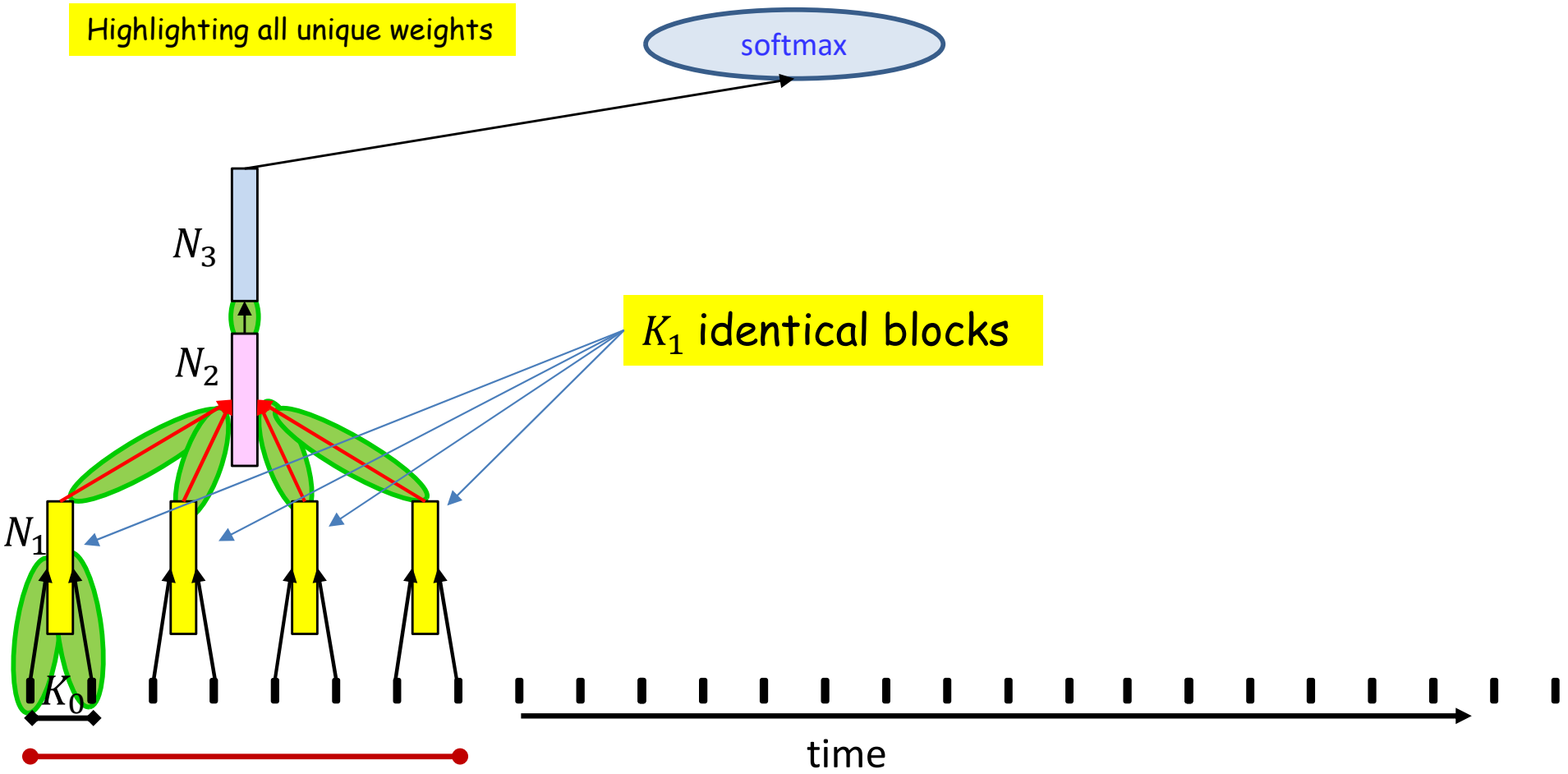
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***

Distributed scanning



- Total parameters: $2DN_1 + 4N_1N_2 + N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + N_2N_3$
  - **Fewer parameters than a non-distributed net with identical number of neurons**

Distributed scanning



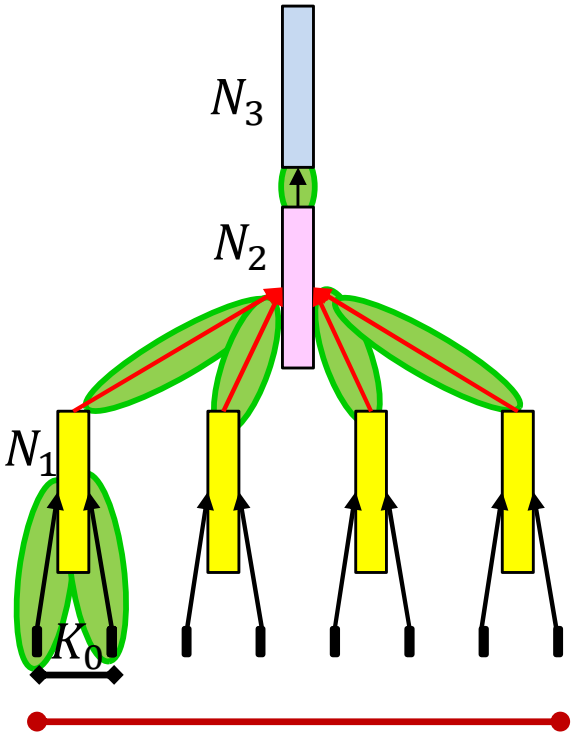
- Total parameters: $2DN_1 + 4N_1N_2 + N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + N_2N_3$
  - **Fewer parameters than a non-distributed net with identical number of neurons**

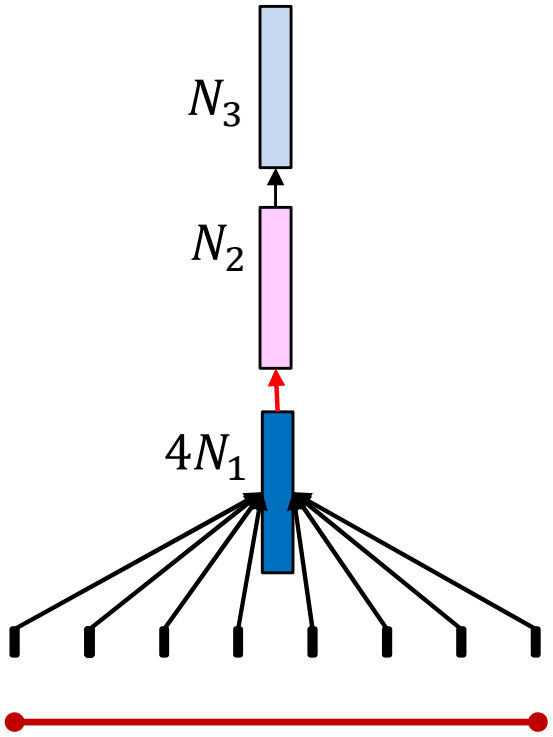
Distributed vs non-distributed scanning

$2DN_1 + 4N_1N_2 + N_2N_3$

Highlighting all unique weights for distributed scan

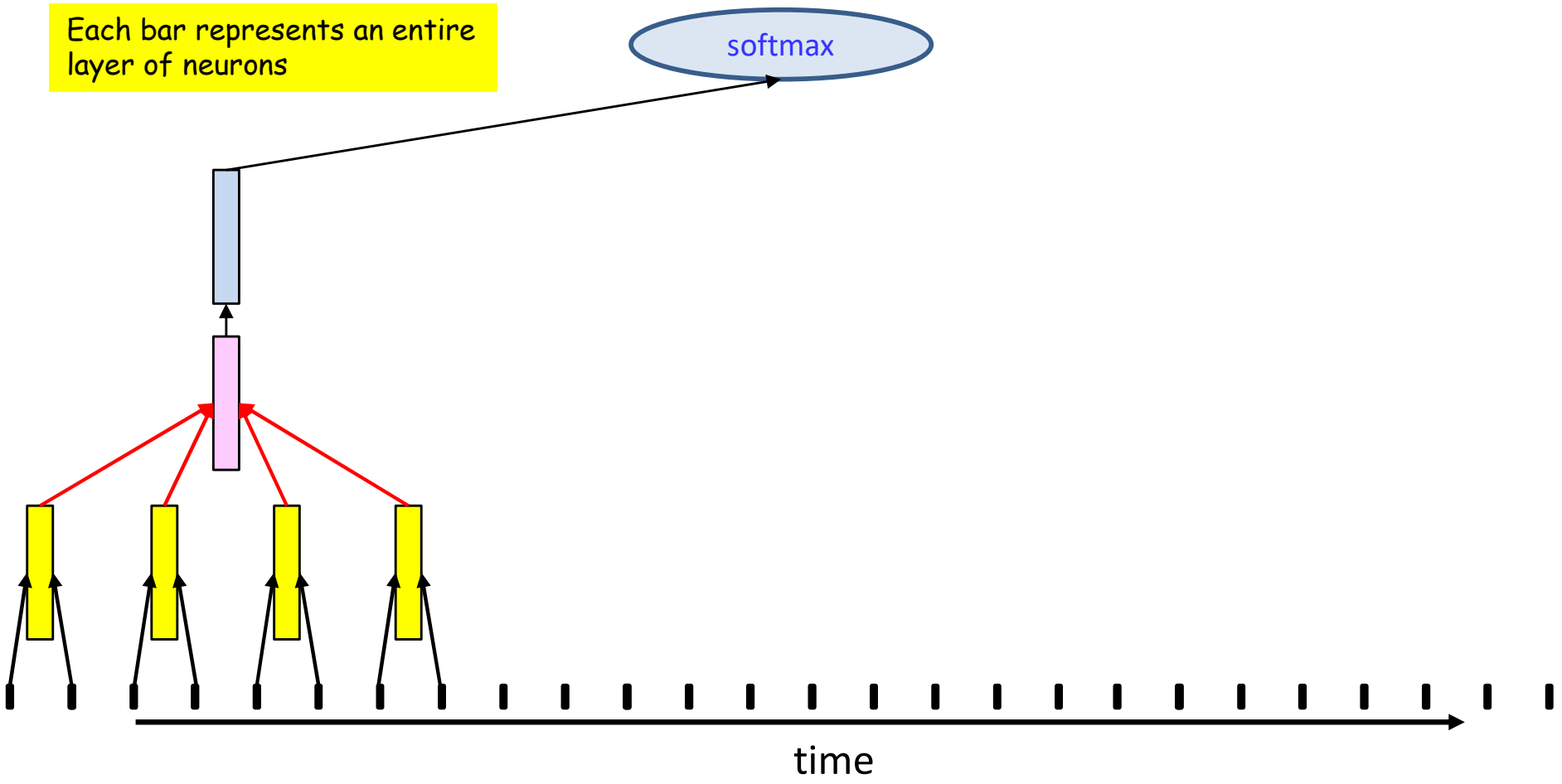
Equivalent non-distributed net has
$8D(4N_1) + 4N_1N_2 + N_2N_3$
parameters (not including bias terms)





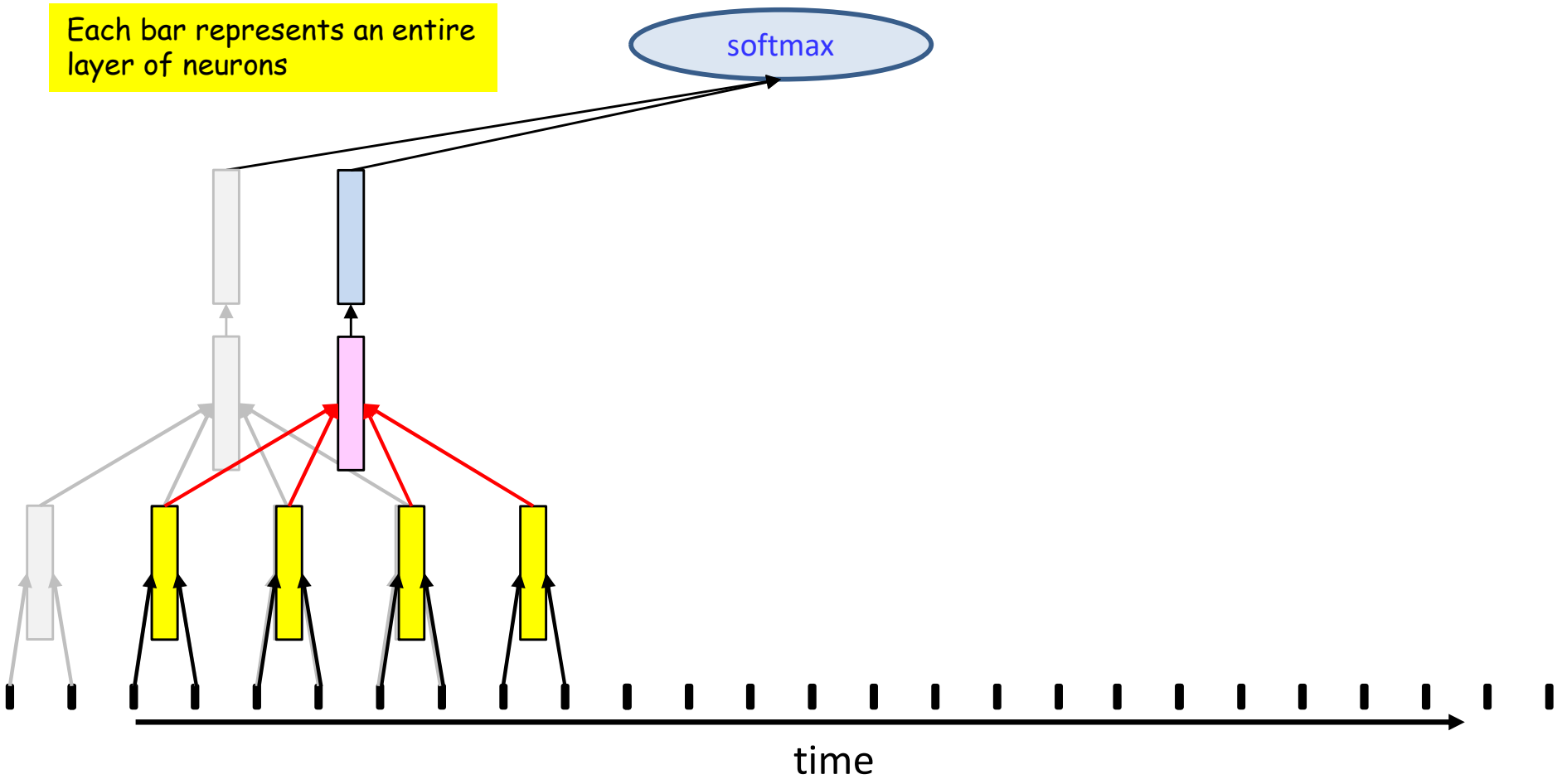
- Total parameters: $2DN_1 + 4N_1N_2 + N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + N_2N_3$
  - **Fewer parameters than a non-distributed net with identical number of neurons**

Distributed scanning



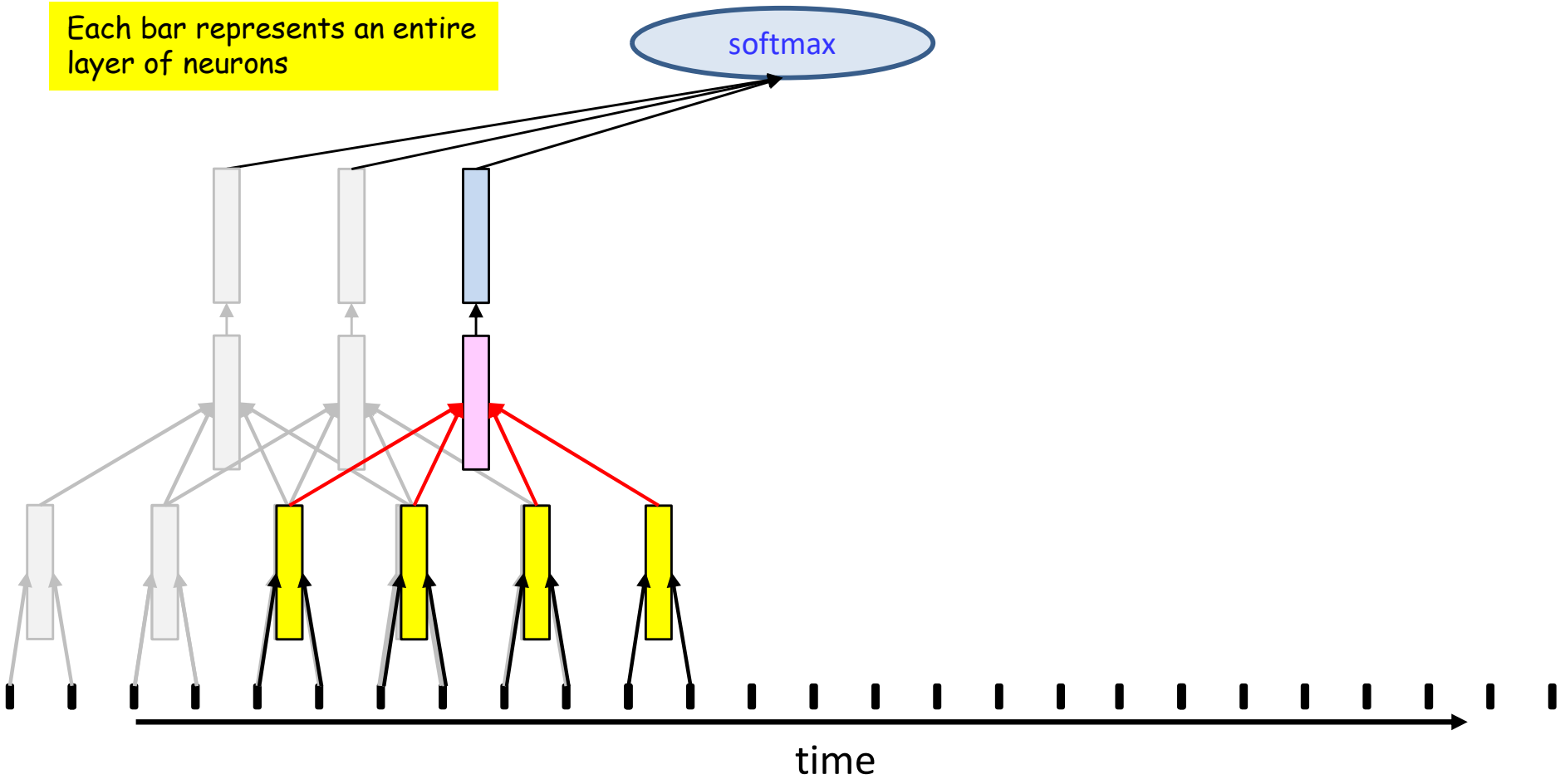
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



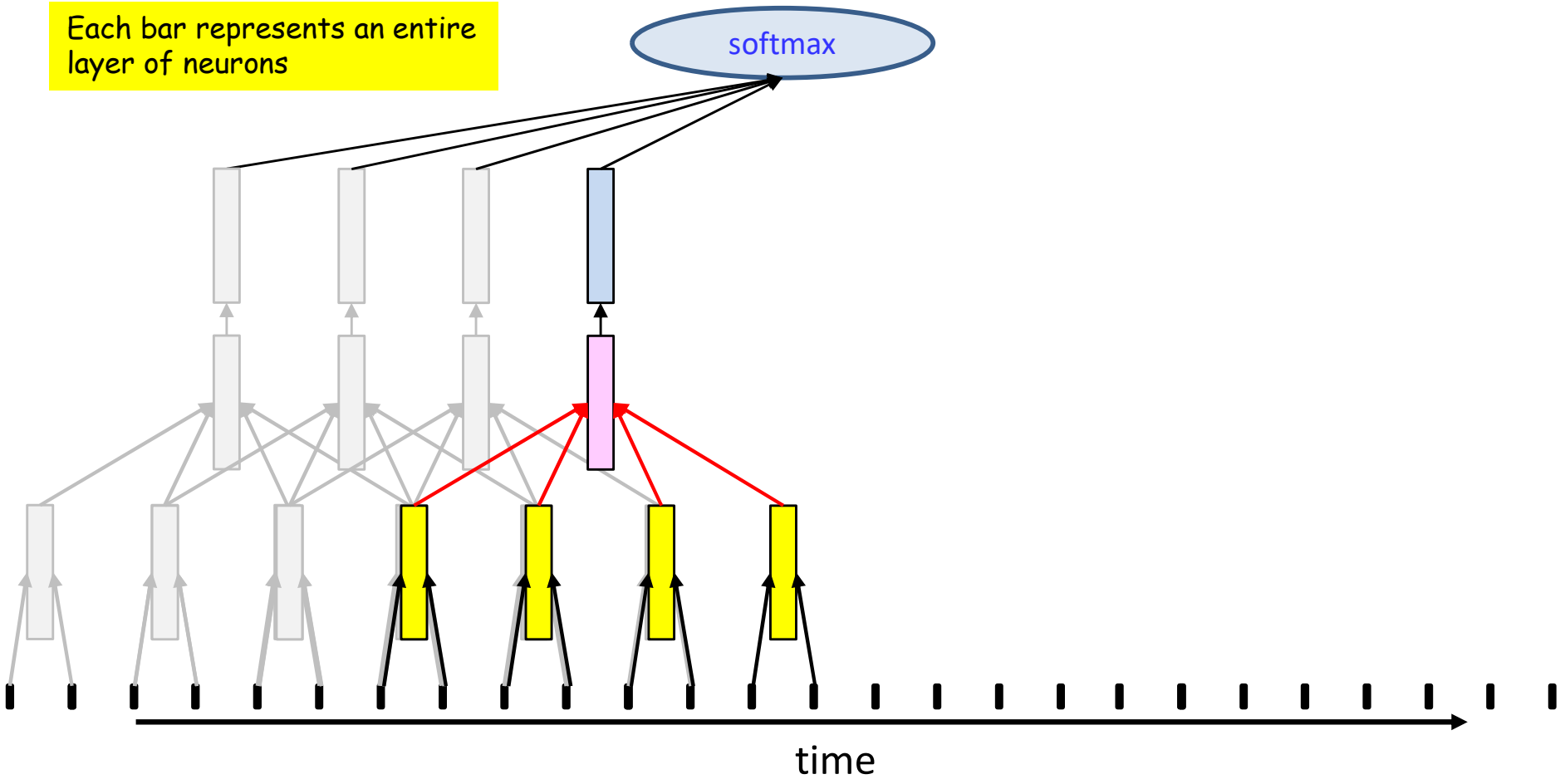
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



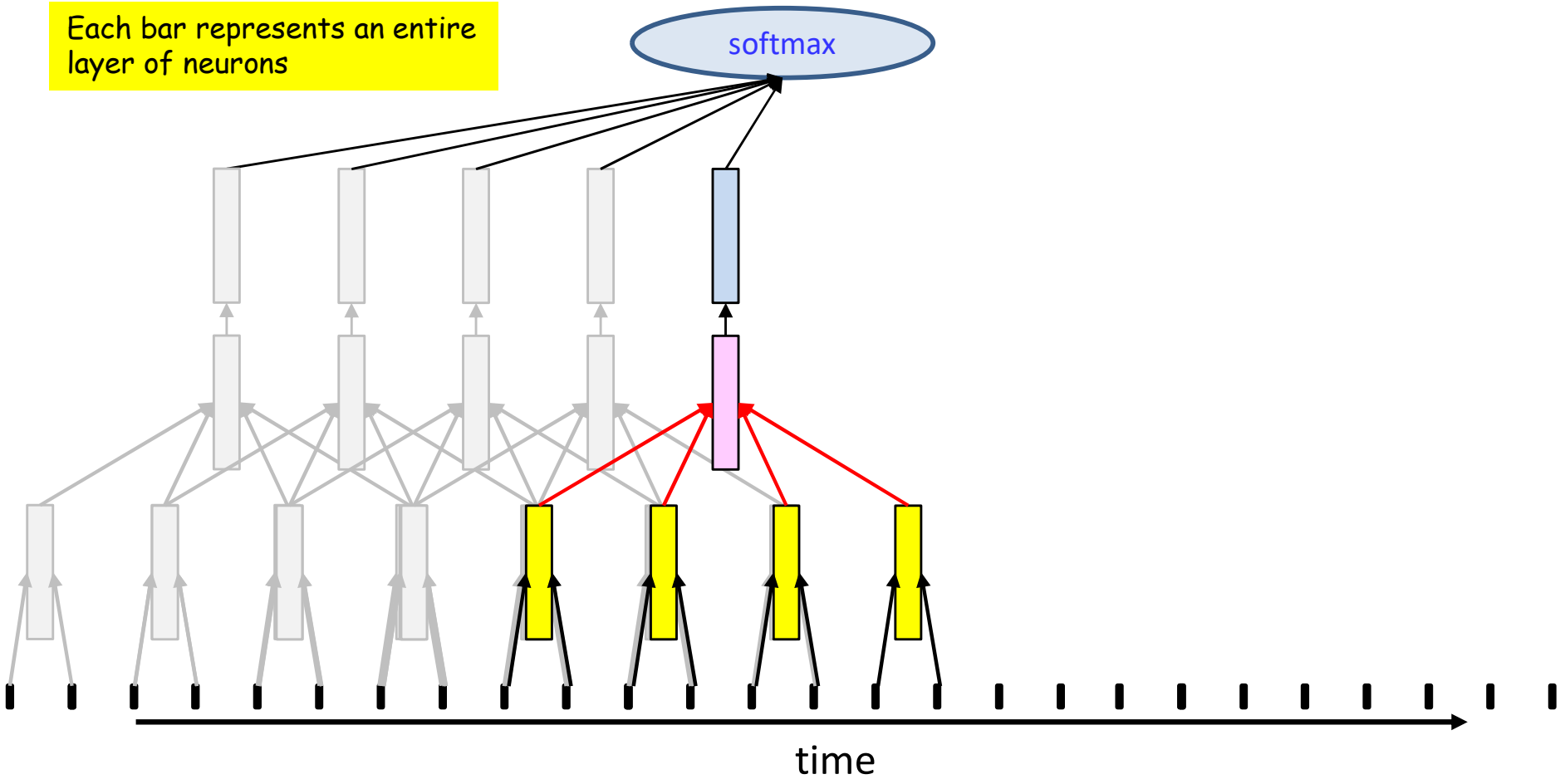
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



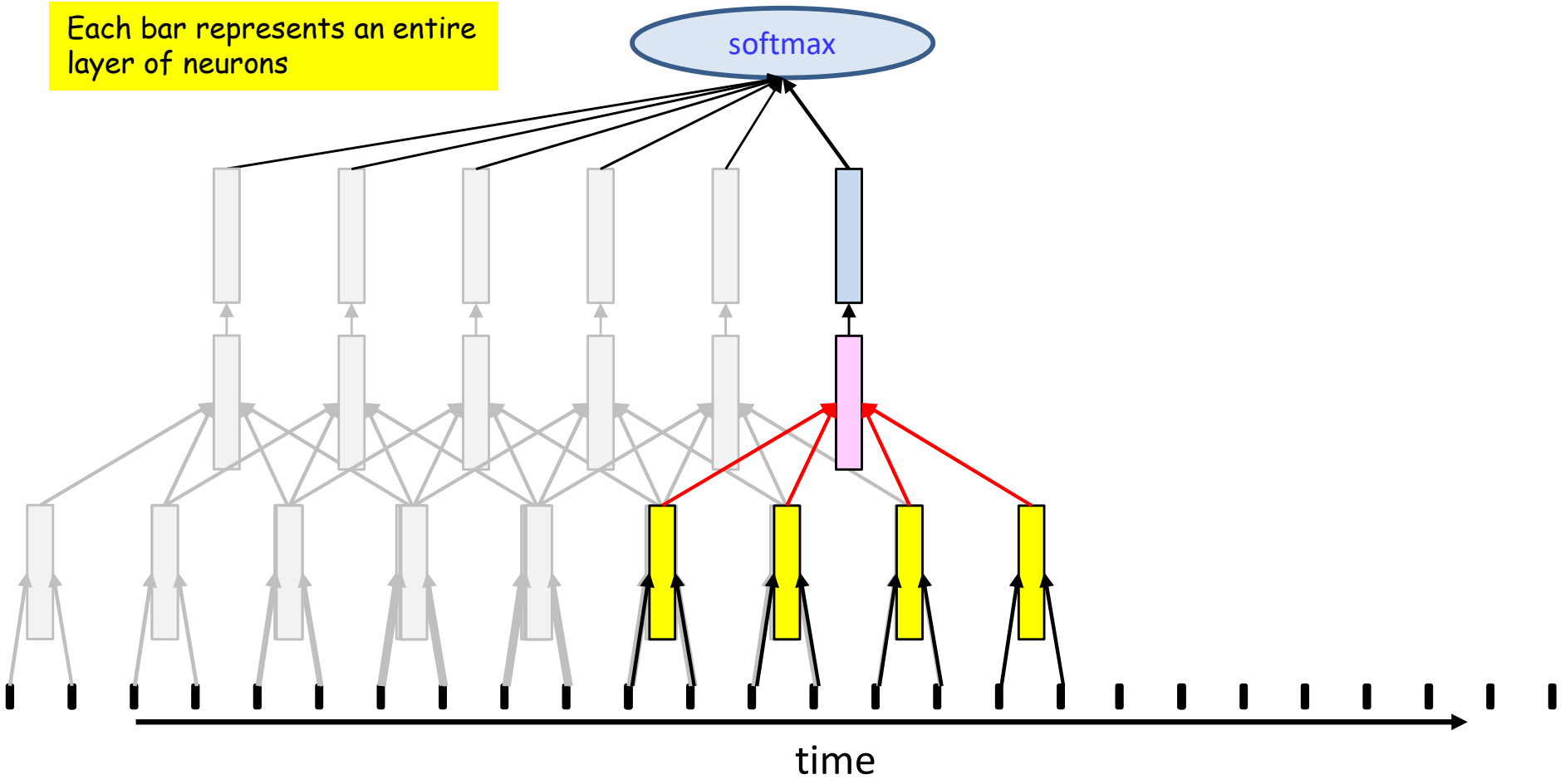
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



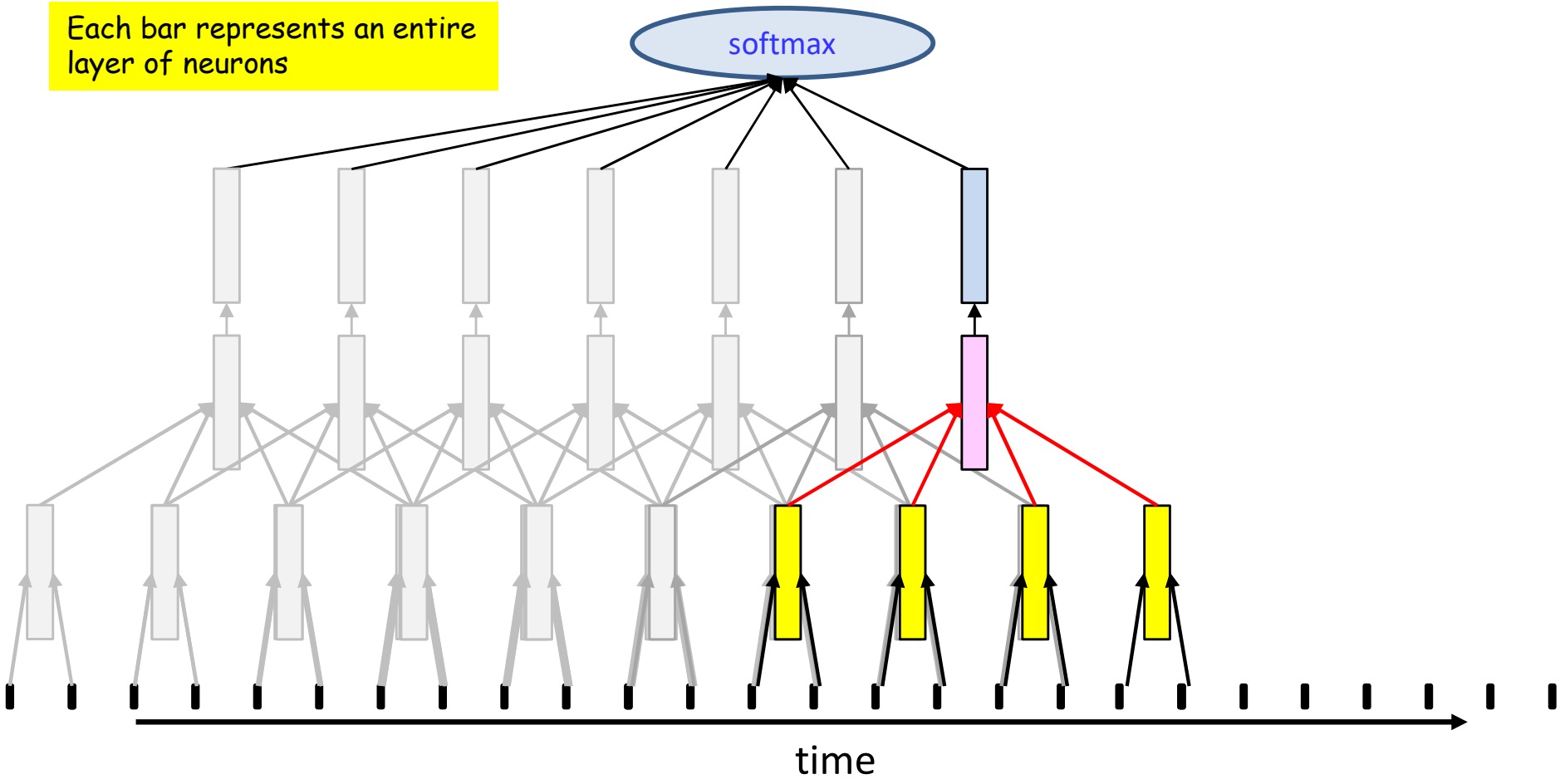
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



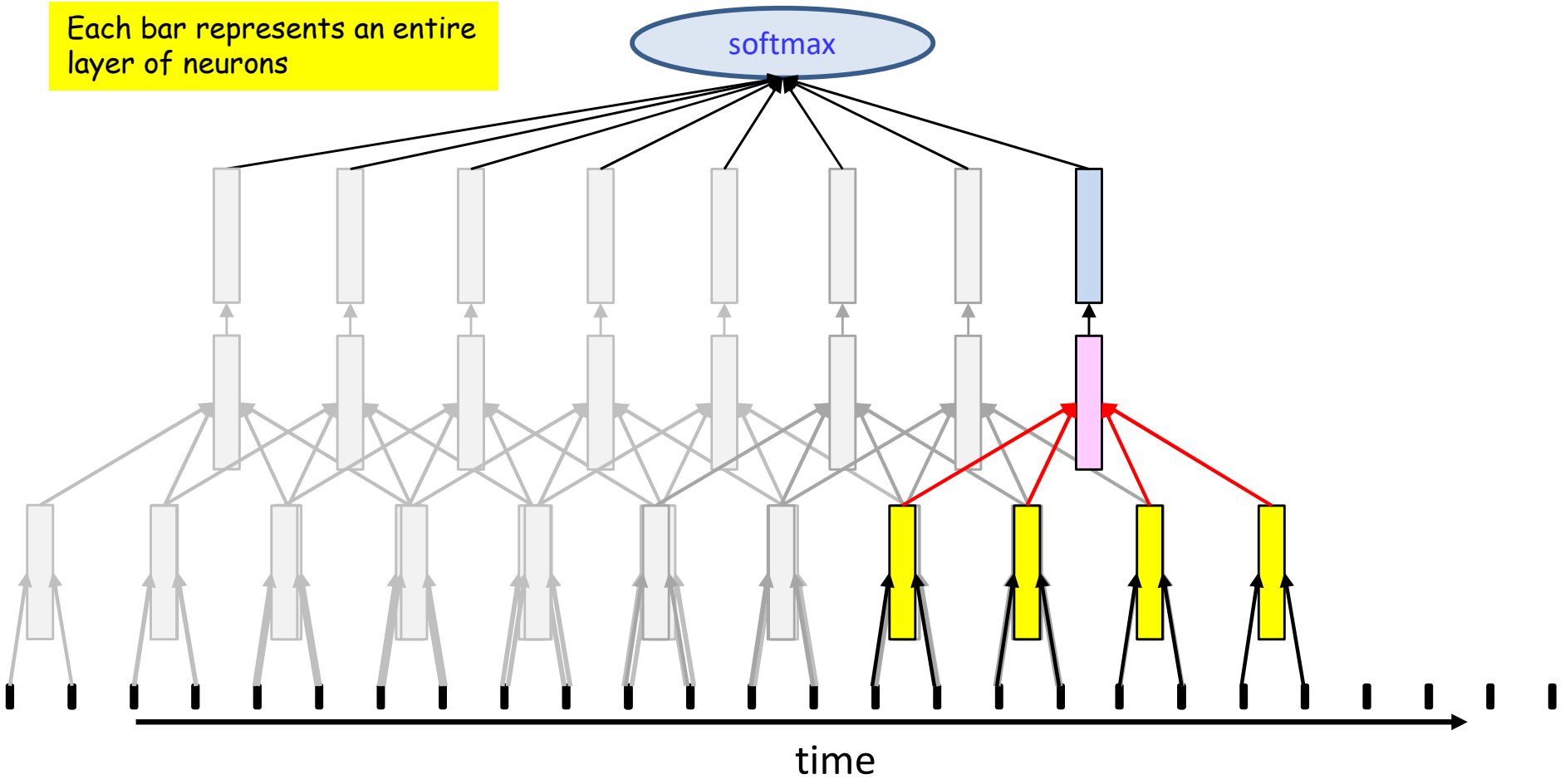
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



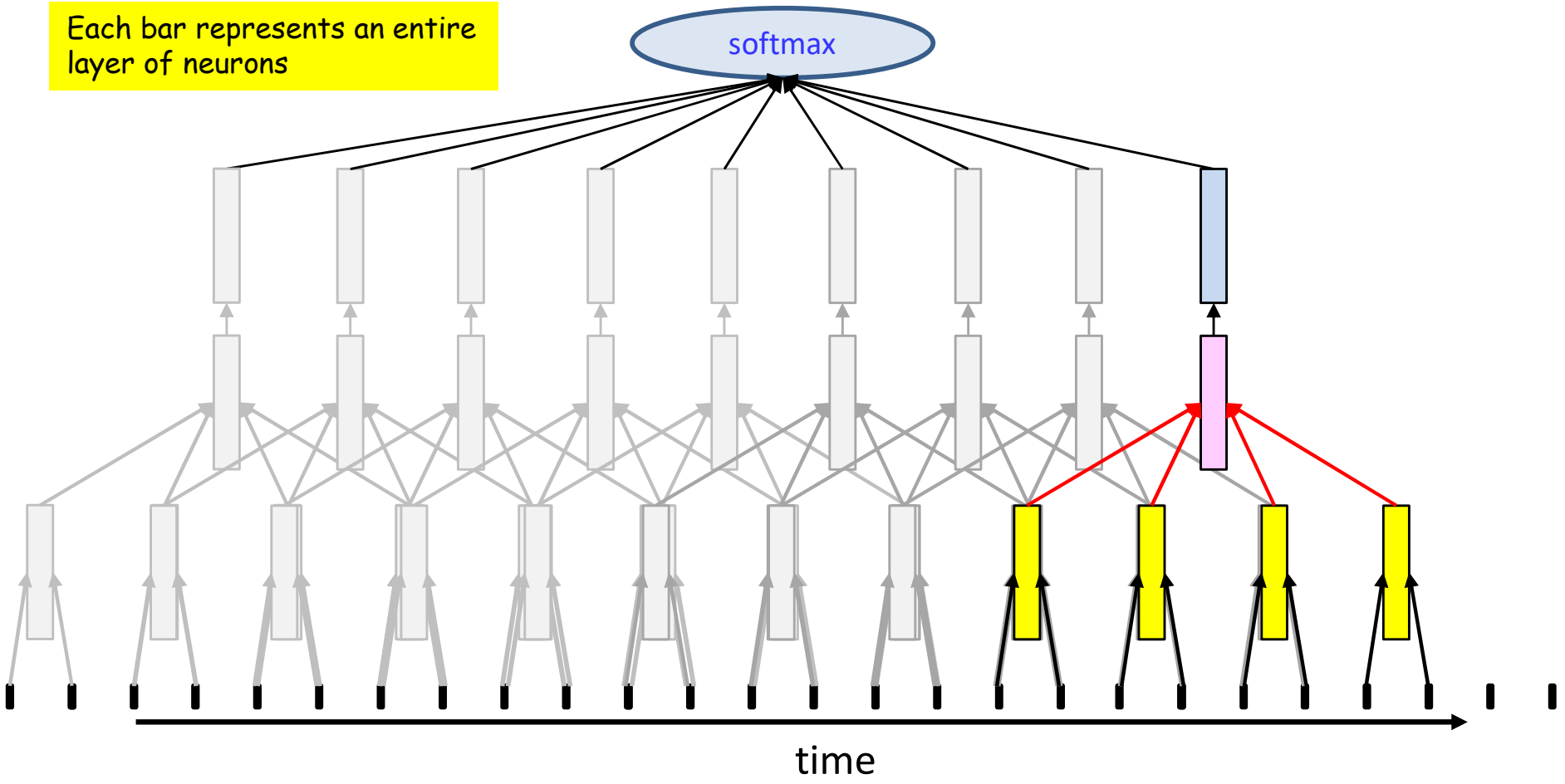
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



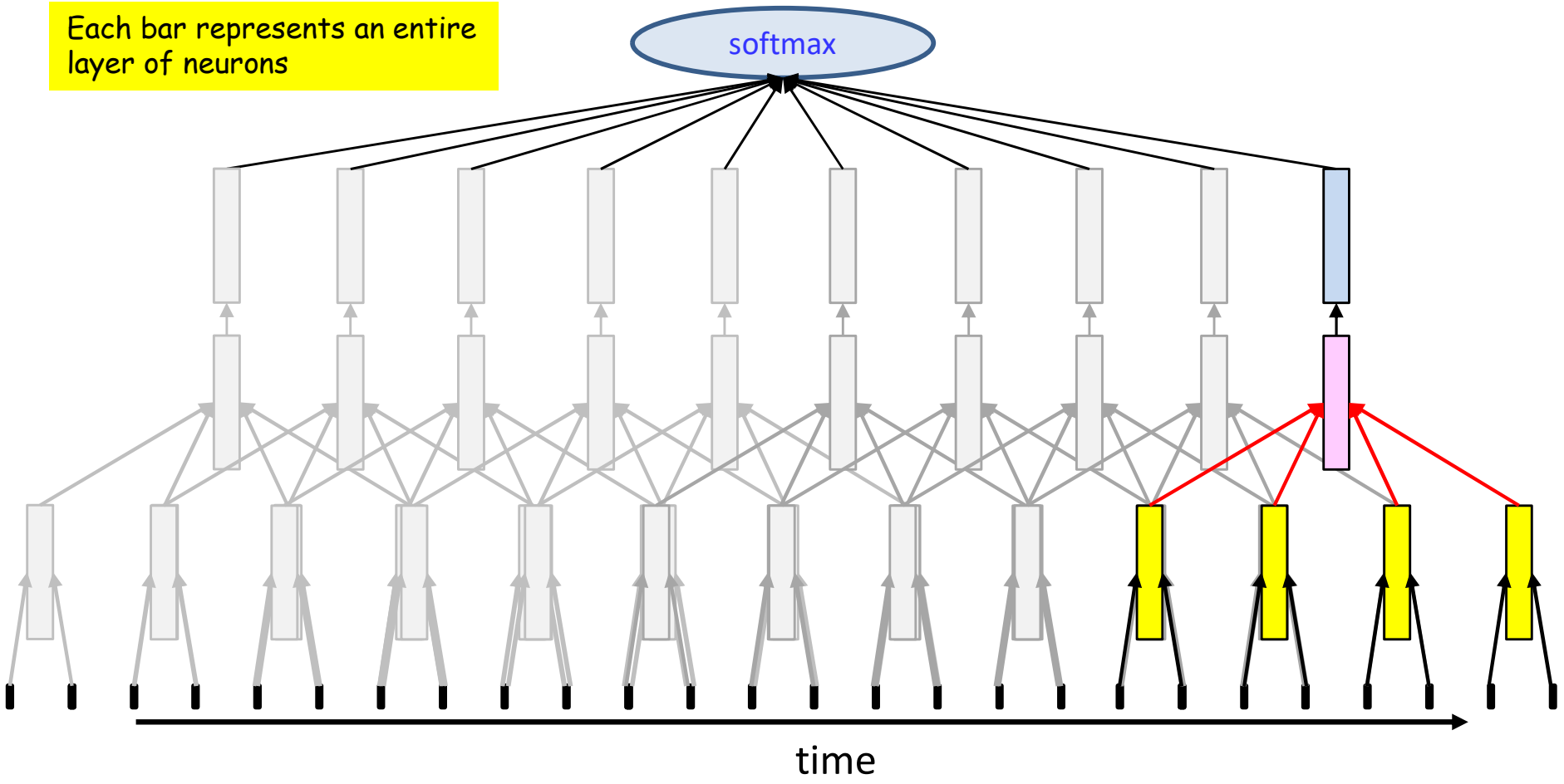
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



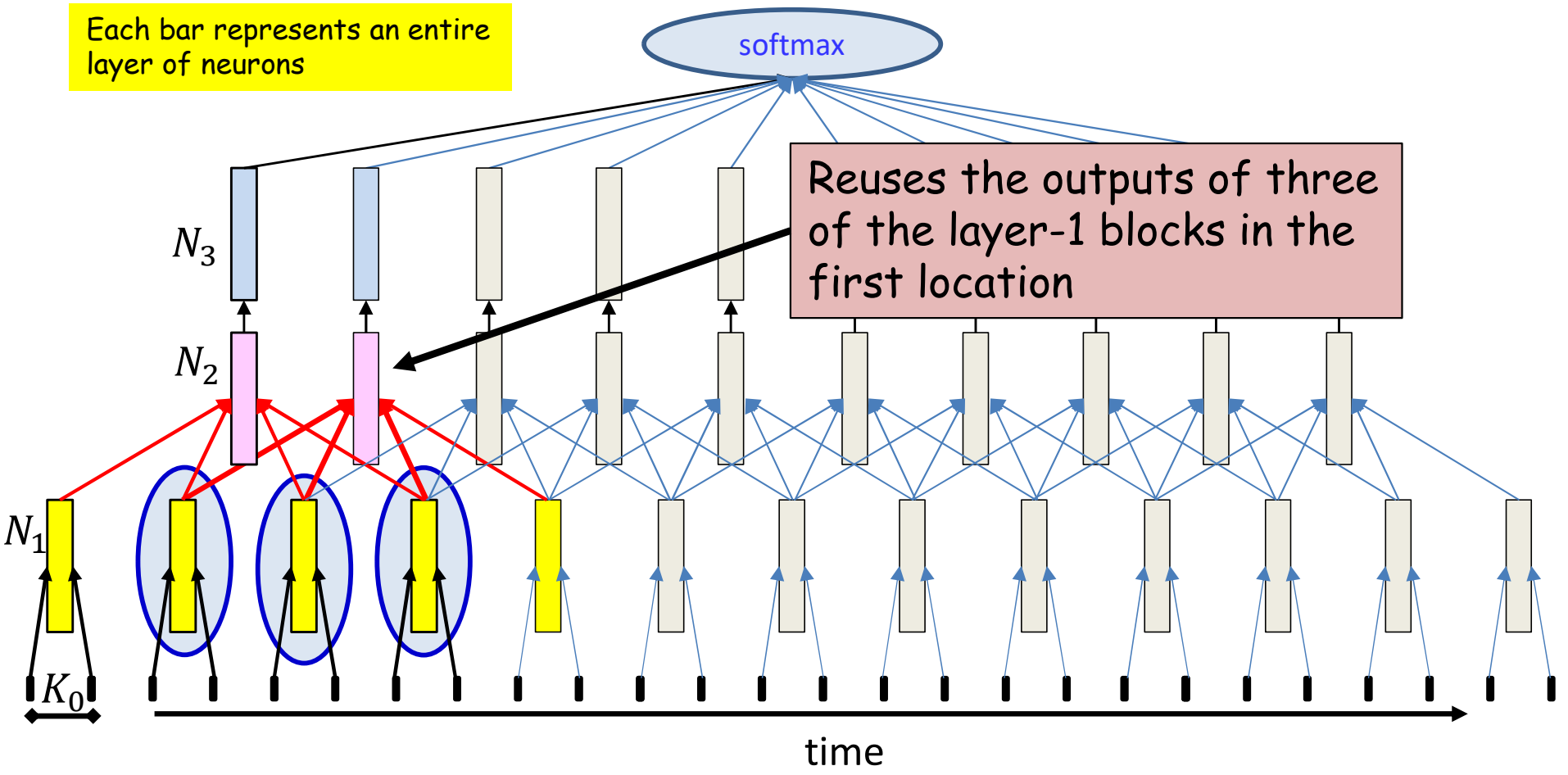
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



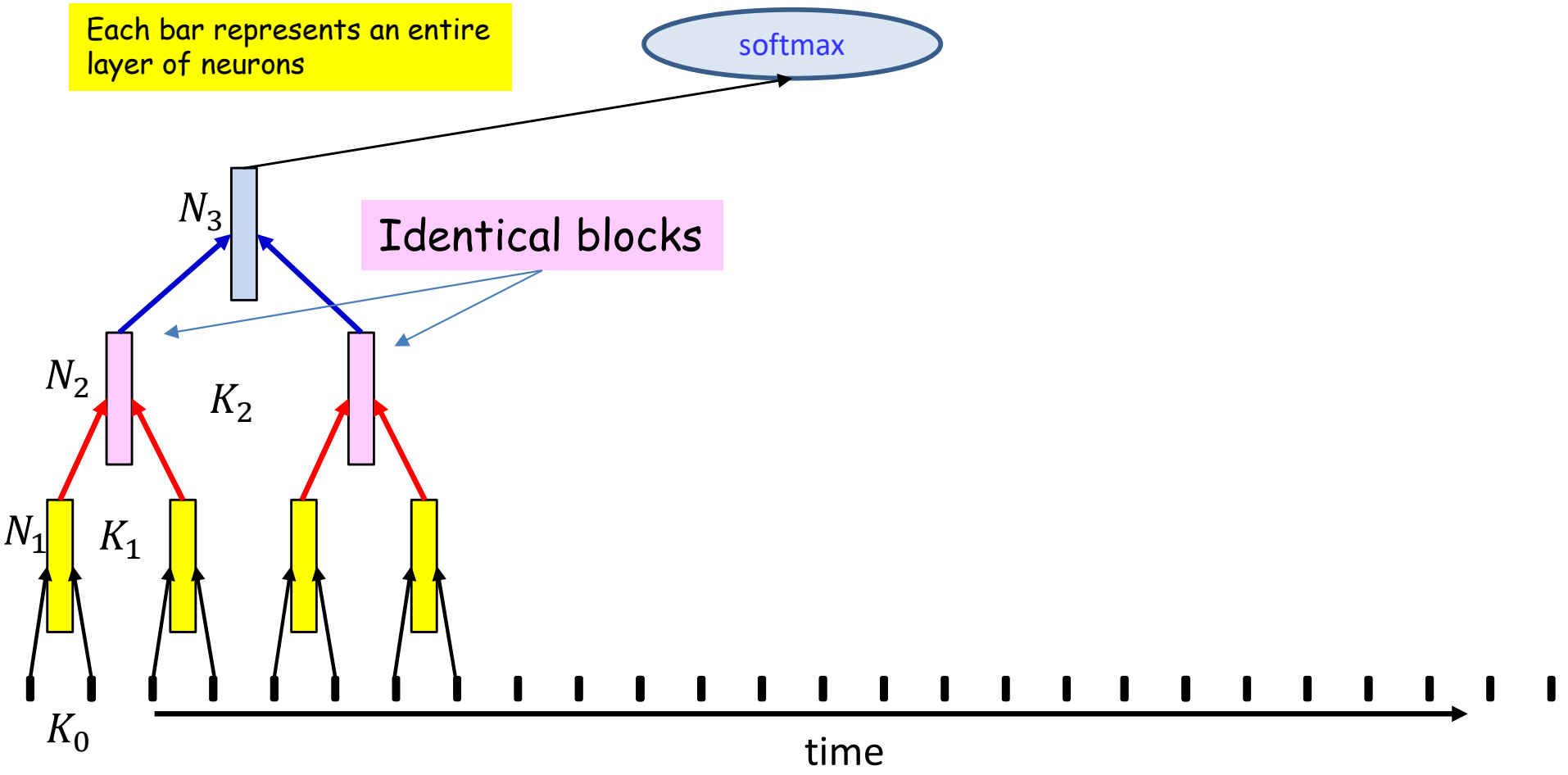
- Scan of 8-time-step wide patterns with a stride of two time steps ***distributed over two layers***
  - **At each position higher level neurons *reuse* some of the computations performed at the previous step(s)!**

Distributed scanning



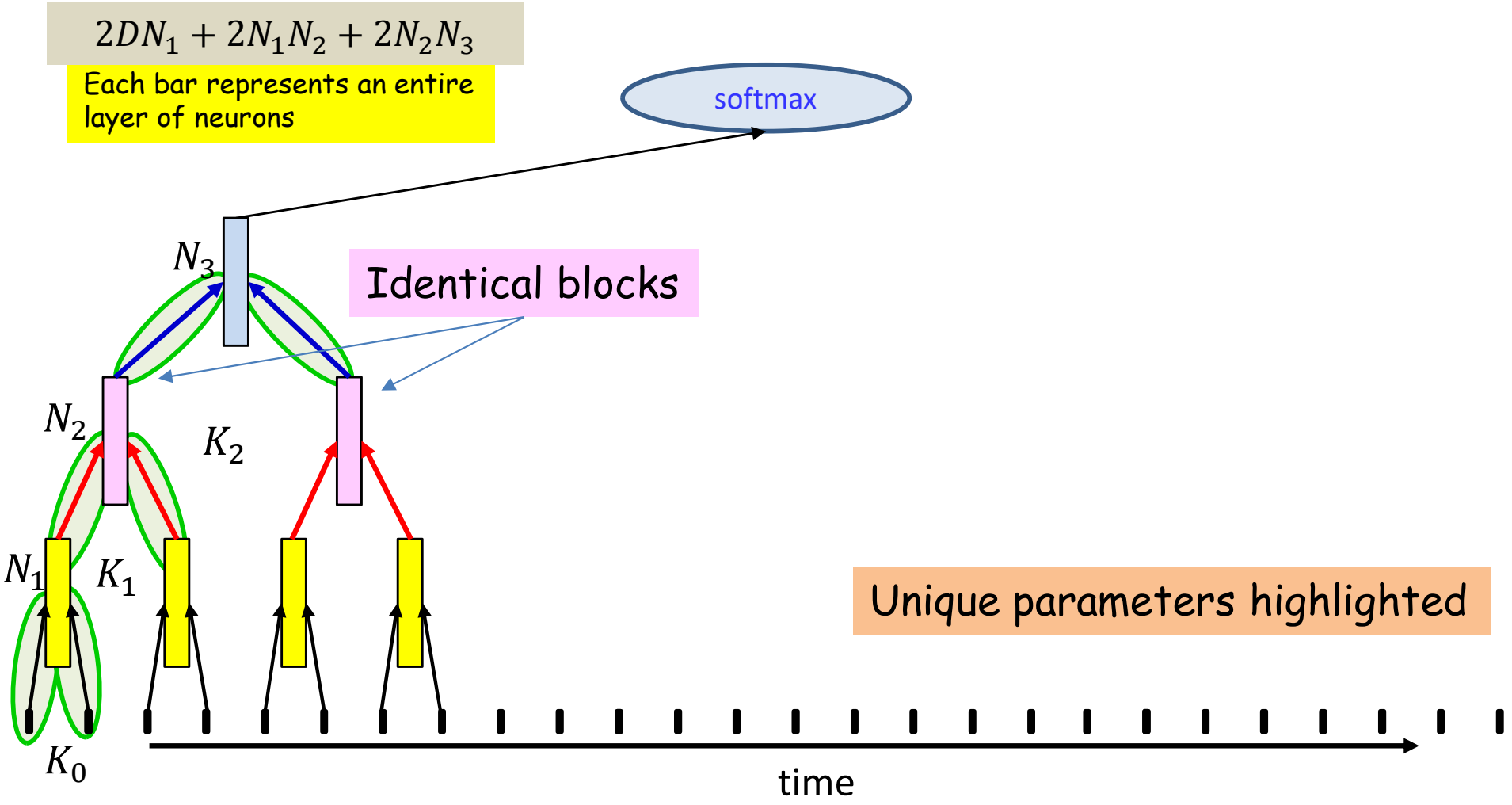
- Much of the computation is reusable
  - No need to recompute the circled yellow blocks for the second position
  - ***Large additional benefit from the fact that scans at neighboring positions share the computation of lower-level blocks!***

Distributed scanning: 3 levels



- Total parameters: $2DN_1 + 2N_1N_2 + 2N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Far fewer parameters than non-distributed scan with network with identical no. of neurons**

Distributed scanning: 3 levels



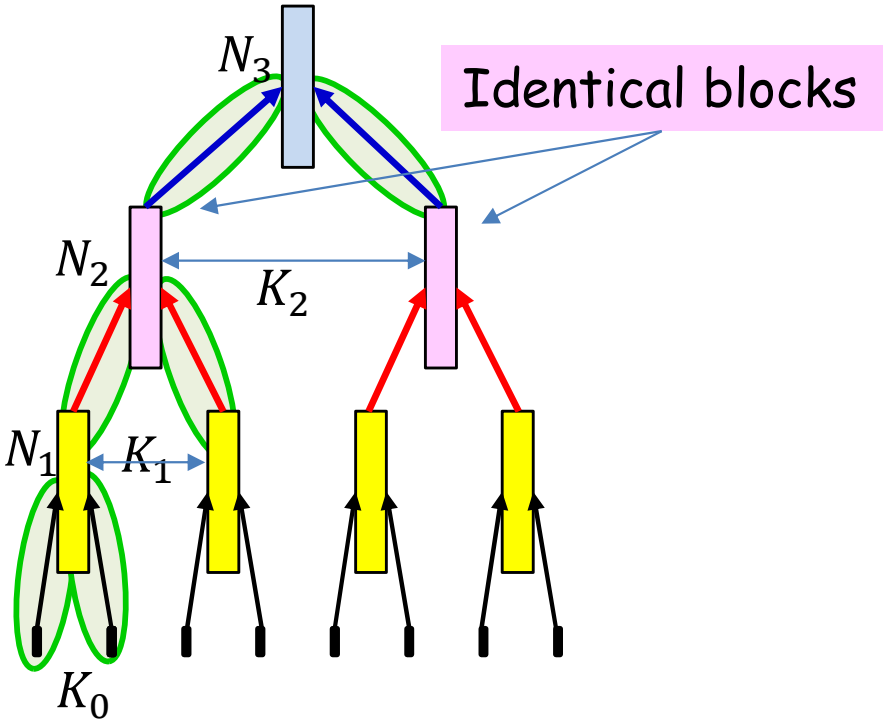
- Total parameters: $2DN_1 + 2N_1N_2 + 2N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Far fewer parameters than non-distributed scan with network with identical no. of neurons**

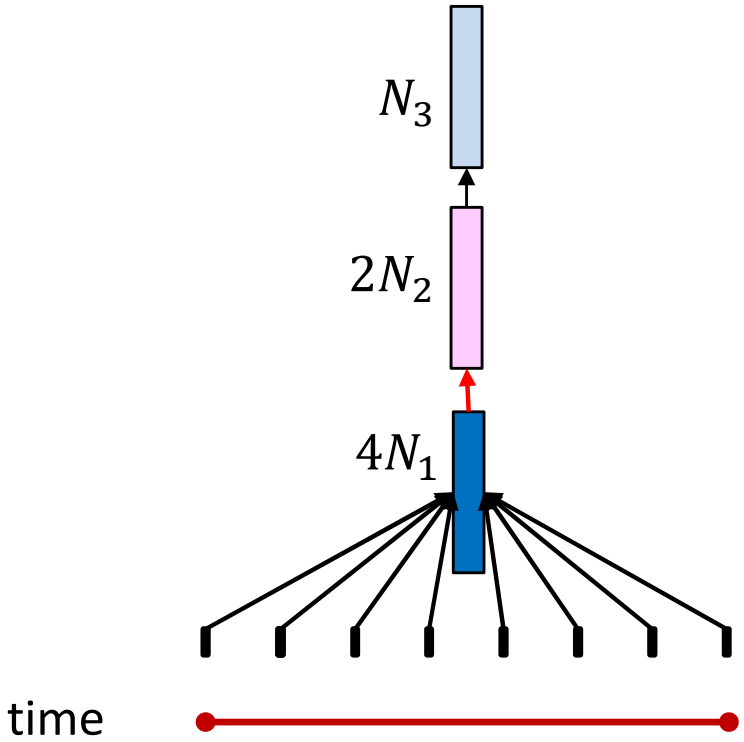
Distributed scanning: 3 levels

$2DN_1 + 2N_1N_2 + 2N_2N_3$

Each bar represents an entire layer of neurons

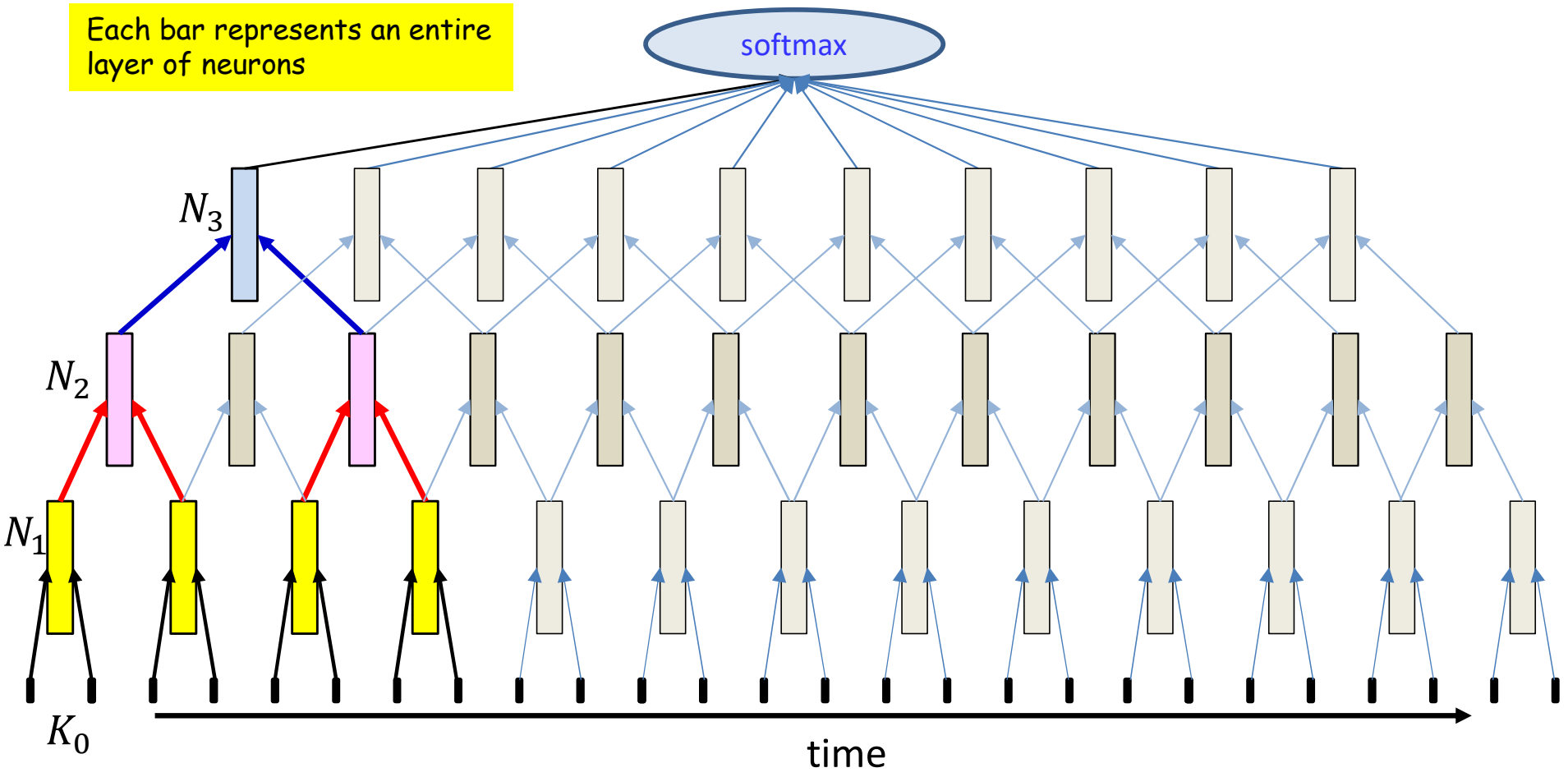
Equivalent non-distributed net has $8D(4N_1) + (4N_1)(2N_2) + (2N_2)N_3$ neurons





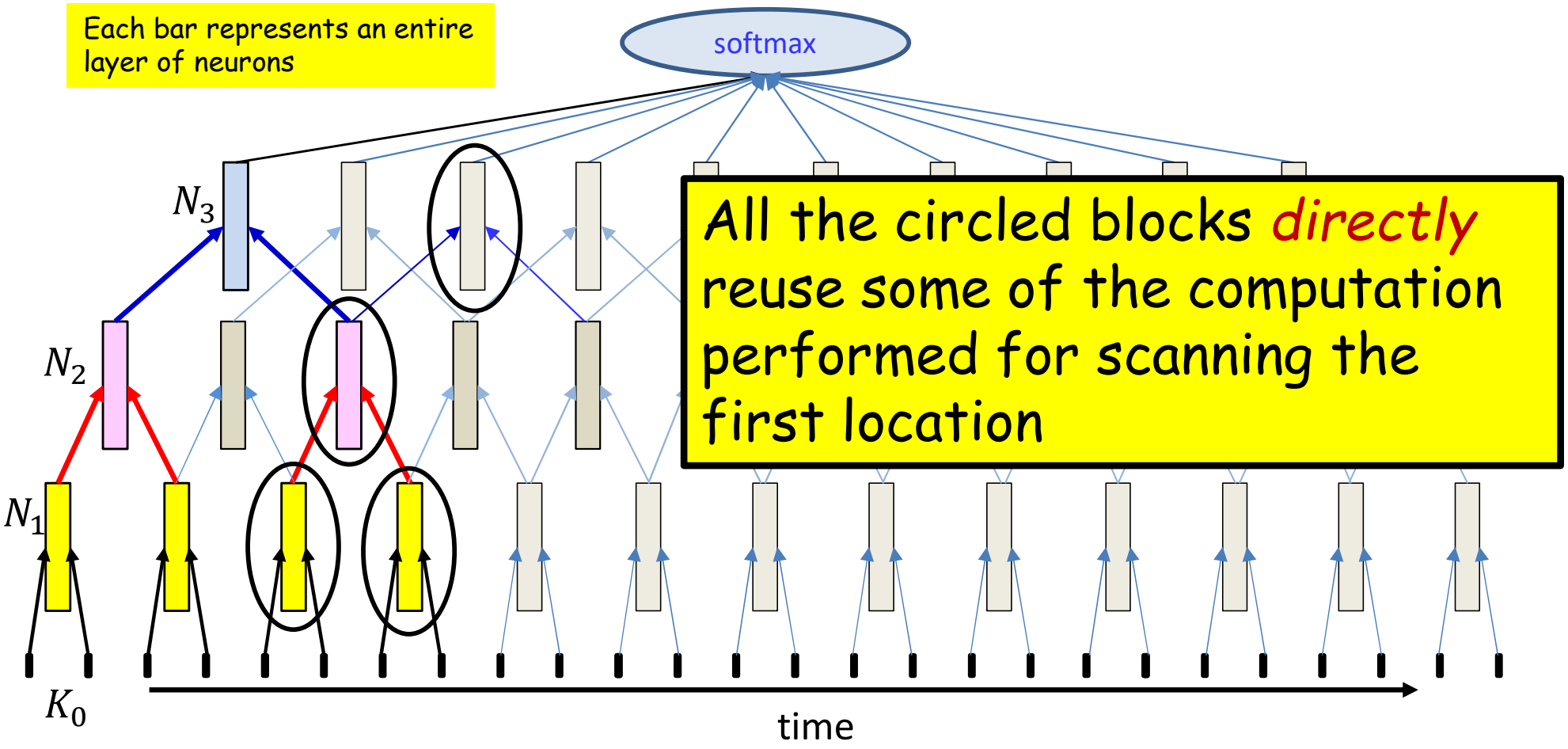
- Total parameters: $2DN_1 + 2N_1N_2 + 2N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Far fewer parameters than non-distributed scan with network with identical no. of neurons**

Distributed scanning: 3 levels



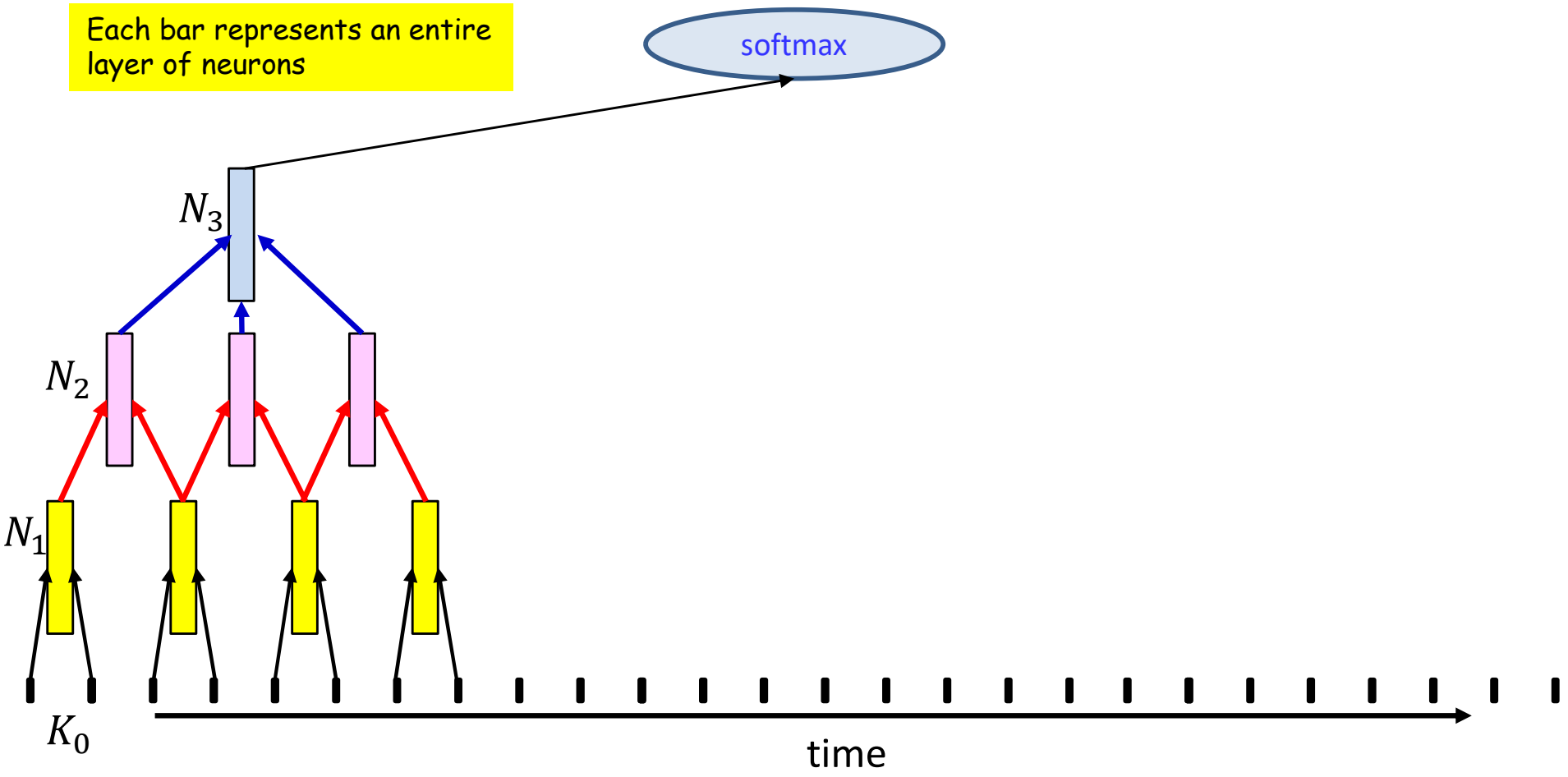
- Total parameters: $2DN_1 + 2N_1N_2 + 2N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Far fewer parameters than non-distributed scan with network with identical no. of neurons**

Distributed scanning: 3 levels



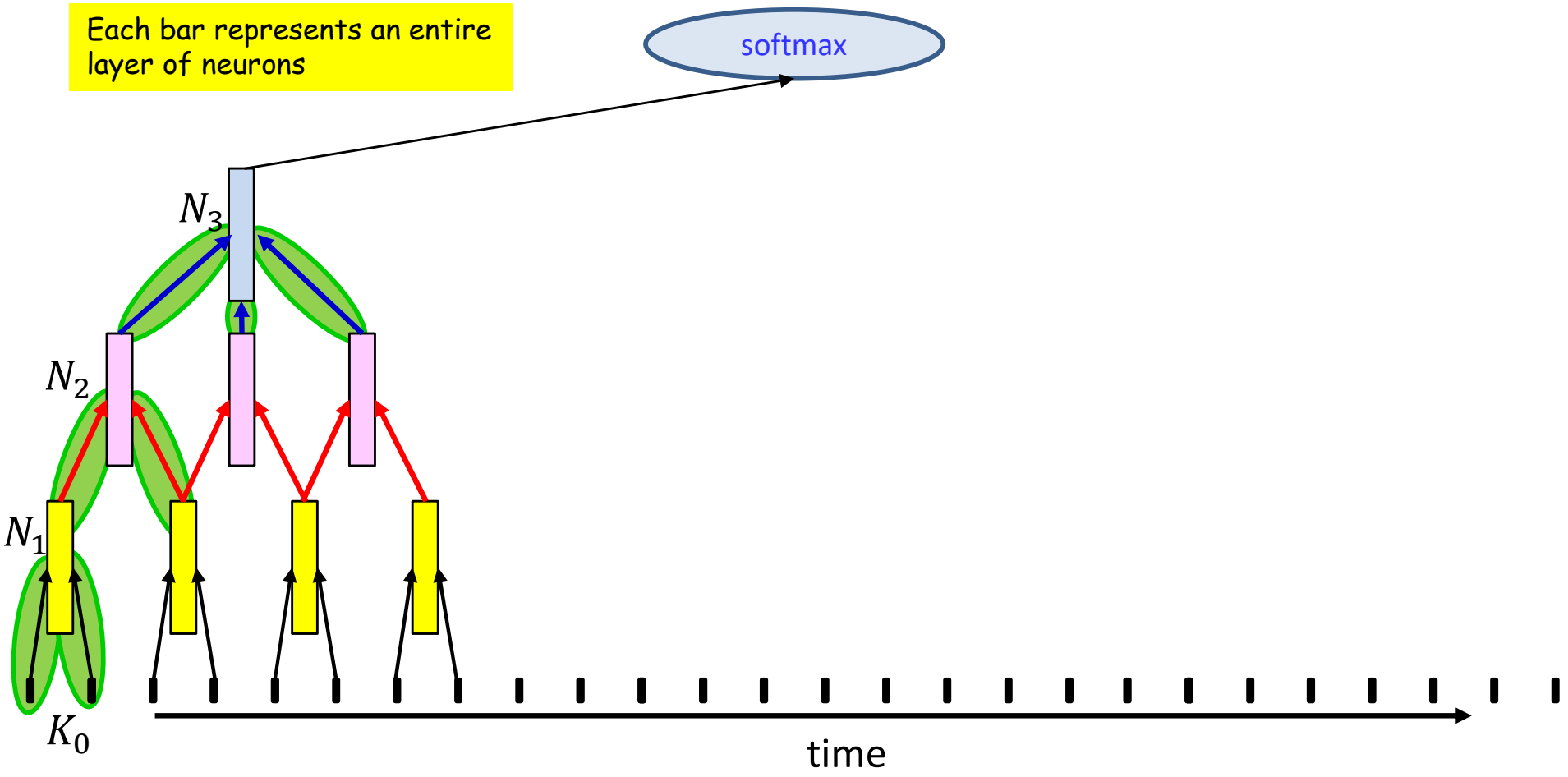
- Total parameters: $2DN_1 + 2N_1N_2 + 2N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Far fewer parameters than non-distributed scan by network with identical no. of neurons**
  - **Large additional gains from reuse of computation!!**

Distributed scanning



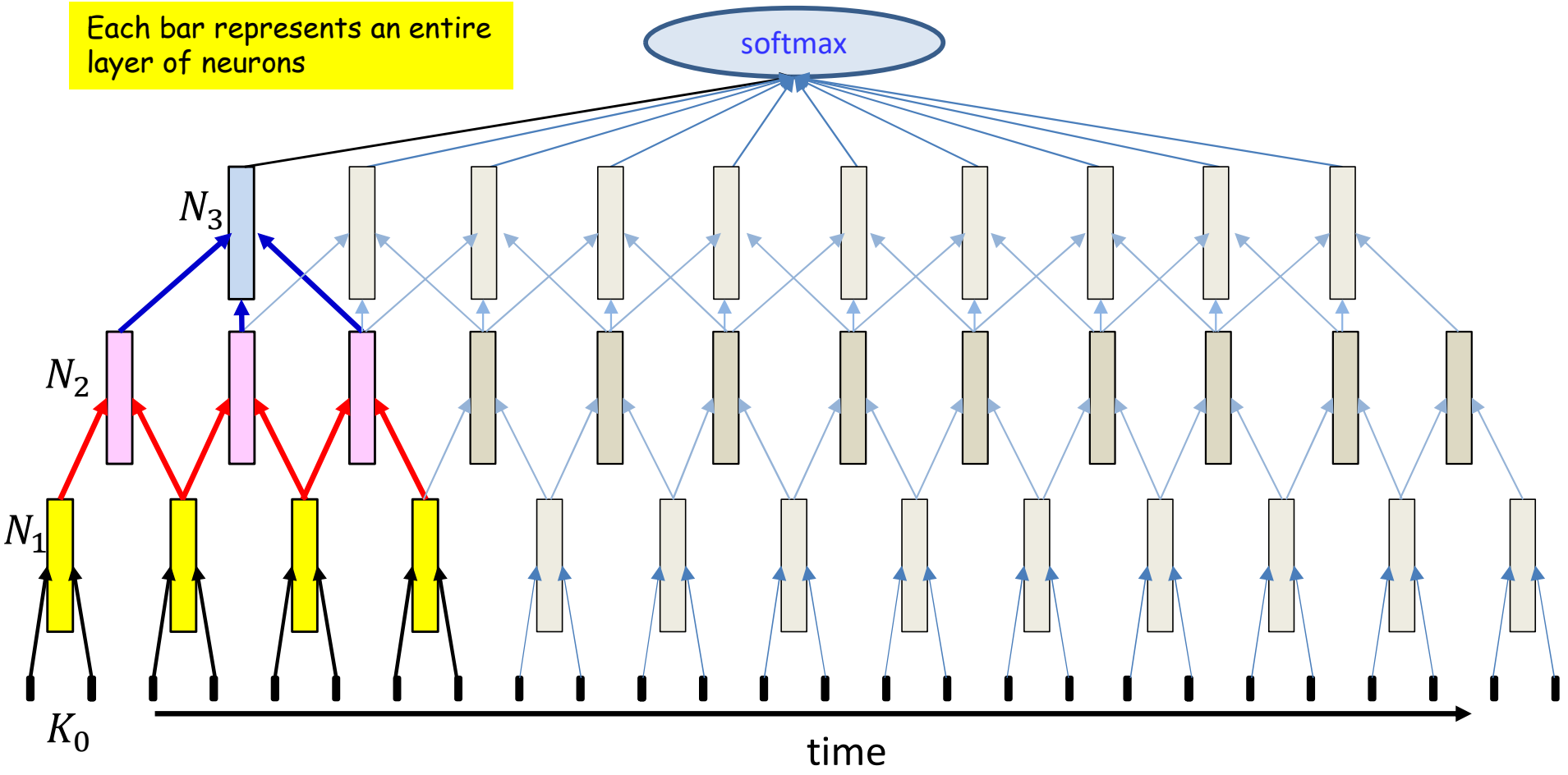
- Total parameters: $2DN_1 + 2N_1N_2 + 3N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Will have fewer parameters than a non-distributed structure with identical numbers of neurons**

Distributed scanning



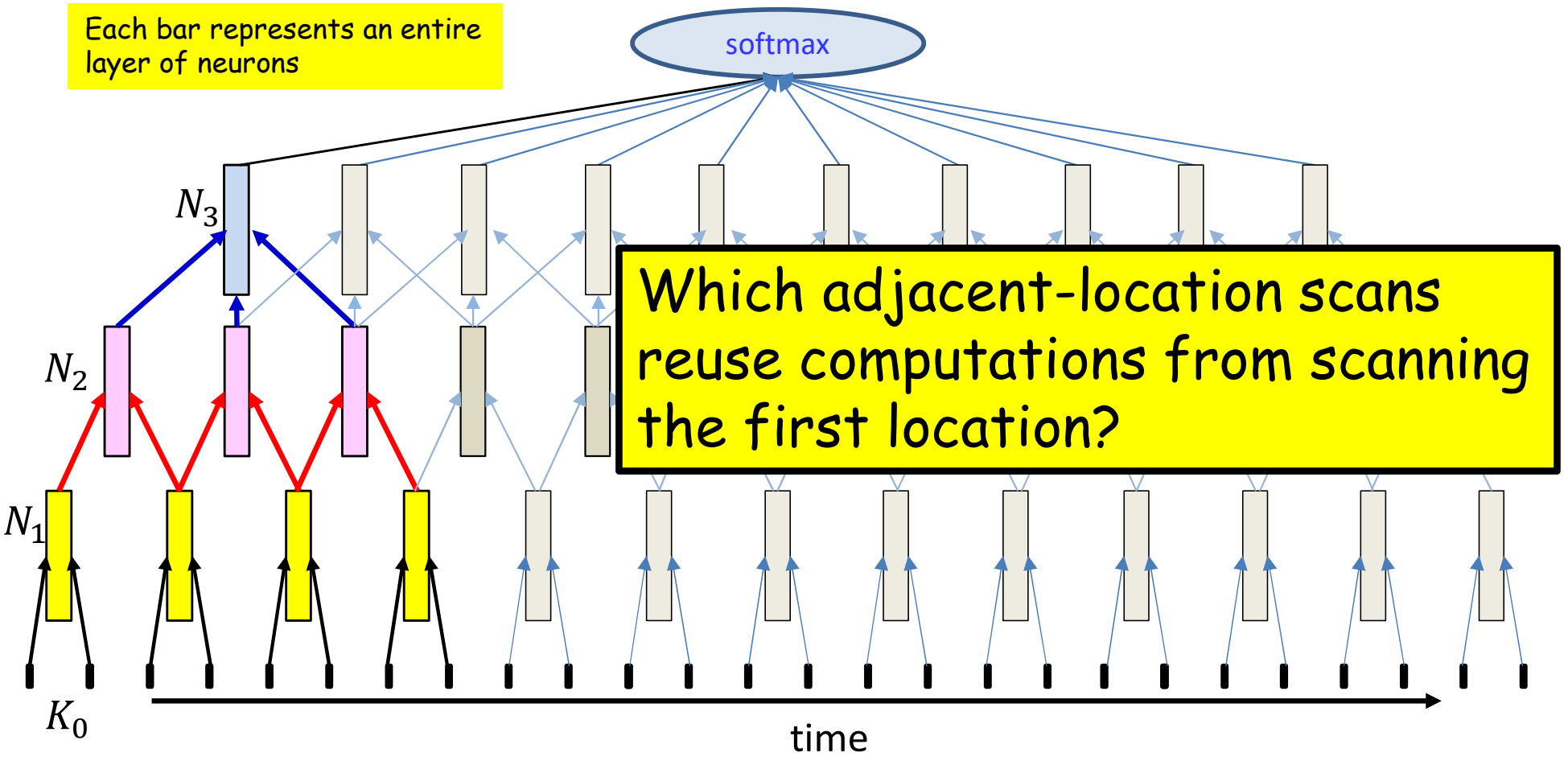
- Total parameters: $2DN_1 + 2N_1N_2 + 3N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Will have fewer parameters than a non-distributed structure with identical numbers of neurons**

Distributed scanning



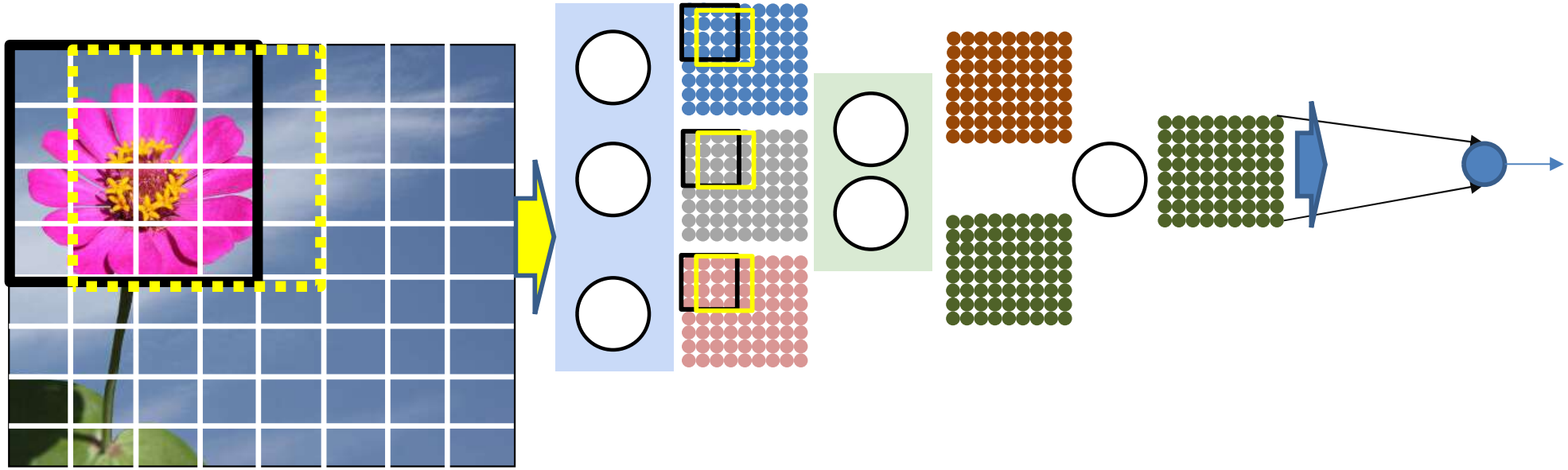
- Total parameters: $2DN_1 + 2N_1N_2 + 3N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Will have fewer parameters than a non-distributed structure with identical numbers of neurons**

Distributed scanning



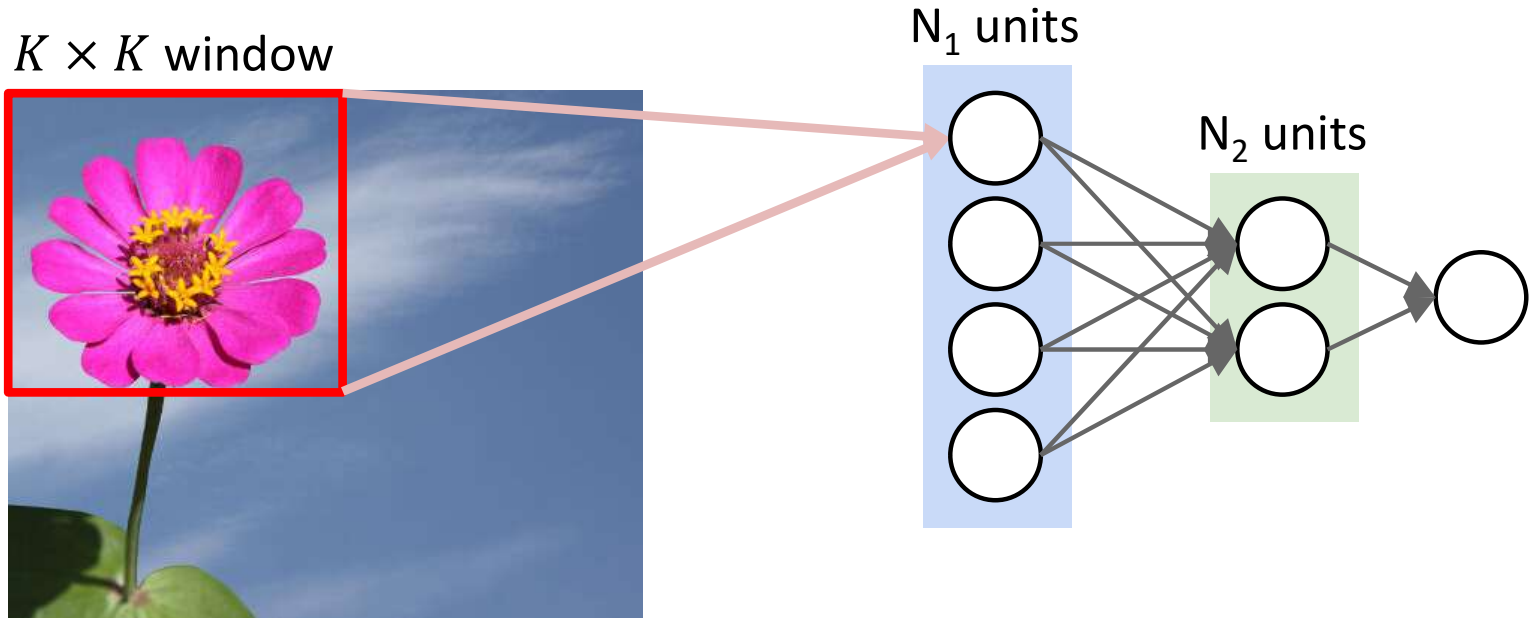
- Total parameters: $2DN_1 + 2N_1N_2 + 3N_2N_3$
  - More generally: $K_0DN_1 + K_1N_1N_2 + K_2N_2N_3$
  - **Will have fewer parameters than a non-distributed structure with identical numbers of neurons**
  - **Also benefits much more from shared computation in the scans of adjacent locations**

The computational benefits of distributed representation



- We reuse computation
  - E.g. for the two adjacent windows shown in black and yellow, the 12 central values are reused
  - The response is computed only once for each smaller block, and reused in many adjacent windows
    - This occurs at each layer of the network

Images: *Undistributed* network



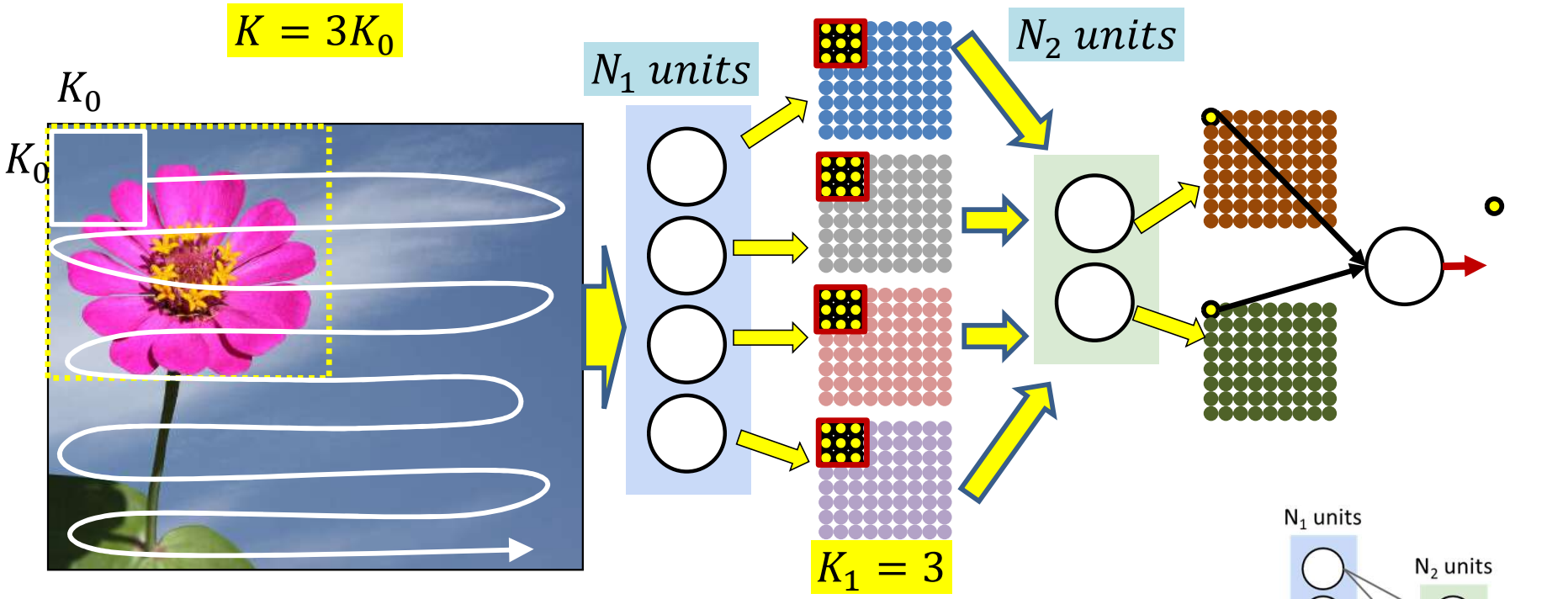
- Only need to consider what happens in *one* block
  - All other blocks are scanned by the same net

2-D version

- $(K^2 + 1)N_1$ weights in first layer
- $(N_1 + 1)N_2$ weights in second layer
  - $(N_{i-1} + 1)N_i$ weights in subsequent ith layer

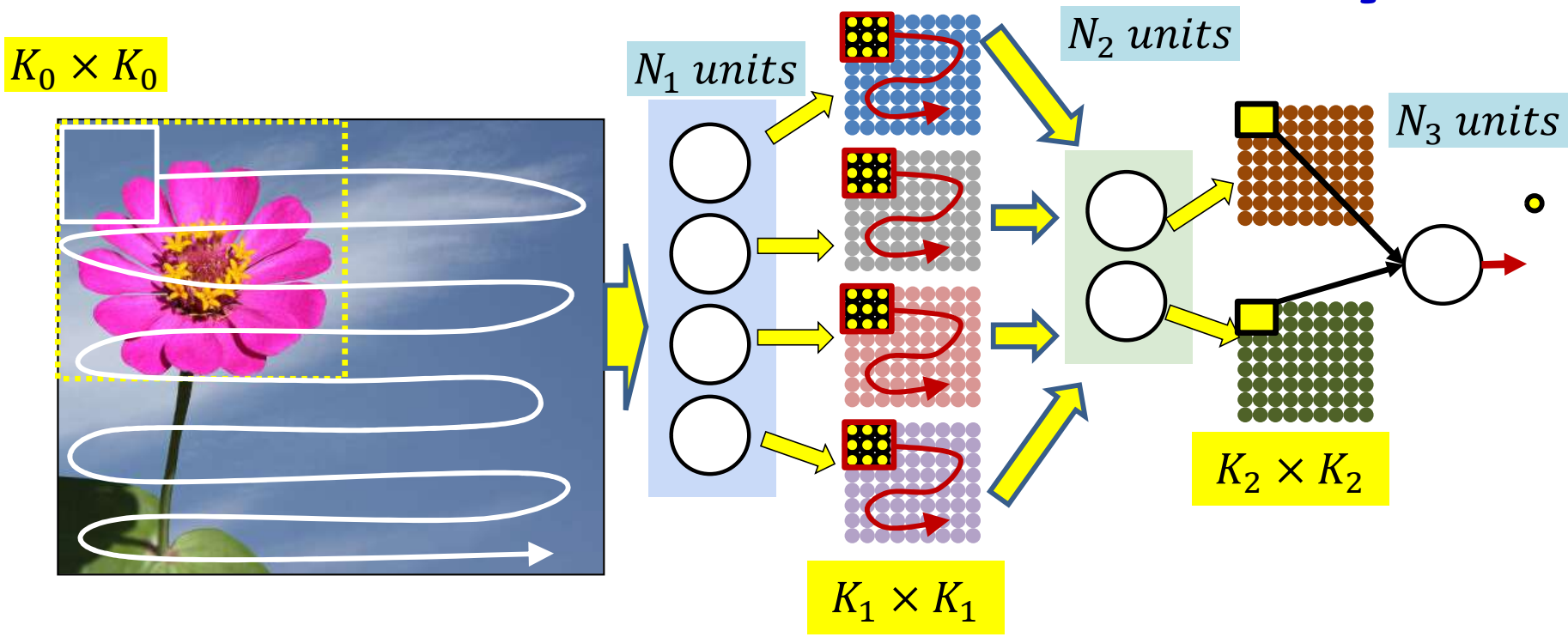
- Total parameters: $\mathcal{O}(K^2 N_1 + N_1 N_2 + N_2 N_3 \ldots)$
  - Ignoring the bias term

When distributed over 2 layers



- First layer: N_1 lower-level units, each looks at K_0^2 pixels
  - $N_1(K_0^2 + 1)$ weights
- Second layer needs $(9N_1 + 1)N_2$ weights
  - $(K_1^2 N_1 + 1)N_2$ weights
- Subsequent layers need $N_{i-1}N_i$ weights when distributed over only 2 layers
  - Total parameters: $\mathcal{O}(K_0^2 N_1 + K_1^2 N_1 N_2 + N_2 N_3 \ldots)$
    - (ignoring bias)

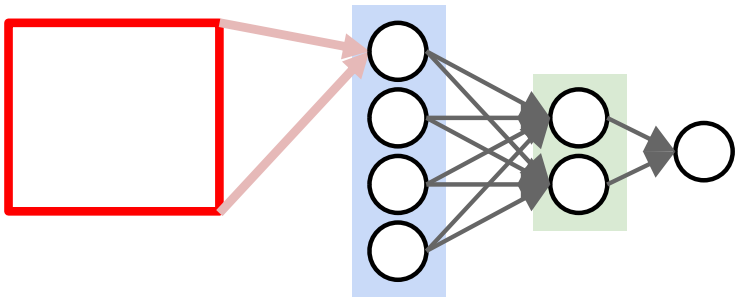
When distributed over 3 layers



- First layer: N_1 lower-level (groups of) units, each looks at K_0^2 pixels
  - $N_1(K_0^2 + 1)$ weights
- Second layer: N_2 (groups of) units looking at groups of $K_1 \times K_1$ connections from each of N_1 first-level neurons
  - $(K_1^2 N_1 + 1)N_2$ weights
- Third layer:
  - $(K_2^2 N_2 + 1)N_3$ weights
- Subsequent layers need $N_{i-1}N_i$ neurons
  - Total parameters: $\mathcal{O}(K_0^2 N_1 + K_1^2 N_1 N_2 + K_2^2 N_2 N_3 + \cdots)$

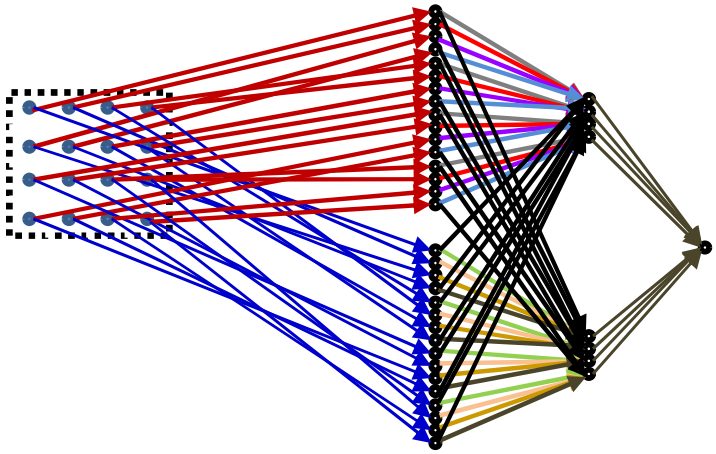
Comparing Number of Parameters

Conventional MLP, not distributed



- $\mathcal{O}(K^2N_1 + N_1N_2 + N_2N_3 \ldots)$
- For this example, let $K = 16$, $N_1 = 4, N_2 = 2, N_3 = 1$
- Total 1034 weights

Distributed (3 layers)



- $\mathcal{O}(K_0^2N_1 + K_1^2N_1N_2 + K_2^2N_2N_3 + \cdots)$
- Here, let $K = 16$, $K_0 = 4$, $K_1 = 4$, $N_1 = 4, N_2 = 2, N_3 = 1$
- Total 64+128+2 = 194 weights

Why distribute?

- Distribution forces *localized* patterns in lower layers
  - More generalizable

- Number of parameters…
  - Large (sometimes order of magnitude) reduction in parameters
    - Gains increase as we increase the depth over which the blocks are distributed
  - Significant gains from shared computation

- **Key intuition: Regardless of the distribution, we can view the network as "scanning" the picture with an MLP**
  - **The only difference is the manner in which parameters are shared in the MLP**

Story so far

- Position-invariant pattern classification can be performed by scanning the input for a target pattern
  - Scanning is equivalent to composing a large network with shared subnets
- The operations in scanning the input with a full network can be equivalently reordered as
  - scanning the input with individual neurons in the first layer to produce scanned "maps" of the input
  - Jointly scanning the "map" of outputs by all neurons in the previous layers by neurons in subsequent layers
- The scanning block can be distributed over multiple layers of the network
  - Results in significant reduction in the total number of parameters

Poll 4 (@629)

- What are the benefits of distributed scanning (mark all that are true)
  - It enables hierarchical composition of patterns, which results in more generalizable models
  - It results in greatly reduced number of parameters
  - It provides computational advantage through reuse of computation
  - It results in reduced memory requirement for intermediate variables during inference

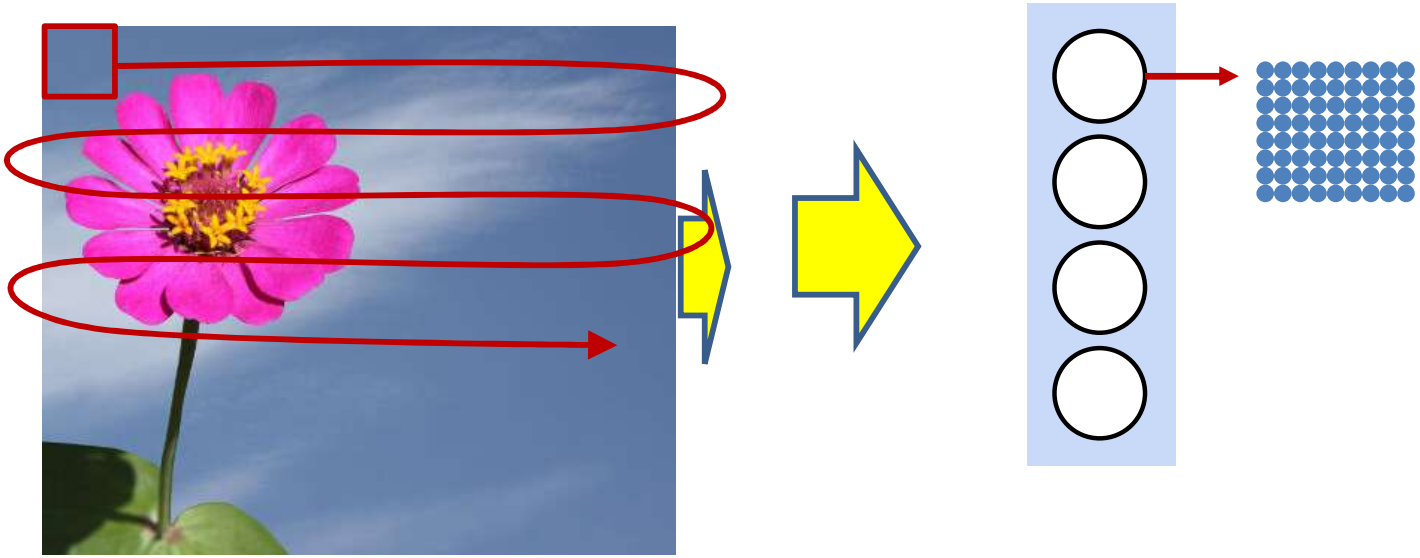
Poll 4

- What are the benefits of distributed scanning (mark all that are true)
  - **It enables hierarchical composition of patterns, which results in more generalizable models**
  - **It results in greatly reduced number of parameters**
  - **It provides computational advantage through reuse of computation**
  - It results in reduced memory requirement for intermediate variables during inference

Some final touches

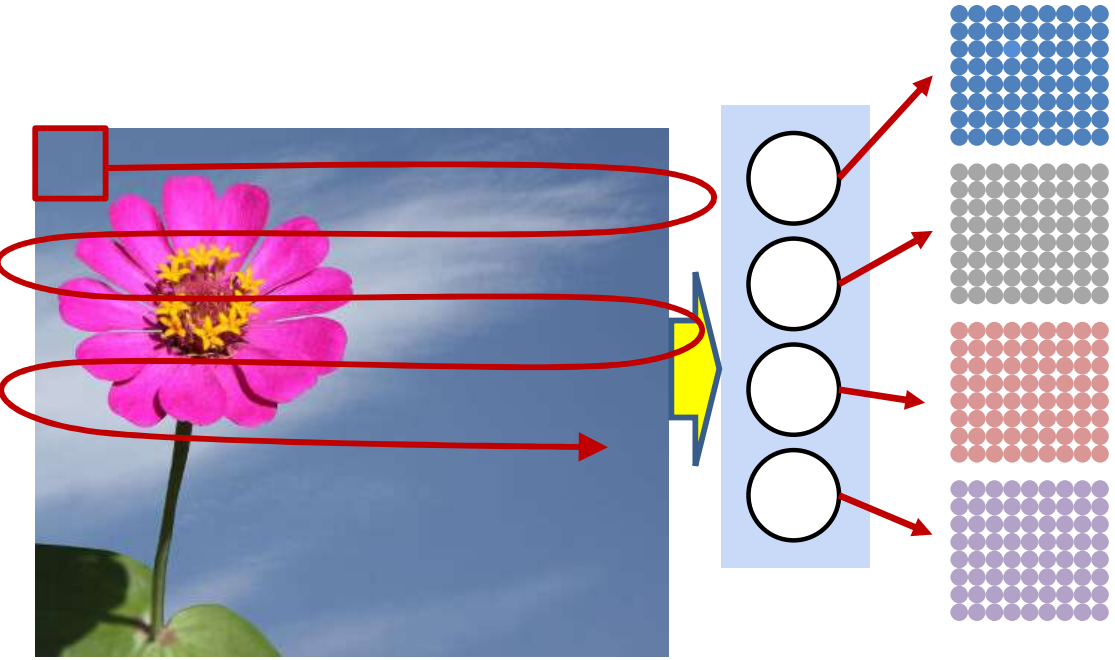
- Terminology
  - Filters and receptive fields
- Shrinking the maps
  - Scanning with strides
- Accounting for jitter
  - Pooling

Hierarchical composition: A different perspective



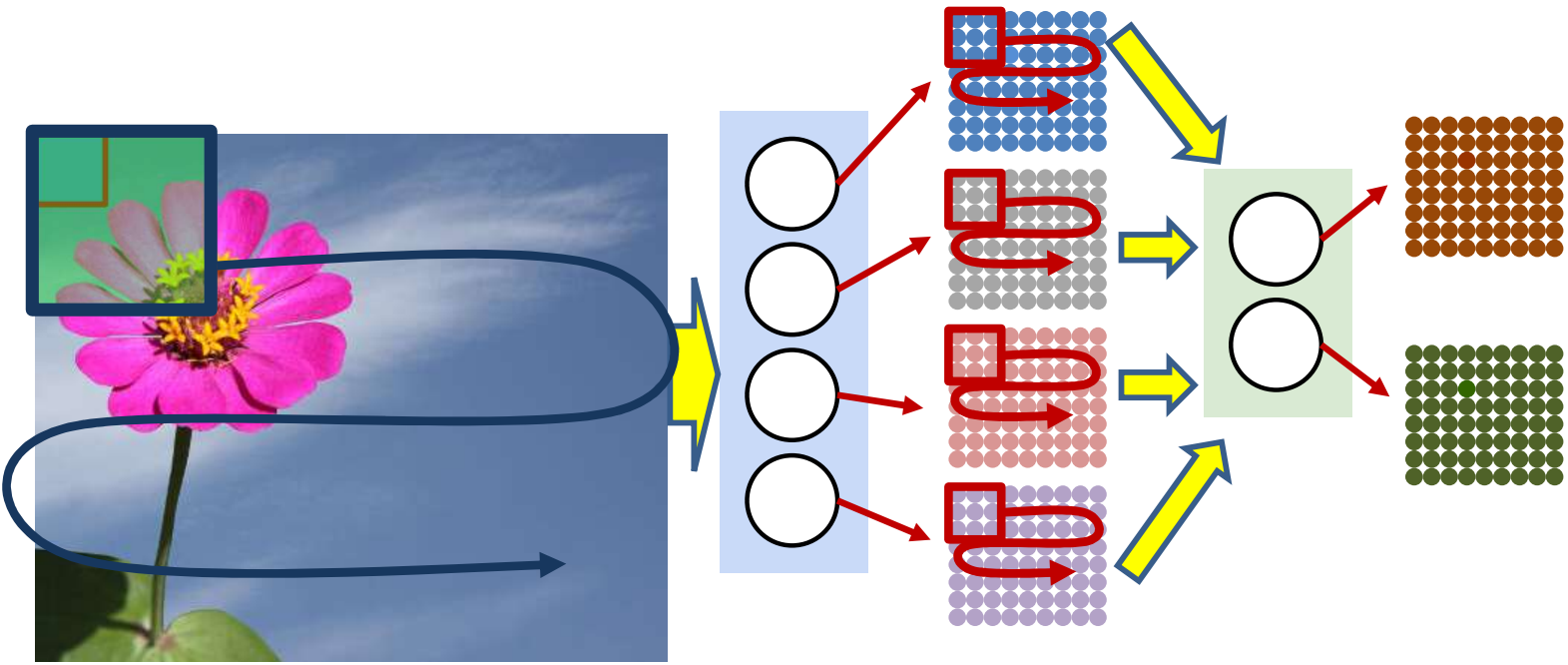
- The entire operation can be redrawn as before as maps of the entire image
- Each neuron scans and “redraws” the input with some features enhanced
  - The specific features that the neuron detects

Building up patterns



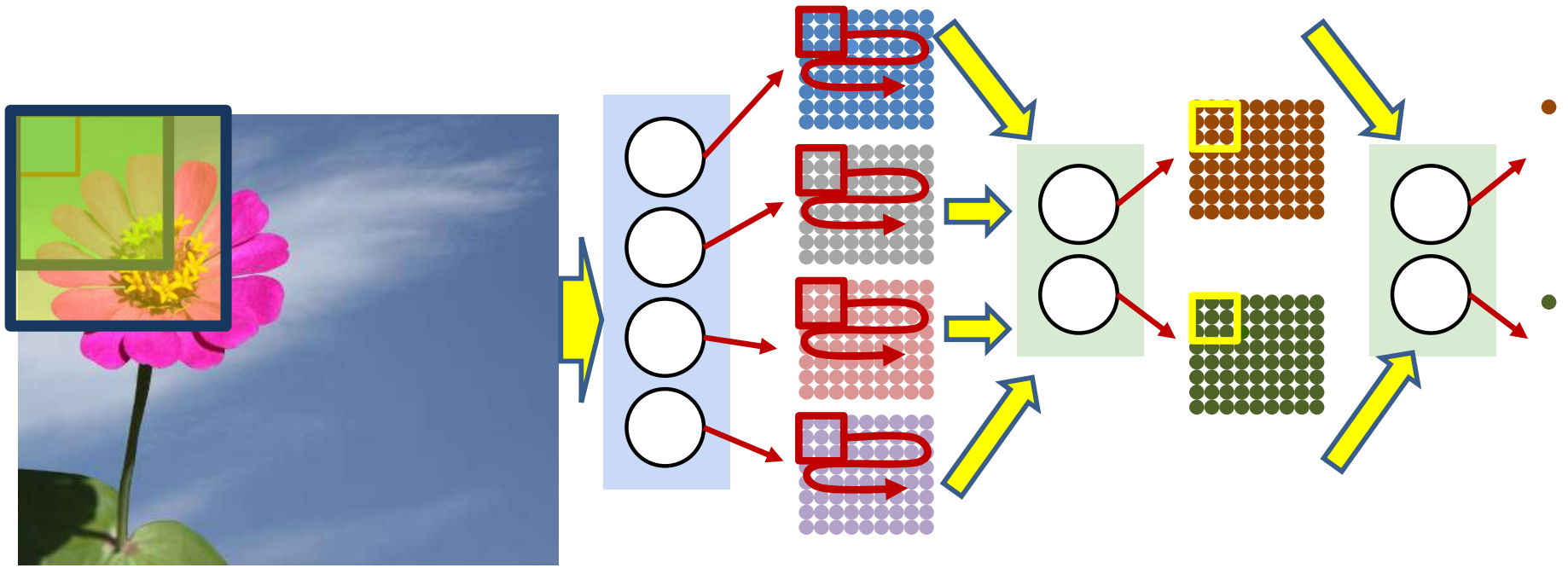
- The first layer looks at small *sub* regions of the input image
  - Sufficient to detect, say, petals
    - And enhances those

The higher-level neurons



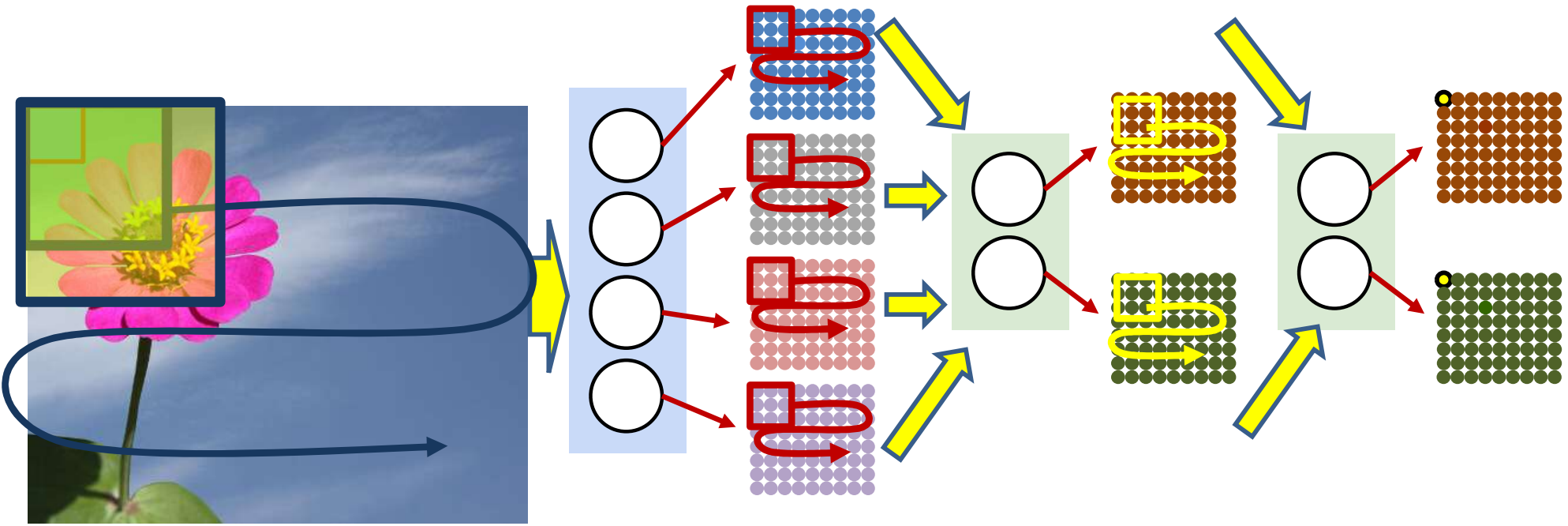
- The first layer looks at *sub* regions of the main image
  - Sufficient to detect, say, petals
- The second layer looks at *regions* of the output of the first layer
  - To put the petals together into a part of a flower
  - This corresponds to looking at a larger region of the original input image

Still-higher level neurons



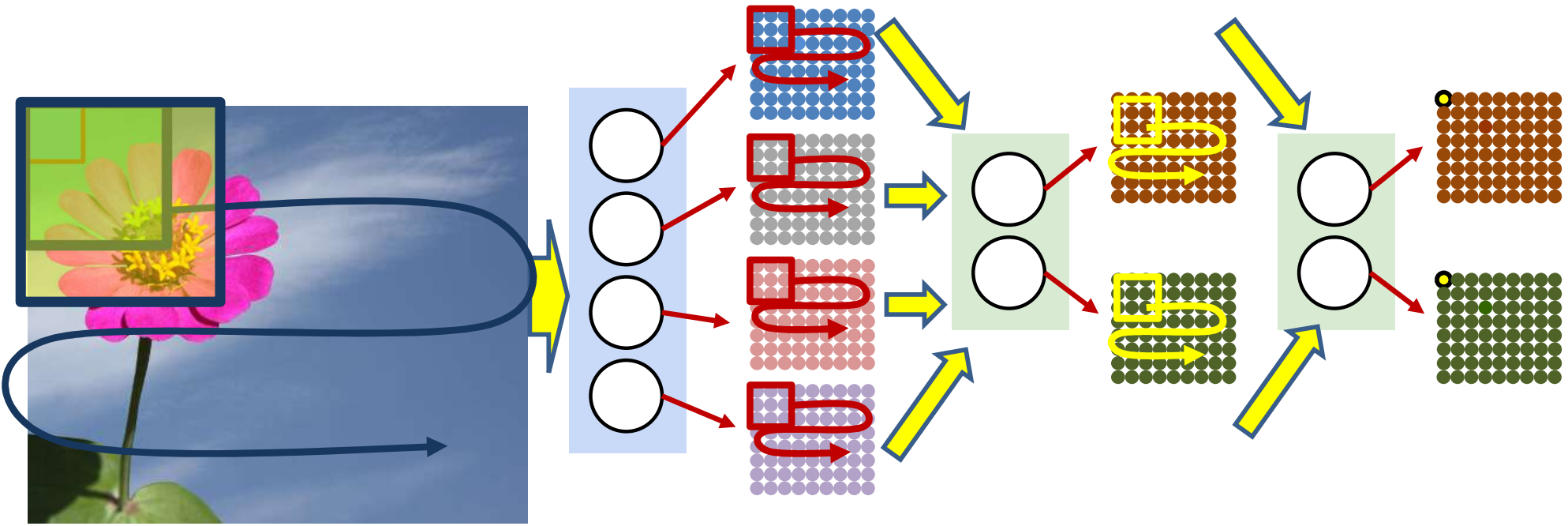
- The first layer looks at *sub* regions of the main image
  - Sufficient to detect, say, petals
- The second layer looks at *regions* of the output of the first layer
  - To put the petals together into a part of a flower
  - This corresponds to looking at a larger region of the original input image
- We may have any number of layers in this fashion

Still-higher level neurons



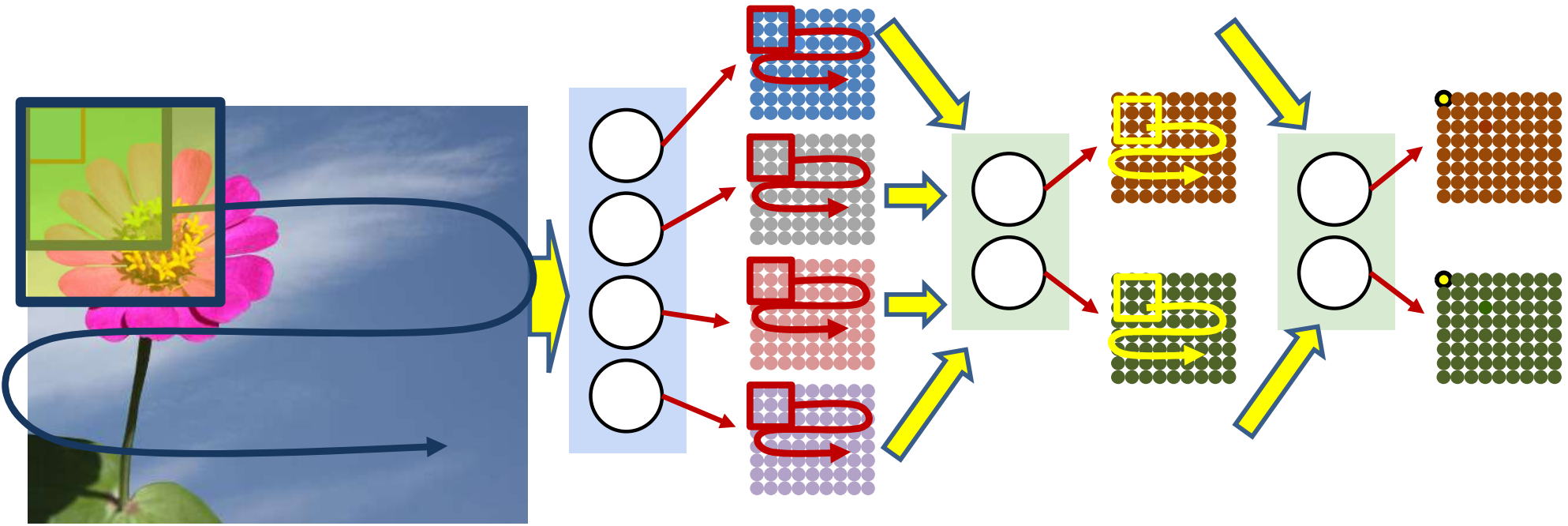
- The first layer looks at *sub* regions of the main image
  - Sufficient to detect, say, petals
- The second layer looks at *regions* of the output of the first layer
  - To put the petals together into a flower
  - This corresponds to looking at a larger region of the original input image
- We may have any number of layers in this fashion

Terminology: Filters



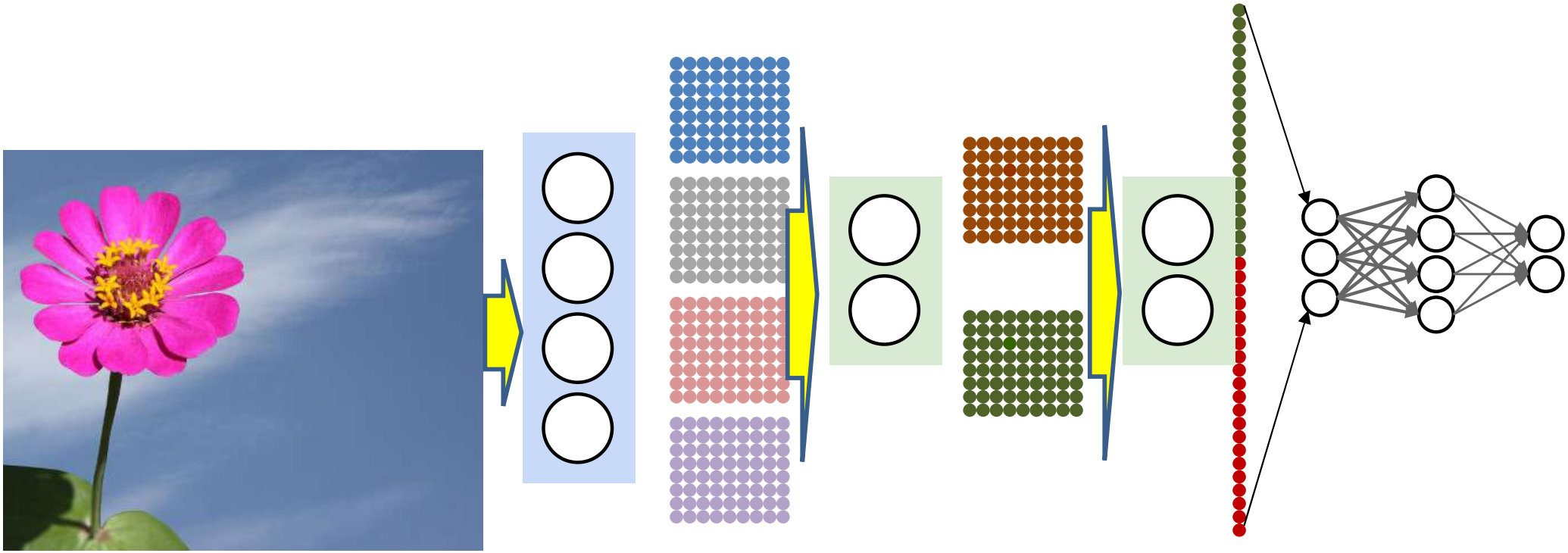
- The set of weights for each of the scanning neurons is generally called a “filter”
  - Each filter scans for a pattern in the map it operates on

Terminology: Receptive fields



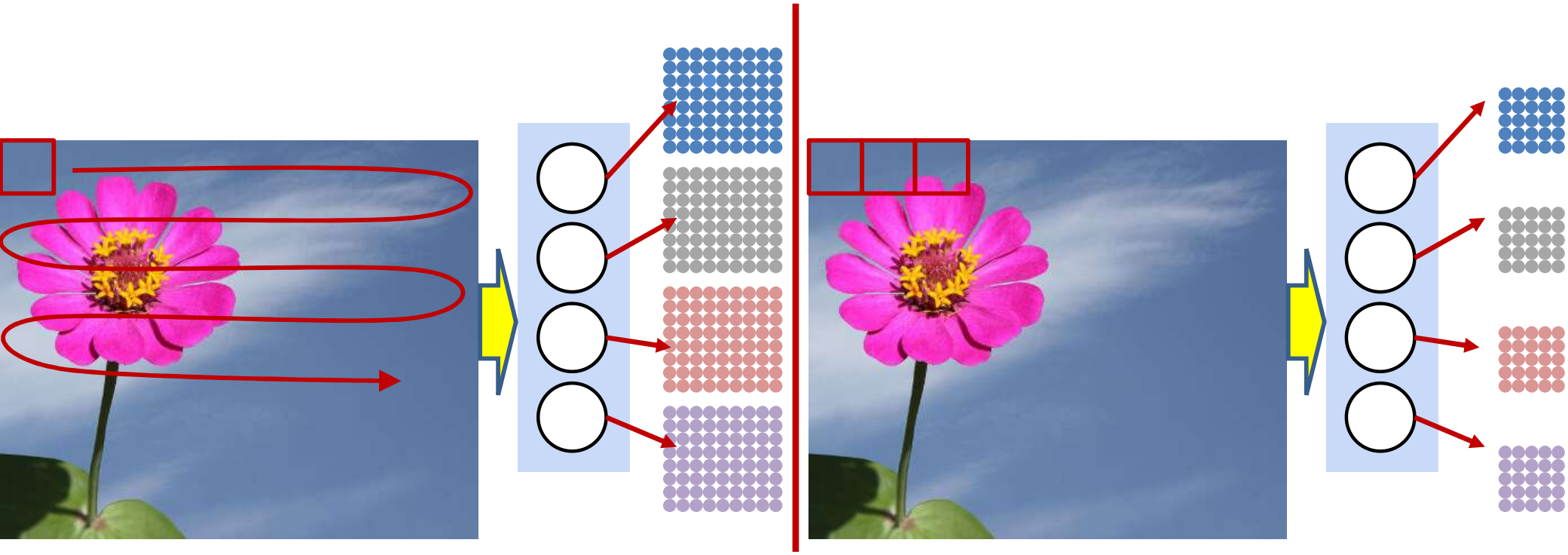
- The pattern in the *input* image that each neuron responds to is its "Receptive Field"
  - The squares show the *sizes* of the receptive fields for the first, second and third-layer neurons
- The actual receptive field for a first layer neuron is simply its arrangement of weights
- For the higher layer neurons, the actual receptive field is not immediately obvious and must be *calculated*
  - What patterns in the input do the filters actually respond to?
  - Will not actually be simple, identifiable patterns like "petal" and "inflorescence"

Terminology: "Flattening"



- The rectangular maps of the neurons in the final layer of the scanning network will generally be reorganized into a vector before passing them to the final softmax or MLP
- This restructuring of the maps is often called "flattening"

Modification 1: Convolutional "Stride"



- The scans of the individual "filters" may advance by more than one pixel at a time
  - The "stride" may be greater than 1
  - Effectively increasing the granularity of the scan
    - Saves computation, sometimes at the risk of losing information
- This will result in a reduction of the size of the resulting maps
  - They will shrink along each axis by a factor equal to the stride
  - To prevent guaranteed loss of information by the shrinking, the number of output maps (neurons) must be S^2 the number of input maps, where S is the stride
- This can happen at any layer

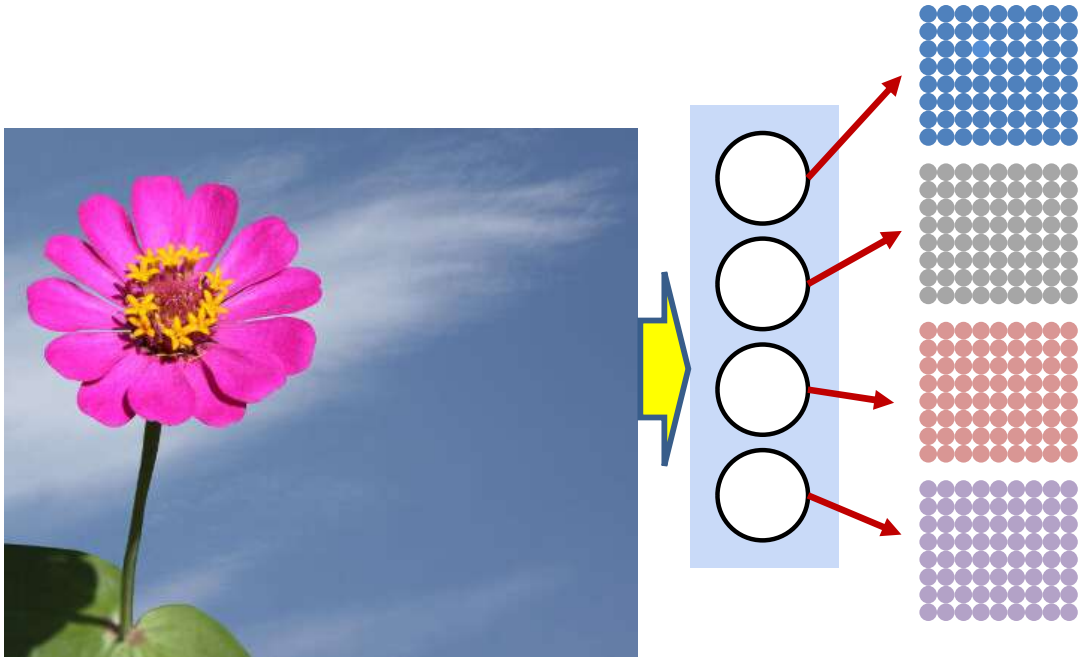
CNN with strides

The weight W(l) is now a 4D $D_l \times D_{l-1} \times K_l \times K_l$ tensor (assuming square receptive fields)

```
Y(0) = Image
for l = 1:L  # layers operate on vector at (x,y)
    m = 1
    for x = 1:stride:W_{l-1}-K_l+1
        n = 1
        for y = 1:stride:H_{l-1}-K_l+1
            segment = Y(l-1,x:x+K_l-1,y:y+K_l-1)
            z(l,m,n) = W(l).segment #tensor inner prod.
            Y(l,m,n) = activation(z(l,m,n))
            n++
        m++

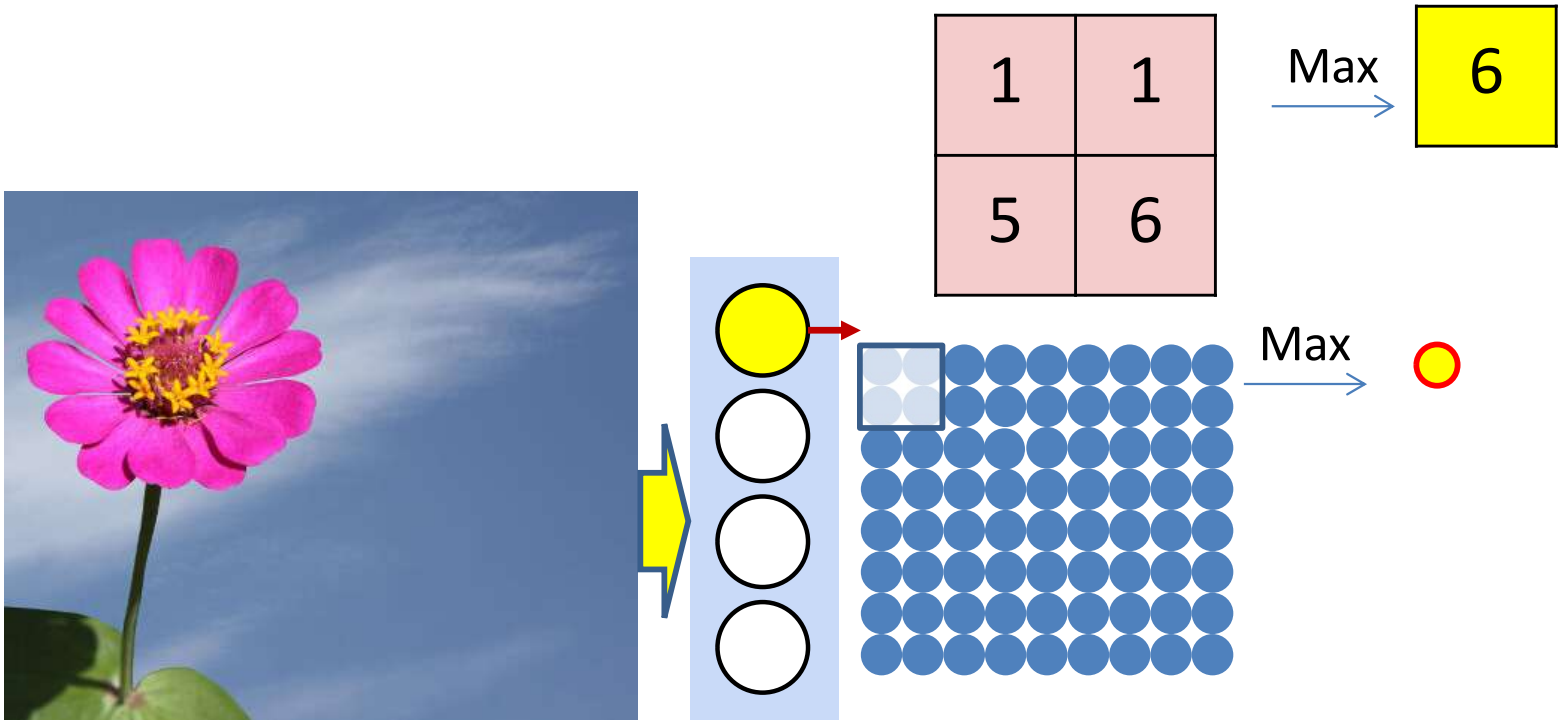
Y = softmax( Y(L) )
```

Modification 2: Accounting for jitter



- We would like to account for some jitter in the first-level patterns
  - If a pattern shifts by one pixel, is it still a petal?
- Solution: “Pooling” neurons
  - Replace $f(\sum_i w_i x_i + b)$ with $pool(x_1, .. x_K)$
  - The pool activation computes a jitter (or permutation) invariant function of its inputs

Pooling Activations



- Max pooling:

$$f(x_1, \cdots, x_K) = \max(x_1, \cdots, x_K)$$

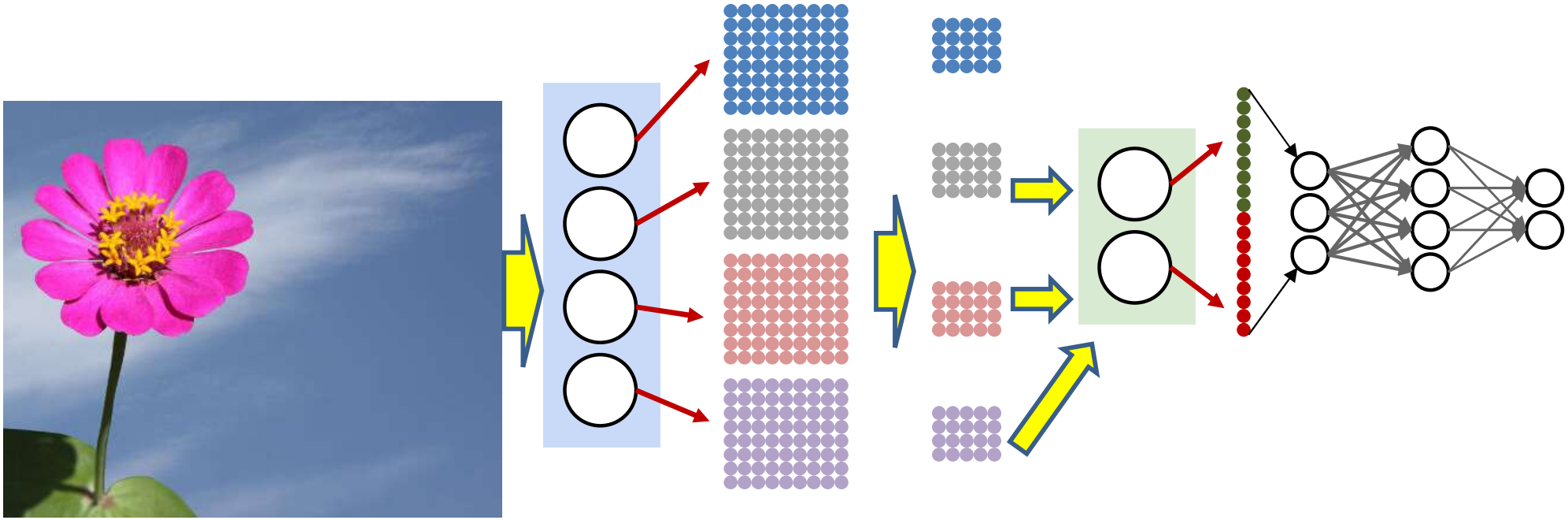
- Mean pooling:

$$f(x_1, \cdots, x_K) = \text{mean}(x_1, \cdots, x_K)$$

- Other options exist

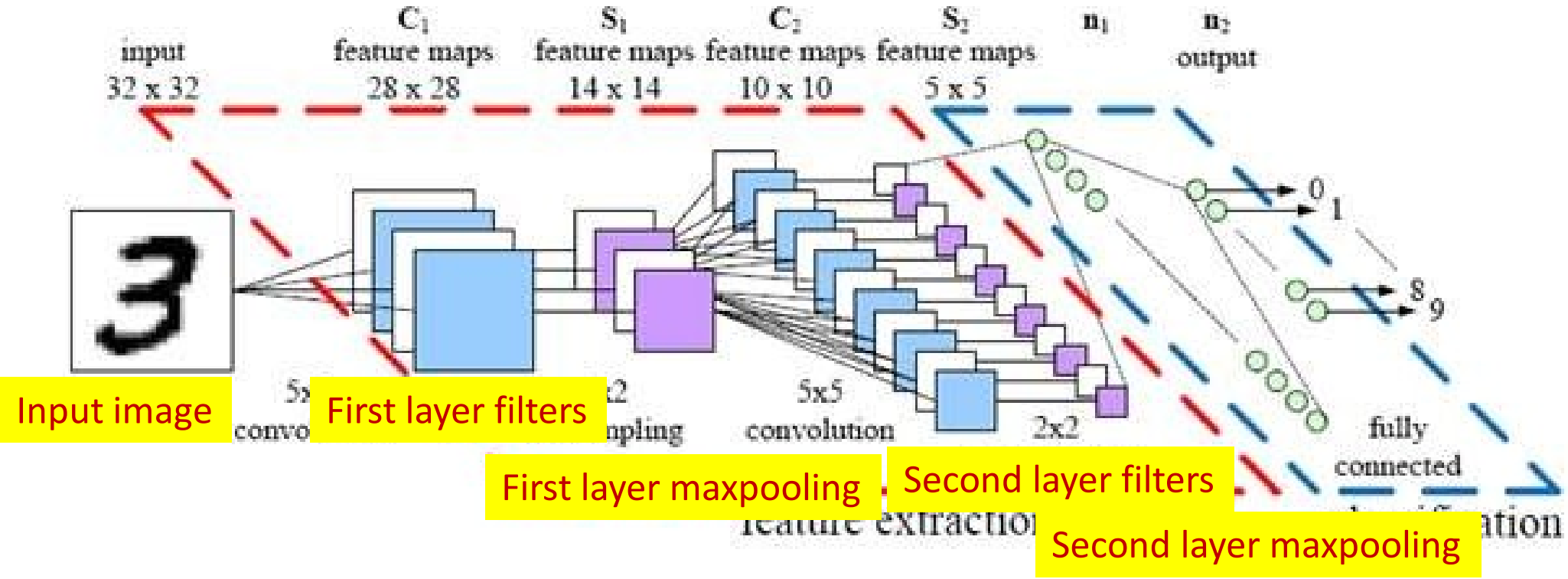
- Must always follow a layer of "regular" neurons
  - The "regular" filters detect patterns; the pooling activations introduce jitter invariance on their outputs
- Typically used with a stride > 1
  - Result in a shrinking, or "downsampling" of the maps

The overall structure



- This entire structure is called a ***Convolutional Neural Network***

Convolutional Neural Network



Story so far

- Neural networks learn patterns in a hierarchical manner
  - Simple to complex
- Pattern classification tasks such as “does this picture contain a cat” are best performed by scanning for the target pattern
- Scanning for patterns can be viewed as classification with a large shared-parameter network

- Scanning an input with a network and combining the outcomes is equivalent to scanning with individual neurons
  - First level neurons scan the input
  - Higher-level neurons scan the “maps” formed by lower-level neurons
  - A final “decision” layer (which may be a max, a perceptron, or an MLP) makes the final decision
- The scanned “block” can be distributed over multiple layers for efficiency
- At each layer, a scan by a neuron may optionally be followed by a “max” (or any other) “pooling” operation to account for deformation

- For 2-D (or higher-dimensional) scans, the structure is called a convnet
- For 1-D scan along time, it is called a Time-delay neural network